

Copy and Paste 2 解説

隈部 壮(DEGwer)

はじめに

- 一日目お疲れ様でした

はじめに

- 一日目お疲れ様でした
- なんだこの問題名は...たまげたなあ...

問題概要

- 次のクエリを順に処理した時の最終的な文字列の最初K文字を求めてください

問題概要

- 次のクエリを順に処理した時の最終的な文字列の最初K文字を求めてください
- クエリ: 文字列のある区間をコピーし、ある場所に張り付ける

問題概要

- 次のクエリを順に処理した時の最終的な文字列の最初K文字を求めてください
- クエリ: 文字列のある区間をコピーし、ある場所に張り付ける
- 文字列の長さがM文字を超えたらM文字になるまで後ろを削る

具体例

debate

$K=16$

$M=18$

具体例

debate

具体例

debate

具体例

debatatee

具体例

debatatee

具体例

debatate

具体例

ebatadebatatee

具体例

ebatadebatatee

具体例

ebatadebatatee

具体例

ebatadebatatadebata
tee

具体例

ebatadebatatadebat

具体例

ebata**debat**atadebat

具体例

batatebatadebatatad
ebat

具体例

batatebatadebatata

具体例

batatebatadebatata

具体例

batate~~ta~~batadebatata

具体例

batatetabatadebata

具体例

batate**tabatadebata**

具体例

tabatadebatabatateta
batadebata

具体例

tabatadebatatate

田端でバタバタ

tabatadebatatate

K=16なので先頭のtabatadebatataが答え

小課題1

- 問題文に書かれている通りにコードを書いてください

小課題1

- 問題文に書かれている通りにコードを書いてください
- 10点が得られる(やったぜ)

小課題1

- 問題文に書かれている通りにコードを書いてください
- 10点が得られる(やったぜ)
- Copy and Pasteの解説を見るとわかりやすいかもです

満点解法

- 永続赤黒木

満点解法

- 赤黒木とは

満点解法

- 赤黒木とは
- 平衡二分探索木の種類です

満点解法

- 赤黒木とは
- 平衡二分探索木の種類です
- Copy and Pasteをちゃんと復習しておきましたか?という問題でした

満点解法

- 実装が大変

満点解法

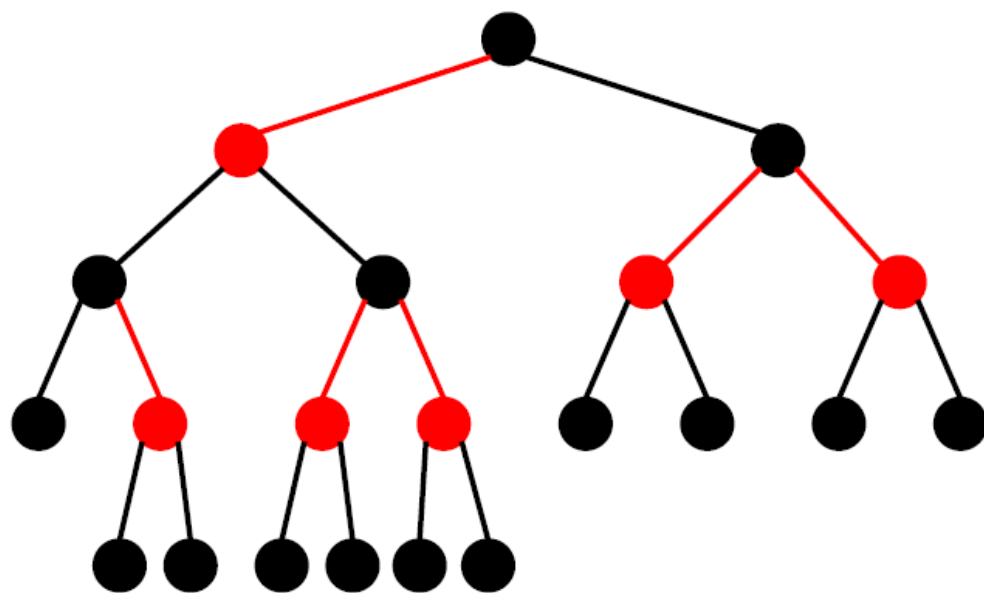
- 実装が大変
- 困ったなあ...困ったなあ...

満点解法

- 実装が大変
- 困ったなあ...困ったなあ...
- 頑張って実装していきましょう

満点解法

赤黒木



満点解法

- というのは嘘です

満点解法

- というのは嘘です
- もう一度制約を見直してみましょう

満点解法

制限

すべての入力データは以下の条件を満たす.

- $1 \leq K \leq 200$.
- $1 \leq M \leq 1\,000\,000\,000$.
- S の各文字は英アルファベットの小文字 ('a' - 'z') である.
- $K \leq (S \text{ の長さ}) \leq \min\{M, 200\,000\}$.
- $1 \leq N \leq 200\,000$.
- i 回目の操作の直前の文字列の長さを L_i とすると, $0 \leq A_i < B_i \leq L_i$ および $0 \leq C_i \leq L_i$ ($1 \leq i \leq N$).

満点解法

制限

すべての入力データは以下の条件を満たす.

- $1 \leq K \leq 200$.
- $1 \leq M \leq 1\,000\,000\,000$.
- S の各文字は英アルファベットの小文字 ('a' - 'z') である.
- $K \leq (S \text{ の長さ}) \leq \min\{M, 200\,000\}$.

満点解法

制限

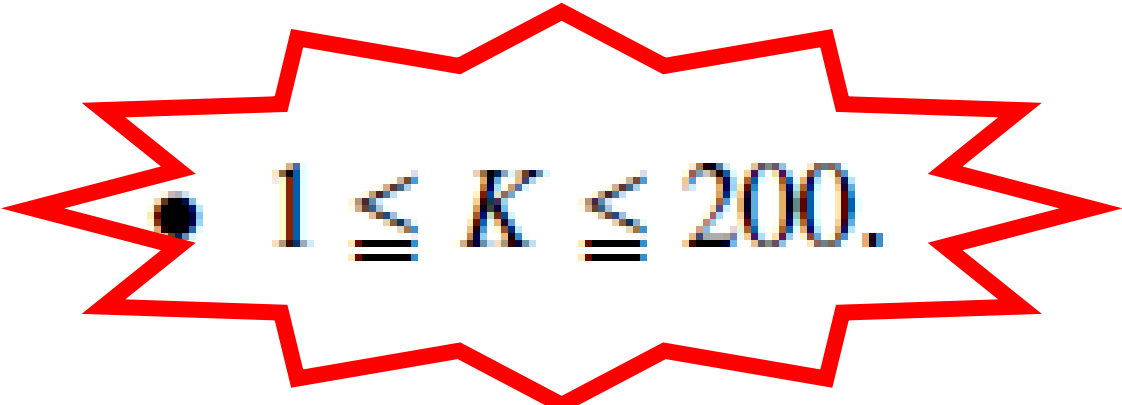
すべての入力データは以下の条件を

- $1 \leq K \leq 200$.
- $1 \leq M \leq 1\,000\,000\,000$.

満点解法

制限

すべての入力データは以下の条件を



- $1 \leq K \leq 200.$

- $1 \leq M \leq 1\,000\,000\,000.$

満点解法

制限

すべての入力データは以下の条件を

- $1 \leq K \leq 200.$

- $1 \leq M \leq 1\,000\,000\,000.$

満点解法

- K が小さい

満点解法

- **Kが小さい!!!!!!**

満点解法

- K が小さい
- $O(KN)$ が間に合いそう？

満点解法

- K が小さい
- $O(KN)$ が間に合いそう？
- 実際に間に合う

満点解法

- 「操作をした後で t 文字目にある文字は、操作の前は何文字目にあったか？」

満点解法

- 「操作をした後で t 文字目にある文字は、操作の前は何文字目にあったか？」
- これが分かれば、逆向きの操作を順番にしてい
くことで t 文字目の文字が $O(N)$ で求められる

満点解法

- 「操作をした後で t 文字目にある文字は、操作の前は何文字目にあったか？」

満点解法

- 「操作をした後で t 文字目にある文字は、操作の前は何文字目にあったか？」
- 3通りの場合がある

満点解法

- 場合分け①: t 文字目の文字が、ペーストによって挿入される区間より前にある場合

満点解法

- 場合分け①: t 文字目の文字が、ペーストによって挿入される区間より前にある場合
- なにもおこらない

満点解法

- 場合分け②: t 文字目の文字が、ペーストによって挿入される区間の中にある場合

満点解法

- 場合分け②: t文字目の文字が、ペーストによって挿入される区間の中にある場合
- t文字目の文字はペーストされてその位置に来たので、t文字目の文字はもともとコピー元の文字列の中にある

満点解法

- 場合分け②: t 文字目の文字が、ペーストによって挿入される区間の中にある場合
- t 文字目の文字はペーストされてその位置に来たので、 t 文字目の文字はもともとコピー元の文字列の中にある
- A 文字目からの文字列を C 文字目からの位置にペーストした時、 t 文字目は $A+(t-C)$ 文字目にあった

満点解法

- 場合分け③: t 文字目の文字が、ペーストによって挿入される区間より後ろにある場合

満点解法

- 場合分け③: t 文字目の文字が、ペーストによって挿入される区間より後ろにある場合
- t 文字目より前に文字列が挿入されるので、長さ x の文字列が挿入される時 t 文字目の文字は $t-x$ 文字目にあった

満点解法

- これを1文字目~K文字目までについてすべて行くと、 $O(KN)$ でこの問題が解ける

満点解法

- これを1文字目~K文字目までについてすべて行くと、 $O(KN)$ でこの問題が解ける
- やったぜ(完全勝利)

得点分布

- 0点: 3人

得点分布

- 0点: 3人
- 10点: 9人

得点分布

- 0点: 3人
- 10点: 9人
- 100点: 7人

En-JOI-able Logo

Design 解説

tozangezan

問題概要

- 円状にJ,O,Iの文字が 4^K 個並んでいる
- いくつかを書き換えてどこから時計回りに読むとレベルKのJOI列になるようにしたい
- レベルKのJOI列 ($K > 1$) は、
(Jが 4^{K-1} 個)(Oが 4^{K-1} 個)(Iが 4^{K-1} 個)(レベルK-1)
と再帰的に定義 (レベル0はJ,O,Iどれでもよい)
- 文字を書き換える回数の最小値は？
- $1 \leq K \leq 10$ (文字列の長さは1048576以下)

小課題1 (30点)

- 実際にレベルKのJOI列を生成
JJJJOOOOIIIIJOI? みたいなもの
- 開始点を 4^K 通り試して全部見る
- 計算量は $O(16^K)$
- 予選通過した皆さんなら書けるはず
- これだけで30点ってやばくない？

満点解法

- ちょっと視点を変える

JJJJ...JJJJ みたいな区間を一まとめにできたら各開始場所に対して $O(K)$ 回で計算できる。

- この区間をまとめて処理したいときは
- 累積和
- 完

満点解法

- 具体的に
- 各開始点を全部試す ... $O(4^K)$
- それぞれに対して、
 - 最初から 4^{K-1} 個のうちJでないものの個数
 - その後 4^{K-1} 個のうち0でないものの個数
 - ...
- この総和を累積和を使って求める ... $O(K)$
- 全体 ... $O(K \cdot 4^K)$

イネージ

• 部分点解法

JJJJJJJJJJJJJJJJ000000000000000000|||

JJJJJJJJJJJJJJ0000000000000000|||

JJJJJJJJJJJJJJJ000000000000000000|||

JJJJJJJJJJJJJJ0000000000000000|||

JJJJJJJJJJJJJJJJJ000000000000000000|||

JJJJJJJJJJJJJJJJJ0000000000000000 | | | |

JJJJJJJJJJJJJJJJJJJ000000000000000000|||

● ● ● ● ● ●

イメージ

- 満点解法

J-----0-----I-----

J-----0-----I-----

J-----0-----I-----

J-----0-----I-----

J-----0-----I-----

J-----0-----I-----

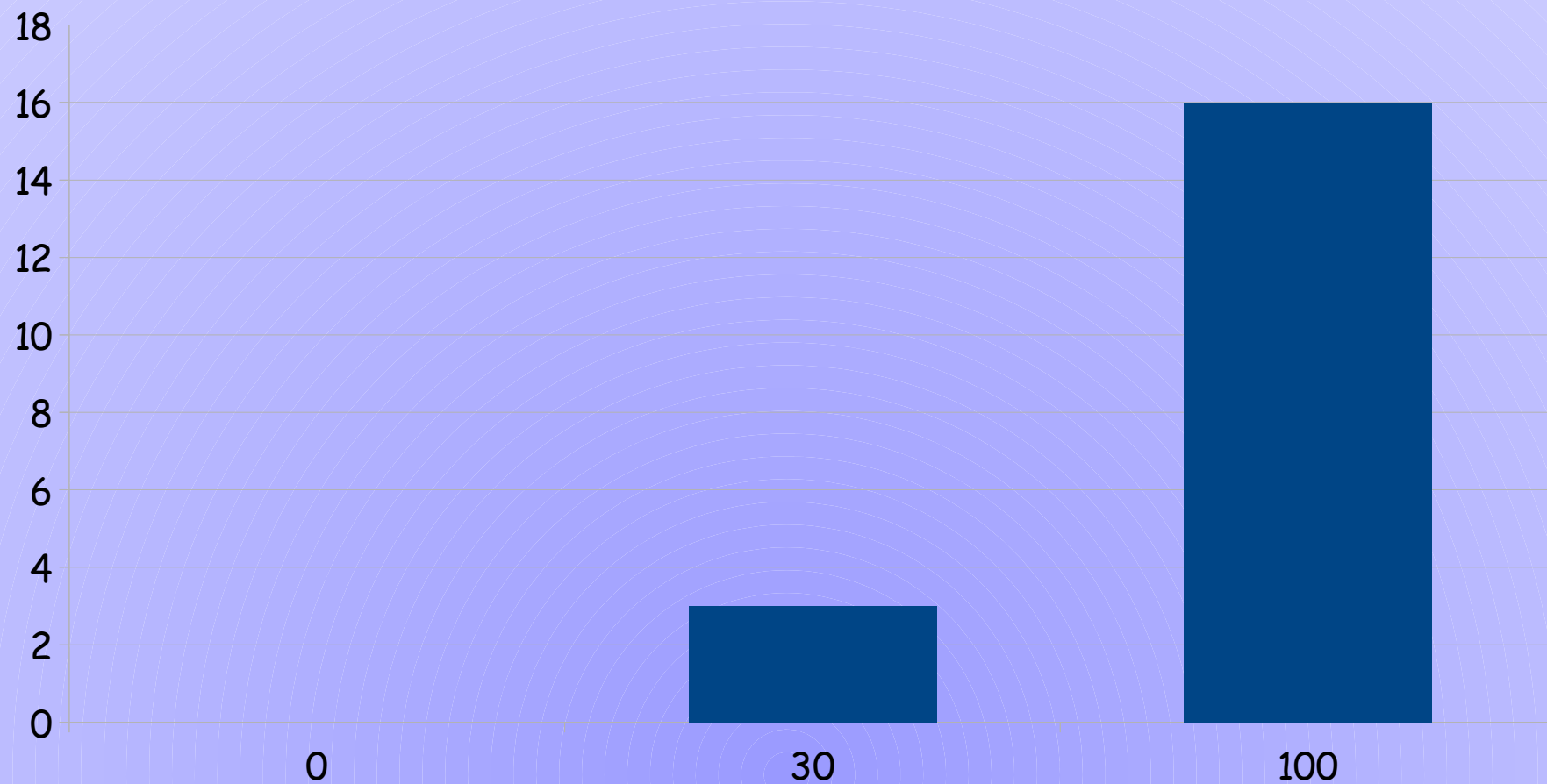
J-----0-----I-----

.....

ところで

- ・区間の和といわれて反射的にBIT書いてTLEするみたいなのはやめましょう
- ・この問題はどうみても更新クエリは飛ばない

得点分布



JOI 春合宿 Day 1

たのしいたのしい家庭菜園 解説

チューター(IOI2011, タイ) 城下慎也(@phidnight)

東京大学 理学部情報科学科 3 年

問題概要

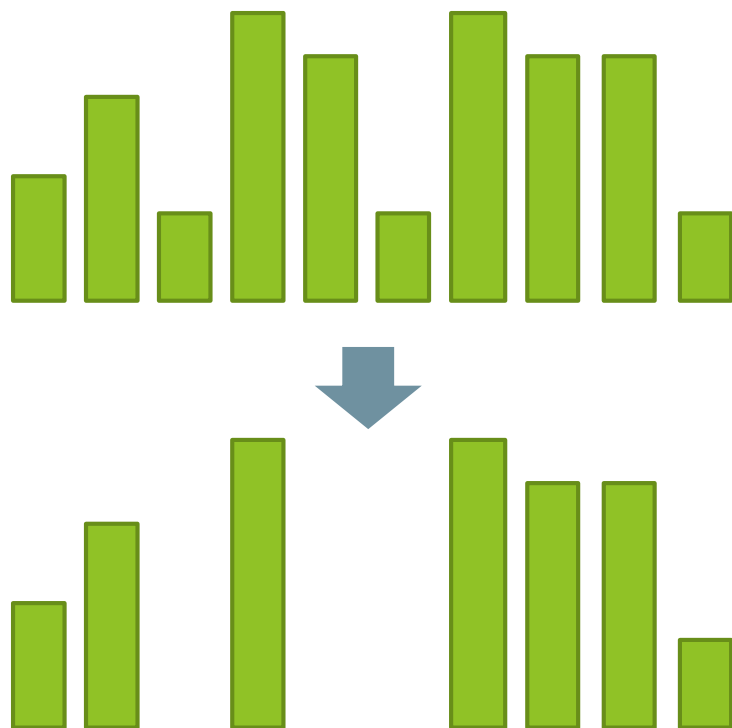
- ▶ N 本の IOI 草が一行に並んでいます。
- ▶ いくつかの IOI 草を抜くことによって、実を付けた IOI 草の合計価値から抜いた費用の合計を引いた値を最大化させてください。
- ▶ $3 \leq N \leq 100,000$
- ▶ その他：でかい

部分点解法 1

- ▶ どの IOI 草を抜くか、という組み合わせをすべて試します。
- ▶ 全部で 2^N 通り考えられるので、各組み合わせについて、どの IOI 草が実を付けるか、を探索します。
- ▶ 計算量は $O(N * 2^N)$ となり、小課題 1 が通ります。

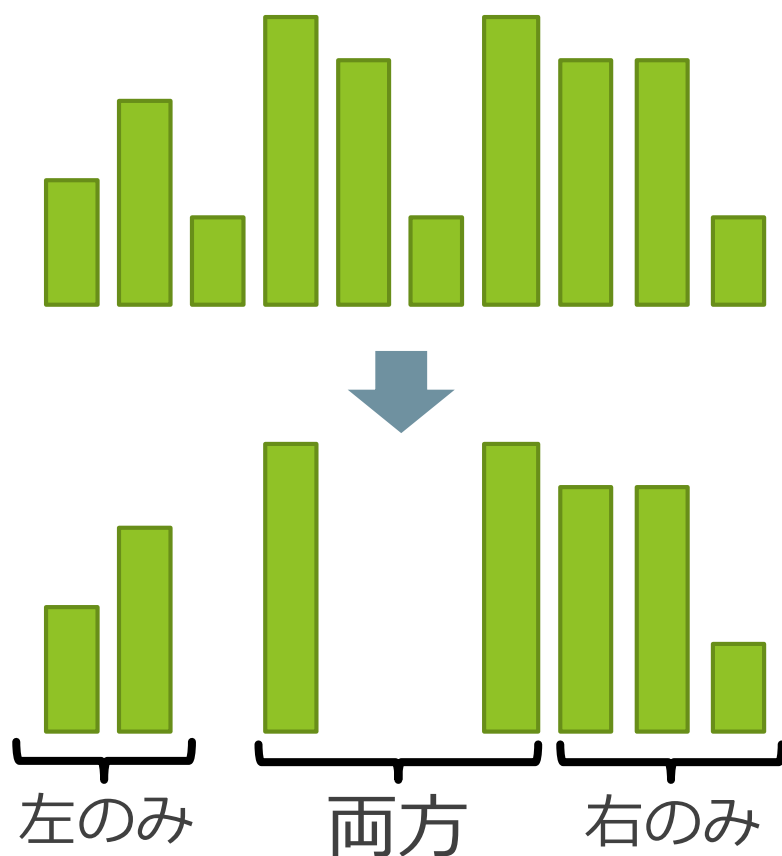
部分点解法 2,3

- ▶ 実をつける IOI 草について考えてみます。



部分点解法 2,3

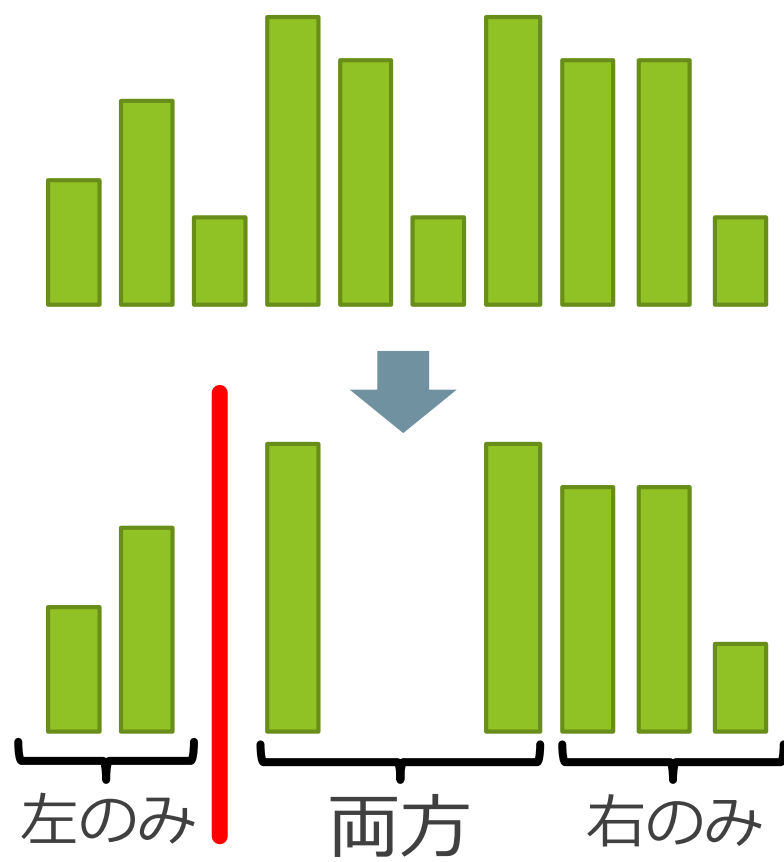
▶ 実をつける 101 草について考えてみます。



▶ 左図のように、どちらからの光が当たるかで分類できます。

部分点解法 2,3

- ▶ 実をつける IOI 草について考えてみます。



- ▶ 左図のように、どちらからの光が当たるかで分類できます。
- ▶ 「左のみ」と「その他」で分けて考えることにします。

部分点解法 2,3

- ▶ ある $|O|$ 草 i について、「 $|O|$ 草 i までで採用したものの高さが最大で j であるものの利益の最大値」を $dpL[i][j]$ とおきます。
- ▶ 「左」を「右」に変えたものを $dpR[i][j]$ と置きます。

部分点解法 2,3

▶ このとき、

$dpL[i][j] = \max(dpL[i-1][l] + P[i] \ (l \leq j, H[i] = j),$

$dpL[i-1][j] \ (H[i] \leq j),$

$P[i] - CSUM(1, i-1) \ (i \text{ より前を抜く}, H[i] = j),$

$dpL[i-1][j] - C[i] \ (i \text{ を抜く}))$

となります (境界部分については省略)。

▶ これは $O(N^2)$ で計算できます。

部分点解法 2,3

- ▶ $dpR[i][j]$ も同様の方針で作ります。
- ▶ $maxL[i]=\max(dpL[i][j])$, $maxR[i]=\max(dpR[i][j])$ とおくと、以下のうちの最大が答えとなります。

1. $maxL[i]$ ($1 \leq i \leq N$)
2. $maxR[i]$ ($1 \leq i \leq N$)
3. $maxL[i]+maxR[i+1]$ ($1 \leq i \leq N - 1$)

満点解法に向けて

- ▶ 先ほどの部分点解法 3 では、dp の状態数が最大 N^2 個あるので、すべての dp を 1 つ 1 つ求めていると $O(N^2)$ よりは良くなりません。
- ▶ 最終的な答えは、maxL, maxR さえ求まっていれば計算できます。
- ▶ maxL, maxR を、dp の 1 つ 1 つを算出せずに ($O(N^2)$ 費やさずに) 求めたいです。

満点解法に向けて

- ▶ 配列 $\{dp[i][0], dp[i][1], \dots, dp[i][M]\}$ (M は高さの上限) から、 $\{dp[i+1][0], dp[i+1][1], \dots, dp[i+1][M]\}$ を求める方法について考えてみます。 $dp[i][j]$ ($0 \leq j \leq M$) を求めたいとき、

※ $dpL[i][j] = \max(dpL[i-1][l] + P[i] \ (l \leq j, H[i] = j),$
 $dpL[i-1][j] \ (H[i] \leq j),$
 $P[i] - CSUM(1, i-1) \ (i \text{ より前を抜く}, H[i] = j),$
 $dpL[i-1][j] - C[i] \ (i \text{ を抜く}))$

満点解法に向けて

▶ $dp[i][j]$ ($0 \leq j \leq M$) を求めたいとき、
第 2 式、第 4 式について、

▶ $dpL[i-1][j]$ ($H[i] \leq j$) → そのまま取ってくる。

▶ $dpL[i-1][j] - C[i] \rightarrow H[i] > j$ で一律に $C[i]$ 引く。

※ $dpL[i][j] = \max(dpL[i-1][l] + P[i] \ (l \leq j, H[i] = j),$
 $dpL[i-1][j] \ (H[i] \leq j),$
 $P[i] - CSUM(1, i-1) \ (i \text{ より前を抜く}, H[i] = j),$
 $dpL[i-1][j] - C[i] \ (i \text{ を抜く}))$

満点解法に向けて

▶ $dp[i][j]$ ($0 \leq j \leq M$) を求めたいとき、
第 1 式、第 3 式については、 $H[i]=j$ となる
ケースのみ考えればよいので、これらの計算
は高々 N 回しかありません。

※ $dpL[i][j]=\max(dpL[i-1][l]+P[i] \ (l \leq j, H[i]=j),$
 $dpL[i-1][j] \ (H[i] \leq j),$
 $P[i]-CSUM(1,i-1) \ (i \text{ より前を抜く}, H[i]=j),$
 $dpL[i-1][j]-C[i] \ (i \text{ を抜く}))$

満点解法に向けて

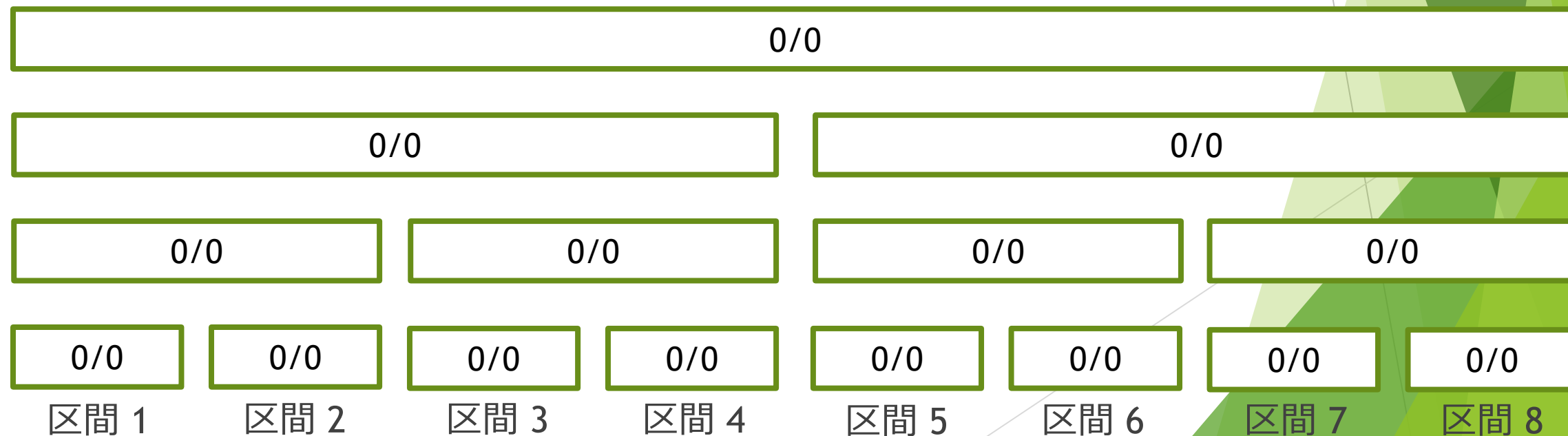
- ▶ 先ほどの計算では、
 - ・ ある値を変更する (第 1 式、第 3 式に登場)
 - ・ ある範囲を一律増減する (第 4 式に登場)
 - ・ ある範囲の最大値を求める
(第 1 式, maxL , maxR に登場)
が登場します。
- ▶ これらを高速に実行するためには？

Segment Tree

- ▶ Segment Tree を使えば計算することができます。
- ▶ RMQ を計算する Segment Tree をベースにして計算することができます。
- ▶ 各ノードには、「その範囲の値 (その範囲全体に足された値については考慮しない)の最大値」「その範囲全体に加算されている値」を覚えておきます。
- ▶ まずは「範囲の増減」を例とともに考えます。

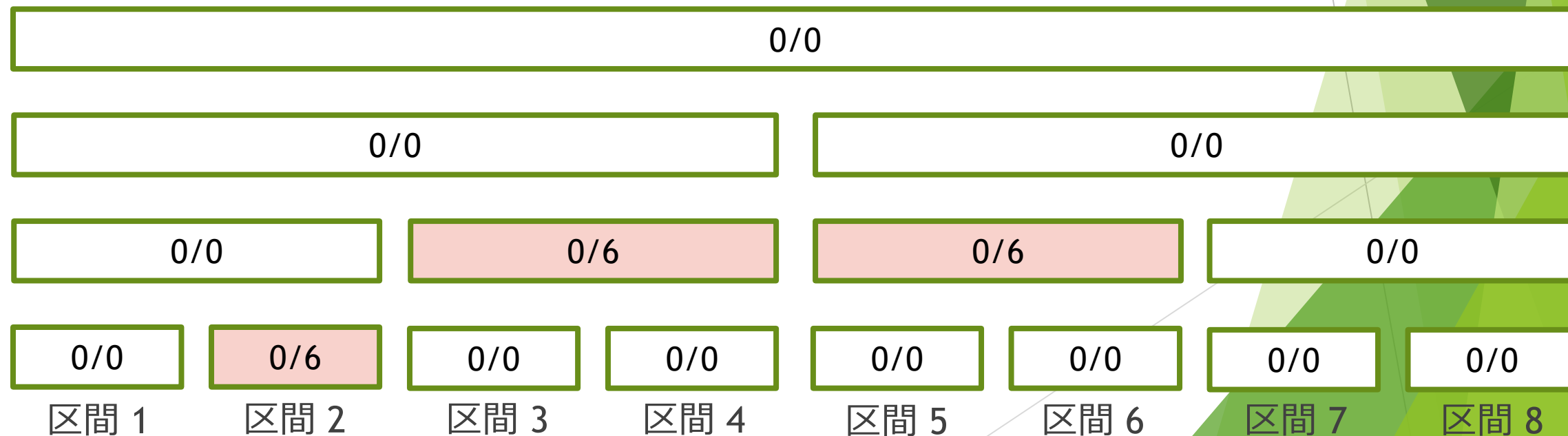
Segment Tree (例)

- ▶ 左辺が区間の最大値、右辺が区間全体に足された値です。
- ▶ 初期状態



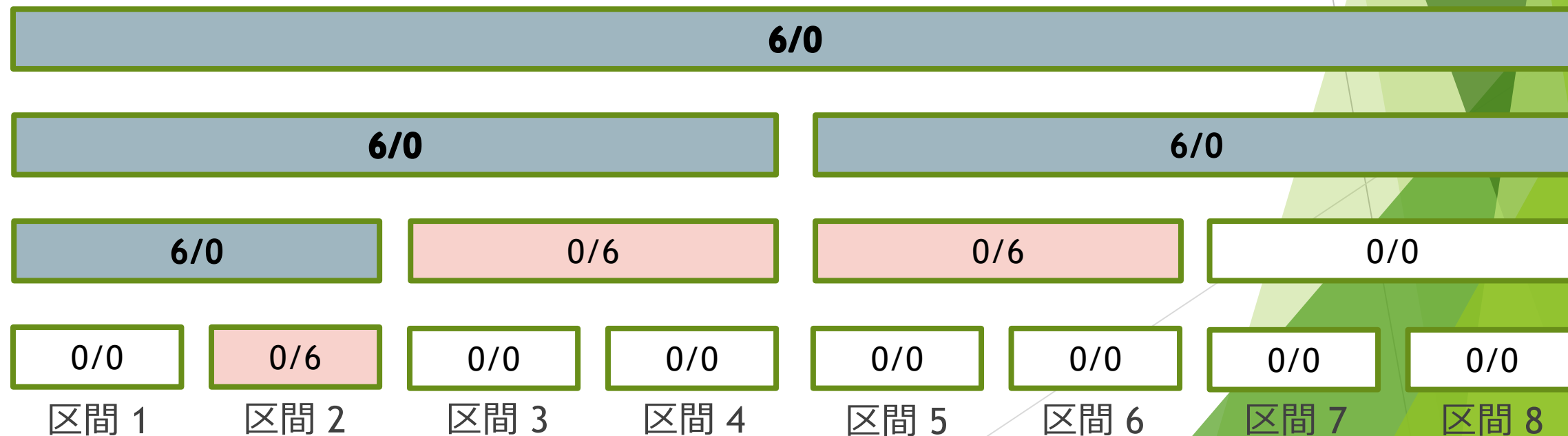
Segment Tree (例)

- ▶ 左辺が区間の最大値、右辺が区間全体に足された値です。
- ▶ 区間 $[2,6]$ に定数 6 を足した後 (左辺未更新時)



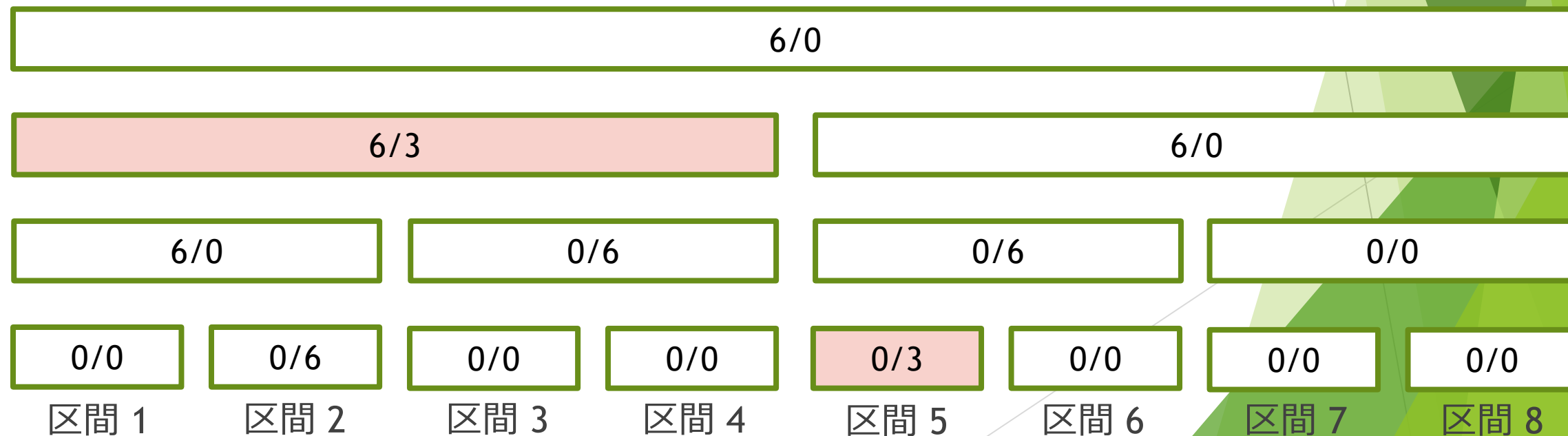
Segment Tree (例)

- ▶ 左辺が区間の最大値、右辺が区間全体に足された値です。
- ▶ 区間 $[2,6]$ に定数 6 を足した後 (左辺更新後)



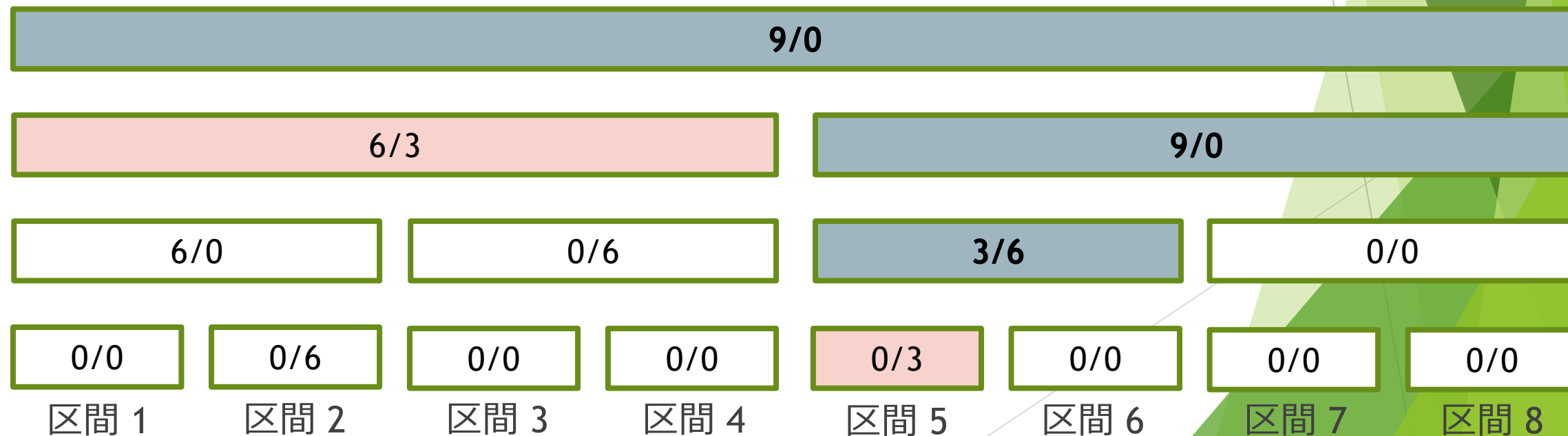
Segment Tree (例)

- ▶ 左辺が区間の最大値、右辺が区間全体に足された値です。
- ▶ さらに、区間 [1,5] に定数 3 を足した後(左辺未更新時)



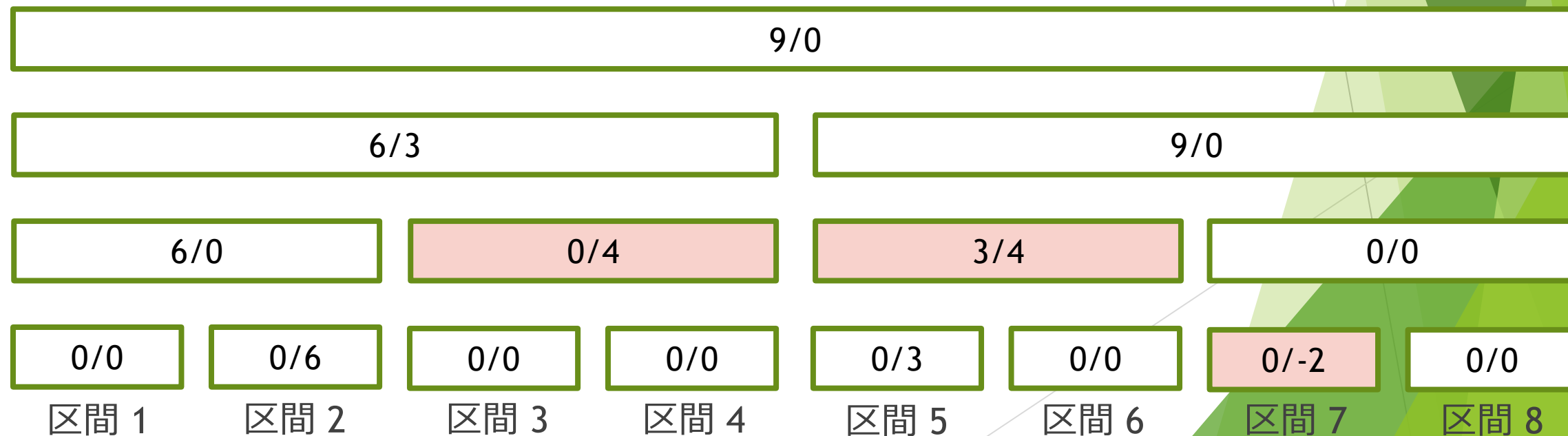
Segment Tree (例)

- ▶ 左辺が区間の最大値、右辺が区間全体に足された値です。
- ▶ さらに、区間 [1,5] に定数 3 を足した後(左辺更新後)



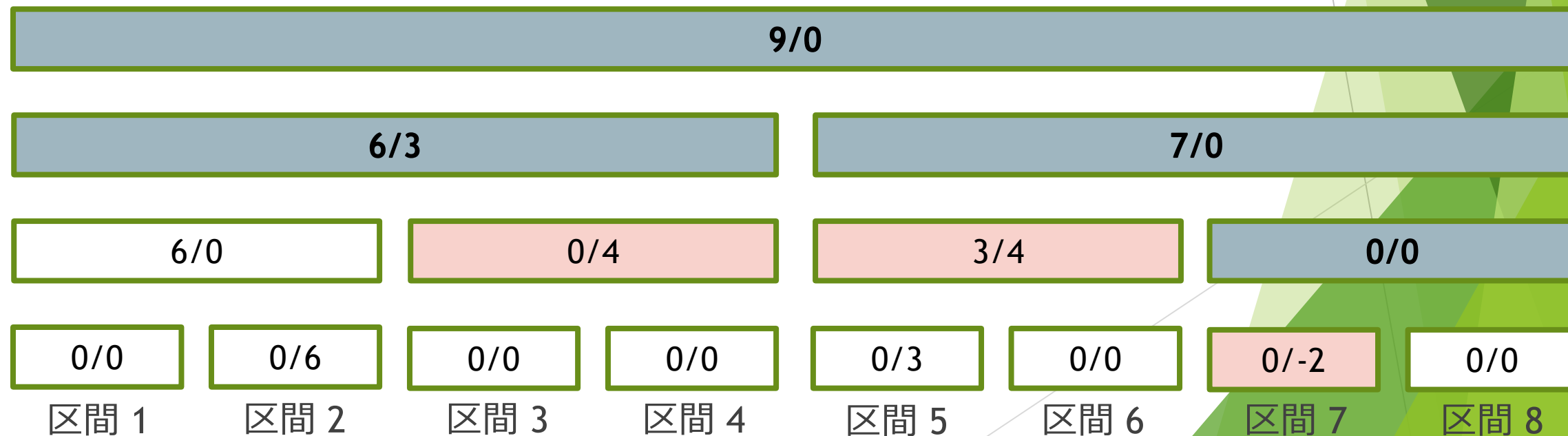
Segment Tree (例)

- ▶ 左辺が区間の最大値、右辺が区間全体に足された値です。
- ▶ さらに、区間 $[3,7]$ に定数 -2 を足した後(左辺未更新時)



Segment Tree (例)

- ▶ 左辺が区間の最大値、右辺が区間全体に足された値です。
- ▶ さらに、区間 $[3,7]$ に定数 -2 を足した後(左辺未更新時)



Segment Tree

- ▶ 例のように Segment Tree に対して、ある範囲に加減算させたいなら、
 1. 「その範囲全体に加算されている値」を増減させる。
 2. その後、変更した値よりも大きな範囲を囲むノードについて、「その範囲の値の最大値」は、 $\max(\text{左の子のmax} + \text{左の子全体に足された数}, \text{右の子のmax} + \text{右の子全体に足された数})$ で実現できます。

Segment Tree

- ▶ 次にある範囲の最大値を求める方法について考えます。
- ▶ 先ほどの加算のように、いくつかの区間に分割した後、
 1. 小さい区間で最大値(計算済)を取り出す。
 2. その区間以上の(覆っている)区間全体に足されている値の合計を求め、それと 1. の値を足す。
 3. 2. で求めた値たちの最大値が答え。

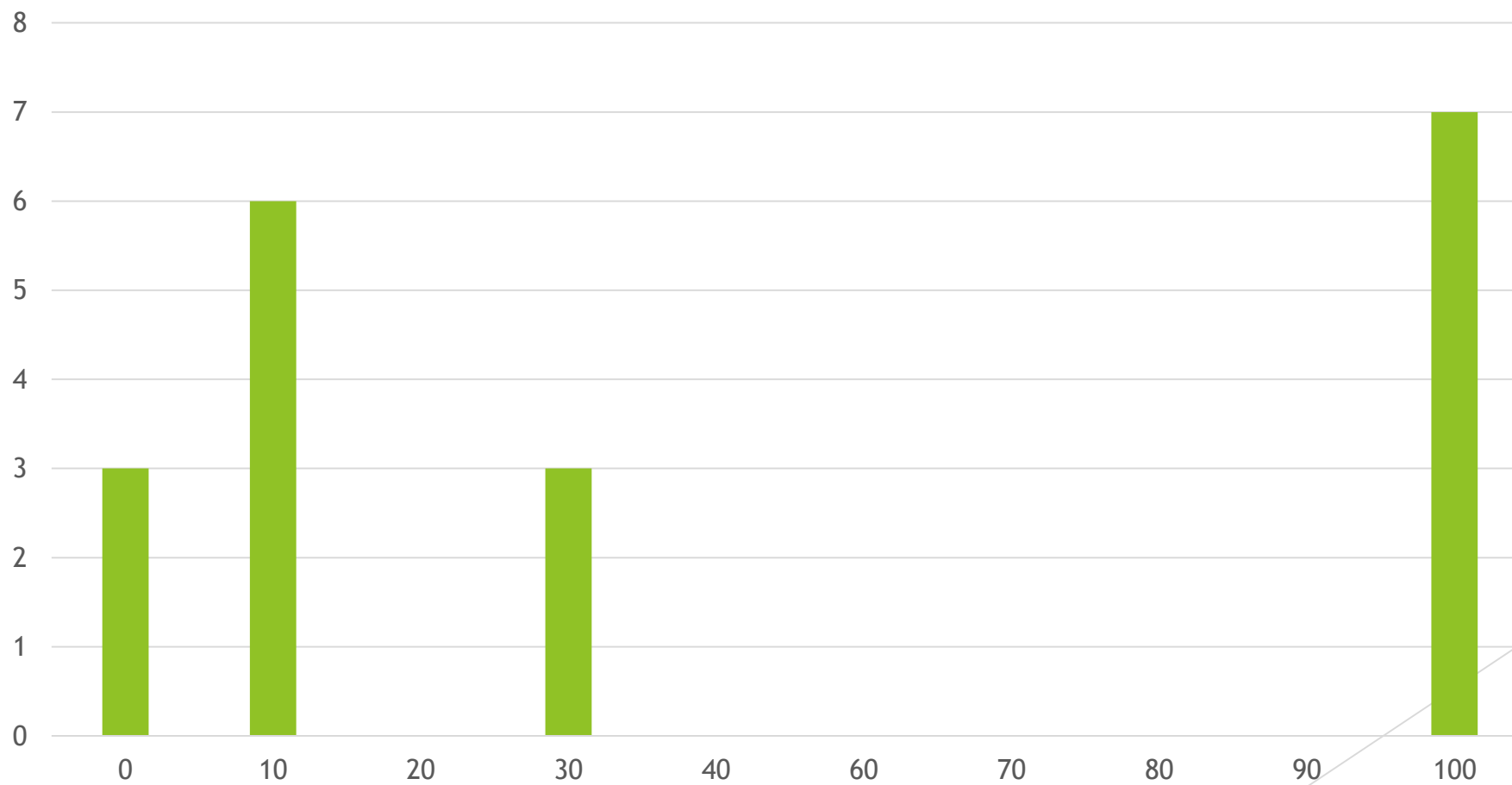
Segment Tree の計算量

- ▶ 値に足す操作では、 $O(\log N)$ 個のノードについて、直接加算が行われ、それ以上のノード(計算すると $O(\log N)$ 個しかないと分かる)について、更新する必要があります。
- ▶ 値を求める場合、上から再帰関数を実装すれば $O(\log N)$ のノードアクセスで実現できます。
- ▶ 以上より $dp[i][[]]$ から $dp[i+1][[]]$ の配列を算出するのに $O(\log N)$ のコストで実現できます。
- ▶ $maxL$, $maxR$ は全体の max を取ります。

おまけ

- ▶ 満点解法には、 H をソートした順にアクセスするようにするという方針があります。
- ▶ この場合、先ほどの立式の高さに関する部分を気にしないで実装することができます。
- ▶ ただし、この方針だと同じ高さが複数ある場合にバグる可能性があります。
- ▶ 小課題 4 のみ通った場合は、この辺りが怪しいなどの情報が得られます。

得点分布



第14回情報オリンピック 春合宿 day1

『○○』カード占い 解説

二階堂建人(snuke)



問題概要

表に $\blacksquare$ 裏に $\odot$ が書かれたカードで占いをする

$\blacksquare$ を A 個、 $\odot$ を B 個、 $\blacksquare$ を C 個、 $\odot$ を D 個、 $\blacksquare$ を E 個並べる

例 : $(A, B, C, D, E) = (1, 2, 3, 4, 5)$ なら、 $\blacksquare \odot \odot \blacksquare \blacksquare \blacksquare \odot \odot \odot \odot \blacksquare \blacksquare \blacksquare \blacksquare$

N 種類の操作を好きな回数だけ好きな順番でできる

i 種類目の操作は、

- ・ 左から L_i 枚目 $\sim R_i$ 枚目のカードをひっくり返す

というもので、この操作には $R_i - L_i + 1$ 秒かかる

全部のカードを $\blacksquare$ にするために必要な時間の最小値は？

小課題 1 (15点)

$A, B, C, D, E \leq 100,000$

$N \leq 10$

操作の種類数が少ない

小課題 1 (15点)

操作の特徴を考察する

- 操作の順番を変えても結果は変わらない
→ 各場所が何回ひっくり返されるかだけが重要
- 同じ操作を 2 回以上しても意味がない
→ 同じ操作を 2 回すると元に戻る

小課題 1 (15点)

つまり、有効な操作の選び方は 2^N 通りしかない
それらを全部試してみて、
全部が $\mathbb{I}$ になったもののうちの操作数の最小値が答え

小課題 1 (15点)

操作を選んだとき、全部が 1 になるかどうかを判定する
ただし、 $O(\text{カード数})$ はかけられないので少し工夫が必要

小課題 1 (15点)

各カードが何回ひっくり返されるかを計算して、

- ・元々が『のカードのひっくり返される回数が偶数回
- ・元々が◎のカードのひっくり返される回数が奇数回

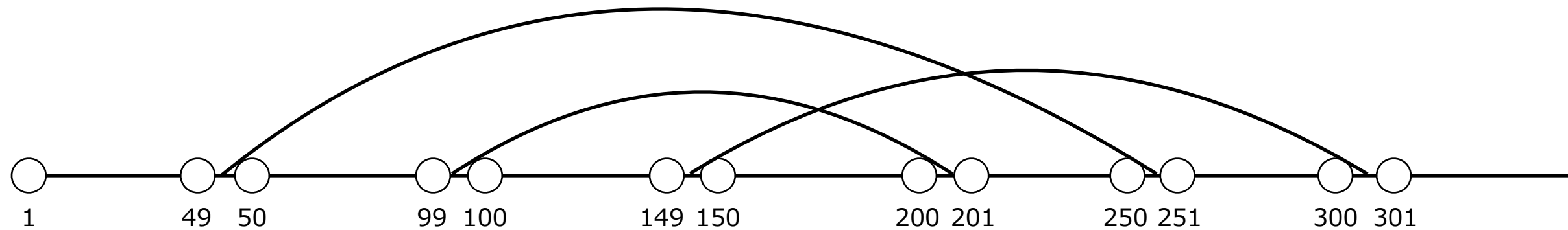
であれば良い

小課題 1 (15点)

ひっくり返す区間が

50~250、100~200、150~300

だったとすると、

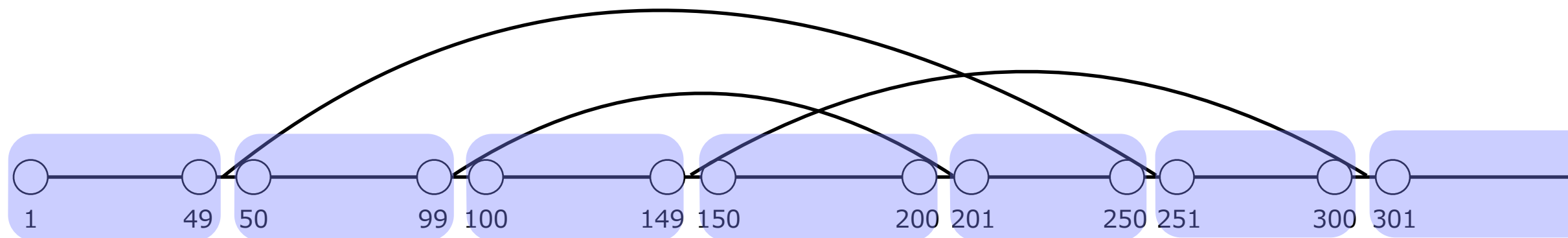


小課題 1 (15点)

青い背景で囲ったような区間で分けて

各部分が何回ひっくり返されたかを計算する

このような区間の個数は高々 21 個とかなので高速



小課題 1 (15点)

- ・ 操作の選び方を 2^N 個試す
- ・ 座標圧縮の要領で操作の選び方が正しいかどうかを判定

$O(2^N * N \log N)$ となり間に合う

考察

␣,◎は少し分かりにくいので、かわりに 1,0 にします

100111000011111

ビット列になりました

考察

差をとってみる (mod 2)

累積和を取れば元に戻ります

(累積和 = 場所 i より前にある数の和)

元の列 : 100111000011111



差の列 : 1101001000100001

考察

区間をひっくり返す操作を、差の列で表すと

元の列： 000000

差の列： 000000



元の列： 001100

差の列： 0010100

元の列： 01111100

差の列： 010000100



元の列： 011000000

差の列： 0101000000

考察

実は、赤い場所のビットが反転するだけ

元の列： 000000

差の列： 000000



元の列： 001100

差の列： 0010100

元の列： 01111100

差の列： 010000100



元の列： 011000000

差の列： 0101000000

考察

「全部1にする」を差の列で言い換えてみると、

100111000011111



こうしたい

111111111111111

考察

差の列にしました

1101001000100001



こうしたい

1000000000000001

考察

赤い点のついたところを全部 0 にしたい
(A+1,B+1,C+1,D+1ビット目)

1101001000100001



こうしたい

100000000001

問題の言い換え

少し言い換えて問題を整理すると

- x_0, x_1, x_2, x_3 ビット目を反転させたい
- N 種類の操作ができ、 i 種類目の操作は
「 a_i ビット目と b_i ビット目を c_i 秒で反転させる」

最小で何秒かかるか？

という問題になる

IOIの場合

- x_0, x_1 ビット目を反転させたい
- N 種類の操作ができ、 i 種類目の操作は
「 a_i ビット目と b_i ビット目を c_i 秒で反転させる」

最小で何秒かかるか？

という問題をまず解いてみます

IOIの場合

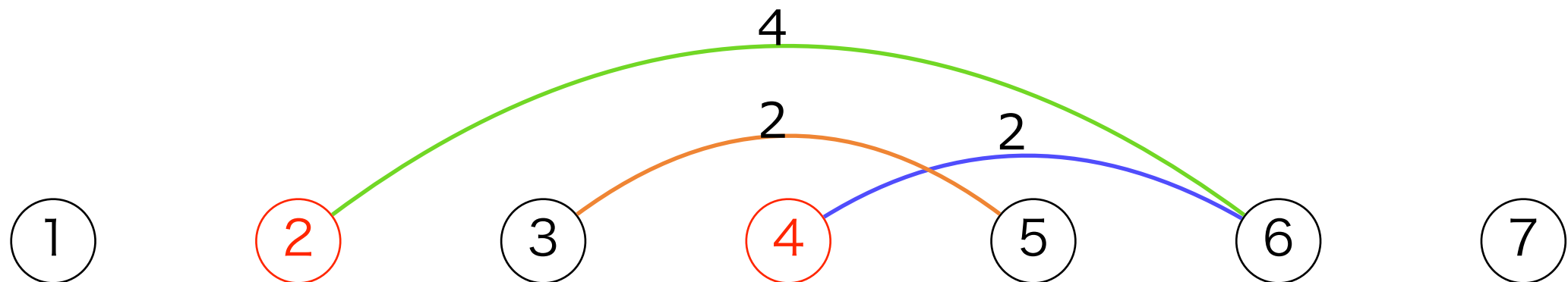
各ビットを頂点にして

頂点 a_i と頂点 b_i の間にコスト c_i 秒の辺を張る

元の列： 100111

差の列： 1101001

1 2 3 4 5 6 7



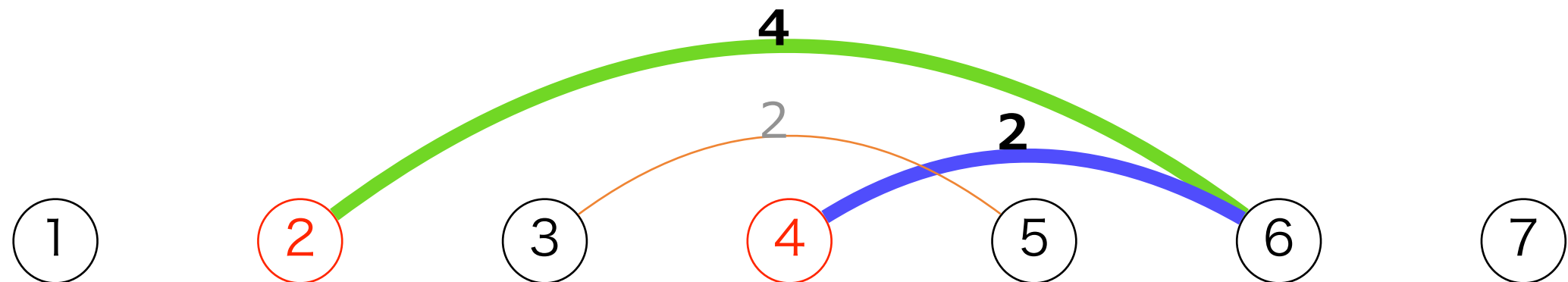
IOIの場合

赤い頂点間のパスを作ればいい！

最小コストのパスを求めたい → **最短経路問題**

元の列： 100111

差の列： 1101001
1 2 3 4 5 6 7



IOIOIの場合

- x_0, x_1, x_2, x_3 ビット目を反転させたい
- N 種類の操作ができ、 i 種類目の操作は
「 a_i ビット目と b_i ビット目を c_i 秒で反転させる」

最小で何秒かかるか？

IOIOIの場合

x_0, x_1, x_2, x_3 をペアに分けて、それぞれの最短経路を計算する

- $(x_0, x_1)(x_2, x_3)$
- $(x_0, x_2)(x_1, x_3)$
- $(x_0, x_3)(x_1, x_2)$

の3通りのペア分けがあるので、全て試す

例えば、 $(x_0, x_1)(x_2, x_3)$ なら、

x_0 から x_1 の最短経路長と x_2 から x_3 の最短経路長を足す

小課題 2 (50点)

$A, B, C, D, E \leq 50$

$N \leq 100,000$

カードの枚数が少ない

Bellman Ford ・ Warshall Floyd で間に合う

小課題 3 (35点)

$A, B, C, D, E \leq 100,000$

$N \leq 100,000$

大きい

Dijkstraを使えば良い

ビルの飾りつけ 3

解説

今西 健介 (@japlj)

問題概要

問題

N 要素の数列に対して LIS (最長増加部分列) を DP で求めたときの配列 $A[i]$ (i 番目で終わる LIS の長さ) から

1 要素取り除いた整数列 B が与えられる.

元の整数列 A としてありうるものは何通りか?

制約

$$2 \leq N \leq 1\,000\,000$$



問題概要

問題

LIS の DP 配列が
できました！



JOI 君

数が $N-1$ 個しかないやん！（憤怒）

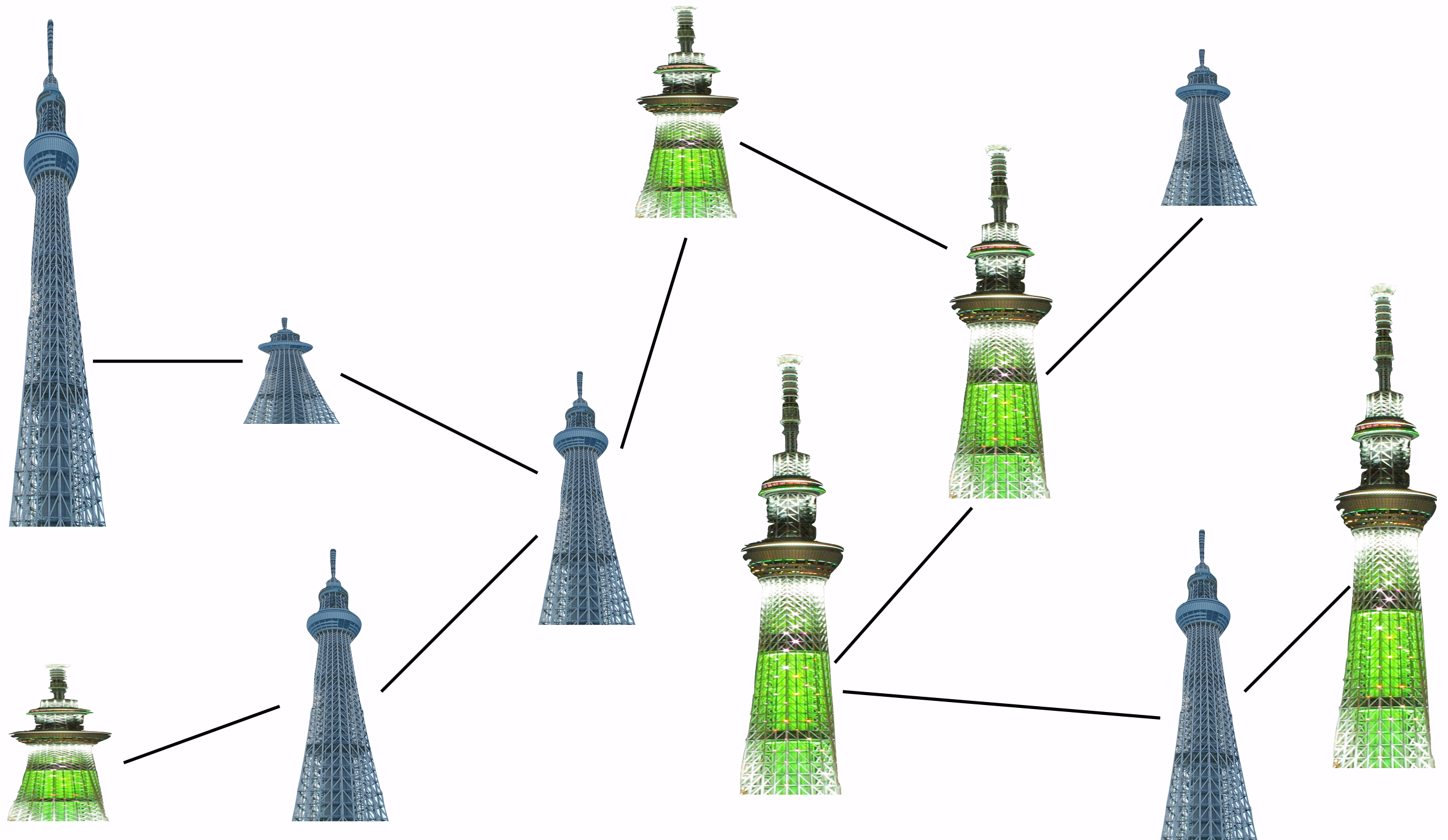


K 理事長

Building



Building 2



Building 3



小課題 1

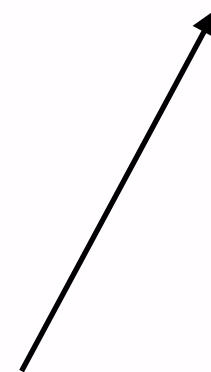
ビルの高さの関係を全探索 → A を実際に求める



$$A = (1, 1, 2, 1)$$



$$A = (1, 2, 1, 2)$$



ビルの高さが違ってても
同じ A が出てくる可能性があるので注意
(サンプルの説明にあります)



小課題 2

A, B の性質

- A から 1 つの要素を取り除いたものが B (という仮定)
- $1 \leq A[i] \leq N$



B に $1 \sim N$ のどれかを挿入してできる列が A の候補

候補は「挿入位置 $\times$ 挿入する数」で $O(N^2)$ 通り

A の候補を全探索

A の候補を $O(N^2)$ 通り全部試すとして……

問題'

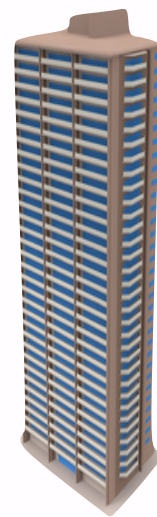
整数列 A' が与えられる. **LIS の DP 配列が A' となるようなビルの高さの列が存在するか？**

という問題が解ければよい.

A' が $O(N^2)$ 通りあるので, $N \leq 300$ だと判定は $O(N)$ ぐらいでやりたい.

A としてありうるものは？

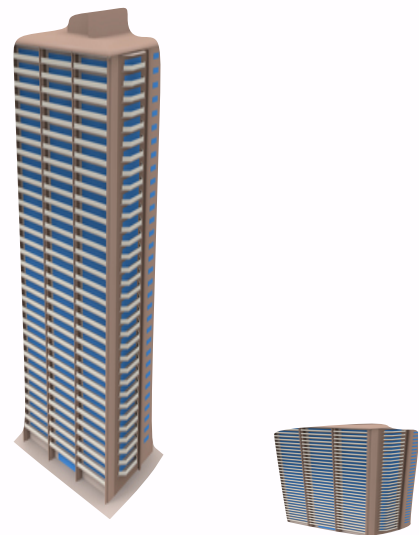
ビル 1 の高さに関わらず $A[1] = 1$



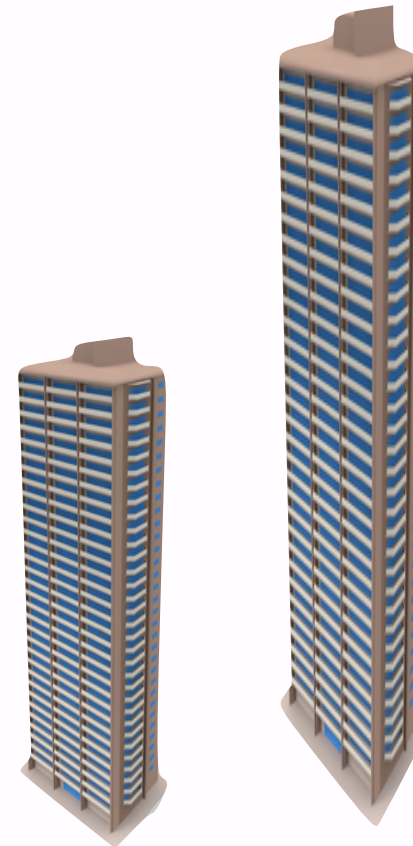
$$A[1] = 1$$

A としてありうるものは？

ビル 2 の高さによって $A[2] = 1 \text{ or } 2$



$$A[1] = 1 \quad A[2] = 1$$

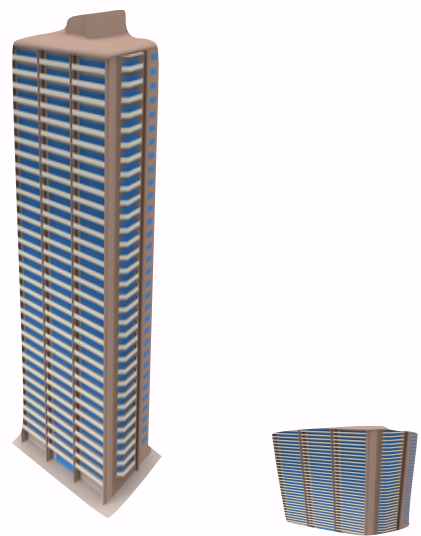


$$A[1] = 1 \quad A[2] = 2$$

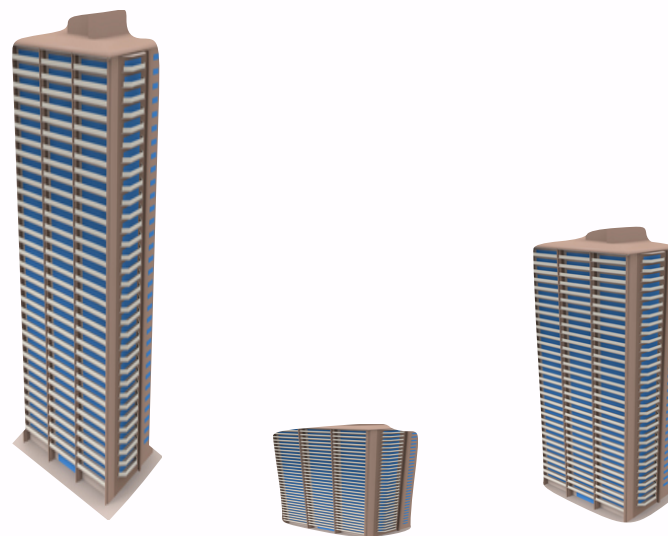
A としてありうるものは？

$A[2] = 1$ なら $A[3] = 1$ or 2

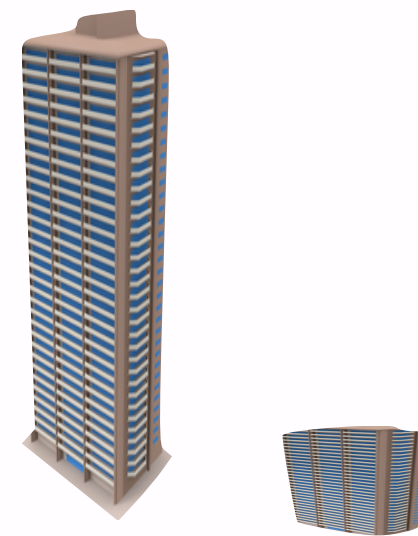
$A[2] = 2$ なら $A[3] = 1$ or 2 or 3



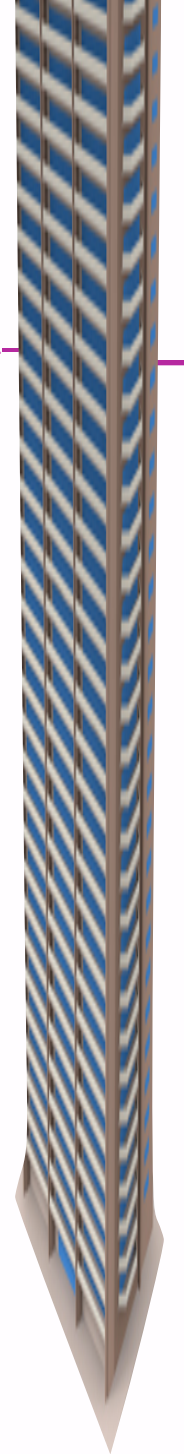
$A[1] = 1$ $A[2] = 1$ $A[3] = 1$



$A[1] = 1$ $A[2] = 1$ $A[3] = 2$



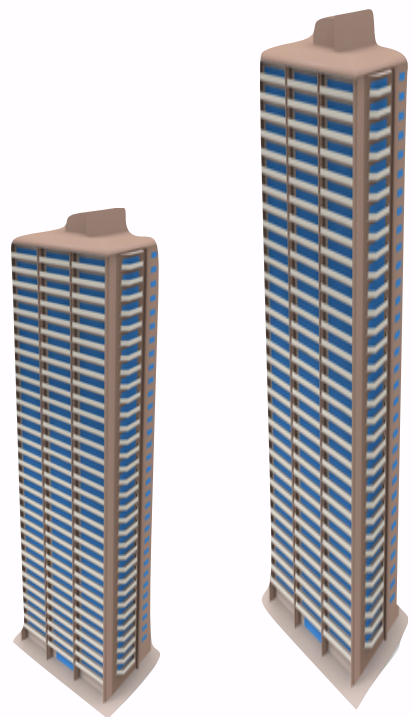
$A[1] = 1$ $A[2] = 1$ $A[3] = 2$



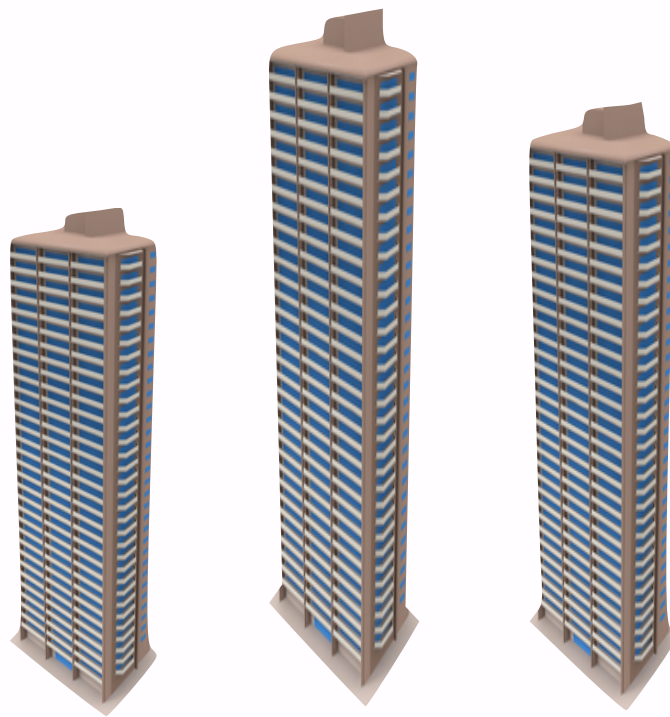
Aとしてありうるものは？

$A[2] = 1$ なら $A[3] = 1$ or 2

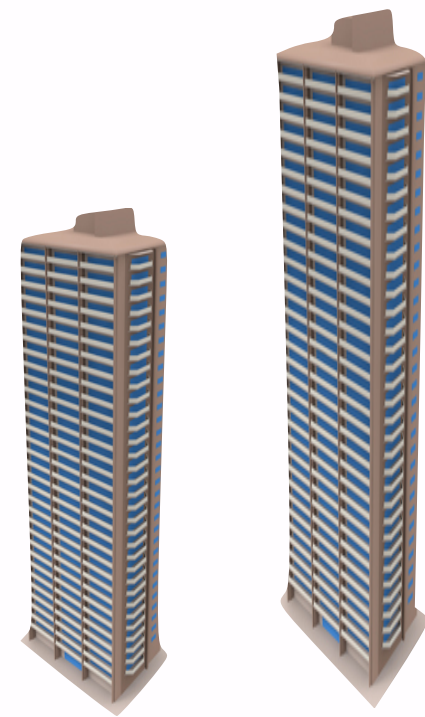
$A[2] = 2$ なら $A[3] = 1$ or 2 or 3



$A[1] = 1$ $A[2] = 2$ $A[3] = 1$



$A[1] = 1$ $A[2] = 2$ $A[3] = 2$



$A[1] = 1$ $A[2] = 2$ $A[3] = 3$

A としてありうるものは？

分かりやすくするため $A[0] = 0$ として……

予想

$A[i]$ は 1 から $\max(A[0], \dots, A[i-1]) + 1$ のうち
どれにでもなれるのでは？

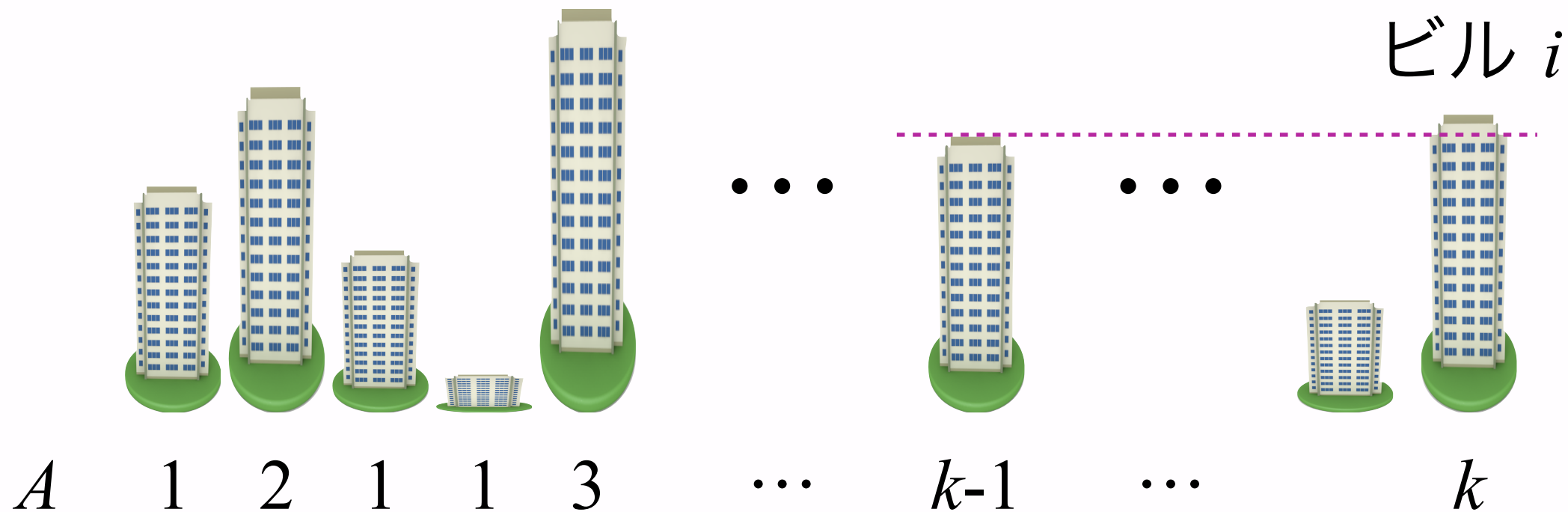
正しい！

→ A の性質は満点解法にも重要になってくるので証明も

略証

$A[i] = k$ とする場合

$A[j] = k-1$ となる最大の j をとってきて、
ビル i の高さをビル j より僅かに高いものとするればよい

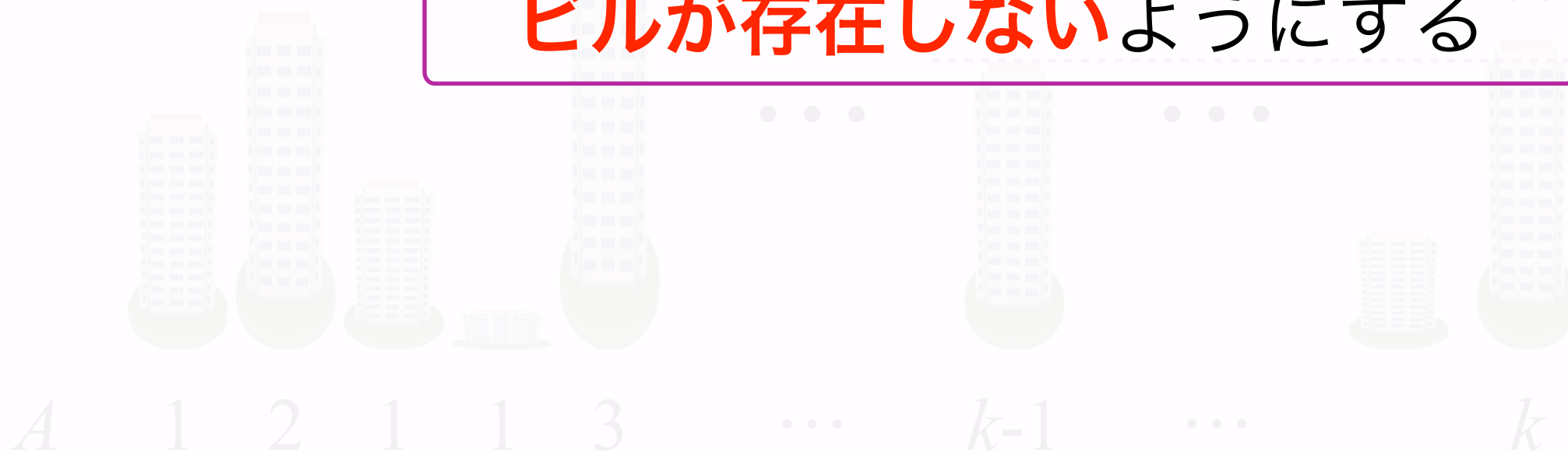


略証

$A[i] = k$ とする場合

$A[j] = k-1$ となる最大の j をとってきて、
ビル i の高さをビル j より僅かに高いものとすればよい

ビル i とビル j の**中間の高さの
ビルが存在しない**ようにする



小課題 2

問題'

整数列 A' が与えられる. LIS の DP 配列が A' となるようなビルの高さの列が存在するか？

$$1 \leq A[i] \leq \max(A[0], \dots, A[i-1]) + 1$$

かどうかをチェックすればよい



小課題 3



結局のところ

$$1 \leq A[i] \leq \max(A[0], \dots, A[i-1]) + 1$$

を満たす A が何個あるかを数えればよい

もう少し直感的に言うと

最大値 $\max(A[0], \dots, A[i])$ が $i = 1, 2, \dots, N$ で

高々 1 ずつ増えていく



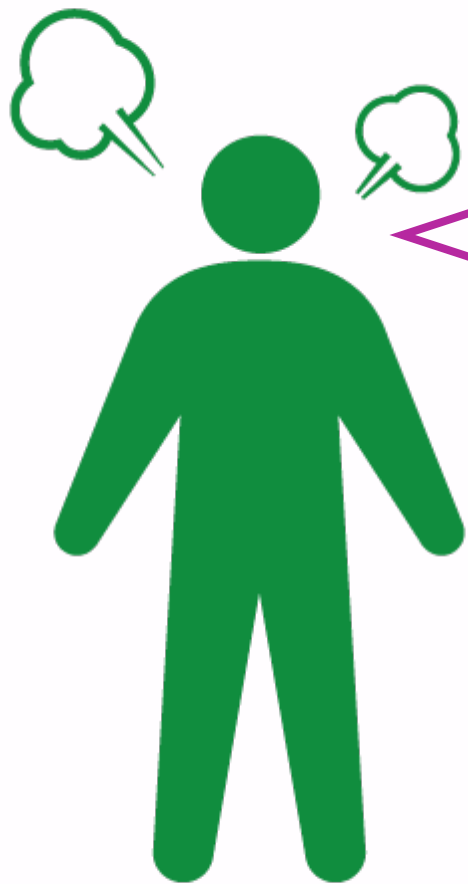
挿入する数が決まる場合

たとえば $B = (1, 2, 1, 4, 3)$ のとき

ここで**最大値が 2 から 4 に**

飛んでるじゃないか！

そんなの有り得るわけないだろ！！



挿入する数は 3 で確定

決まる場合 → 場合分け

$B = (1, \textcolor{red}{2}, 2, 1, \textcolor{red}{4}, 2, 4, 3)$ 3 を入れたいけど 2 の後じゃないとダメ
ここに入れられる

$B = (1, 2, 1, \textcolor{red}{4}, 3, \textcolor{red}{6})$
3 も 5 も入れたい → ダメ (0 通り)

$B = (1, 2, 1, \textcolor{red}{5})$
3 も 4 も入れたい → ダメ (0 通り)



挿入する数が決まらない場合

たとえば $B = (1, 2, 1, 3, 2)$ のとき

そのままでも正しい！ヤッター！
でもこういう時は
何を挿入すればいいんだろう？



条件さえ満たしていれば何でも OK！



決まらない場合 → 数える

$B[i]$ の後には

1 から $\max(B[1], \dots, B[i]) + 1$ までの数を挿入できる

$$B = (1, 2, 1, 3, 2)$$

たとえばここには 1, 2, 3 が入れられる



重複に注意



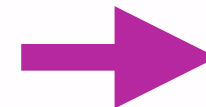
ここに 1 を入れるのと

$$B = (1, 2, 1, 3, 2)$$

ここに 1 を入れるのは同じ



$B[i]$ の後に $B[i+1]$ を入れるのと
 $B[i+1]$ の後に $B[i+1]$ を入れるのが被る



答えから
 $N-1$ を引く

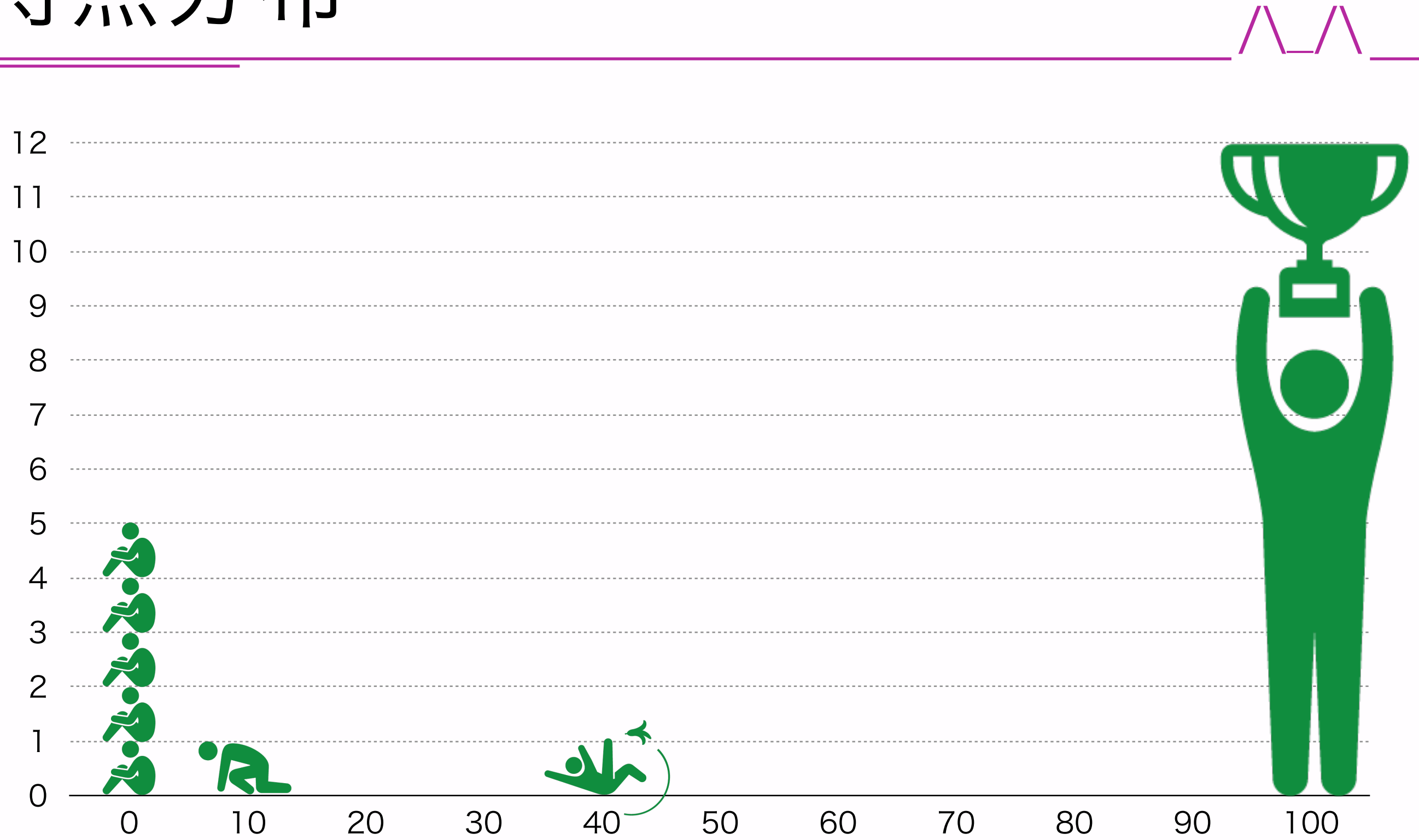
解法まとめ



- B に数をひとつ挿入して A の候補を作る
- $1 \leq A[i] \leq \max(A[1], \dots, A[i-1]) + 1$ なら OK
- 挿入する数が決まる場合と決まらない場合に分ける
- それぞれさらに場合分け + 数え上げ
- $O(N)$



得点分布



合鍵 (Keys)

IOI 2009 Bulgaria 銅メダル

東京大学医学部医学科5年

平野湧一郎 (nai)

2015年3月21日 情報オリンピック春合宿

問題概要

- 社員 N 人中 K 人に合鍵を配る
- 全員1回ずつ外出する
- 錠が閉まっている時間を最大化しよう

小課題

1. $N \leq 20$, $M \leq 100$ 万

2. $N \leq 2000$

小課題

1. $N \leq 20$, $M \leq 100$ 万

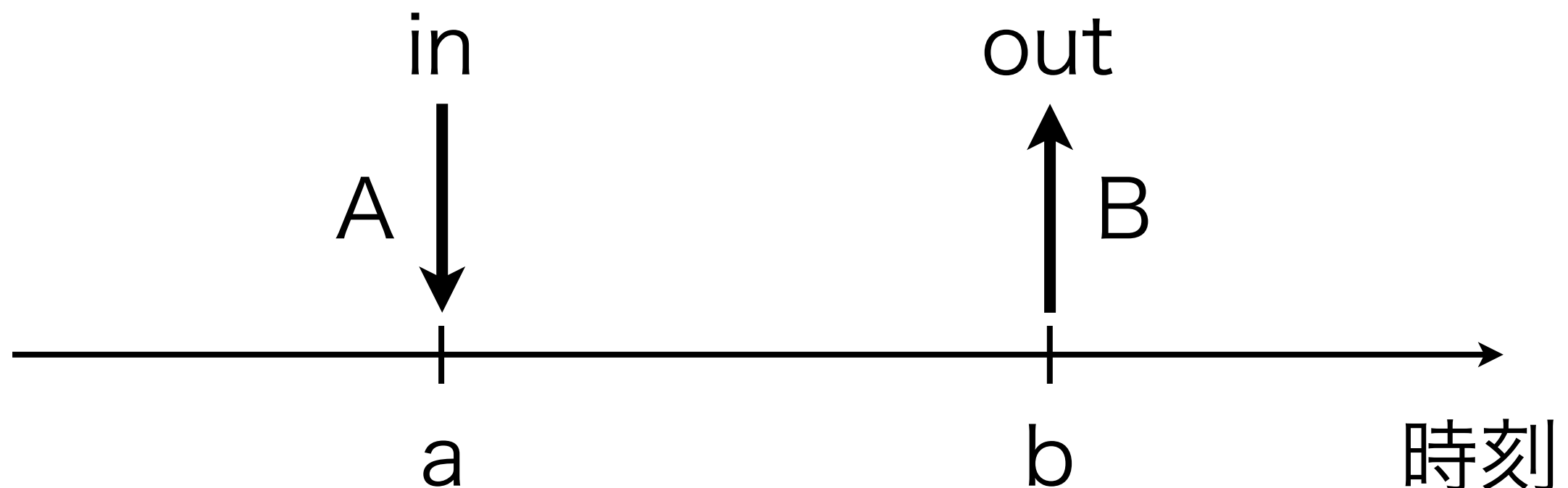
2. $N \leq 2000$

部分点解法1

- 合鍵の渡し方を全部試す
- 合鍵の渡し方を決めたときに、錠の開け閉めを最適化しよう！

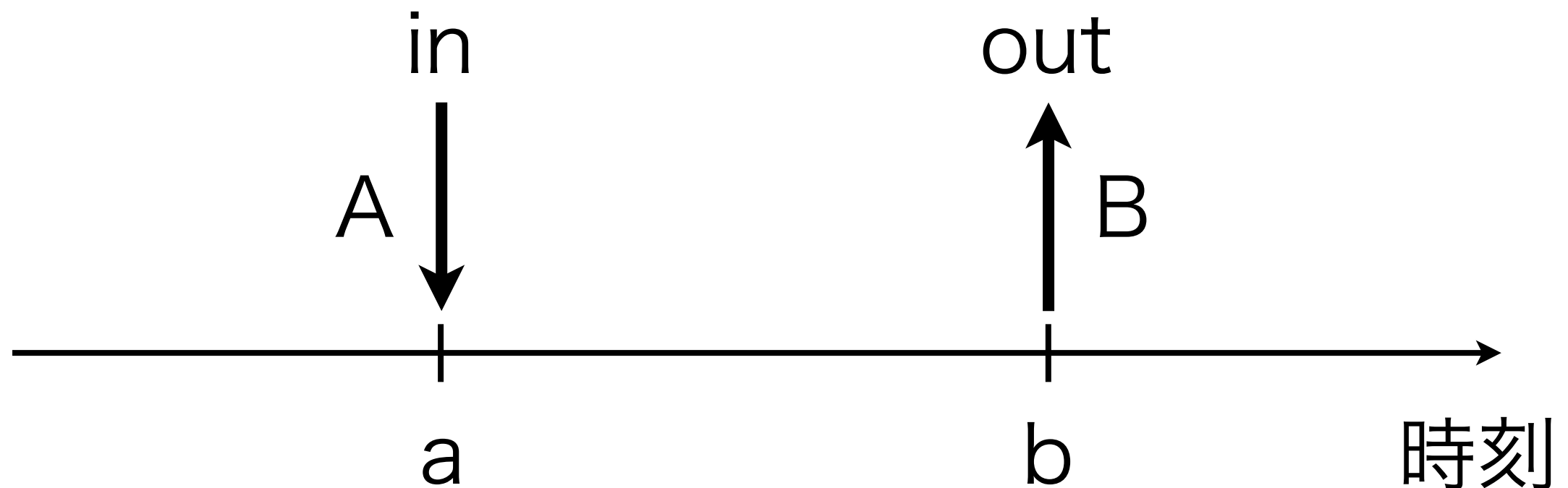
考察

- 隣り合った2人の出入りを考える
- 時刻**a**から時刻**b**まで錠を閉めることのできる条件は？
- $\Rightarrow$ 社員AとBの出入りで場合分け



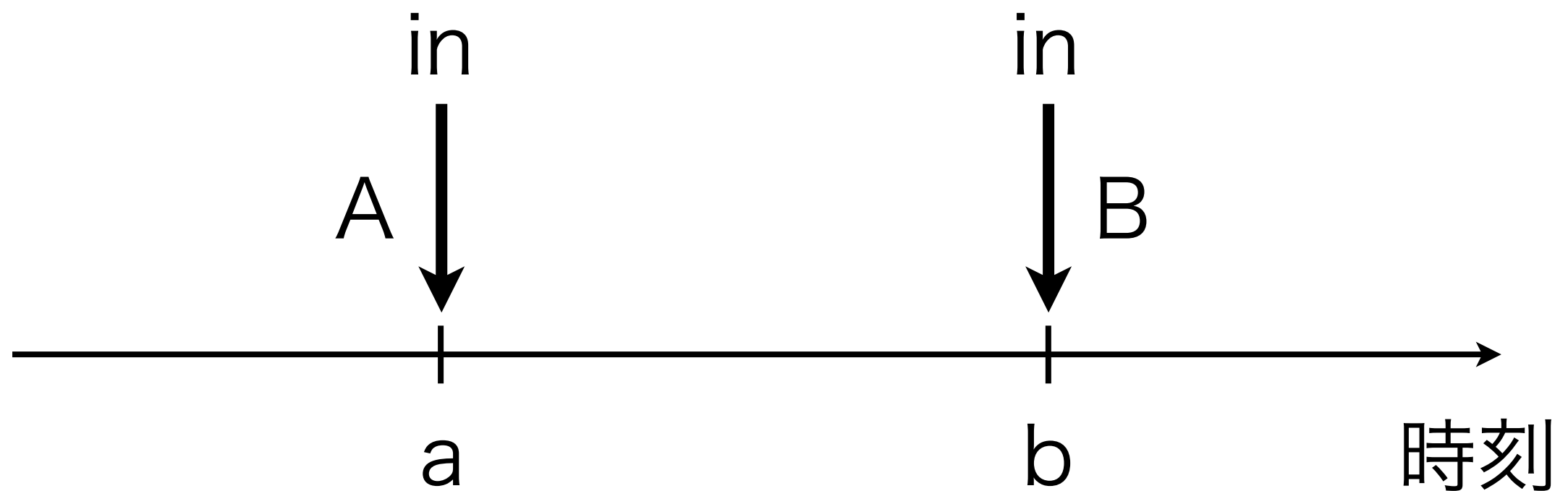
(1) in-out 型

- A, Bが合鍵を持っているかどうかによらず, 常に錠を閉めることができる



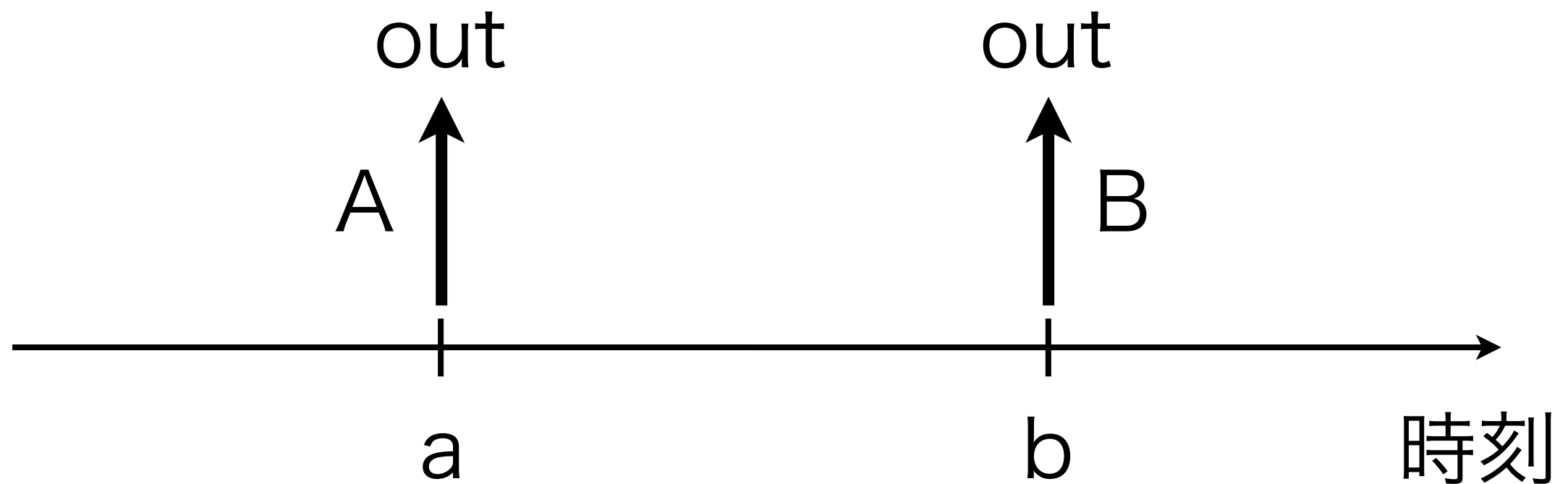
(2) in-in 型

- Bが合鍵を持っていればOK



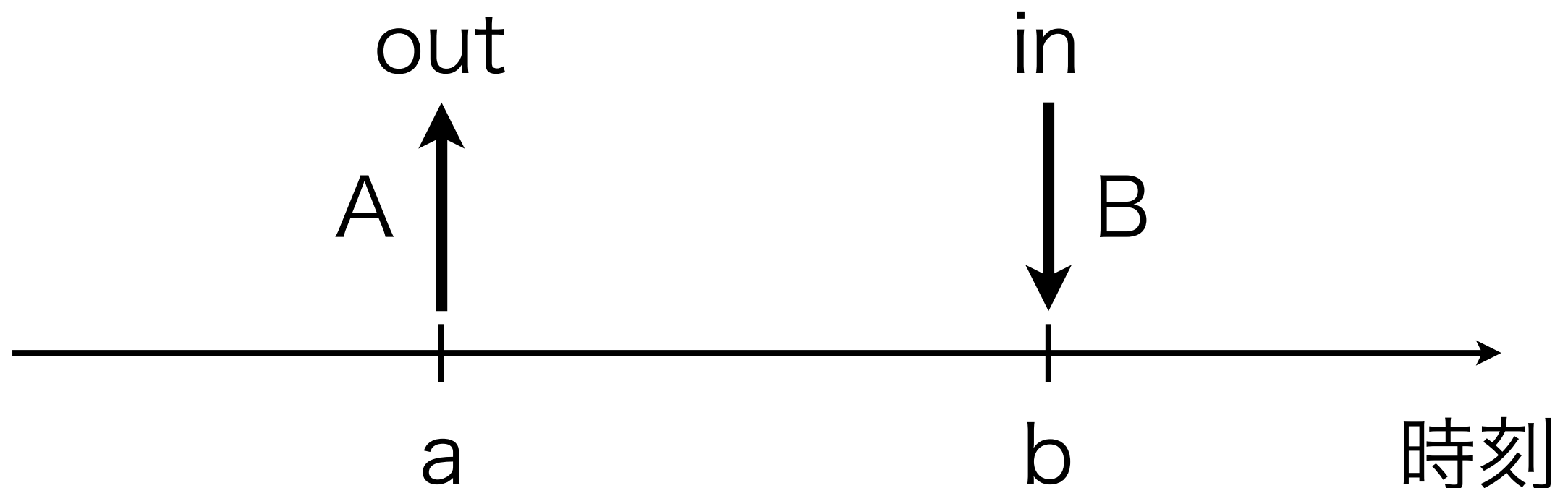
(3) out-out 型

- Aが合鍵を持っていればOK



(4) out-in 型

- AとBの**両方**が合鍵を持っている必要がある



まとめ

A	B	錠を閉められる条件
in	out	いつでも
in	in	Bに合鍵
out	out	Aに合鍵
out	in	A, B両方に合鍵

部分点解法1

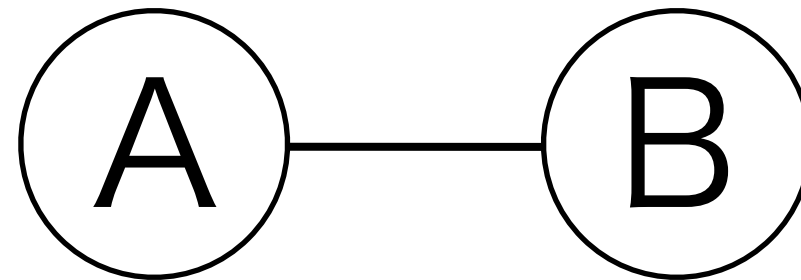
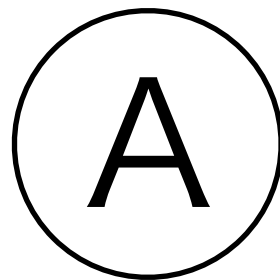
- 合鍵の渡し方を全て試す
- 隣り合う出入り全てに対し， 上述の場合分けにより錠が閉められるかを判定

さらに考察

- 合鍵を渡すことで, 「錠を閉められる時間」が伸びるのは2パターン

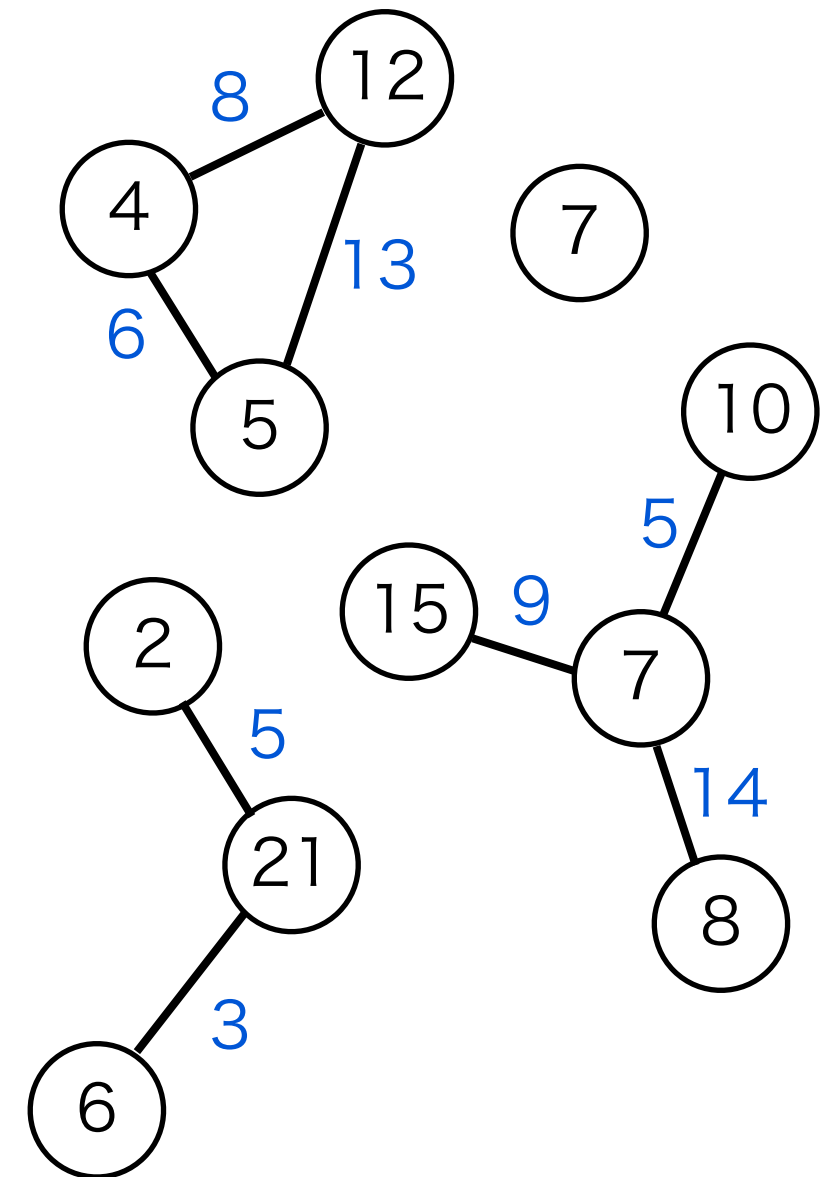
(1)社員Aに合鍵を渡す

(2)社員A, Bの両方に合鍵を渡す



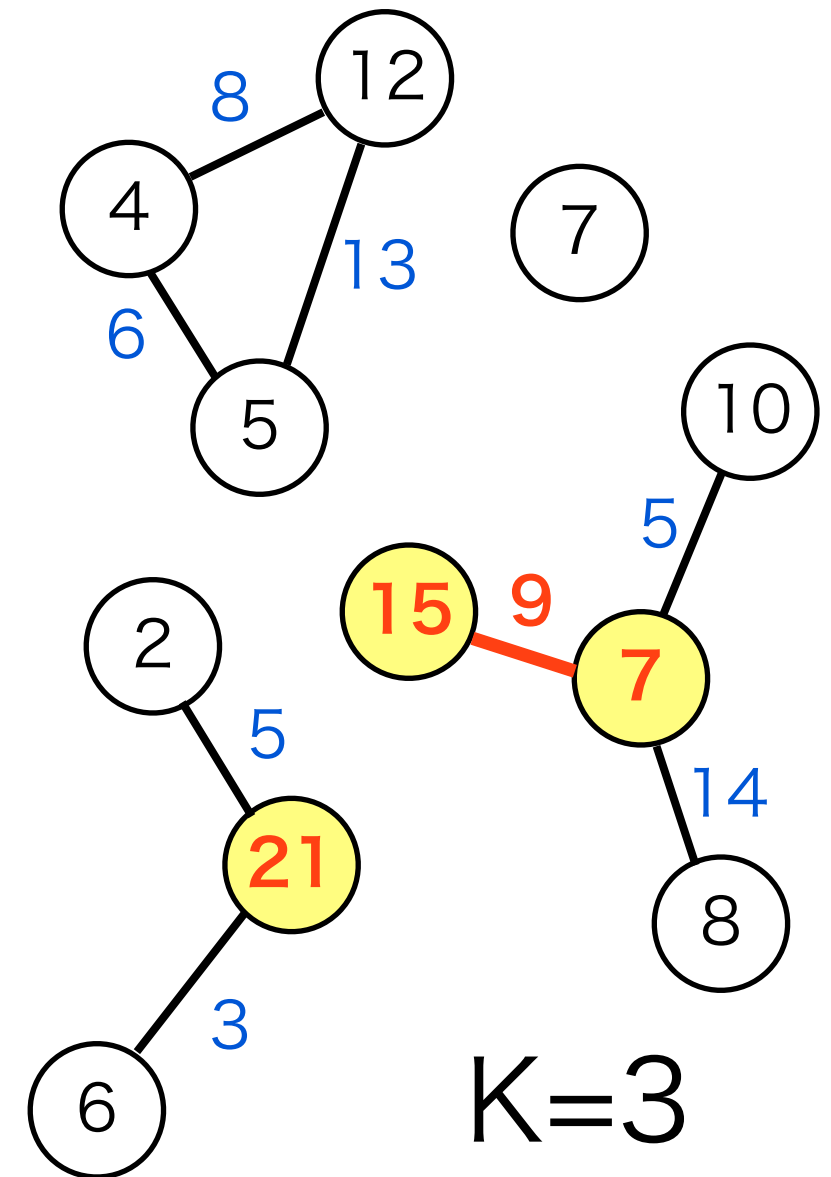
グラフにする

- 社員それぞれが頂点の
N頂点のグラフを作る
- 頂点：(1)で得する時間
- 辺：(2)で得する時間



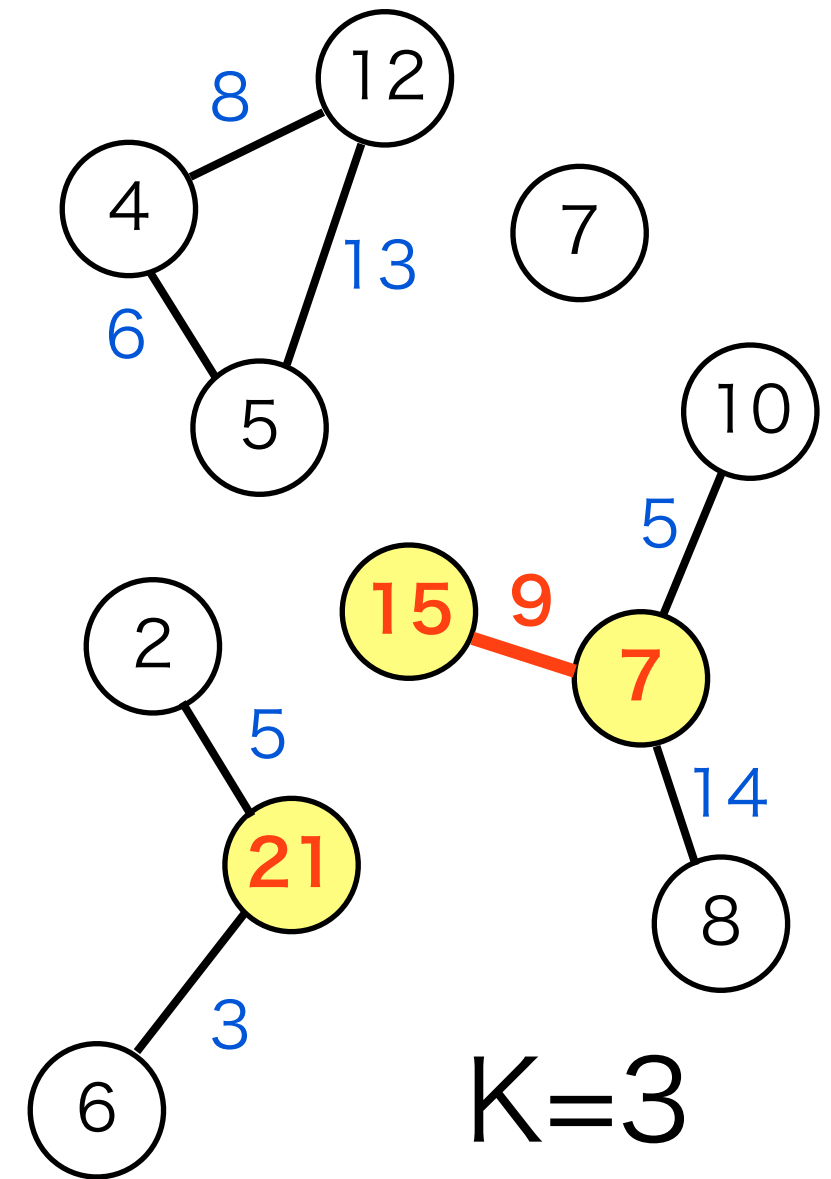
問題の言い換え

- N 頂点のうち K 点に色を塗り,
- 頂点に書いてある数 + 両端点が塗られた辺に書いてある数
- の総和を最大化！



問題の言い換え

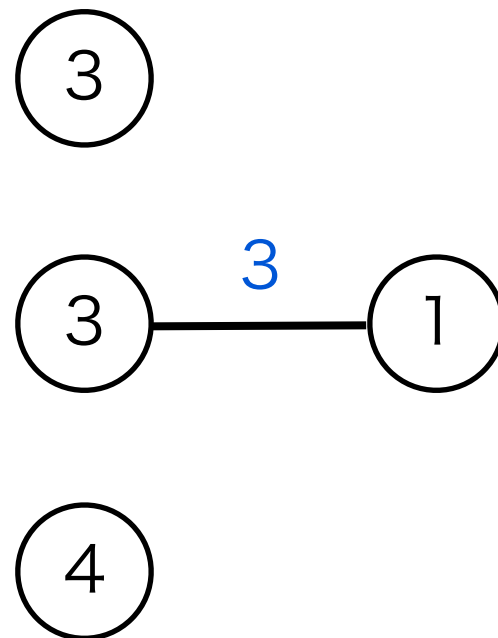
- N 頂点のうち K 点に色を塗り,
- 頂点に書いてある数 + 両端点が塗られた辺に書いてある数
- の総和を最大化!



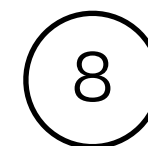
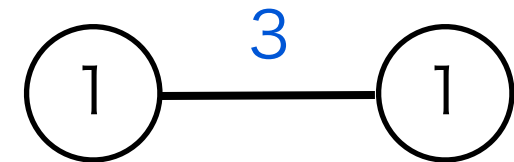
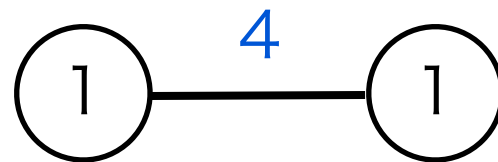
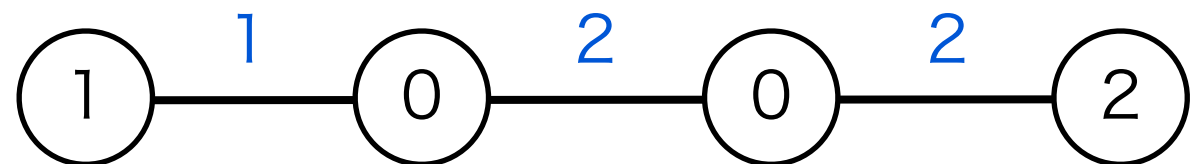
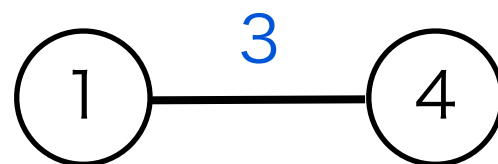
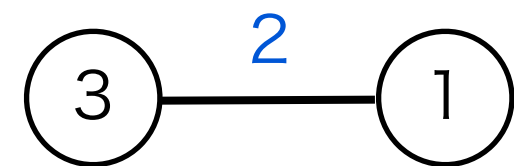
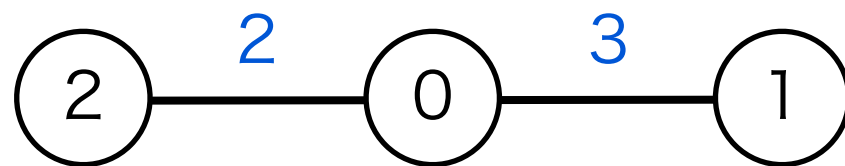
→ 解けない  ('ω')

試しに描いてみよう！

サンプル1



とあるinput



問題文をよく読む

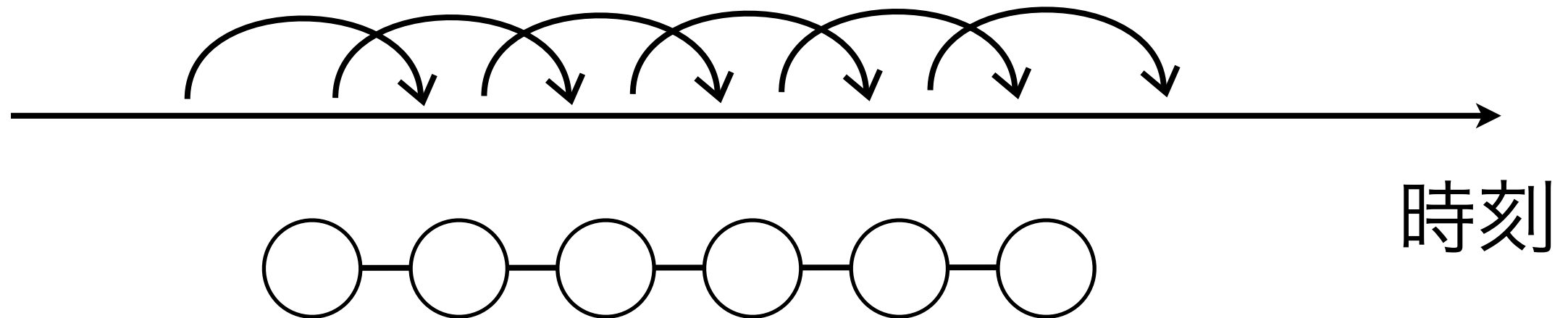
- 今日**は偶然にも**,

どの社員もちょうど

1回ずつ外出する。

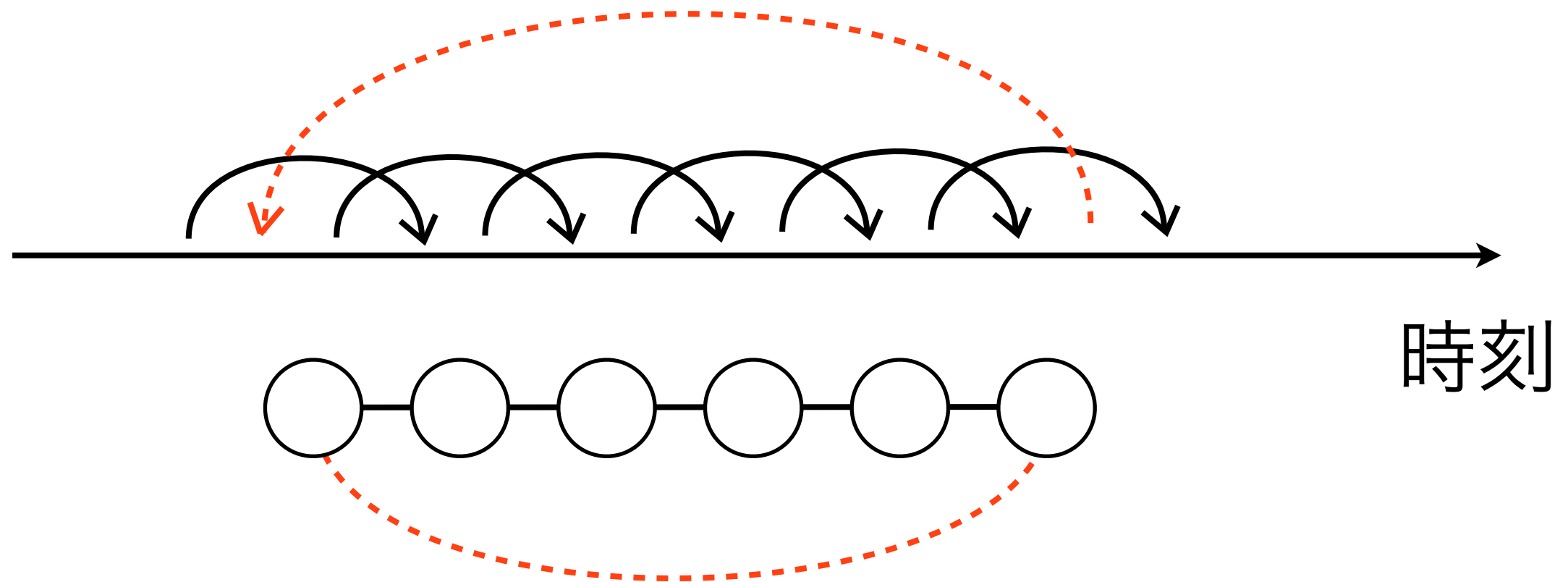
何が嬉しいか

- 頂点の次数は高々2
- グラフに閉路が存在しない



何が嬉しいか

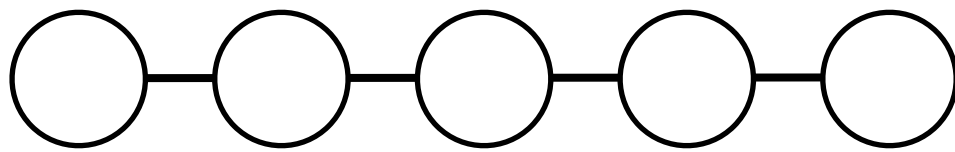
- 頂点の次数は高々2
- グラフに閉路が存在しない



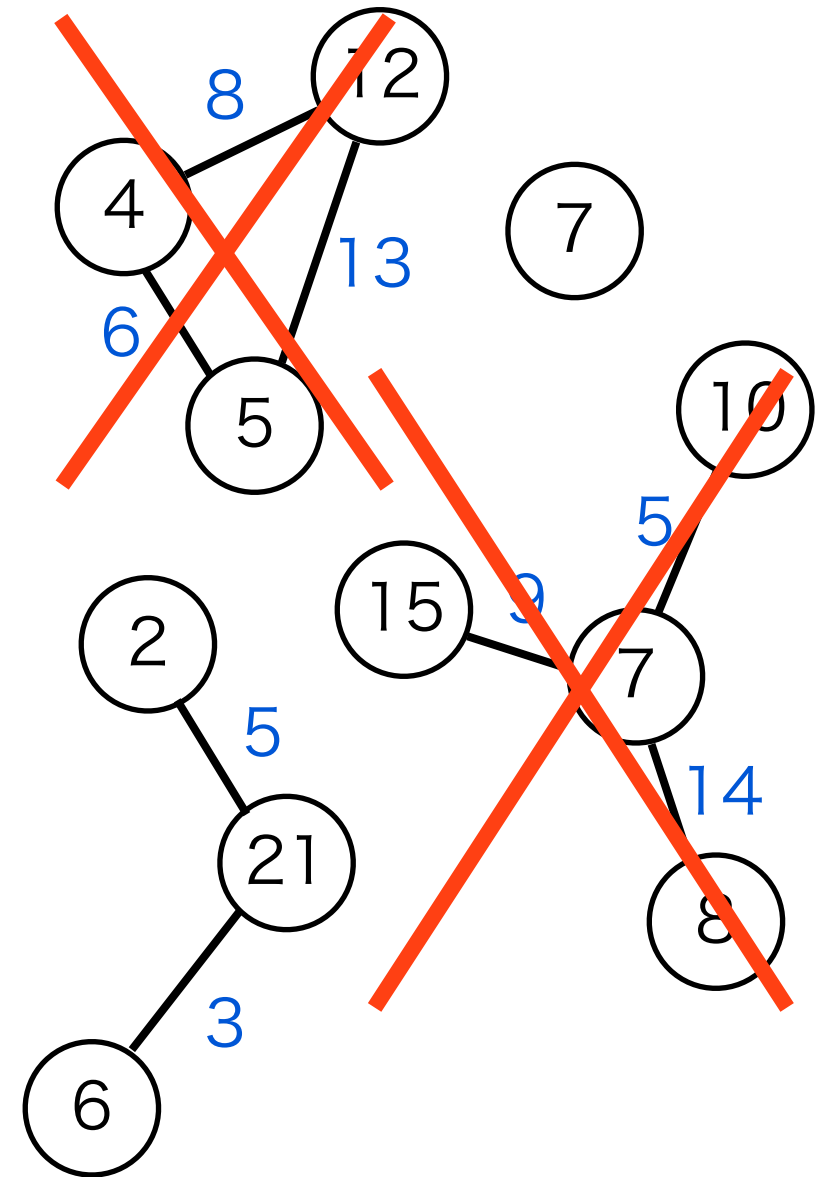
わかったこと

- グラフに閉路や分岐なし
→ グラフはパスの集合

- ※パス：

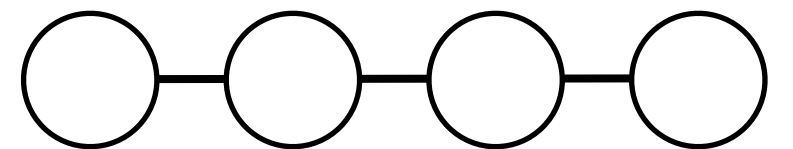
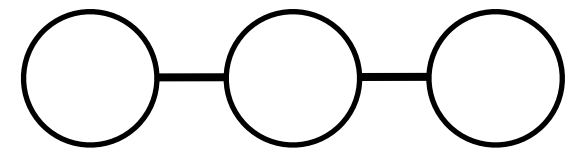
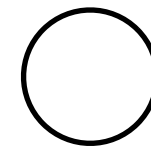
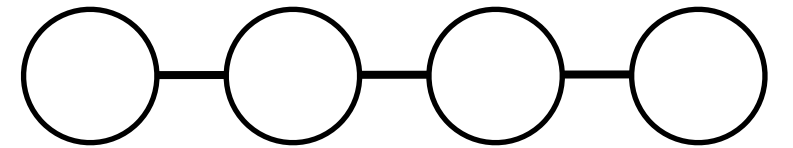


みたいなもの



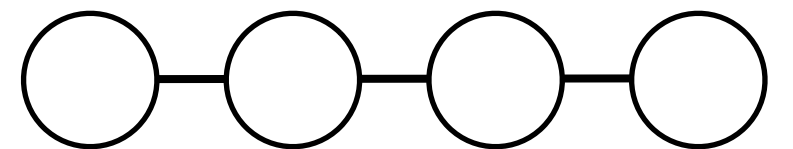
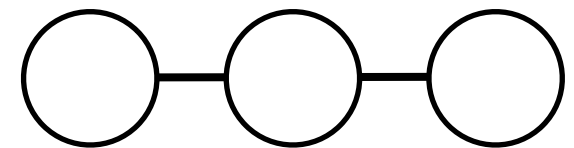
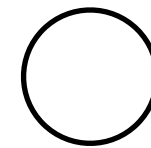
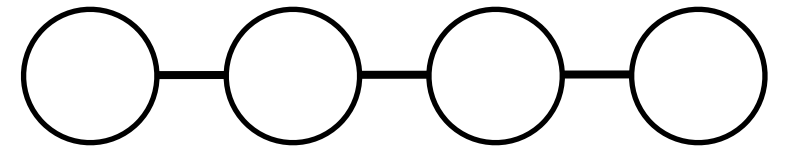
さらに言い換え

- パスがいくつかある
- 頂点と辺に数が書いてある
- N 頂点のうち K 個に色を塗る
- 利益の合計を最大化しよう



さらに言い換え

- パスがいくつかある
- 頂点と辺に数が書いてある
- N 頂点のうち K 個に色を塗る
- 利益の合計を最大化しよう



→ 解けそう 🍷('ω'🍷)≡🍷('ω')🍷≡(🍷'ω')🍷

パスが1本の場合

- 頂点に左から順に1 .. Nの番号を付ける
- 「今見ている頂点」
「今まで塗った頂点の数」
「最後の頂点を選んだか」
でDP

パスがC本ある場合

- パスに1 .. Cの番号を付ける
- それぞれのパスに対し, 「K個塗った時の最大利益」を各Kについて求めておく
- 「今見ているパス」
「今まで塗った頂点の数」
でDP

満点解法

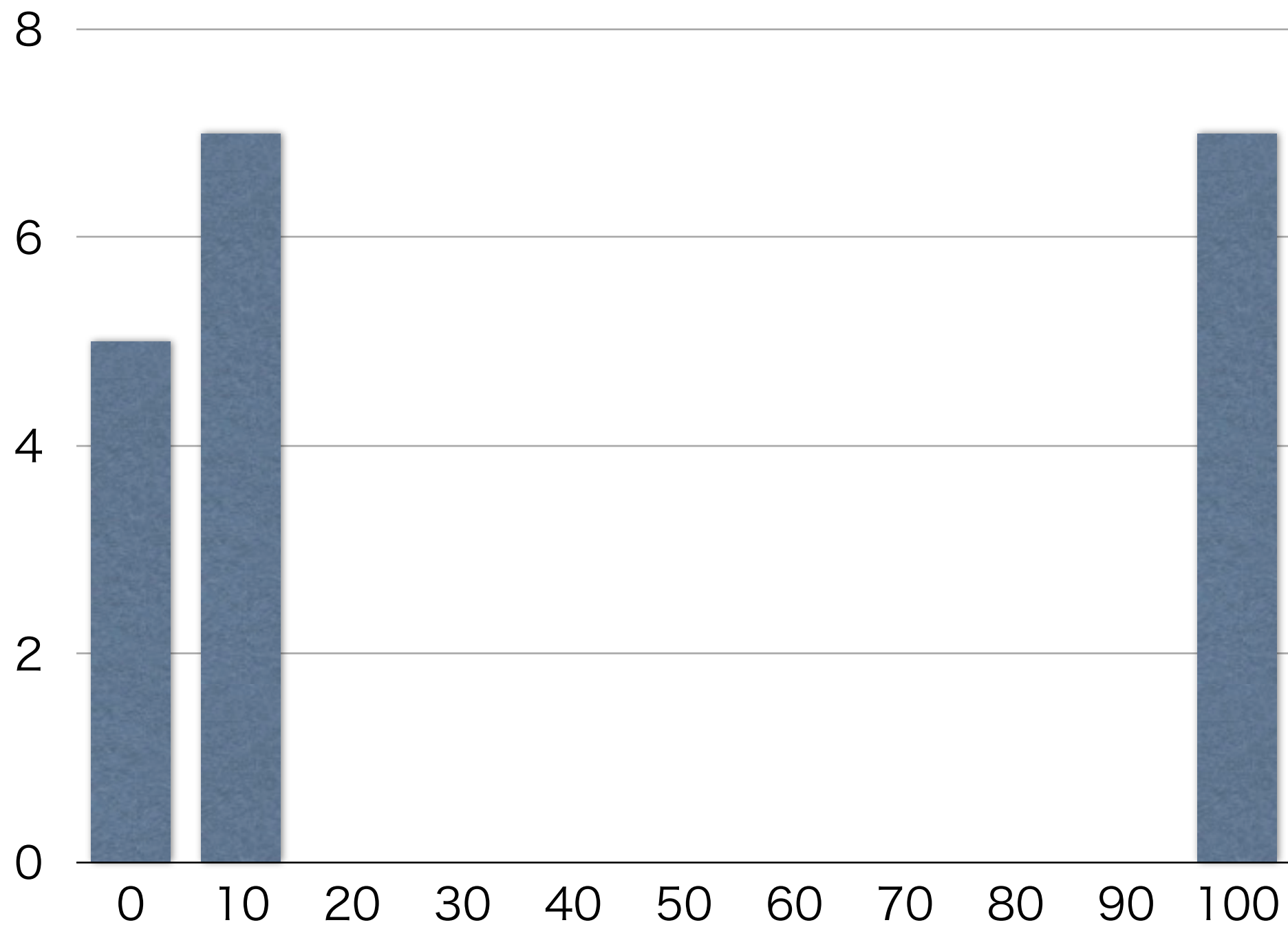
- 考察でパスの問題に落としこむ
- DP2回で解ける
- $O(N^2)$

結論

- **考察が本質！**
 - 条件をうまく利用
 - 小さいケースで実験
 - グラフに落としこむ



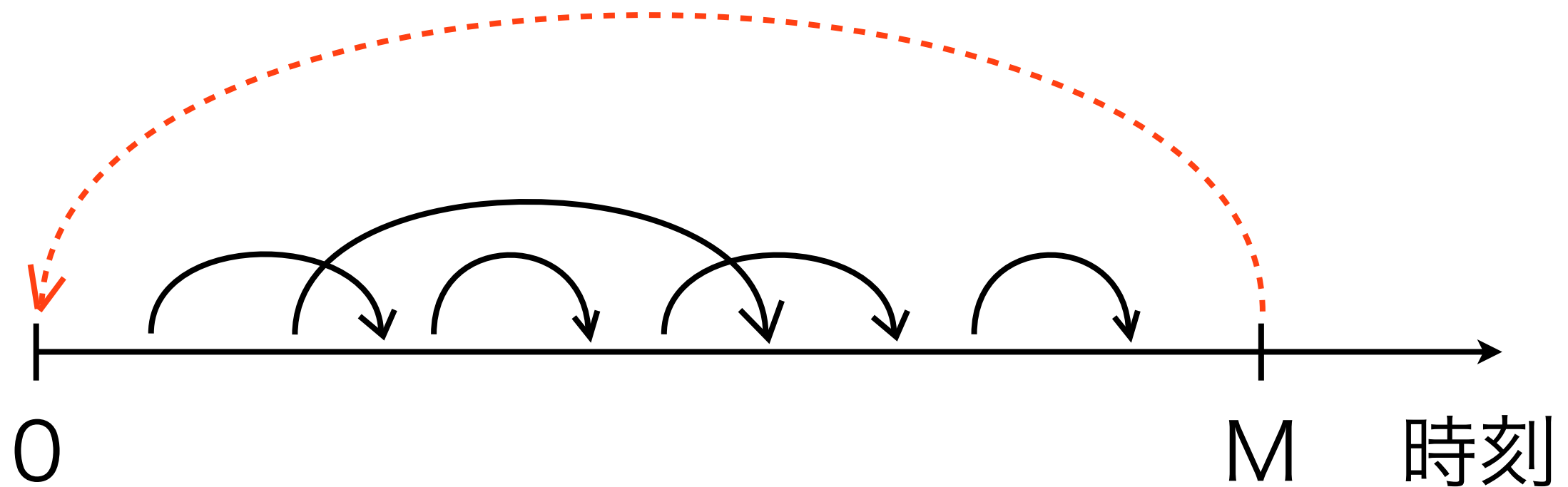
得点分布



おまけ

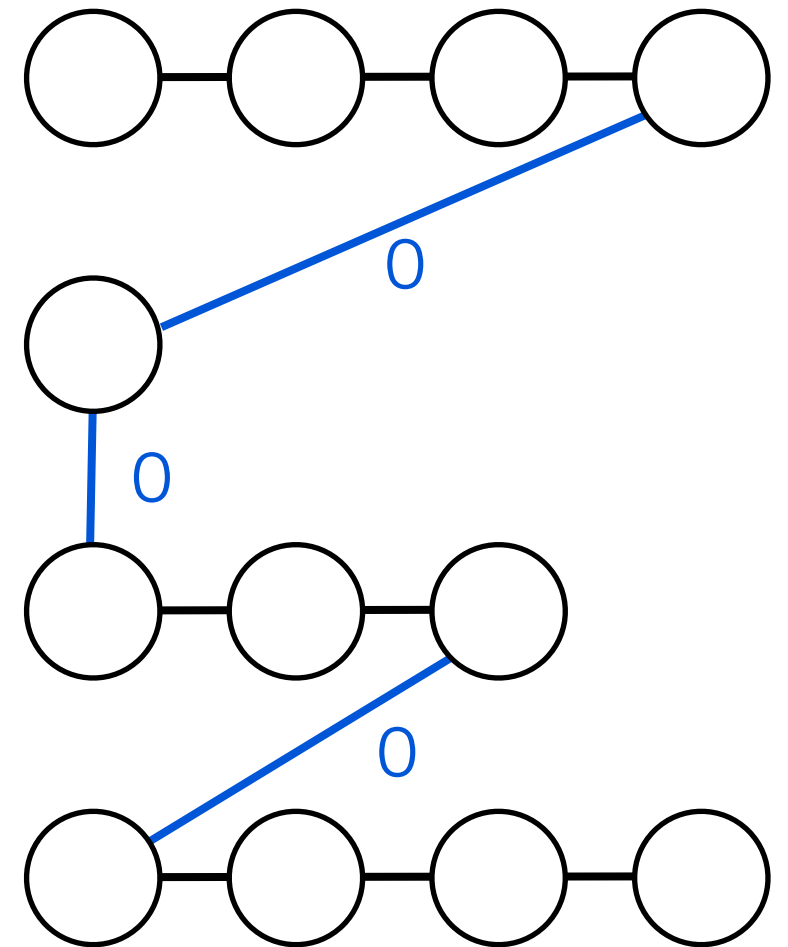
端点の扱い

- 時刻**M**に会社を出て、
時刻**0**に会社に戻る
変な社員**N+1**を作ればよい



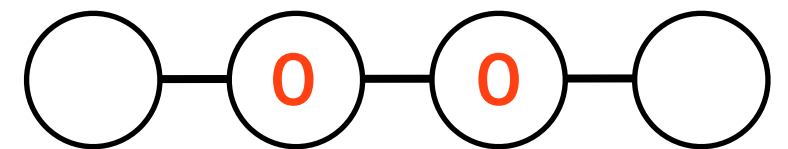
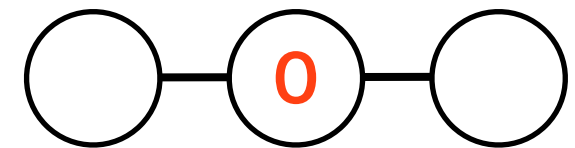
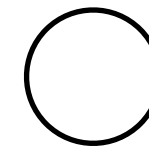
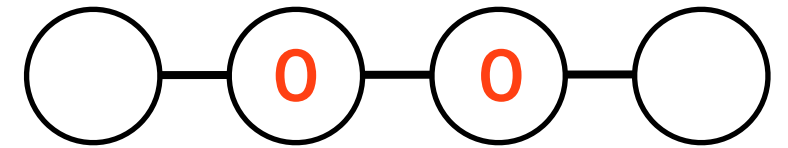
DP2回もしたくない

- パスとパスを0の辺で結ぶ
- パスが1本だと思ってDP
- 楽



驚愕の事実

- パスの端点以外に書かれた値はすべて0
- これを利用して計算量を下げることが可能？
(できるとは言っていない)



JOI2014-2015 春合宿 Day2
道路整備 (Road Development)
解説

三谷 庸 (wo)

問題概要

- N 頂点のグラフがある
- 最初は辺はない
- グラフの更新クエリと出力クエリを処理せよ
- $N \leq 100,000$
- $Q \leq 300,000$

問題概要

- 更新クエリ (タイプ 1 のクエリ) (u, v)
 - u と v が非連結ならば、 u と v をコスト 1 の辺で結ぶ
 - u と v が連結ならば、それらの最短路上の辺のコストをすべて 0 にする
- 出力クエリ (タイプ 2 のクエリ) (u, v)
 - u と v の最短路のコストを出力する

小課題 1 (10 点)

- $N \leq 1,000$
- $Q \leq 3,000$

小課題 1 (10 点)

- クエリごとに $O(N)$ や $O(N \log N)$ かけてよさそう
- クエリごとにその時点のグラフ上で最短路問題を解くことで処理できる

小課題 1 (10 点)

- 更新クエリ (u, v)
- 頂点 u と頂点 v が非連結な場合は辺を張るだけ
- 頂点 u から頂点 v への最短路に辺 (x, y) が含まれる条件は以下のいずれかが成り立つこと
 - $\text{dis}(u, x) + \text{cost}(x, y) + \text{dis}(y, v) = \text{dis}(u, v)$
 - $\text{dis}(u, y) + \text{cost}(y, x) + \text{dis}(x, v) = \text{dis}(u, v)$

小課題 1 (10 点)

- 更新クエリ (u, v)
- u からの最短路と v からの最短路を求めた後、すべての辺について u から v への最短路に含まれるか判定し、含まれるならばコストを 0 にする

小課題 1 (10 点)

- 出力クエリ (u, v)
- 最短路問題を解いて結果を出力すればよい

グラフの形は？

- 小課題 2 以降は N, Q がとても多い
- 特殊な構造を見つける必要がある
- できるグラフの形について考察する

グラフの形は？

- グラフの更新クエリを再検討
- u と v が非連結ならば、辺を張る
- u と v が連結ならば、 $\dots$ のコストを 0 にする

グラフの形は？

- グラフの更新クエリを再検討
- u と v が非連結ならば、 u と v の間に辺を張る
- u と v が連結ならば、 $\dots$ のコストを 0 にする
- u と v が連結ならば、 u と v の間には**辺を張らない**

グラフの形は？

- グラフの更新クエリを再検討
- よって、グラフに閉路はできない
- 最終的なグラフの形は森になる (非連結な場合もあります)

グラフの形は？

- 更新クエリで「すべての最短路に対して」などと言っているが、実際には最短路は一つしかない

小課題 2 (25 点)

- 全ての更新クエリが来てから出力クエリがくる
- 全ての更新クエリが来た後のグラフの状態が分かればよい

辺のコストは？

- 最終状態において頂点 u と頂点 v の間にコスト 0 の辺が張られている条件を考える
 - u と v が非連結な状態で更新クエリ (u, v) が呼ばれる
 - その後、あるコストを 0 にするクエリ (w, x) に対して、 w から x への最短路で辺 (u, v) を用いる
- もう少しわかりやすく言い換えられないか？

小課題 2 (25 点)

- 全ての更新クエリ (u, v) に対して、頂点 u と頂点 v の間に辺を張る
- 頂点 u と頂点 v の間をコスト 0 で行き来できる条件は、このグラフ上で頂点 u から頂点 v に 2 通り以上で行けること

小課題 2 (25 点)

- 逆に言うと、このグラフ上で辺 (u, v) のコストが 1 である条件は、頂点 u から頂点 v へ 1 通りの方法でしか行けないこと

小課題 2 (25 点)

- 逆に言うと、このグラフ上で辺 (u, v) のコストが 1 である条件は、頂点 u から頂点 v へ 1 通りの方法でしか行けないこと
- グラフ理論の用語でいうと、辺 (u, v) が橋になっていること

橋・二重辺連結成分

- グラフの辺 (u, v) が橋であるとは、辺 (u, v) を取り除いた時にグラフの連結成分が増えることをいう
- 橋でない辺でつながった成分を二重辺連結成分という
- これは DFS を用いて $O(N)$ で検出できる
- 詳しくは調べてください

小課題 2 (25 点)

- 全ての更新クエリ (u, v) に対して (u, v) に辺を張ったグラフを考える
- 二重辺連結成分分解する (多重辺に注意)
- その結果は森になる
- 森の上のパスの長さを求めたい

最近共通祖先 (LCA)

- 木の根を (適当に一つ) 決める
- 頂点 u と頂点 v の共通の先祖であって、最も深い (u や v に最も近い) ものを最近共通祖先 (LCA) という
- $O(\log N)$ 時間で求まる
- 詳しくはプログラミングコンテストチャレンジブック (蟻本) などを参照

小課題 2 (25 点)

- 頂点 u と頂点 v (が属する二重辺連結成分) のLCA を求めると、 u と v の距離は $\text{dep}(u) + \text{dep}(v) - \text{dep}(\text{lca}) * 2$ と高速に求まる
- これで出力クエリが処理できた

グラフの形は？ (続き)

- 辺のコストに関する考察は後回しにして最終的にできる森を構成する
- $T[i] = 1$ となるクエリをすべて見て行って、 $A[i]$ と $B[i]$ が非連結ならばそれらの間に辺を張ることを $i = 1, 2, \dots, Q$ に対して順に行う

グラフの形は？ (続き)

- クエリはオフライン (最初にすべて見れる) なのでこのようなことができる
- 今後、この森の上で考える
- つまり、最初にクエリをすべて見て森を構成した後、もう一度最初からクエリを見て出力クエリに答えていく

グラフの形は？ (続き)

- 更新クエリ (u, v)
 - (u, v) が森の辺になっている場合は無視
 - そうでない場合、 u から v へのパス上の辺のコストをすべて 0 にする
 - 森の作り方より、このクエリよりも後に追加される辺が u, v 間の最短路に影響することはない
- 出力クエリ (u, v)
 - u から v へのパス上の辺のコストの和を求める

グラフの形は？ (続き)

- 以下の解説では次のことを仮定します
 - できる森は木になる
 - 出力クエリ (u, v) が来るとき u と v は連結である
- 森が木でない (非連結である) 時はそれぞれに対して処理すればよい
- 出力クエリの二頂点の連結性は Union Find 木で判定できる

小課題 3 (25 点)

- 全ての更新クエリ (u, v) に対して、次のいずれかが成り立つ
 - u と v は非連結
 - u と v は距離 200 以下

小課題 3 (25 点)

- 言い換えると、
 - 全ての「辺のコストを 0 にするクエリ (u, v) 」に対して、 u と v の距離は 200 以下
- 辺のコストを 0 にするクエリは一つずつ辺を見ていけばよい
- 出力クエリを高速に処理する必要がある
- 木のパス上の辺のコストの和を高速に求めたい

小課題 3 (25 点)

- まとめると、やりたいことは
 - 木の辺のコストを 1 から 0 にする
 - 木の上のパス上の辺のコストの和を求める
- ほぼ同じ問題が蟻本にも載っています

小課題 3 (25 点)

- 出力クエリ
 - $\text{dis}(u, v) = \text{dis}(r, u) + \text{dis}(r, v) - \text{dis}(r, \text{lca}) * 2$
 - 各頂点 v に対して根と頂点 v の間の辺のコストの和が分かれば、任意のパスのコストが分かる

小課題 3 (25 点)

- 更新クエリ
 - 辺を列挙する
 - u と v から 1 つずつ親にさかのぼっていけばよい
 - LCA を求めてもよい

小課題 3 (25 点)

- 辺 (c, p) (p は c の親)のコストを 1 から 0 にすると $\text{dis}(r, v)$ はどう変わるか
- v が c の子孫でないとき
 - 何も変わらない
- v が c の子孫であるとき
 - $\text{dis}(r, v)$ は 1 減る
- 「 c を根とする部分木」をうまく扱いたい

DFS 順序

- 木を根から DFS する
- 頂点を、初めて訪れた時間が早い順に並べる
- この時、「頂点 v を根とする部分木」は、「頂点 v から始まるある区間」に対応する

DFS 順序

$c = 0$

$\text{dfs}(v, p)$

$\text{pre}[v] = c$

$c = c + 1$

 for each u in (v に隣接している p 以外の頂点)

$\text{dfs}(u, v)$

$\text{pos}[v] = c$

$\text{dfs}(r, -1)$

DFS 順序

- この時、頂点 v は $\text{pre}[v]$ 番目に訪れたことになる
 - このような順序のつけ方を行きがけ順と言います
- v を根とする部分木の頂点は $[\text{pre}[v], \text{pos}[v])$ に含まれる

実現したいことを再確認

- 木がある $\rightarrow$ 数列がある
 - $\text{pre}[v]$ 番目の要素は根から頂点 v までの距離 $\text{dis}(r, v)$
 - 初期値は v の深さ
- v を根とする部分木の各頂点と根との距離が 1 減る $\rightarrow$ 数列のある (v に対応する) 区間の各要素が 1 減る
- v と根の距離を求める $\rightarrow$ 数列のある (v に対応する) 要素を求める

Segment Tree

- これは、Segment Tree を用いて実現できる
- Segment Tree の各ノードには、「そのノードに対応する区間に含まれる要素すべてに一律に足された数値」を持っておく

Segment Tree

[1,6) の要素を -1 したいときは、以下のノードに -1 と書きこむ

[0, 8)							
[0, 4)				[4, 8)			
[0, 2)		[2, 4)		[4, 6)		[6, 8)	
[0, 1)	[1, 2)	[2, 3)	[3, 4)	[4, 5)	[5, 6)	[6, 7)	[7, 8)
0	1	2	3	4	5	6	7

Segment Tree

要素 2 の値を知りたいときは、以下のノードの値の和を求める

[0, 8)							
[0, 4)				[4, 8)			
[0, 2)		[2, 4)		[4, 6)		[6, 8)	
[0, 1)	[1, 2)	[2, 3)	[3, 4)	[4, 5)	[5, 6)	[6, 7)	[7, 8)
0	1	2	3	4	5	6	7

Segment Tree

- このようにすると、区間の更新クエリもある要素の値を求めるクエリも $O(\log N)$ で処理できる

小課題 3 (25 点)

- これらのことを使うと小課題 3 が解ける
- 初期化
 - 木の頂点に DFS 順序をつける
 - Segment Tree を用意し、 $\text{pre}[v]$ 番目の要素を v の深さで初期化する

小課題 3 (25 点)

- 更新クエリ (u, v)
 - u から v へのパスに含まれる辺(c, p)を一つずつ見ていく
 - u および v から LCA までたどっていけばよい
 - $\text{cost}(c, p) = 1$ であつた場合
 - $\text{cost}(c, p)$ を 0 に更新
 - Segment Tree を用いて c を根とする部分木に対応する区間 $[\text{pre}[c], \text{pos}[c])$ の要素を -1 する

小課題 3 (25 点)

- 更新クエリ
 - 一つの辺の処理に $O(\log N)$ 時間かかるので、一つのクエリ (u, v) の処理に $O((u, v) \text{ の距離} * \log N)$ 時間かかる
 - 距離が 200 以下であることから十分高速

小課題 3 (25 点)

- 出力クエリ (u, v)
 - u と v の LCA を求める
 - $\text{dis}(r, u) + \text{dis}(r, v) - \text{dis}(r, \text{lca}) * 2$ を出力
 - $\text{dis}(r, v)$ は Segment Tree の $\text{pre}[v]$ 番目の要素として求められる
 - 一回あたり $O(\log N)$ 時間
 - 十分高速

小課題 4 (25 点)

- 出力クエリは 200 個以下である
- 出力クエリ (u, v) に対しては、 u から v へ一本ずつ辺をたどっていけばよい
 - どの辺のコストが 1 でどの辺のコストが 0 かを持っておく必要がある

小課題 4 (25 点)

- この小課題は小課題 3 とは独立に解けます
 - 小課題 3 では更新クエリが良心的で、出力クエリを高速化
 - 小課題 4 では出力クエリが良心的で、更新クエリを高速化
- このようなこともあるので、小課題の制約も全部読みましょう

小課題 4 (25 点)

- 辺のコストは、
 - 最初 1
 - 更新クエリが来ると 0 になる
 - その後ずっと 0

小課題 4 (25 点)

- 辺のコストが変わるのは高々 $N - 1$ 回
- それぞれの更新クエリ (u, v) に対して、毎回 u から v へのパス上の辺をすべて見るのは無駄
- コスト 0 の辺をうまく飛ばしたい

小課題 4 (25 点)

- やりたいことは以下の通り
 - 頂点 v からその祖先の頂点 a に向かうパスの上のコスト 1 の辺を列挙する
 - 辺のコストを 0 にする

Union Find 木

- Union Find 木は集合の併合および二つの要素が同じ集合に属するかの判定ができるデータ構造です
- うまく工夫することでこの問題に応用できます

Union Find 木

- コスト 0 の辺でつながっている頂点を Union Find でくっつける
- Union Find の各成分の根はその成分に属する頂点のうち最も浅いものにする
 - 普通の Union Find では二点 x, y をくっつける時にどちらを親にするかは自由だが、今回は元の木で浅い方を根にする

Union Find 木

- これでもクエリあたりのならし計算量が $O(\log N)$ になることが知られています
 - もちろんパス圧縮は行う
 - 普通の Union Find 木で、どちらを親にするかを rank などと管理すると $O(\alpha(N))$ になります

小課題 4 (25 点)

- Union Find の v を含む成分の根を t とすると、 $(t, \text{par}[t])$ は v から親に向かってたどっていったときに最初に出てくるコスト 1 の辺になっている
 - t が木全体の根でない場合

小課題 4 (25 点)

- Union Find を以上のようにして用いると、小課題 4 は次のように解ける

小課題 4 (25 点)

- 更新クエリ
 - v からその先祖 a へのパス上の辺のコストを 0 にする処理
 - v からはじめて以下のことを繰り返す
 - その頂点を含むコスト 0 の辺でつながった成分の根に行く
 - a より浅くなったら終了
 - 今の頂点と親を結ぶ辺のコストを 0 にする
 - 親に移動
 - Union Find を更新

小課題 4 (25 点)

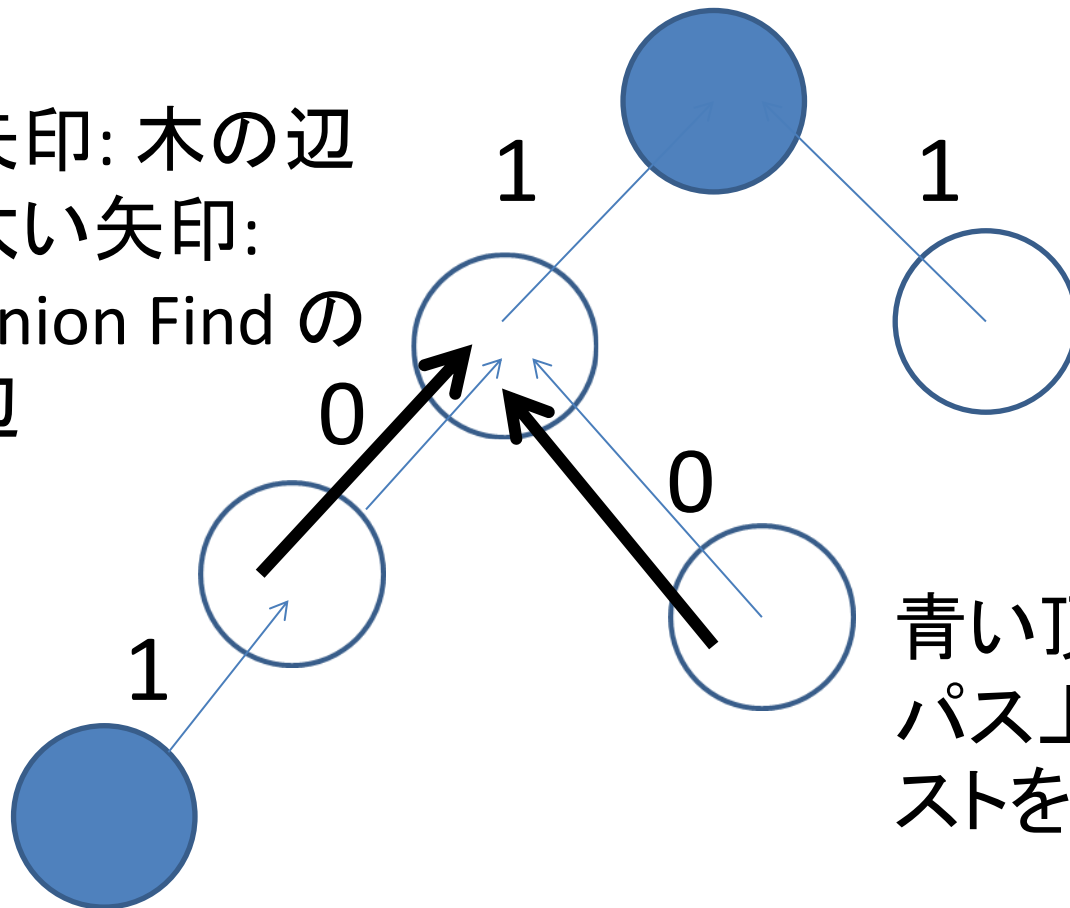
数字: 辺のコス

ト

矢印: 木の辺

太い矢印:

Union Find の
辺



青い頂点を結ぶ
パス上の辺のコ
ストを 0 にする

小課題 4 (25 点)

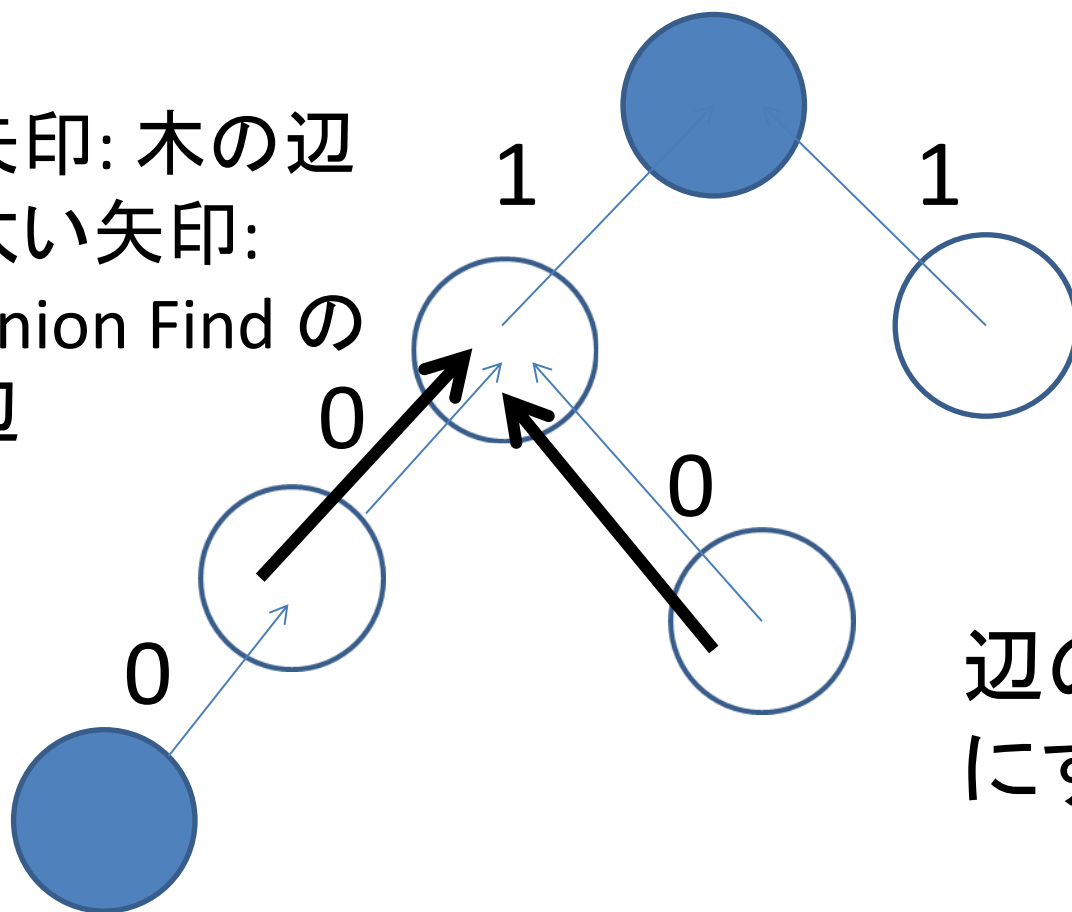
数字: 辺のコス

ト

矢印: 木の辺

太い矢印:

Union Find の
辺



辺のコストを 0
にする

小課題 4 (25 点)

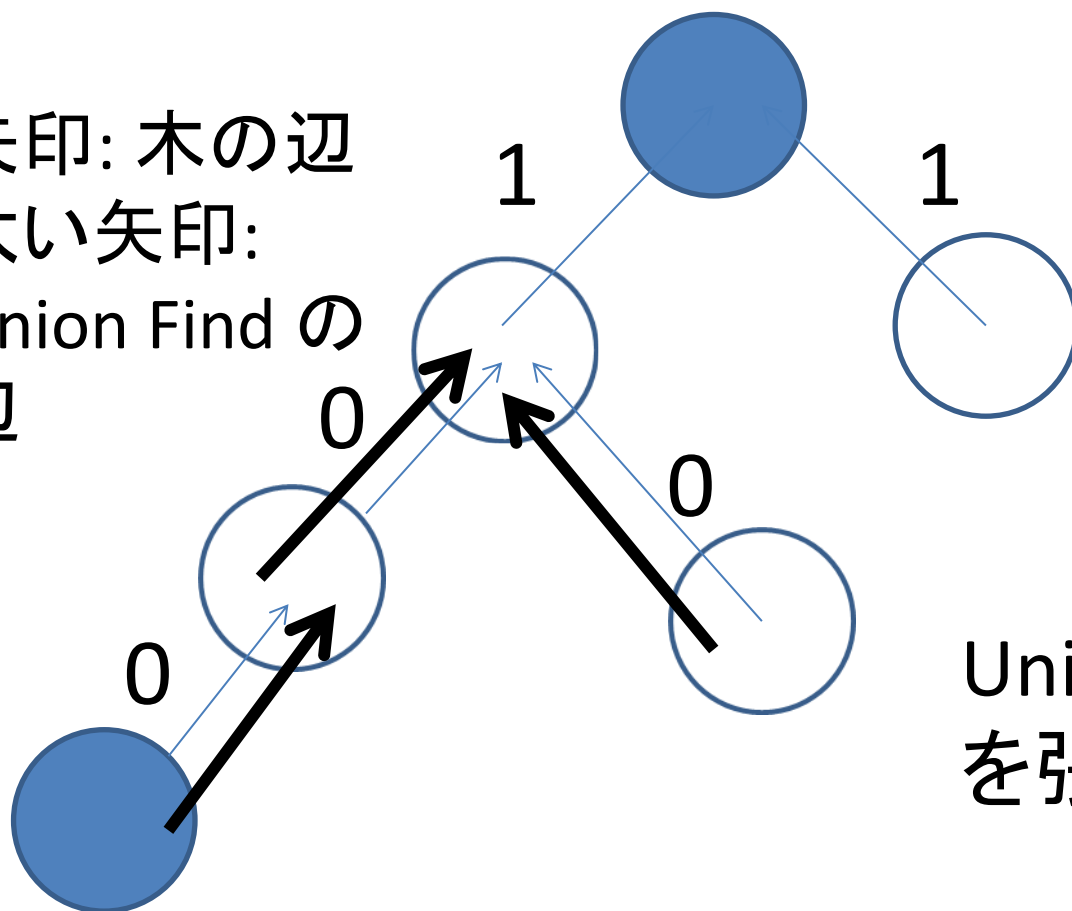
数字: 辺のコスト

ト

矢印: 木の辺

太い矢印:

Union Find の
辺



Union Find で辺
を張る

小課題 4 (25 点)

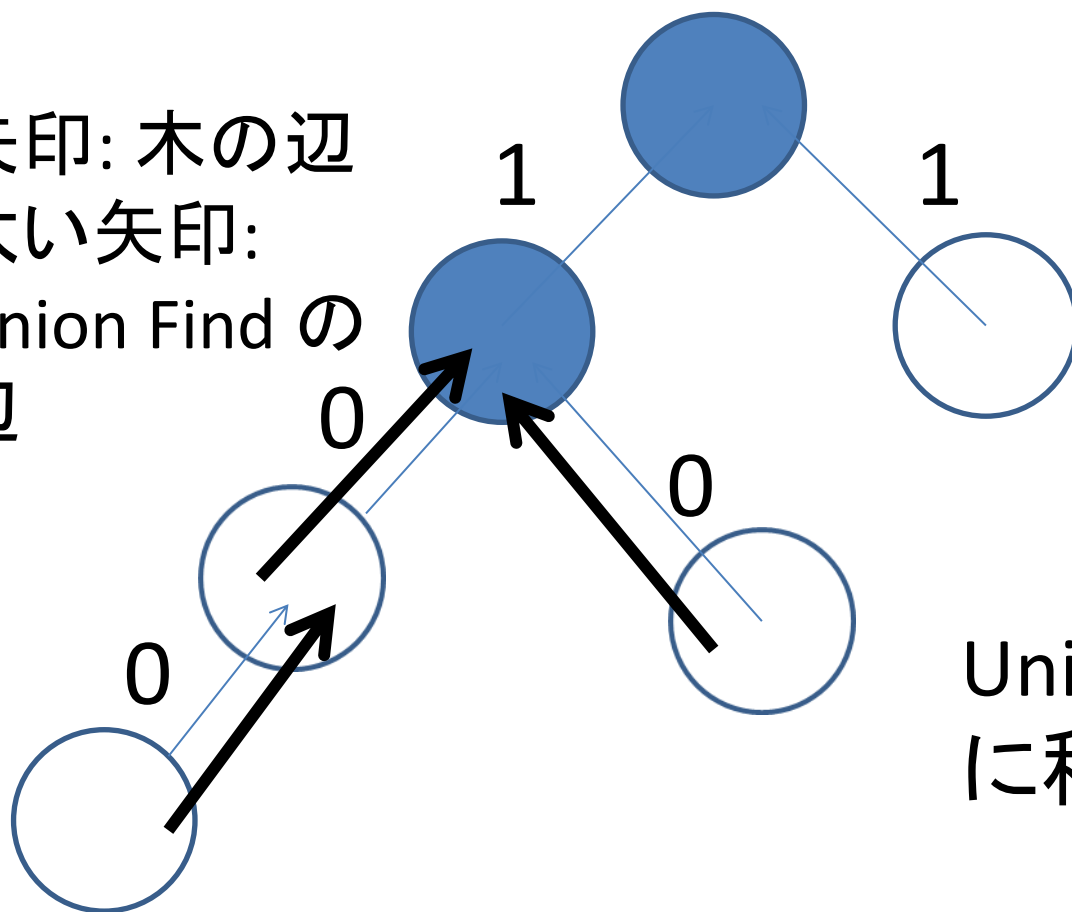
数字: 辺のコスト

ト

矢印: 木の辺

太い矢印:

Union Find の
辺



小課題 4 (25 点)

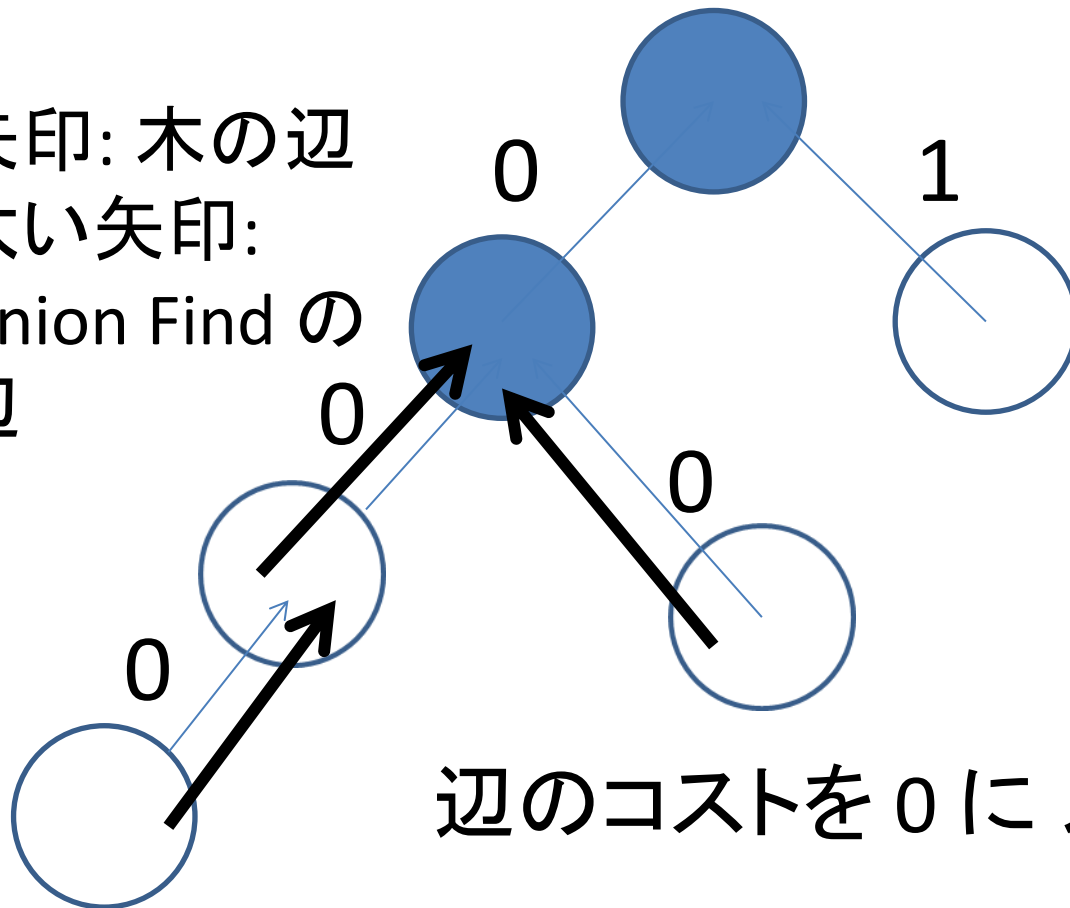
数字: 辺のコス

ト

矢印: 木の辺

太い矢印:

Union Find の
辺



辺のコストを 0 に 以下繰り返し

小課題 4 (25 点)

- 辺のコストの更新は $O(N)$ 回しかないことから、更新クエリは全体で $O(N \log N)$ 以下の時間で処理できる

小課題 4 (25 点)

- 出力クエリ
 - 毎回 $O(N)$ かけて木のパスを求める
 - 最短路上の辺のコストを一つずつ足していく
 - 全部で 200 回以下しか来ないのでこれで間に合う

小課題 5 (15 点)

- 追加の制限はない
- と言っても、ここまでくれば解けているようなものです

小課題 5 (15 点)

- 小課題 3 で、出力クエリが高速に処理できるようになった
- 小課題 4 で、更新クエリが高速に処理できるようになった
- これらの高速化を片方しか行ってはいけない、なんていう理由はない
- 両方やれば小課題 5 が解ける

別解

- Heavy Light Decomposition
 - 木の各辺を Heavy Edge と Light Edge に分ける
 - 木を Heavy Edge でつながったパスをくっつけたものとみなす
 - このとき、木の任意のパスが $O(\log N)$ 個の Heavy Path (Heavy Edge からなるパス) に分けられる
 - Segment Tree と組み合わせることで、木のパスのいろいろな操作が $O(\log^2 N)$ で実現できる
- 詳しくは調べてください

別解

- 実装はそれなりに重いです
- 書ける自信がない人は手を出すかよく考えましょう

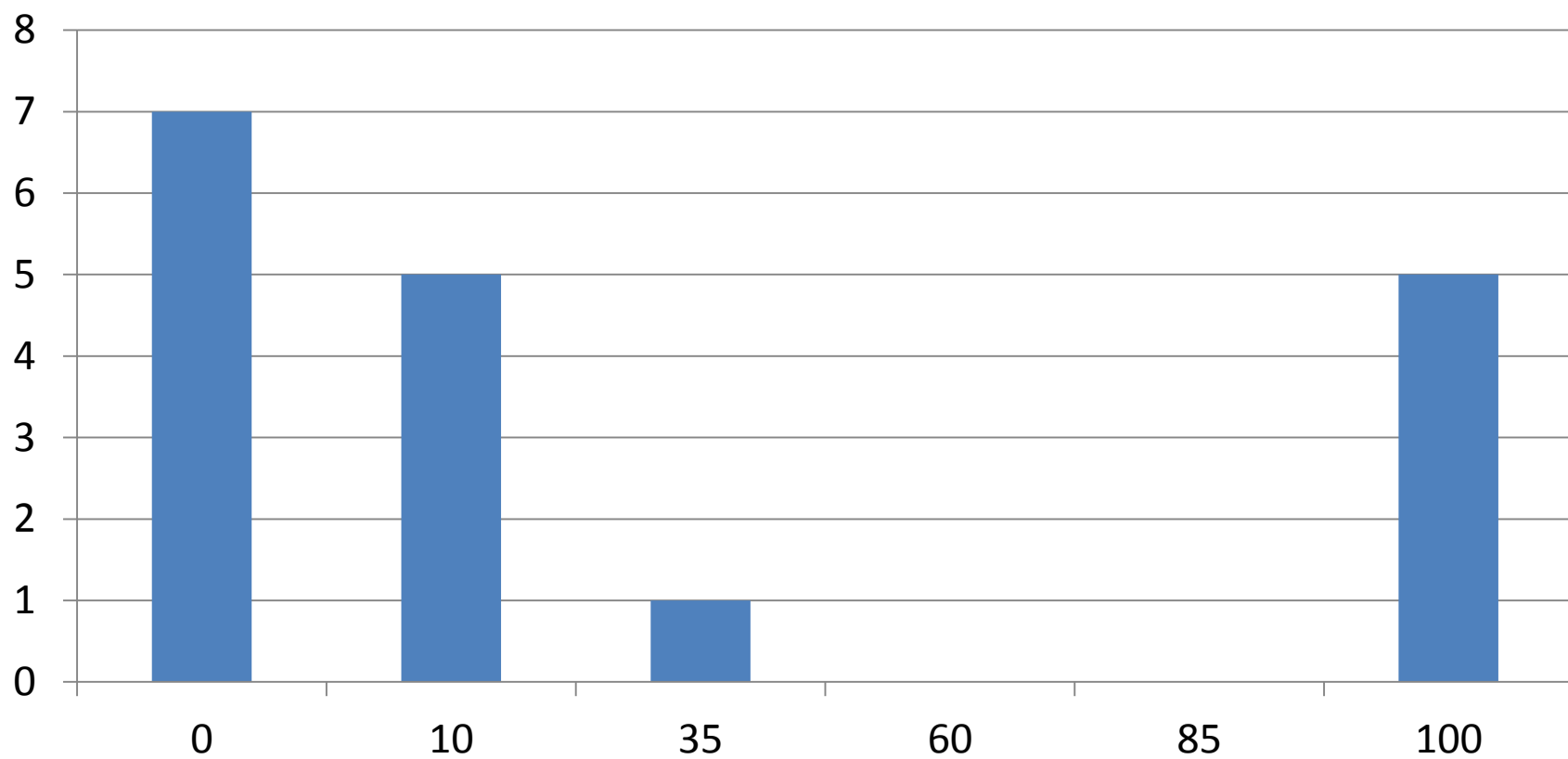
まとめ

- この問題では以下のテクニックを使いました
 - 橋・二重辺連結成分 (部分点解法)
 - 最近共通祖先 (LCA)
 - 木の頂点の DFS 順序
 - Segment Tree
 - Union Find (向きをつける) ですでに処理したところを飛ばす

まとめ

- よく知らなかった人は、蟻本やその他の資料で勉強してください
- どれも重要なテクニックです

得点分布





2015 JOI春合宿 Day3

AAQQZ(AAQQZ) 解説

2015/03/22
山下 洋史
@utakiyoshi

概要



- 文字列が与えられる
- ある区間をソートする
- 回文を探す
- うまい区間を見つけて最長の回文を作る
- 文字の種類数 = 長さ $\leq 3\ 000$

例



ZQQQAZAQSTAN

例



ZQQQAZAQSTAN

区間を選ぶ ZQQ**QAZAQ**STAN

例



ZQQQAZAQSTAN

区間を選ぶ ZQQ**QAZAQ**STAN

ソートする ZQQ**AAQQZ**STAN

例



ZQQQAZAQSTAN

区間を選ぶ

ZQQQ**AZA**QSTAN

ソートする

ZQQ**AAQQZ**STAN

回文発見

ZQQAAQQZSTAN

小課題1

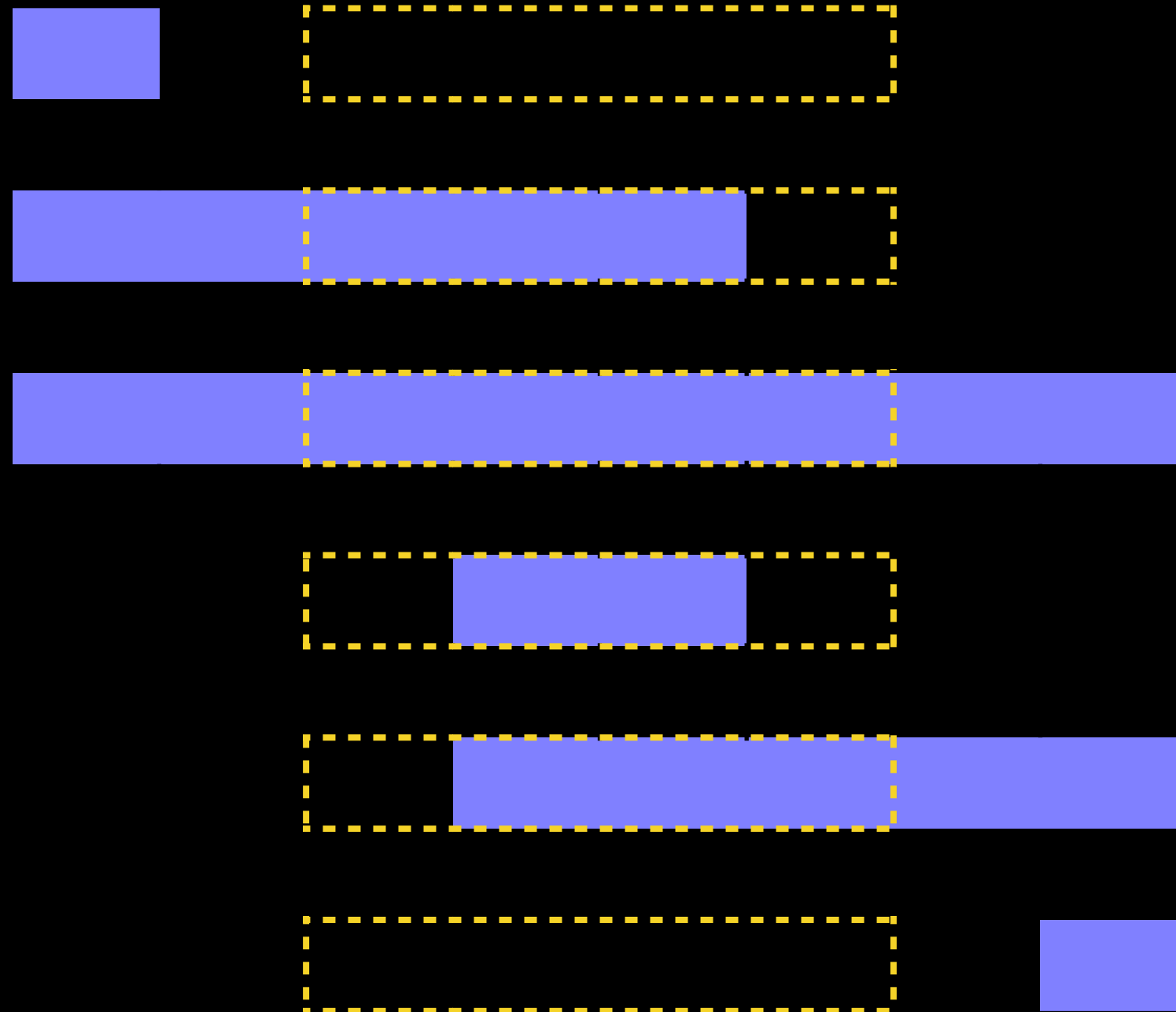
- $O(N^4)$ ($N \leq 50$)
 - 区間を $O(N^2)$ 通り試す
 - ソートして, $O(N^2)$ かけて最長回文を探す
- $O(N)$ の最長回文アルゴリズムなど使うと $O(N^3)$ になるけど.....
 - $N = 3\ 000$ は解けそうにない
 - 「Manacher のアルゴリズム」各自調べること
 - (例えば <http://snuke.hatenablog.com/entry/2014/12/02/235837>)



ここから本題 (満点解法)



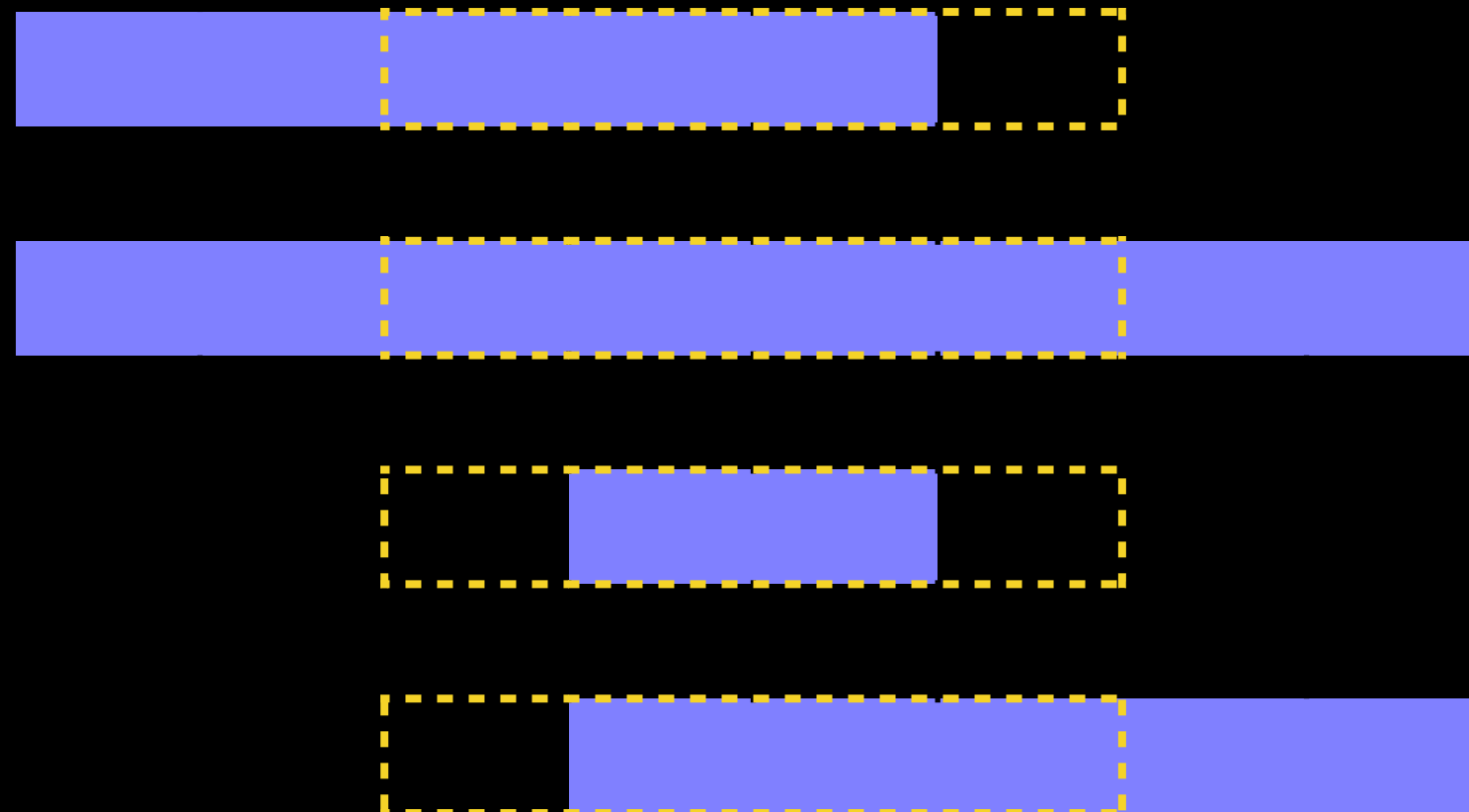
ソート列の 左右の端の位置で分類





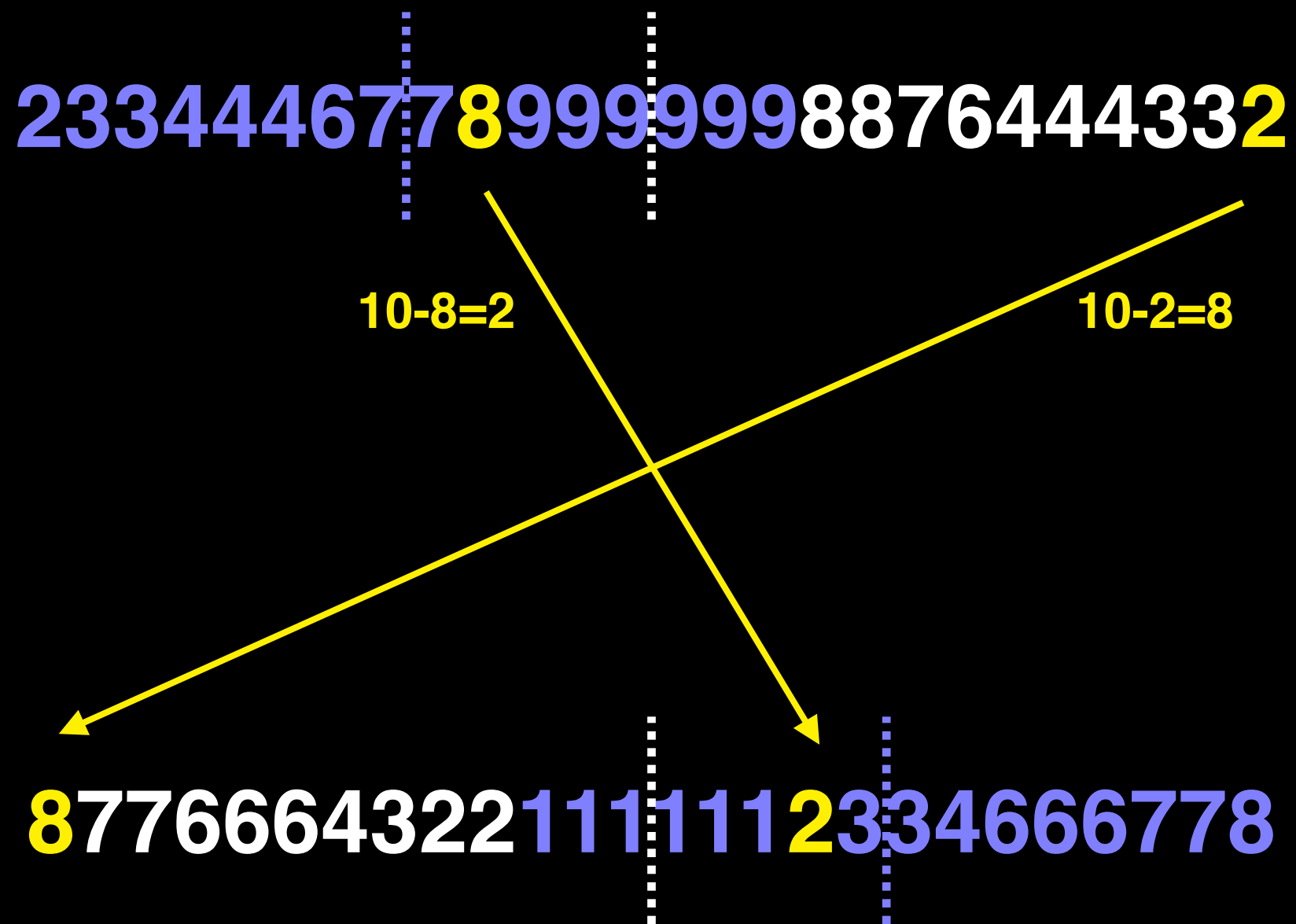


ソート列が回文の 外側に来るものは不要





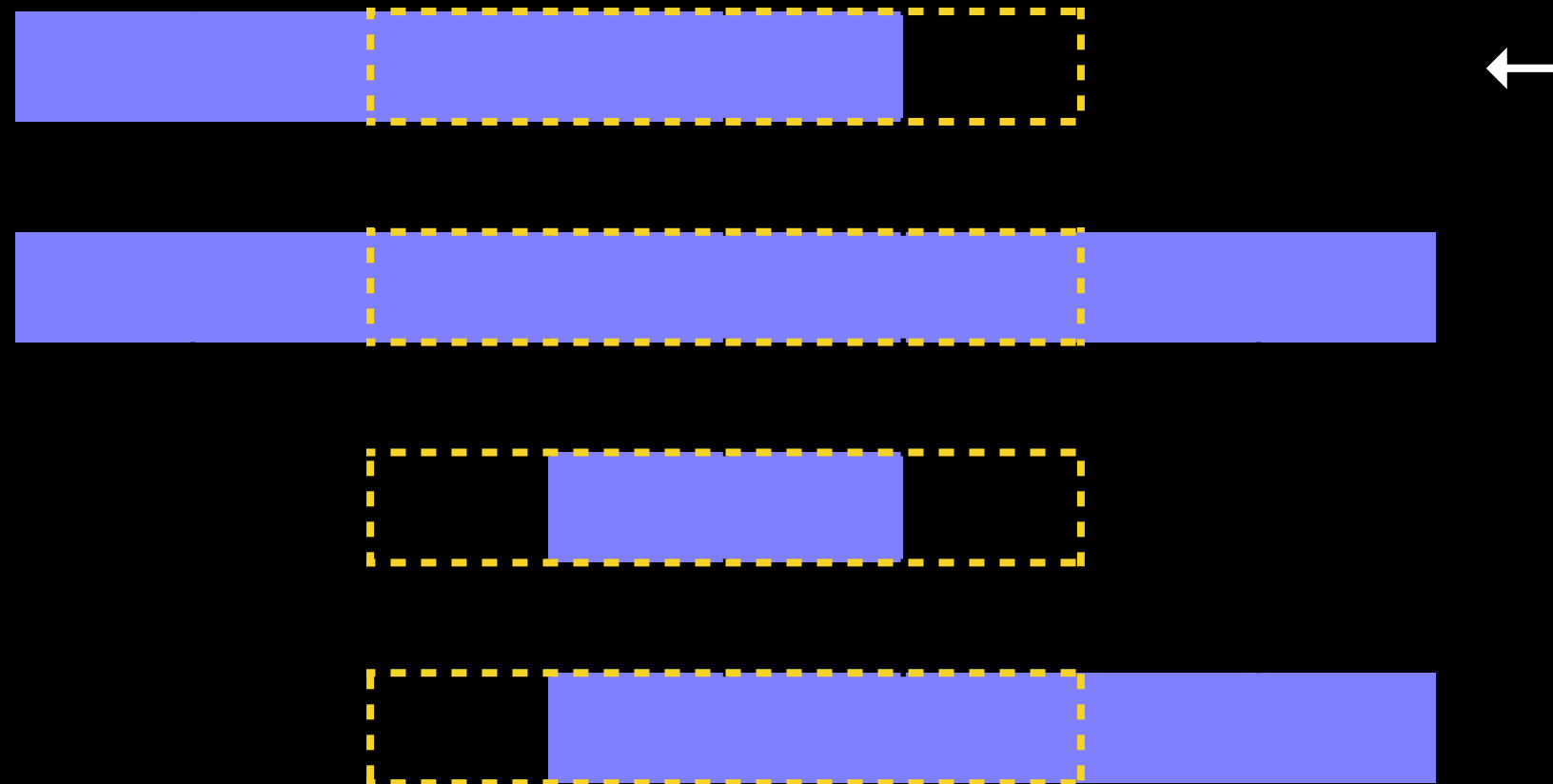
「反転」したものと
合わせて2回解く



ソート列の中心 | が 回文の中心 | より右の場合だけ考えれば良い

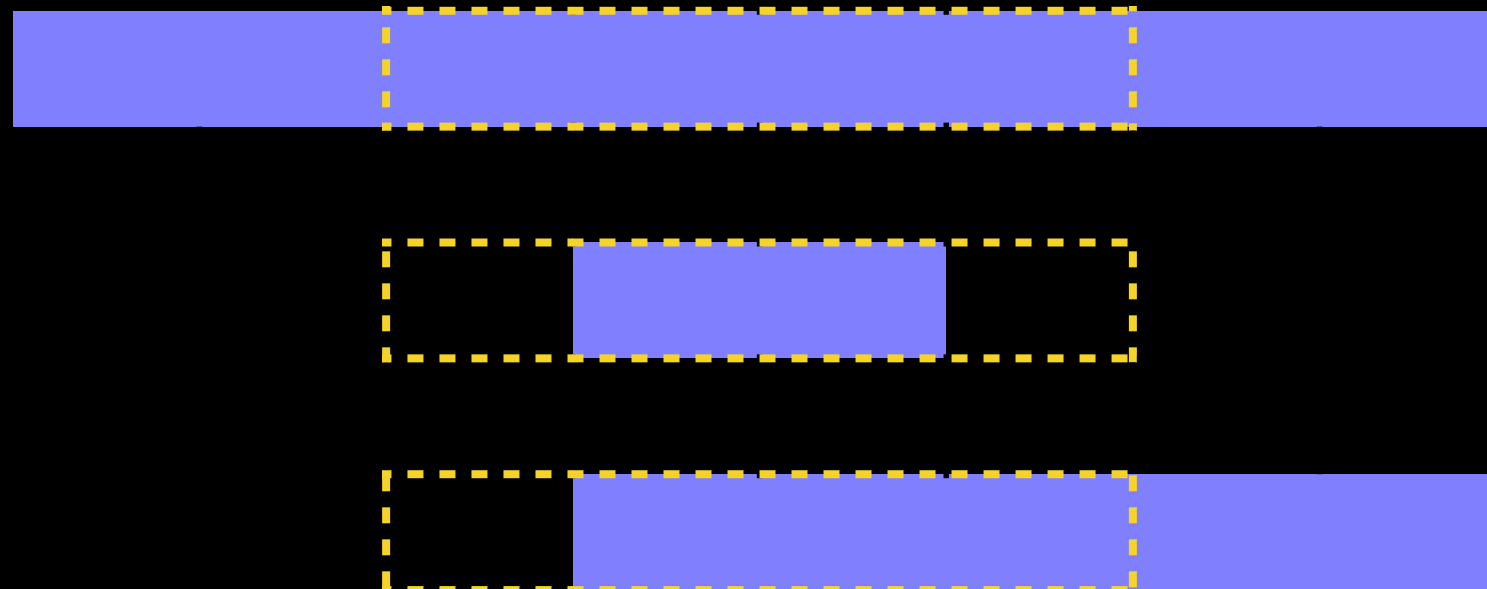


ソート列の中心が 回文の中心より左にあるもの

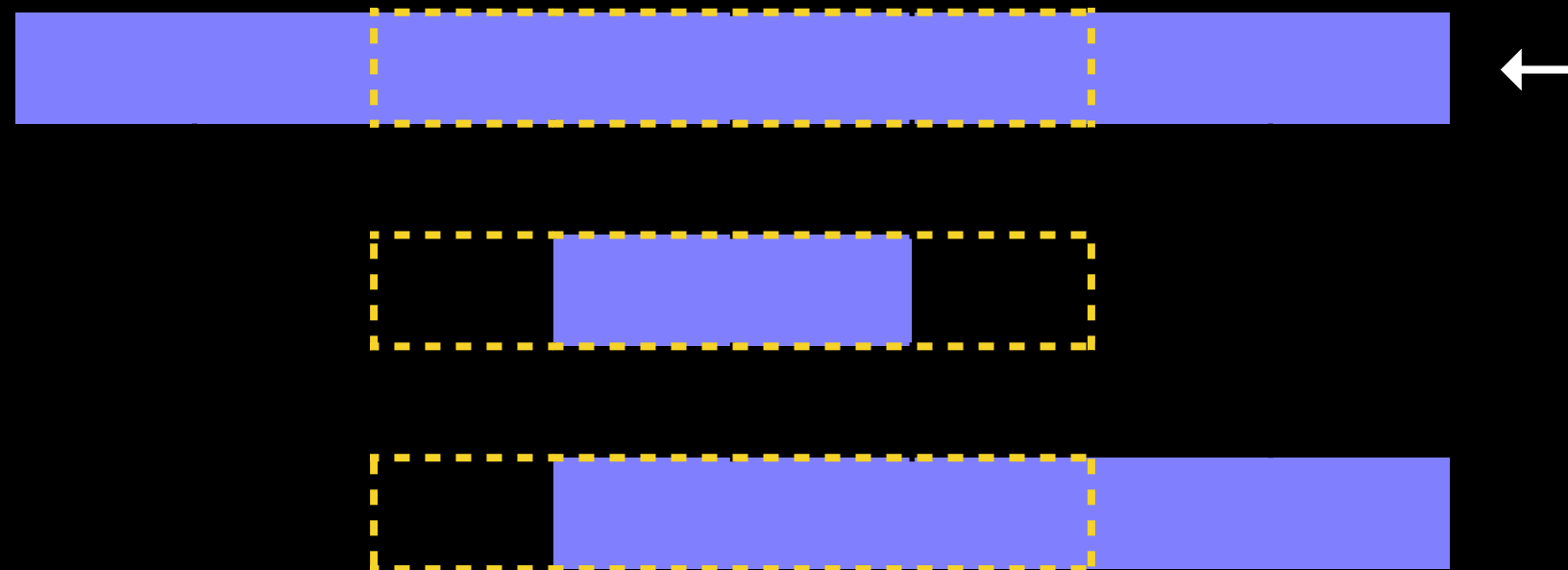




ソート列の中心が 回文の中心より左にあるもの

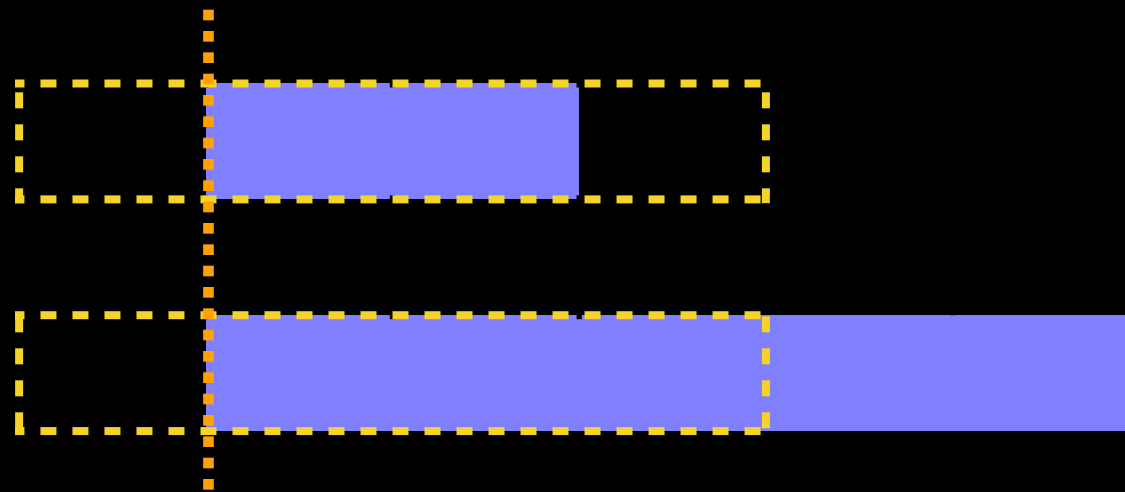


ソート列が回文を覆うものは後回し





なので、
いま考えるべきなのは

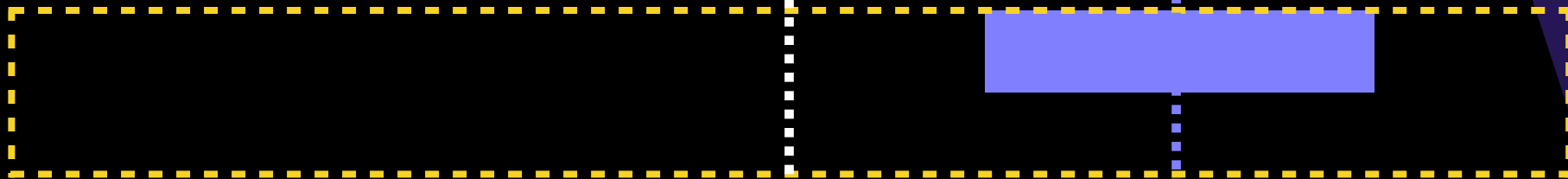


ソート列の左端が回文内に入っているパターン

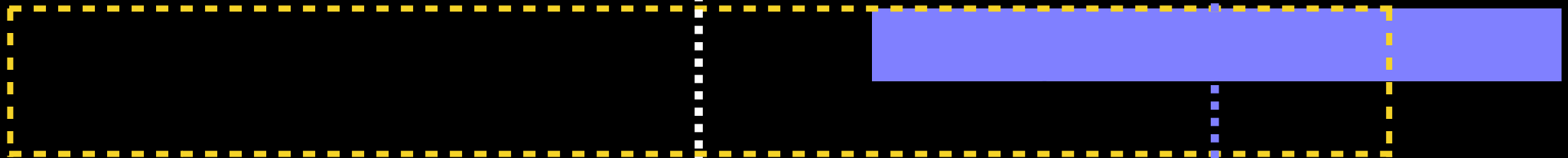


ソート列の 左右の端の位置で分類

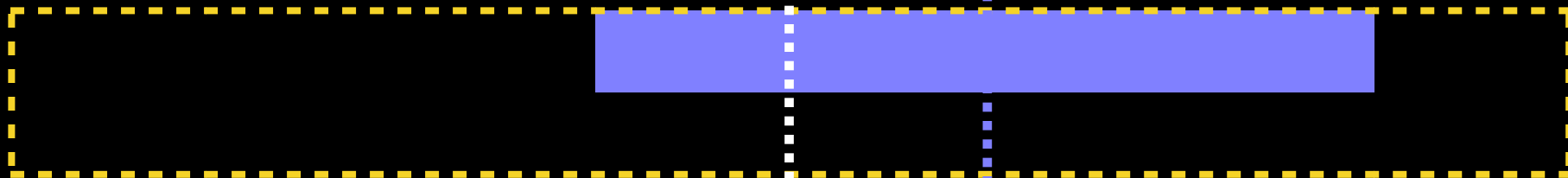
(1)



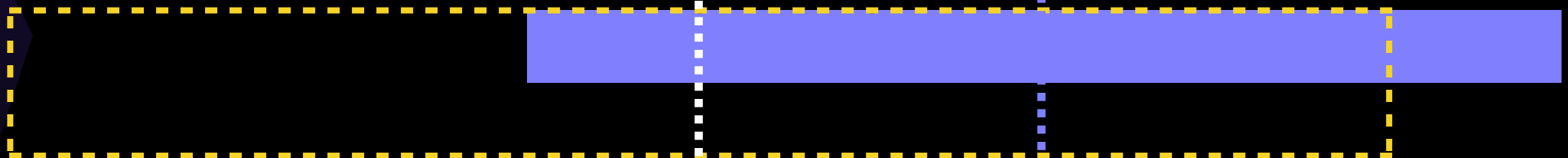
(2)



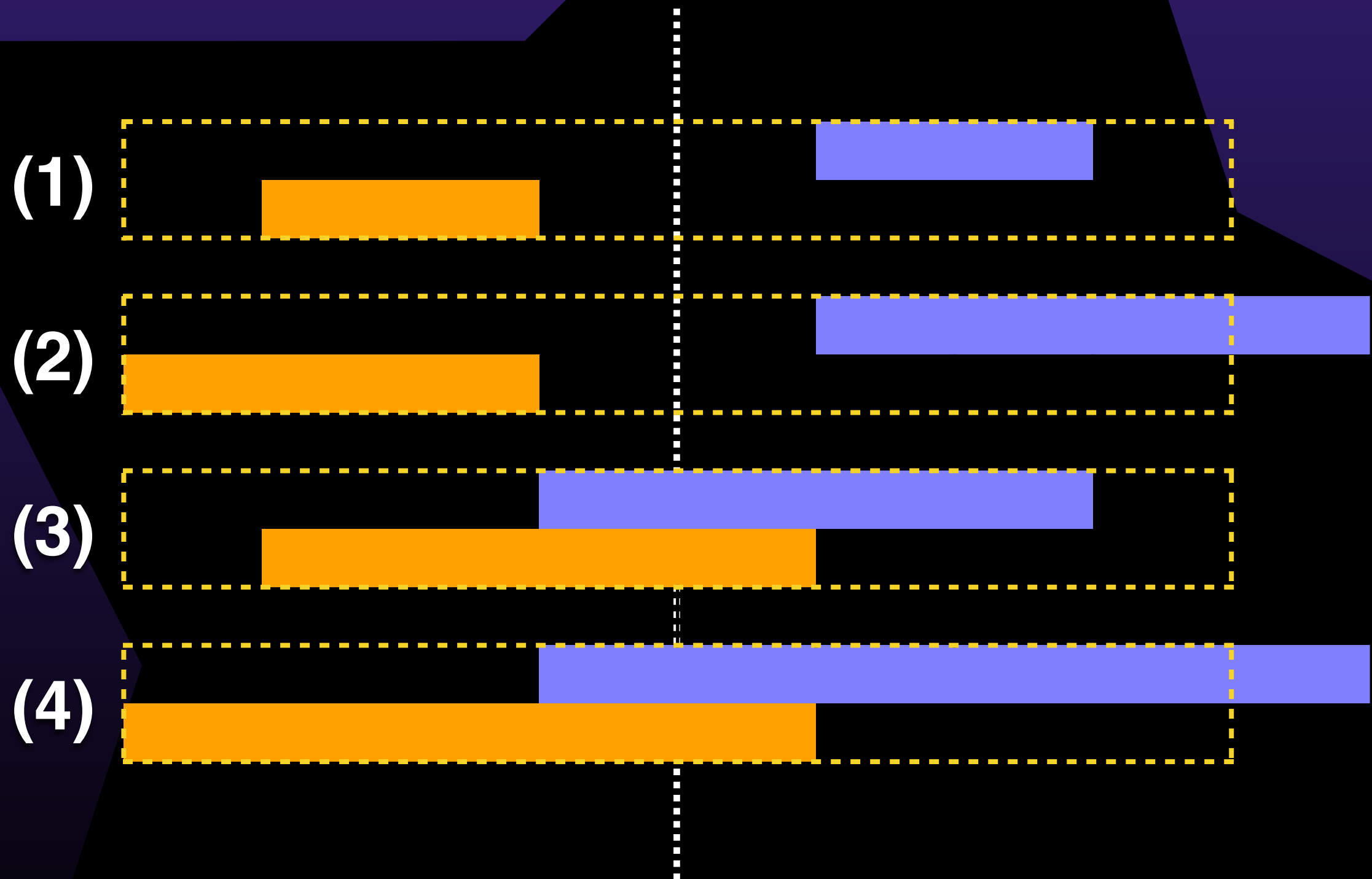
(3)



(4)



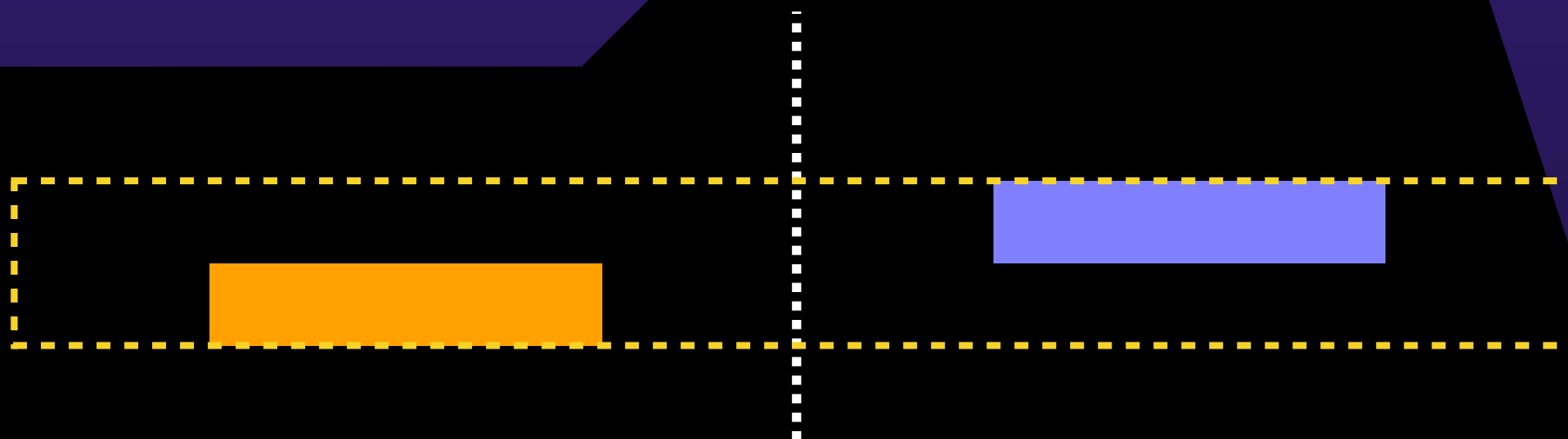
中心に関して対称に



パターン(1)



(1)

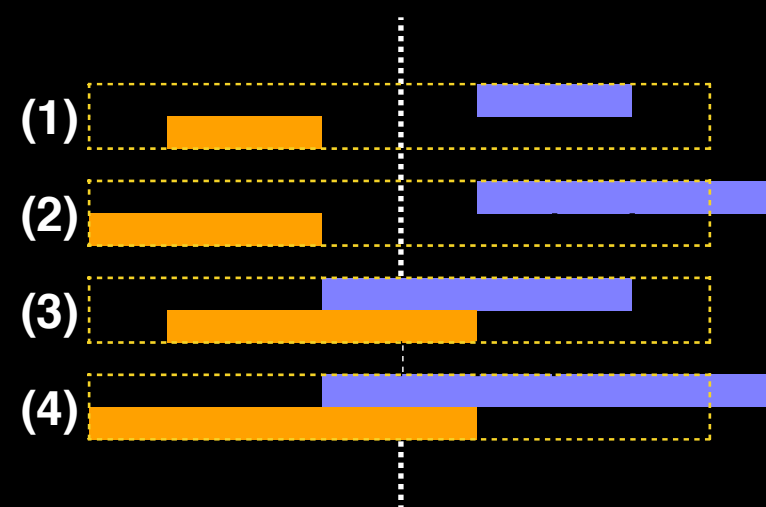


114 664332 514415 233466 411

降順

回文

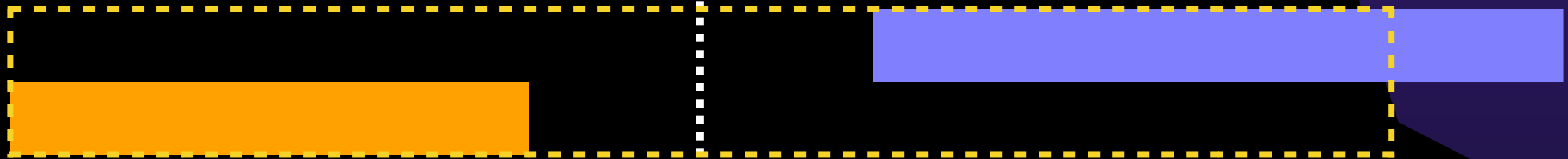
昇順



パターン(2)



(2)

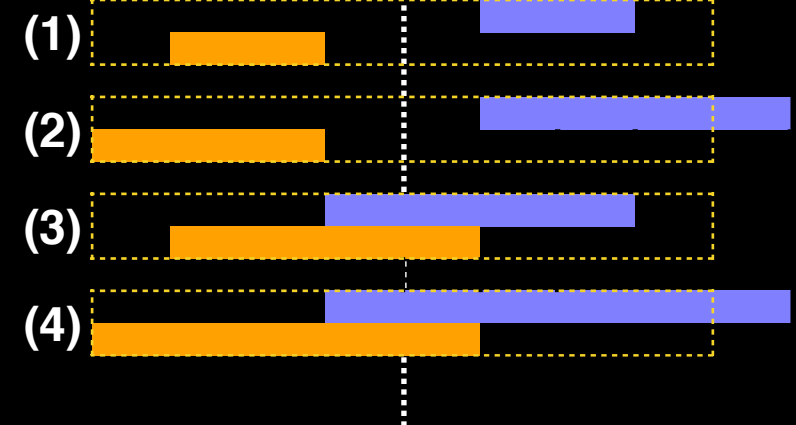


8776664332 514415 2334666778 899

降順

回文

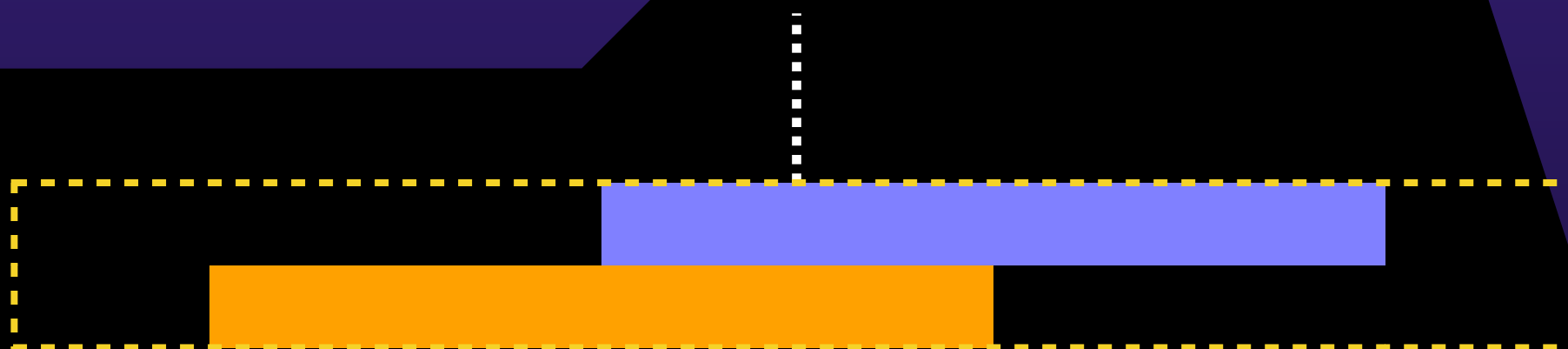
昇順



パターン(3)



(3)

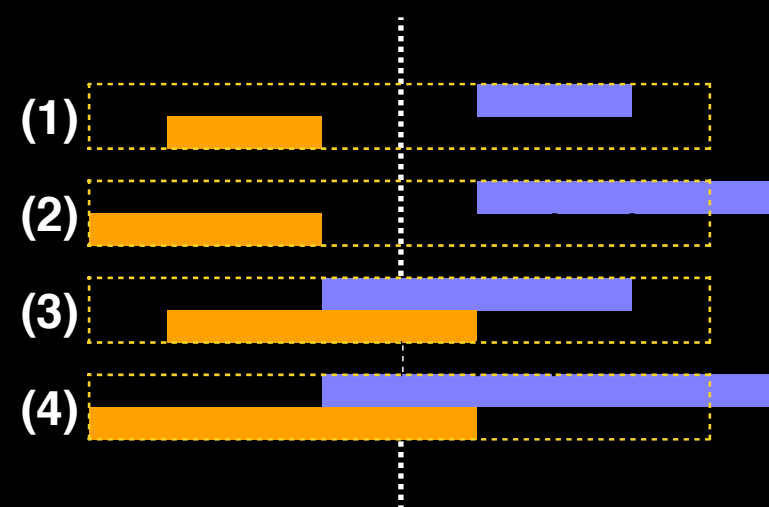


114 664332 111111 233466 411

降順

一定

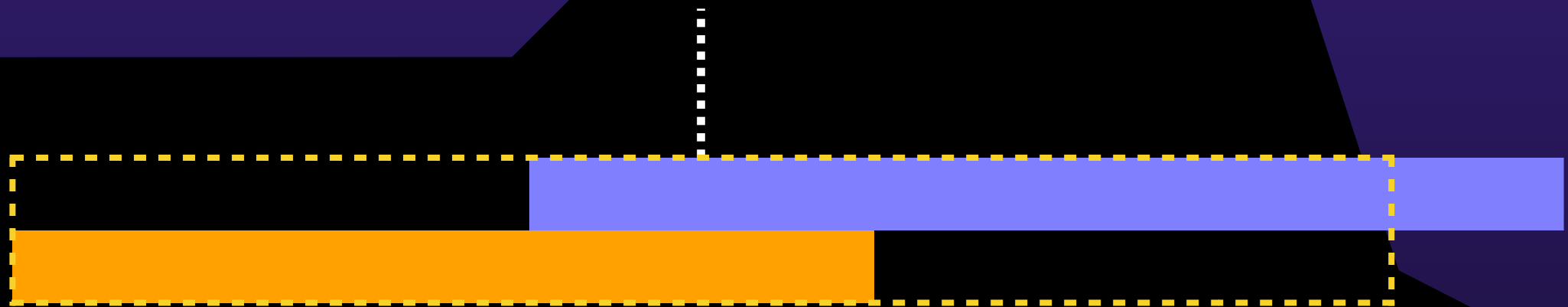
昇順





パターン(4)

(4)



8776664332

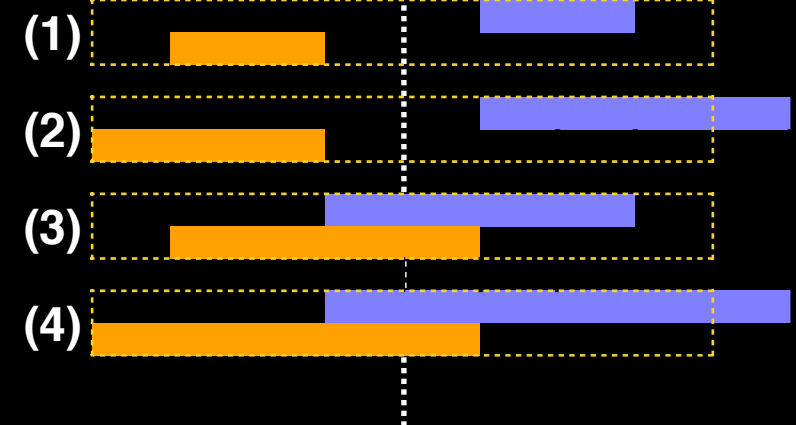
111111

2334666778 899

降順

一定

昇順



パターンのまとめ

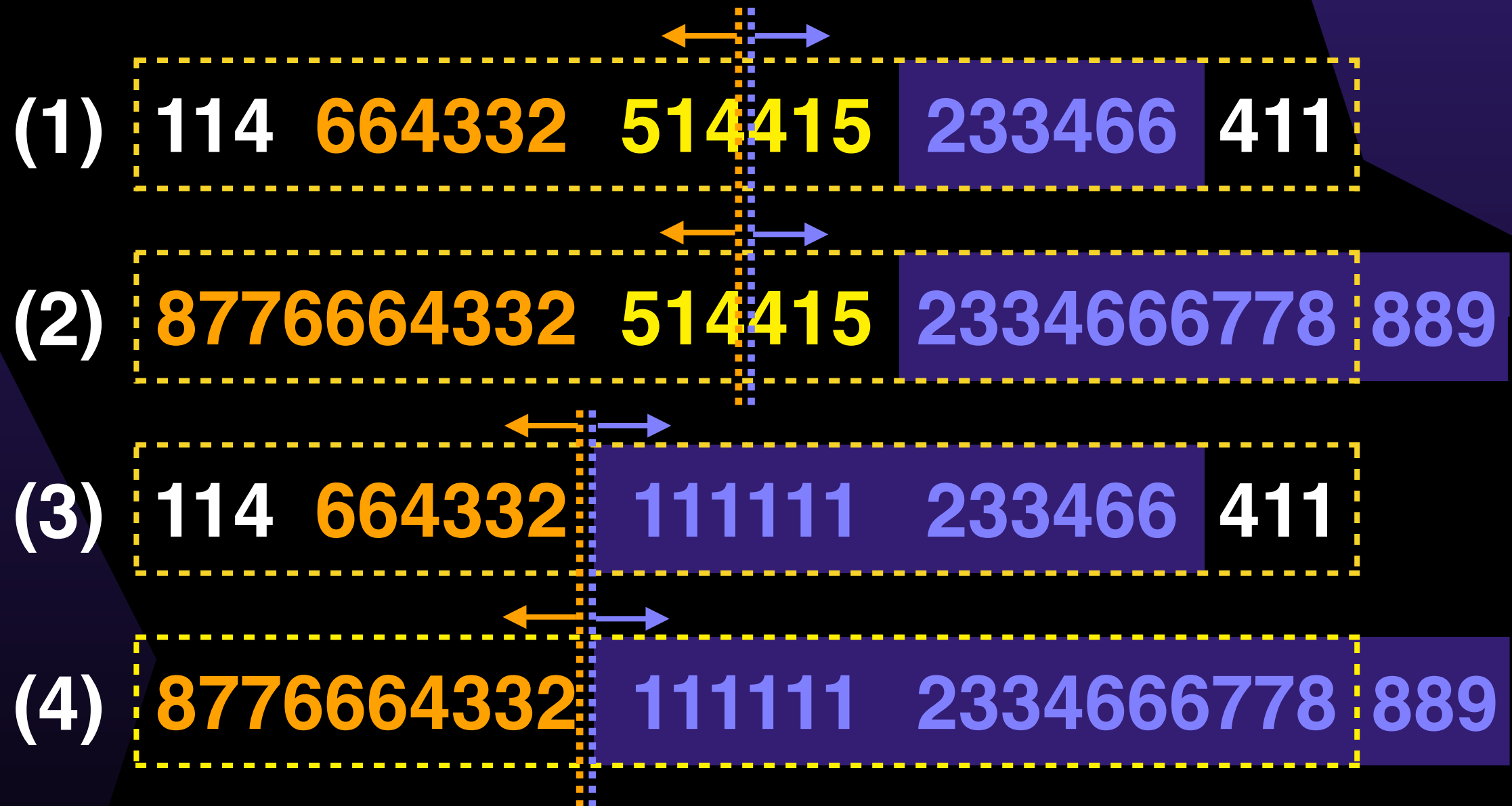
(1) 114 664332 514415 233466 411

(2) 8776664332 514415 2334666778 889

(3) 114 664332 111111 233466 411

(4) 8776664332 111111 2334666778 889

開始位置をO(N)通り試す

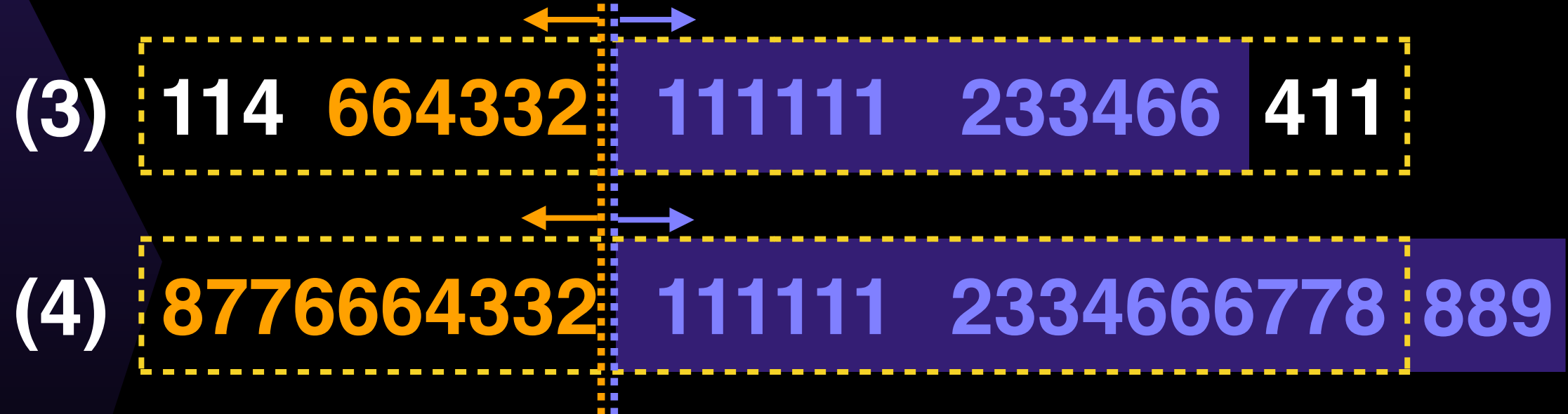




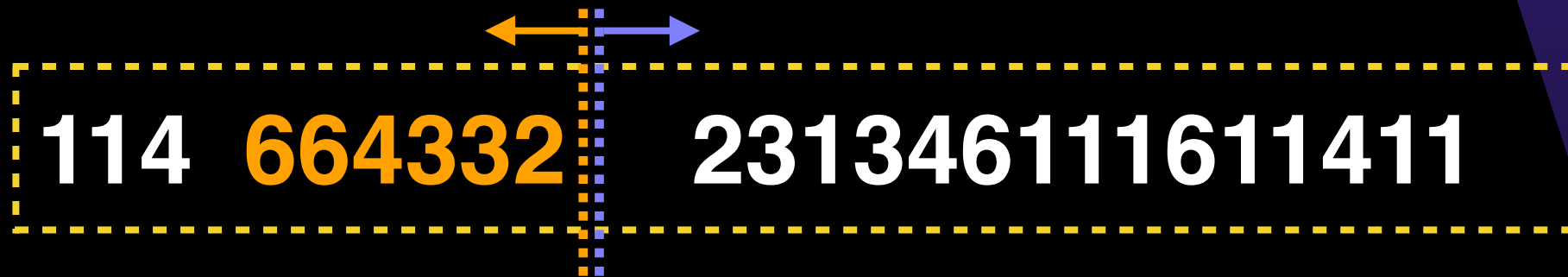
(1)(2)はまず
できるだけ長い回文をとる



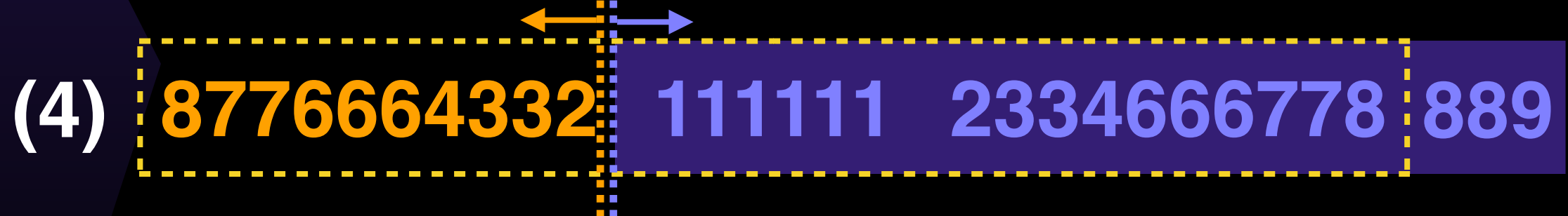
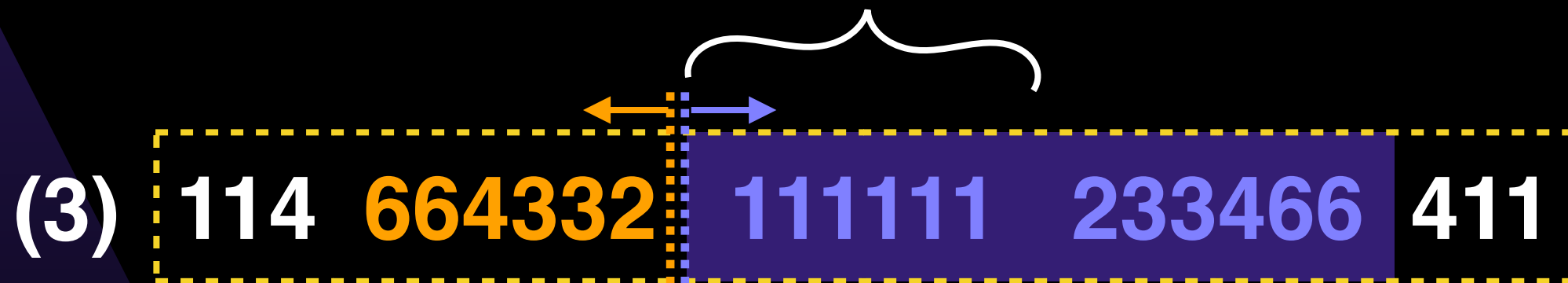
(3)(4)は？



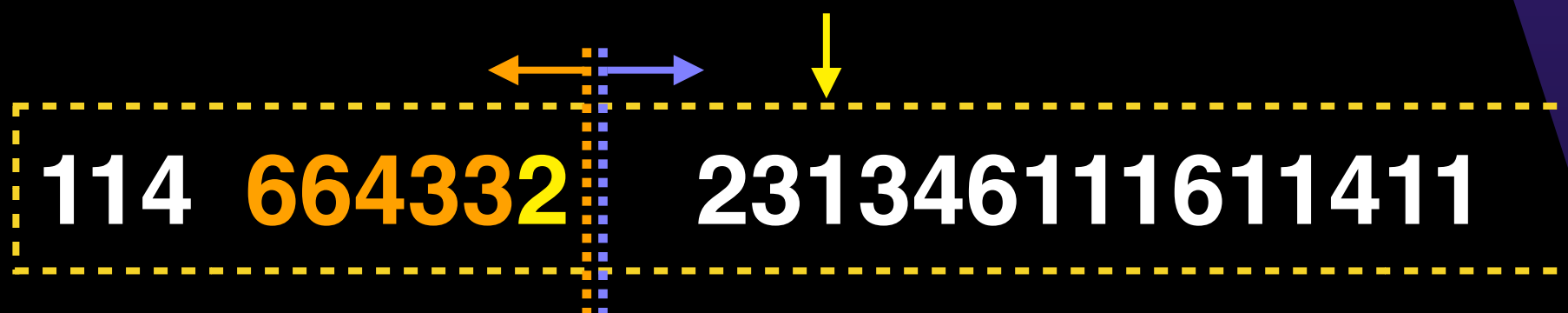
(3)(4)は？
中心はどんな数？



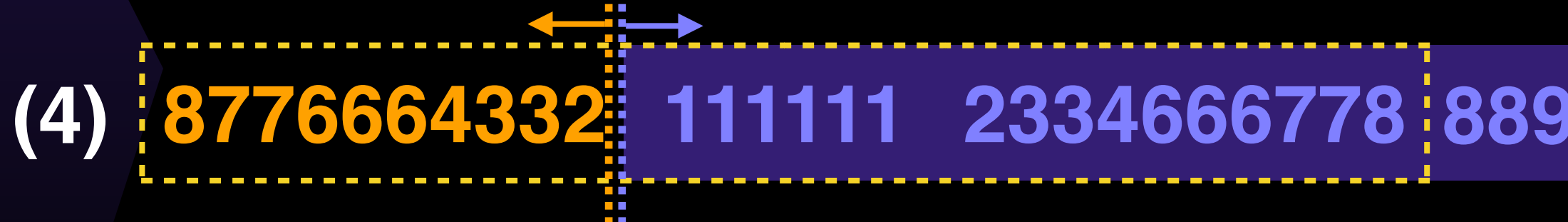
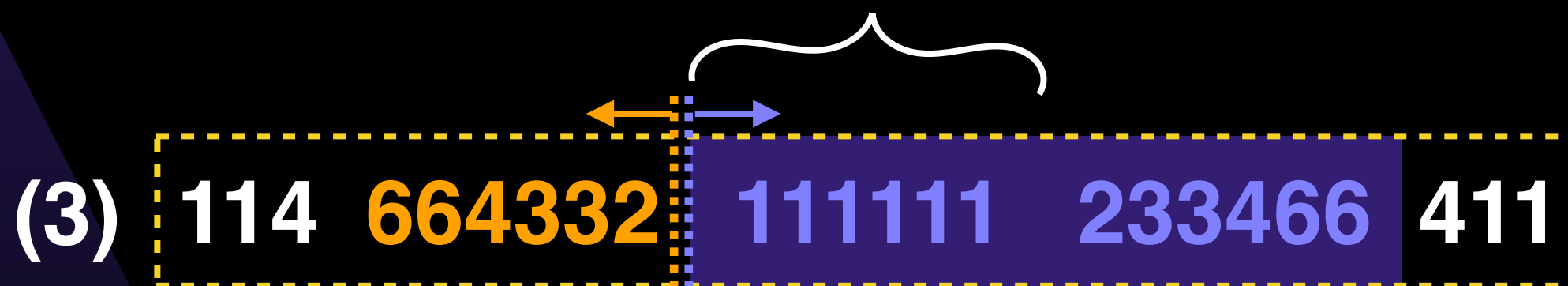
どんな数？



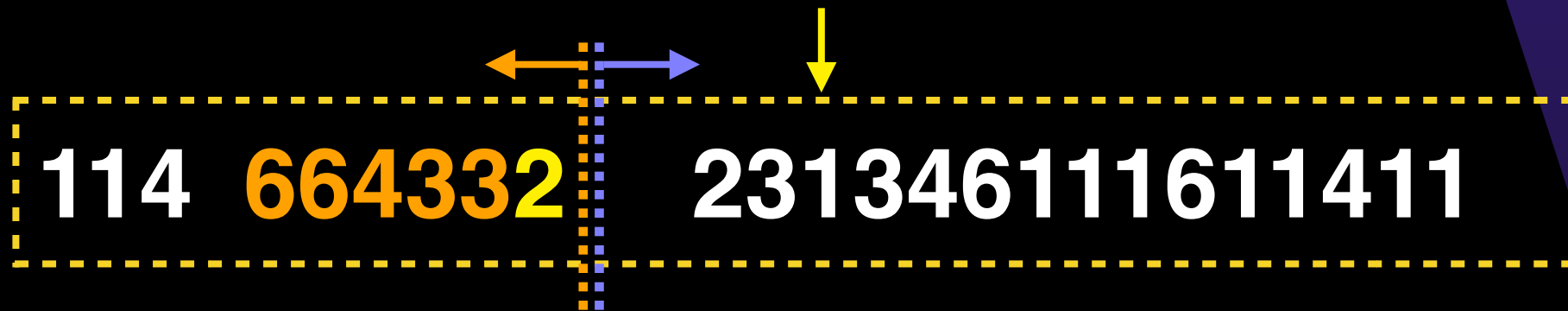
(3)(4)は？
中心はどんな数？



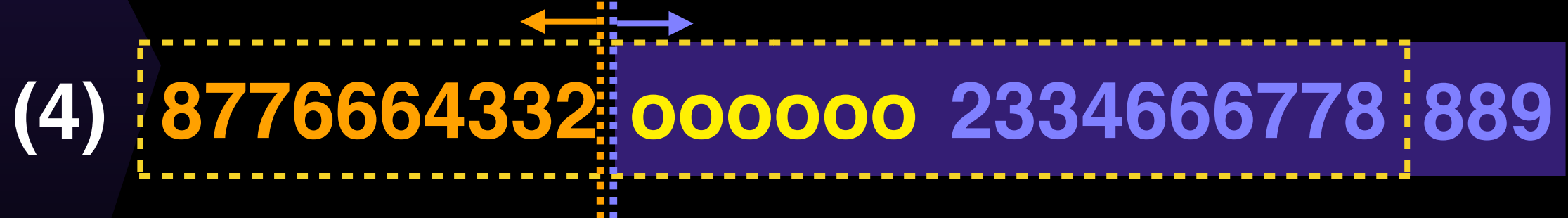
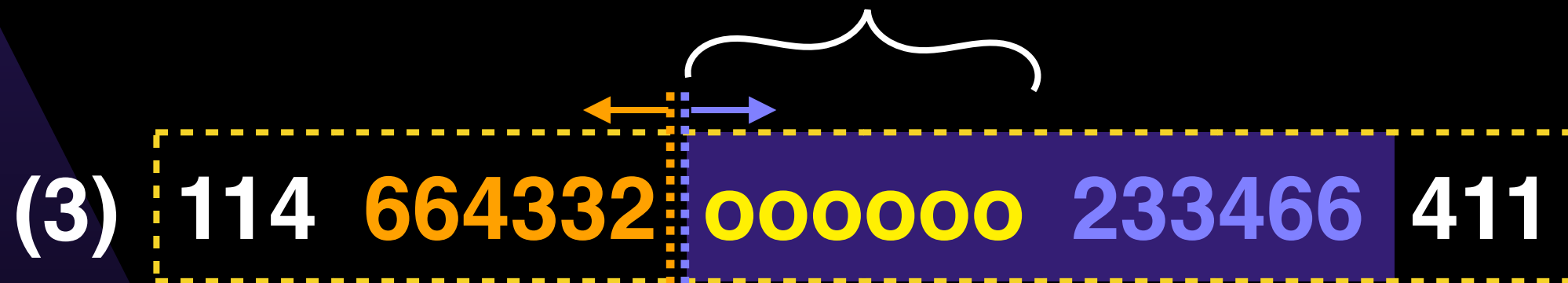
から右に見ていって初めに見つかる, の1つ左より小さい数



(3)(4)は？
中心をoとおく



から右に見ていって初めに見つかる, の1つ左より小さい数



現在の状況



Solve()をつくるう

- Solve(**Lr**, **o**, **RI**)
- 中心を $O(N)$ 個試す ←(1)(2)
 - できるだけ長い回文を見つけ, **Lr=左端**, **RI=右端**
 - **o = -1 (なし)**
- **Lr,RI**を $O(N)$ 通り試す ←(3)(4)
 - **o = RI** から右に見ていって初めに見つかる, **S[Lr]** より小さい数
- Solve() を $O(N) \sim O(N \log N)$ ぐらいで





Solve(): パターン(2)(4)

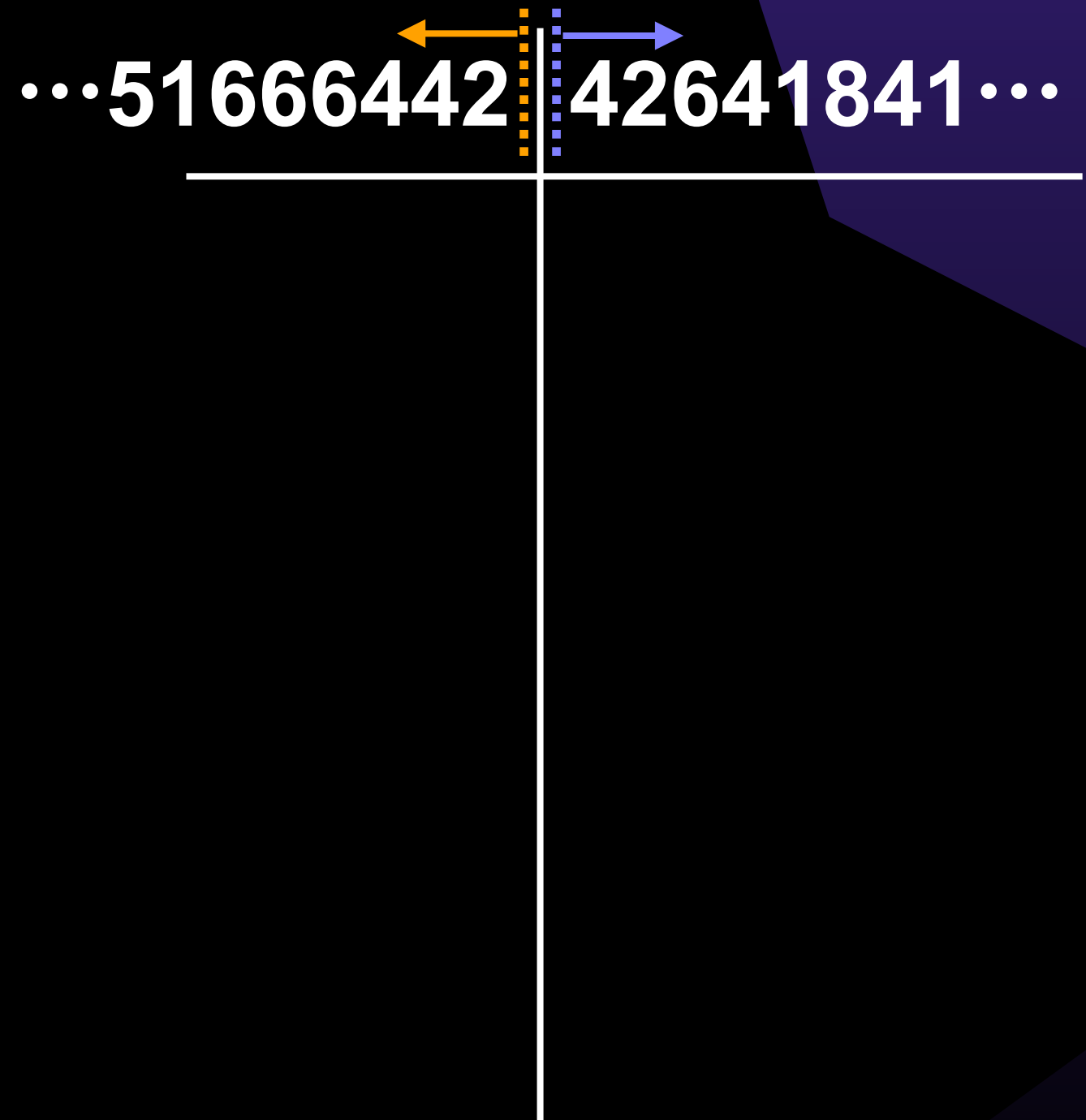
(ソート列が回文をはみ出る)

(1)	114	664332	514415	233466	411
(2)	8776664332	514415	2334666778	889	
(3)	114	664332	000000	233466	411
(4)	8776664332	000000	2334666778	889	



左側の文字を数える

○ = 1

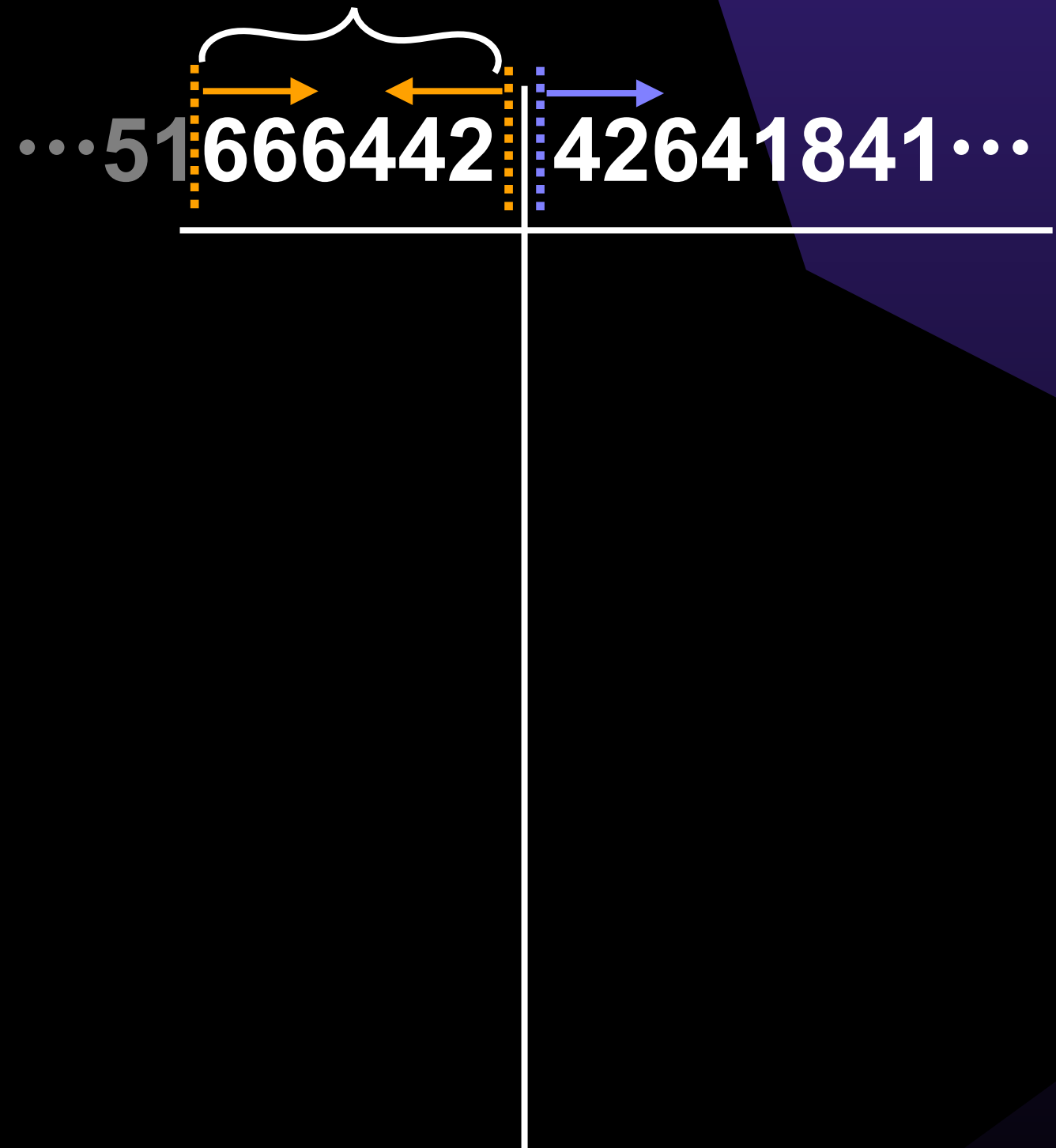




左側の文字を数える

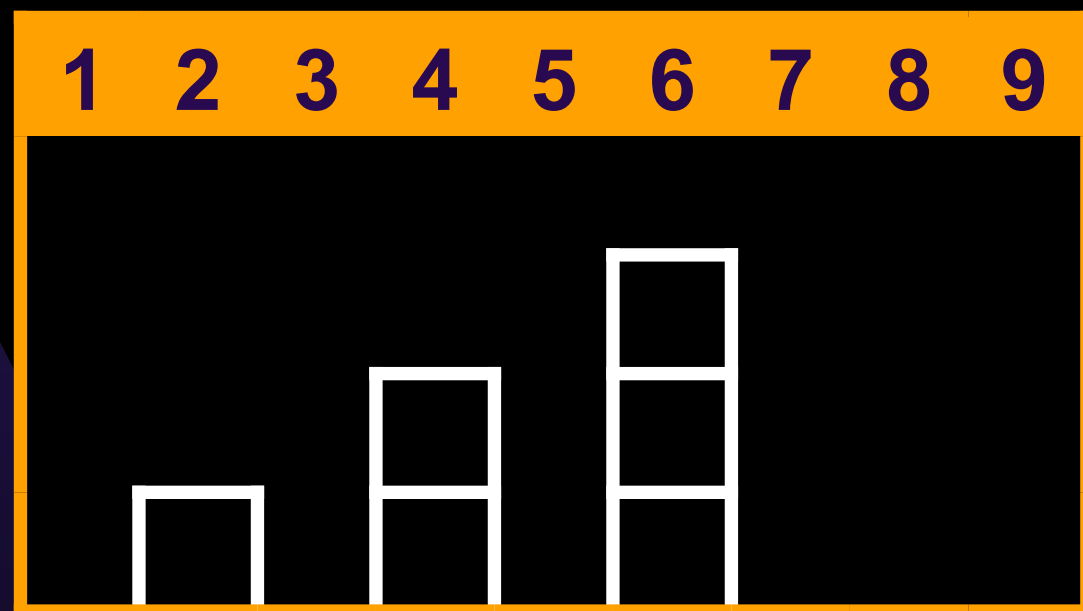
0 = 1

降順の部分数を数える



左側の文字を数える

○ = 1



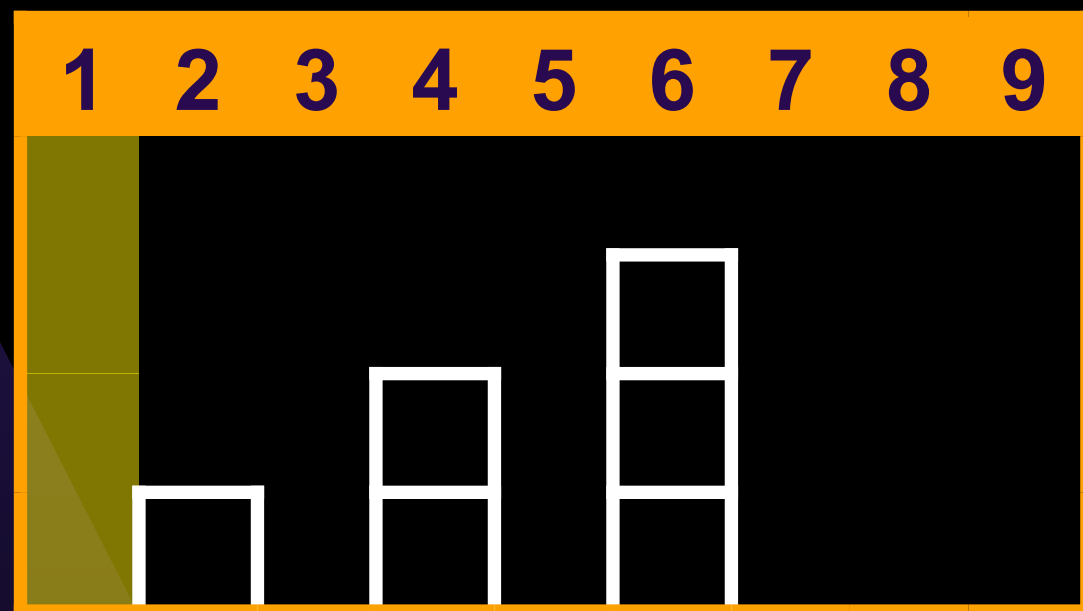
降順の部分の数える

...51666442 | 42641841...



左側の文字を数える

○ = 1



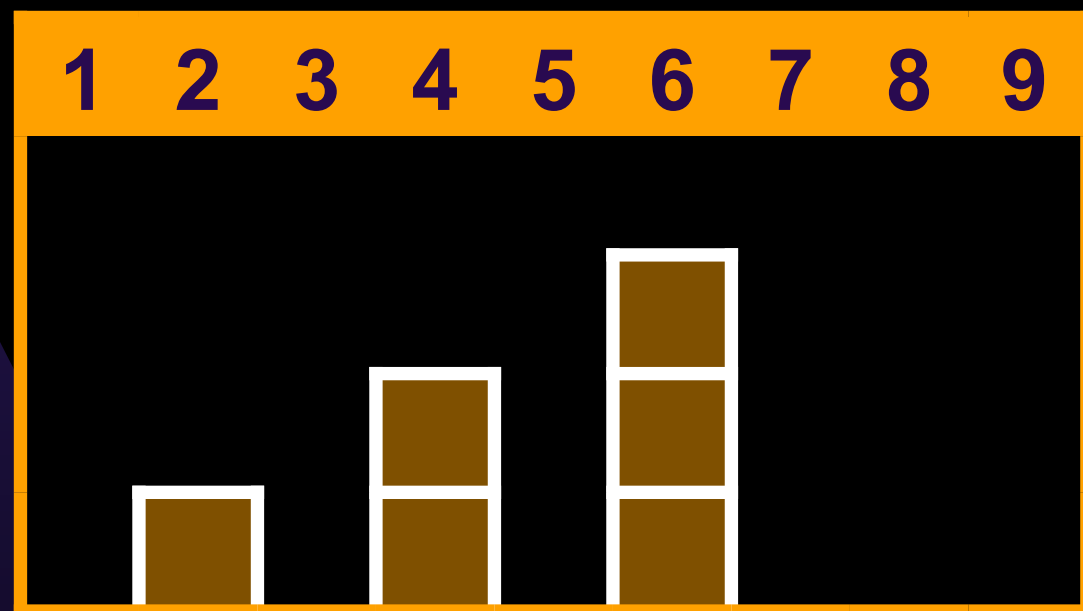
どれだけ入れてもOK

...5166644242641841...



左側の文字を数える

○ = 1

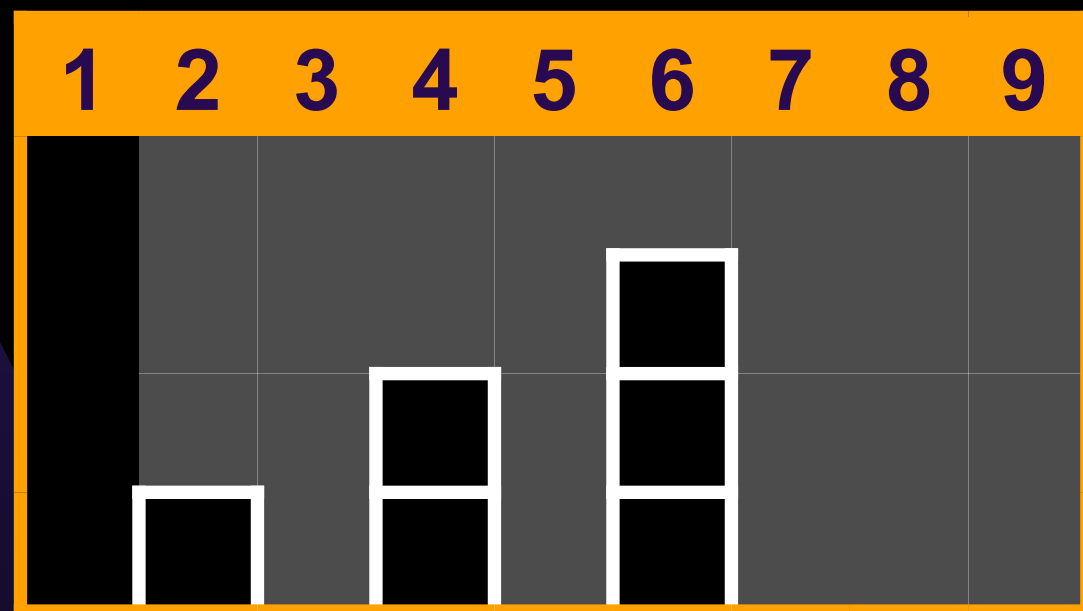


マッチする可能性アリ

...51666442|42641841...

左側の文字を数える

○ = 1



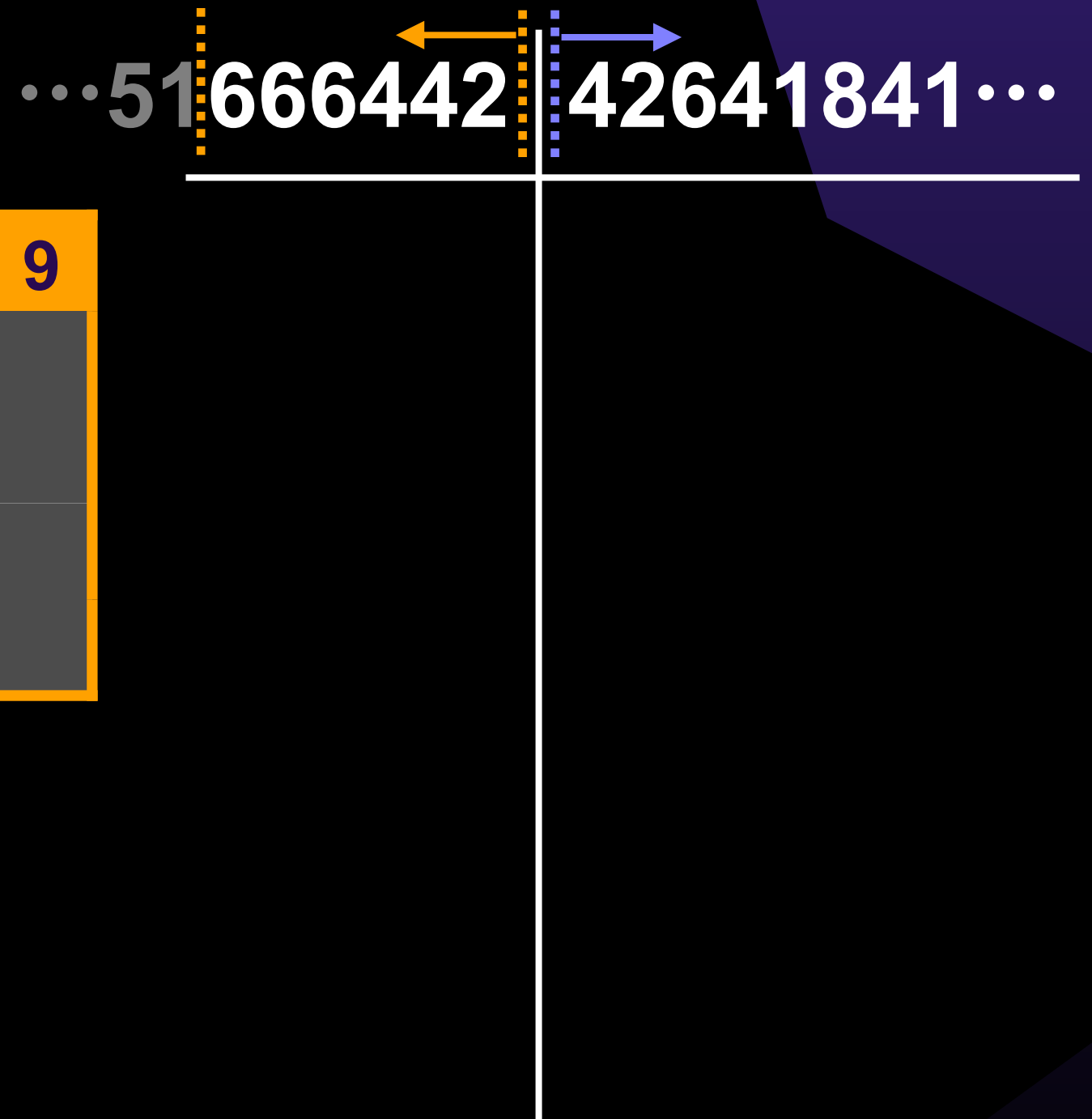
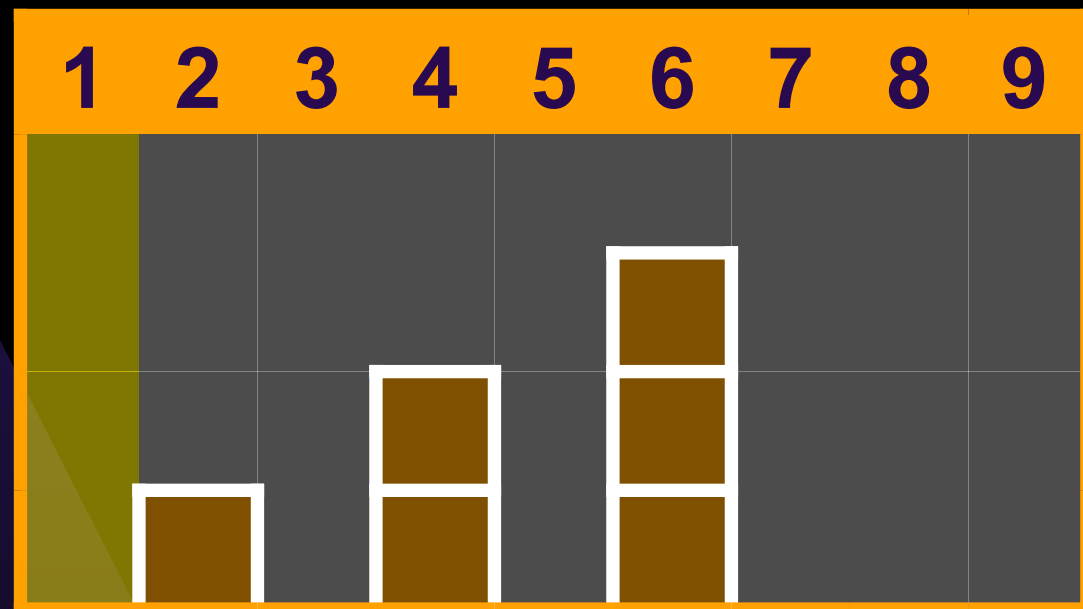
マッチしない

...5166644242641841...



左側の文字を数える

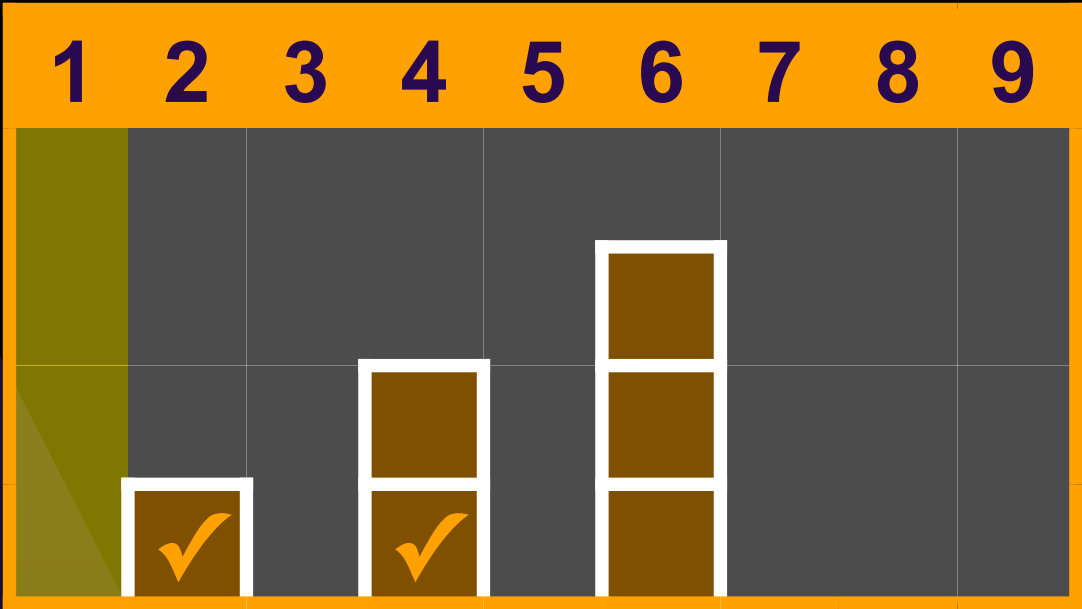
○ = 1





右側を伸ばしていく：
2がマッチして4もマッチする

○ = 1

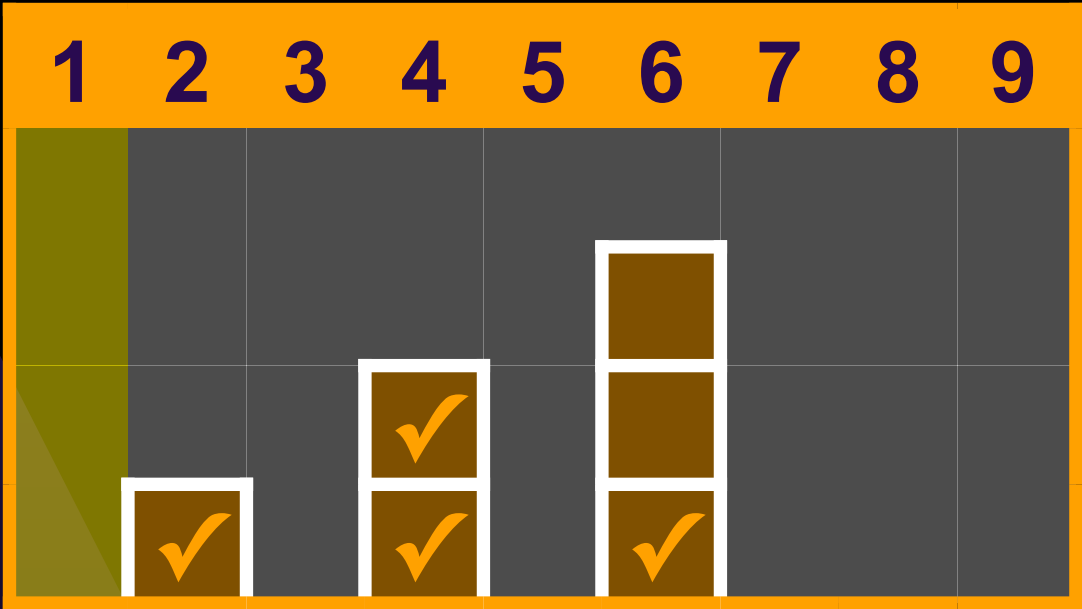


666442	42641841...
666442	4
666442	24



右側を伸ばしていく：
4がマッチして6がマッチする

○ = 1

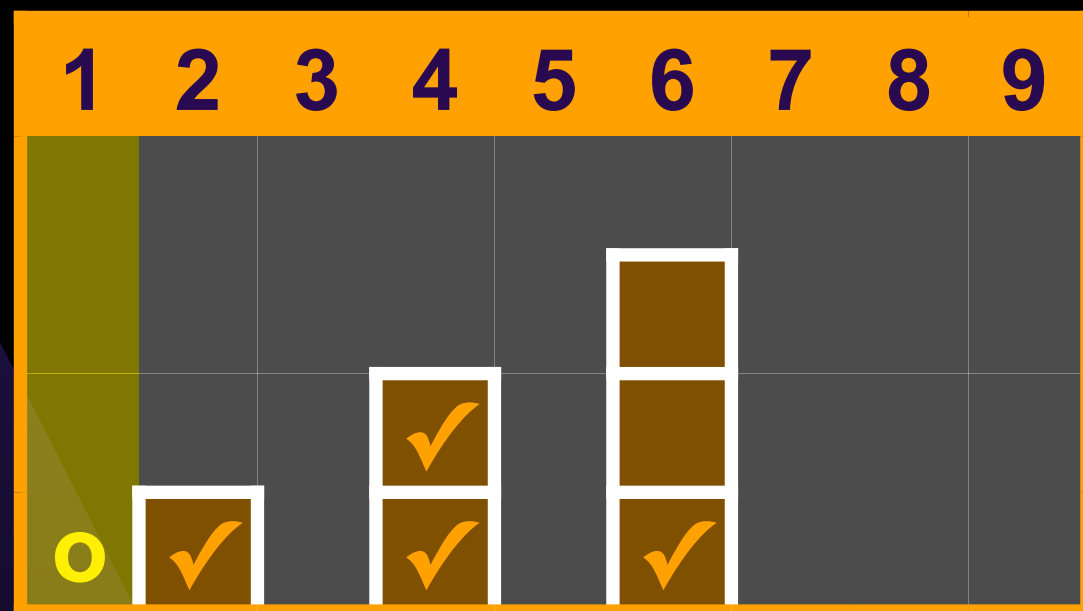


666442	42641841...
666442	4
666442	24
666442	246
666442	2446



右側を伸ばしていく：
1は○なので中心が伸びる

○ = 1

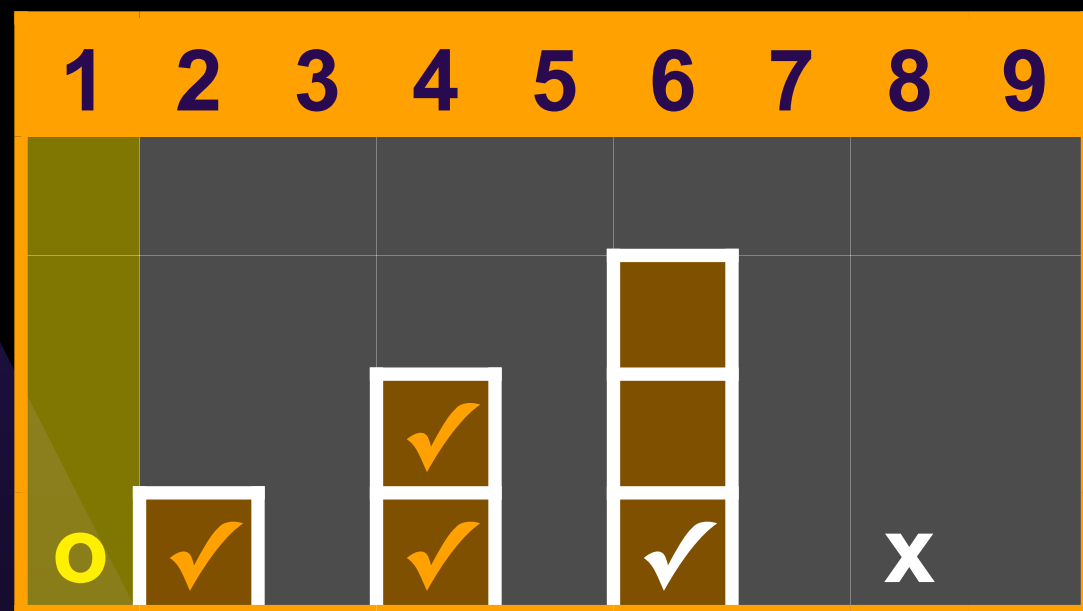


666442	42641841...
666442	4
666442	24
666442	246
666442	2446
666442	12446

右側を伸ばしていく：
8はゴミ



○ = 1

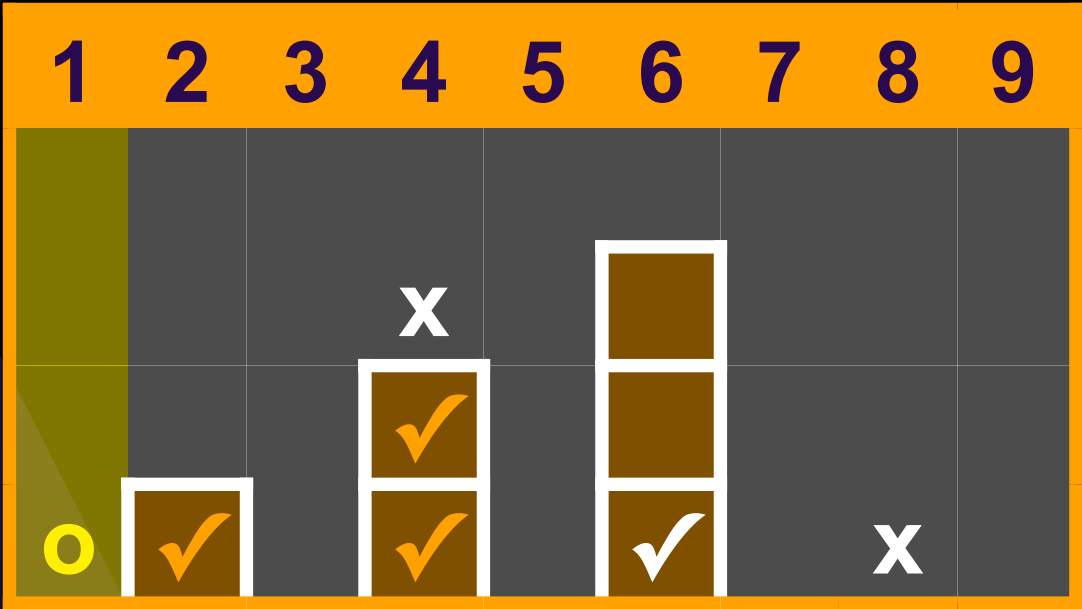


666442	42641841...
666442	4
666442	24
666442	246
666442	2446
666442	12446
666442	124468



右側を伸ばしていく：
4はゴミ，大きい6もゴミに

○ = 1

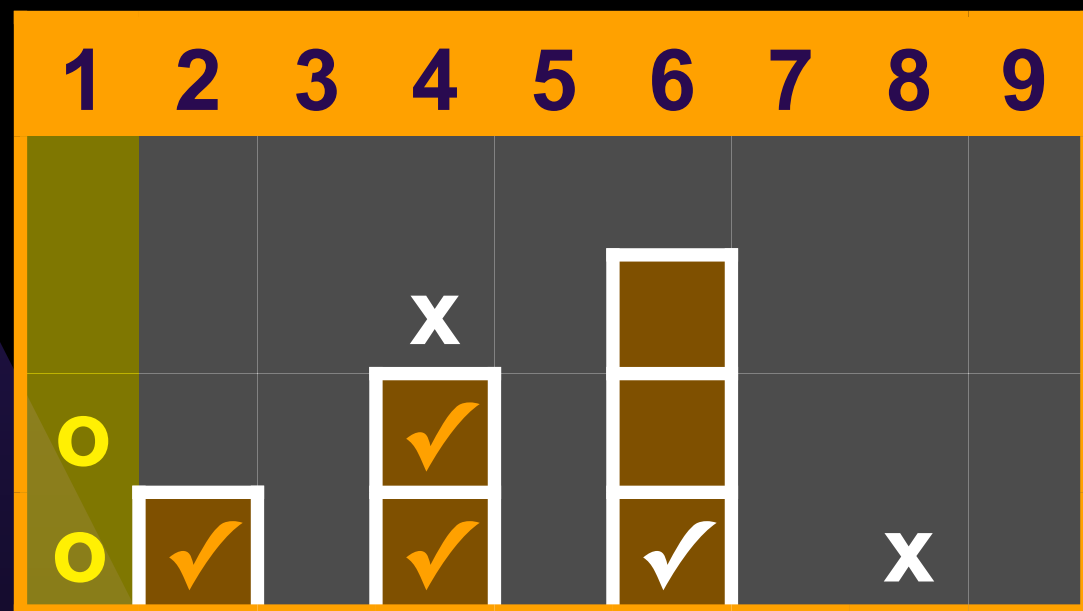


666442	42641841...
666442	4
666442	24
666442	246
666442	2446
666442	12446
666442	124468
666442	1244468



右側を伸ばしていく：
ゴミが入っても○は伸びうる

○ = 1

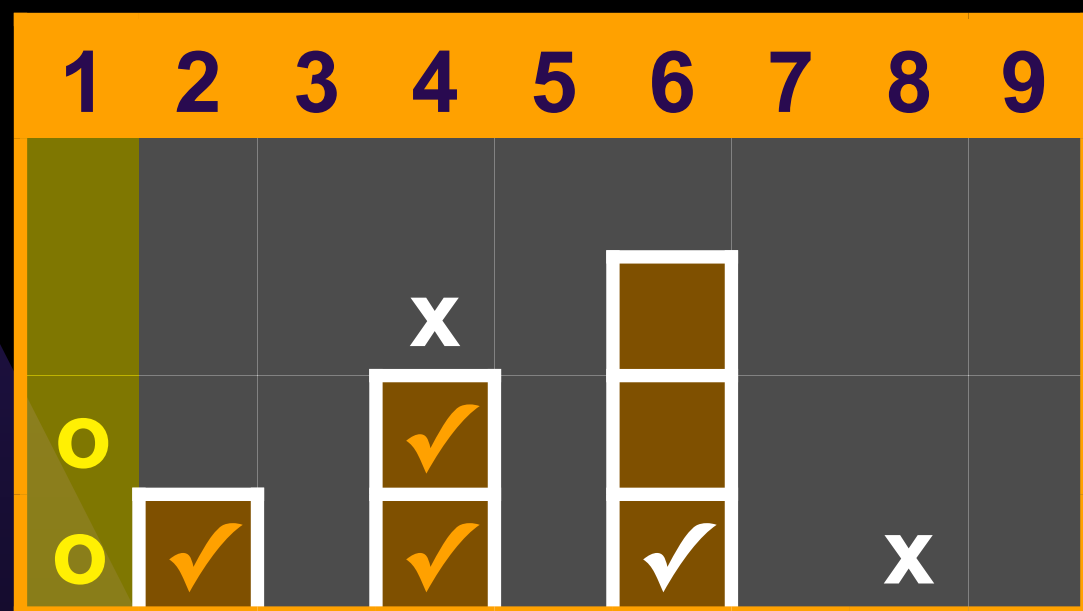


666442	42641841...
666442	4
666442	24
666442	246
666442	2446
666442	12446
666442	124468
666442	1244468
666442	11244468
⋮	⋮

こういうのを カウントする



○ = 1



○ の個数 = 2

✓ の右端までの個数 = 4

x の左端までの個数 = 3

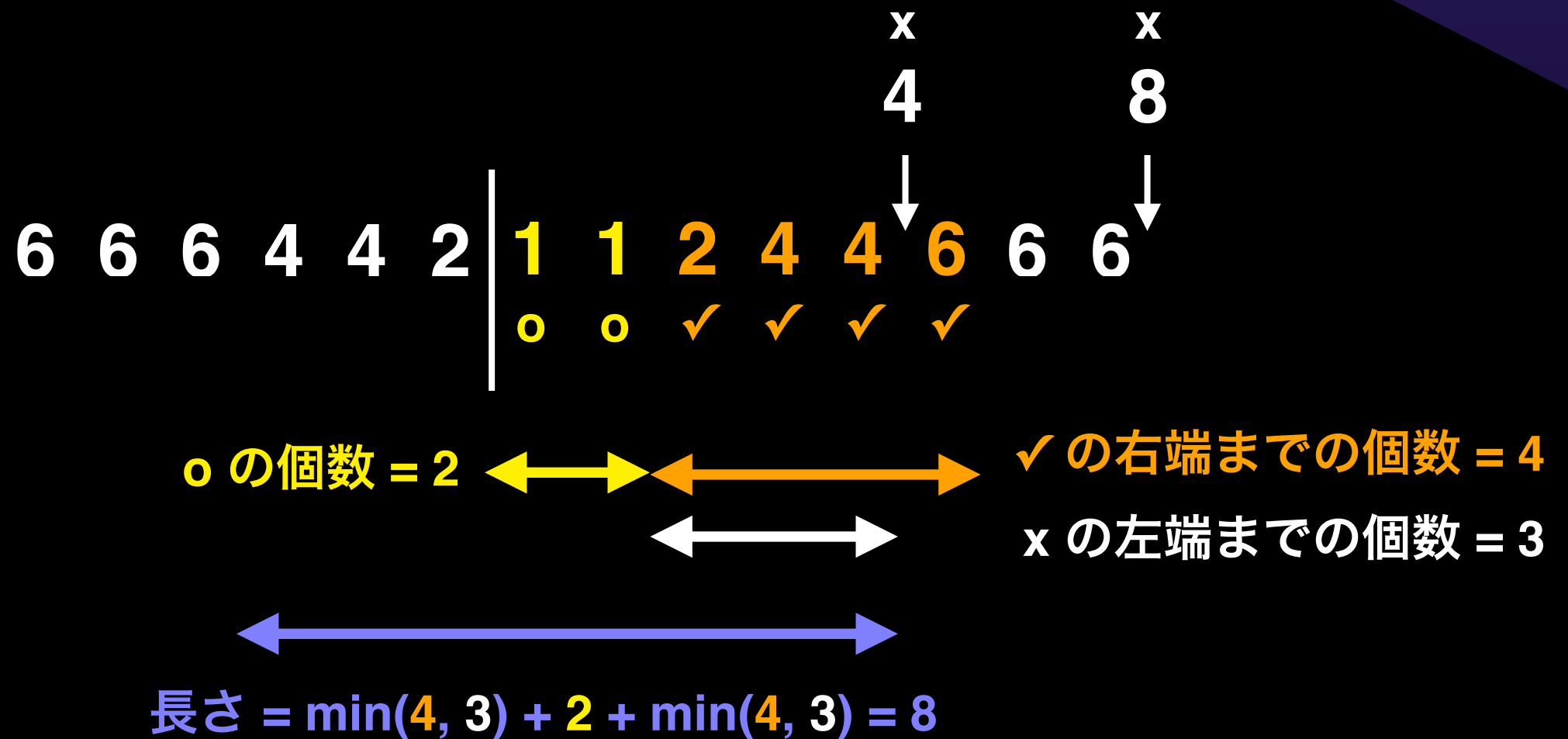
長さ = $\min(4, 3) + 2 + \min(4, 3) = 8$

666442	42641841...
666442	4
666442	24
666442	246
666442	2446
666442	12446
666442	124468
666442	1244468
666442	11244468
⋮	⋮

こういうのを
カウントする



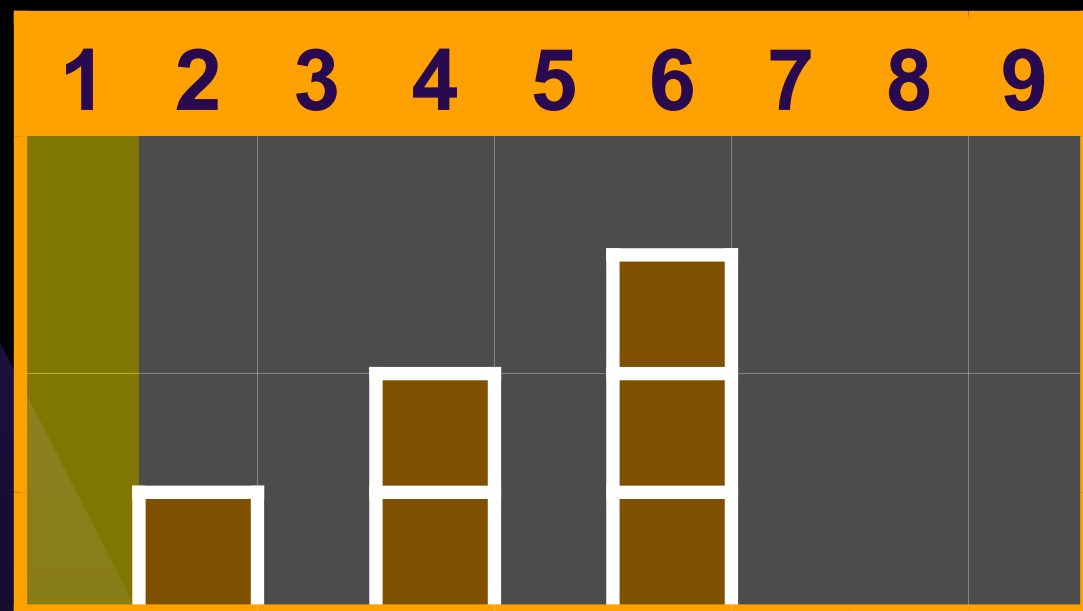
66644211244468



右側を伸ばしていく：
左側の文字を数える



o = 1

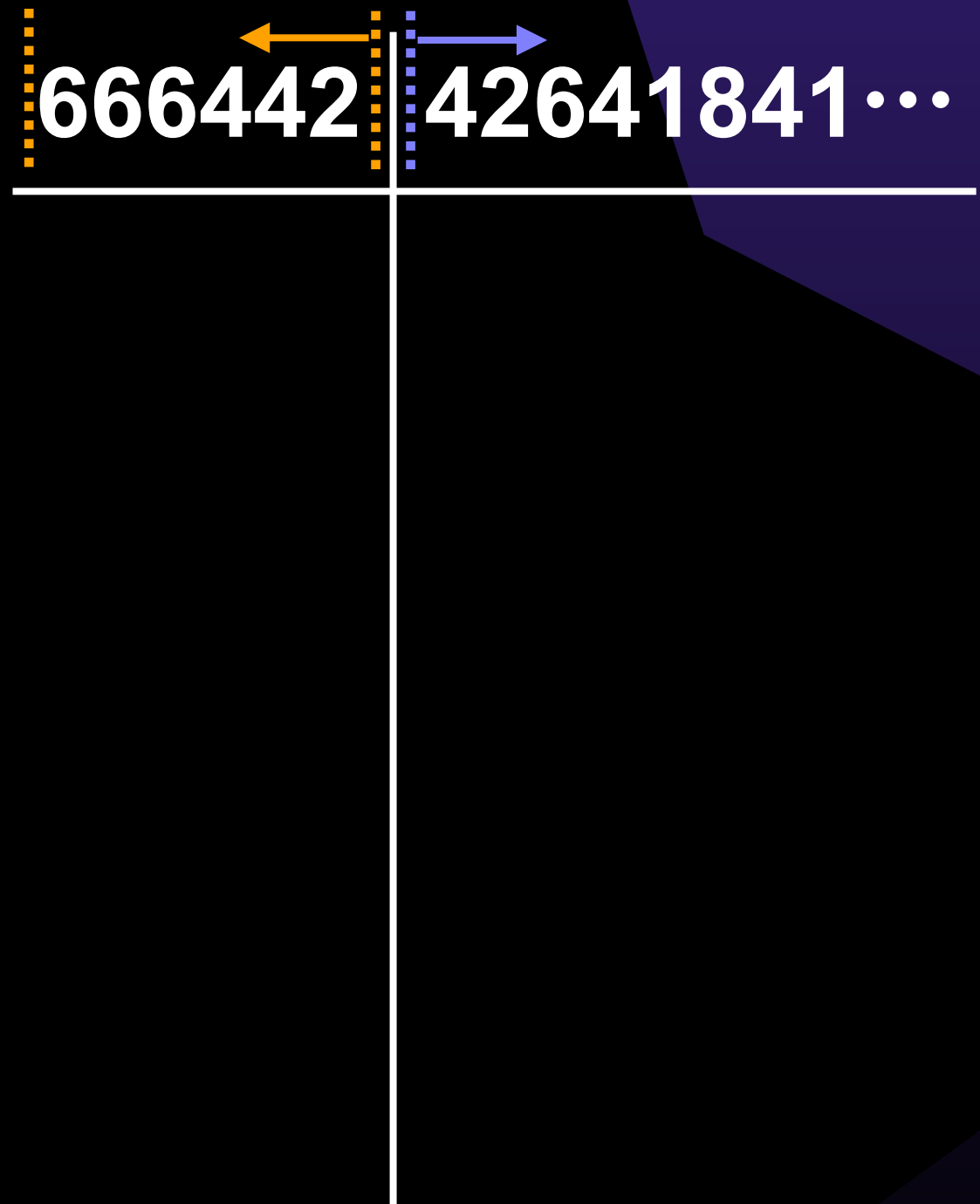


o の個数 = 0

✓ の右端までの個数 = 0

x の左端までの個数 = 6

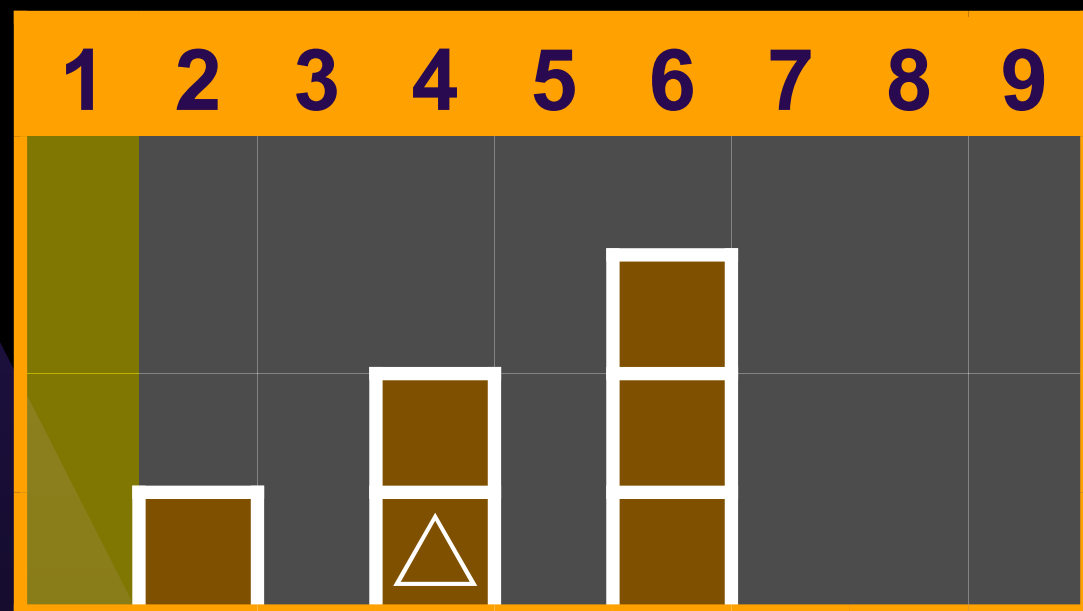
長さ = $\min(0, 6) + 0 + \min(0, 6) = 0$



右側を伸ばしていく：
4はマッチしない



o = 1

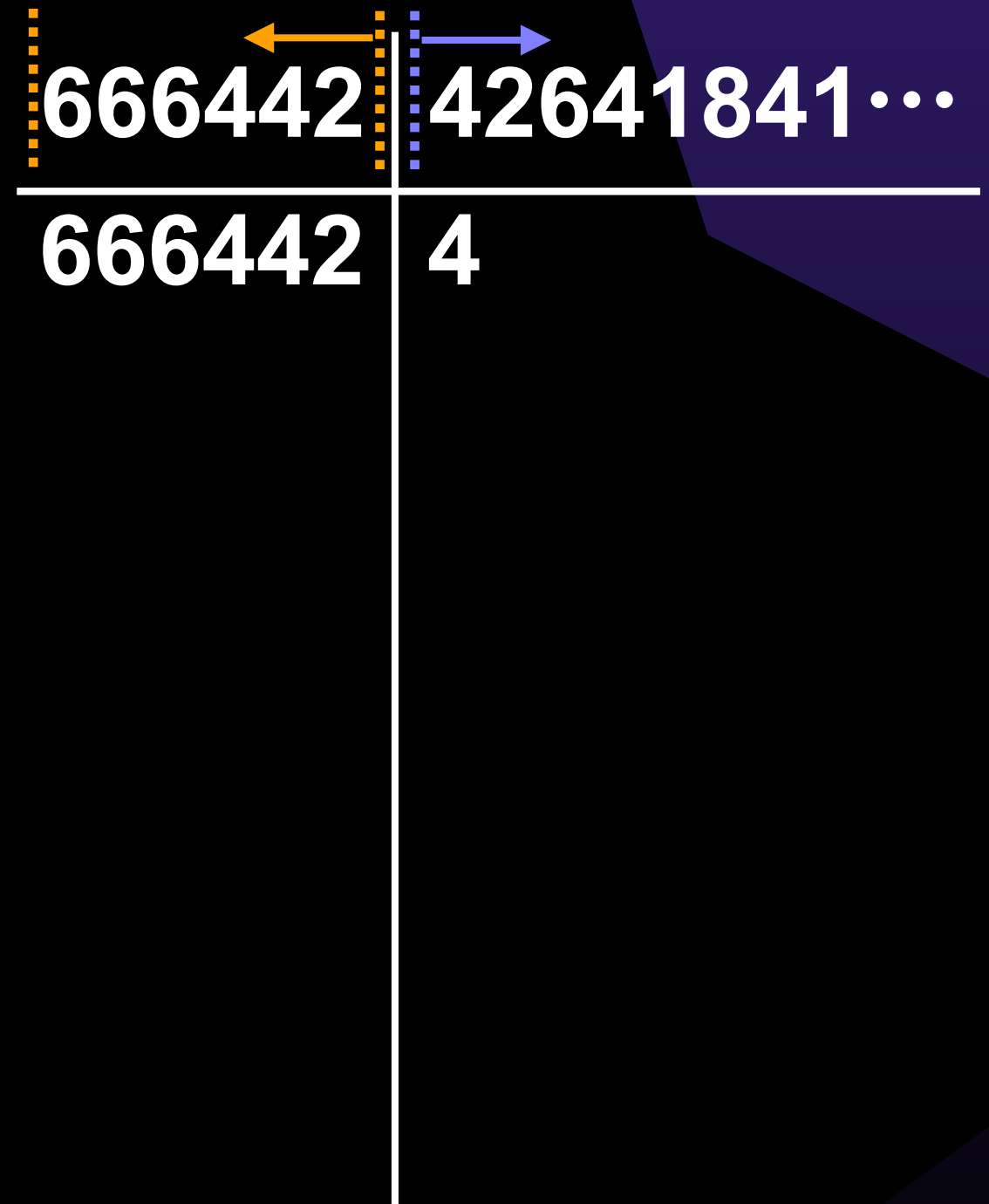


o の個数 = 0

✓ の右端までの個数 = 0

x の左端までの個数 = 6

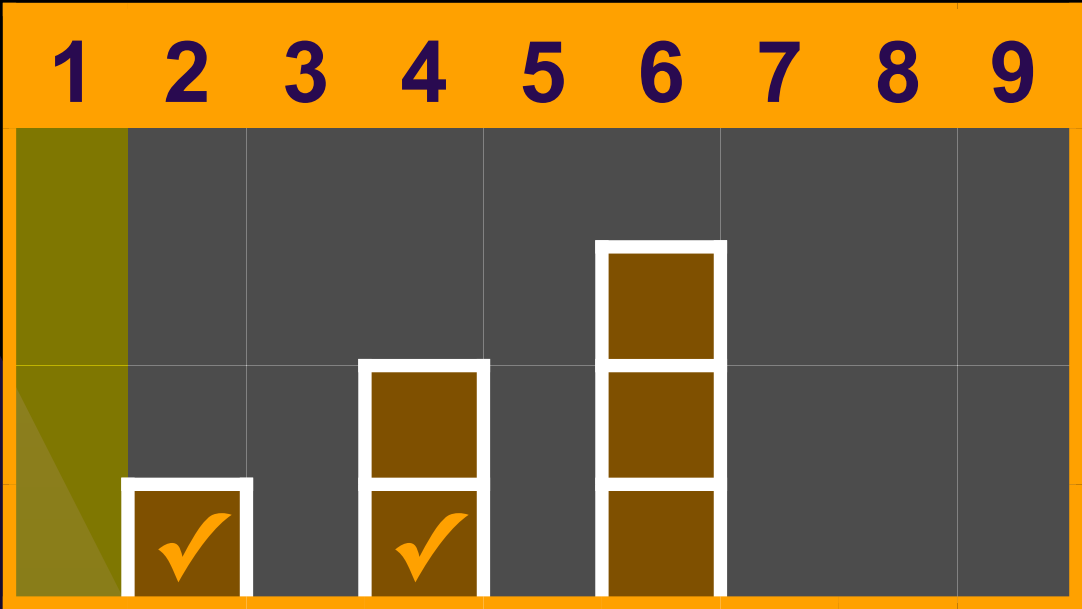
長さ = $\min(0, 6) + 0 + \min(0, 6) = 0$





右側を伸ばしていく：
2がマッチして4もマッチする

o = 1



o の個数 = 0

✓ の右端までの個数 = 2

x の左端までの個数 = 6

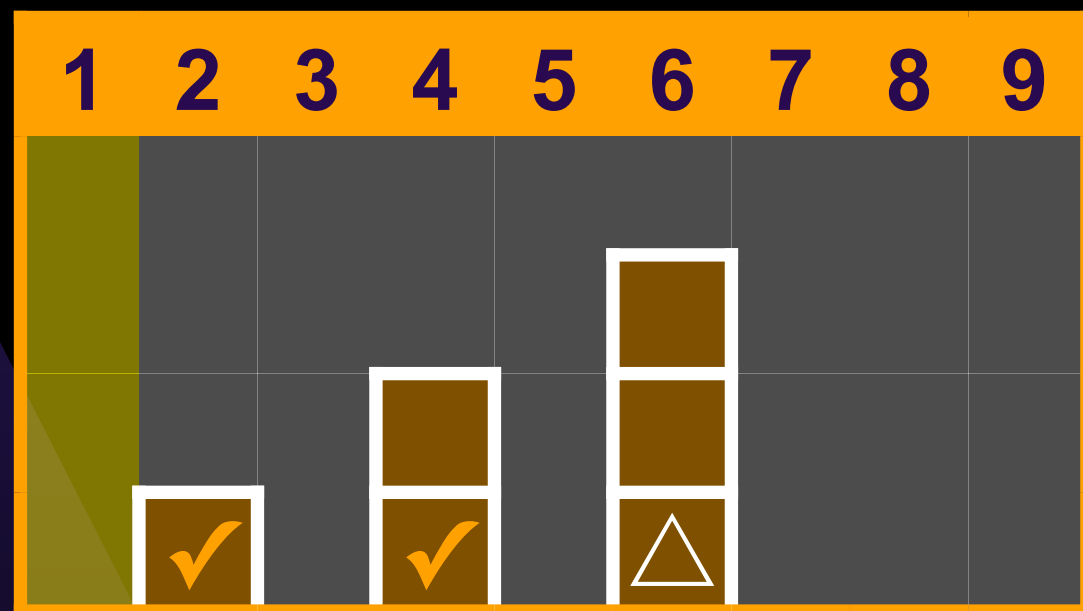
長さ = min(2, 6) + 0 + min(2, 6) = 4

666442	42641841...
666442	4
666442	24



右側を伸ばしていく：
6はマッチしない

o = 1



o の個数 = 0

✓ の右端までの個数 = 2

x の左端までの個数 = 6

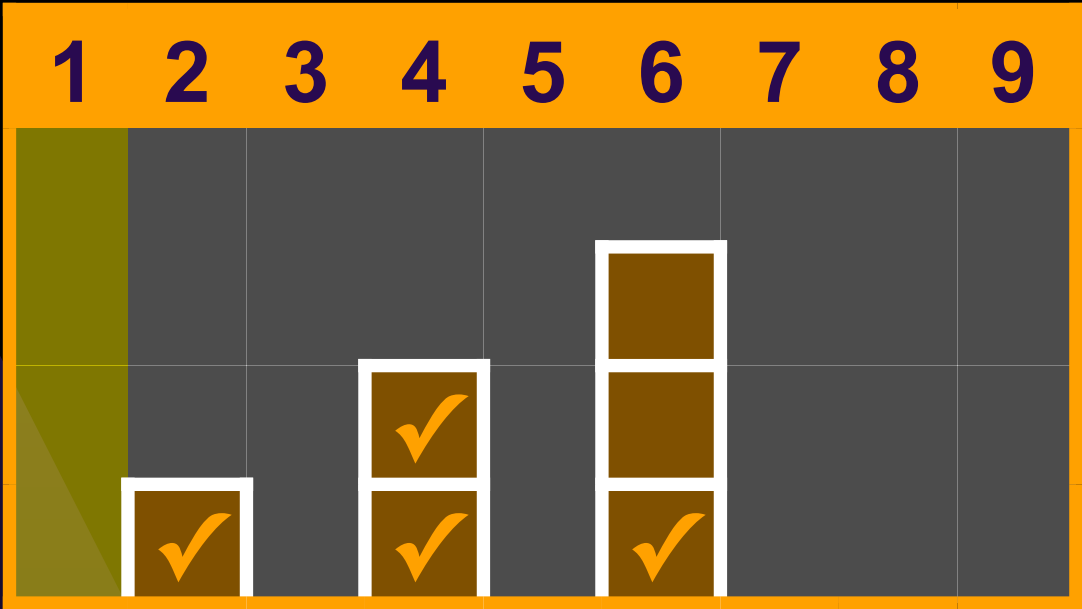
長さ = $\min(2, 6) + 0 + \min(2, 6) = 4$

666442	42641841...
666442	4
666442	24
666442	246



右側を伸ばしていく：
4がマッチして6がマッチする

o = 1



o の個数 = 0

✓ の右端までの個数 = 4

x の左端までの個数 = 6

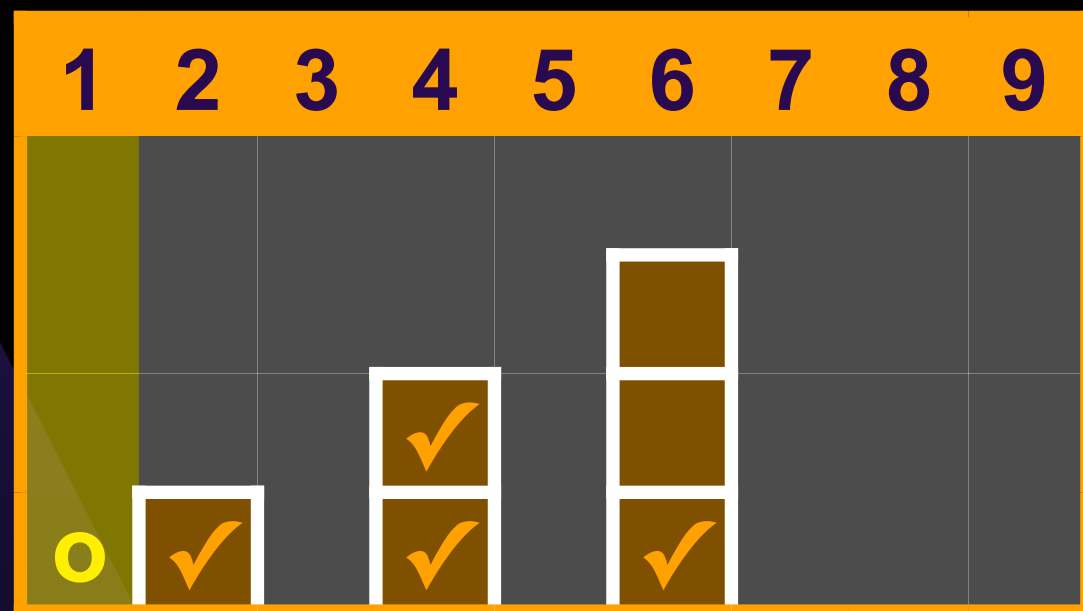
長さ = min(4, 6) + 0 + min(4, 6) = 8

666442	42641841...
666442	4
666442	24
666442	246
666442	2446



右側を伸ばしていく：
1は○なので中心が伸びる

○ = 1



○ の個数 = 1

✓ の右端までの個数 = 4

x の左端までの個数 = 6

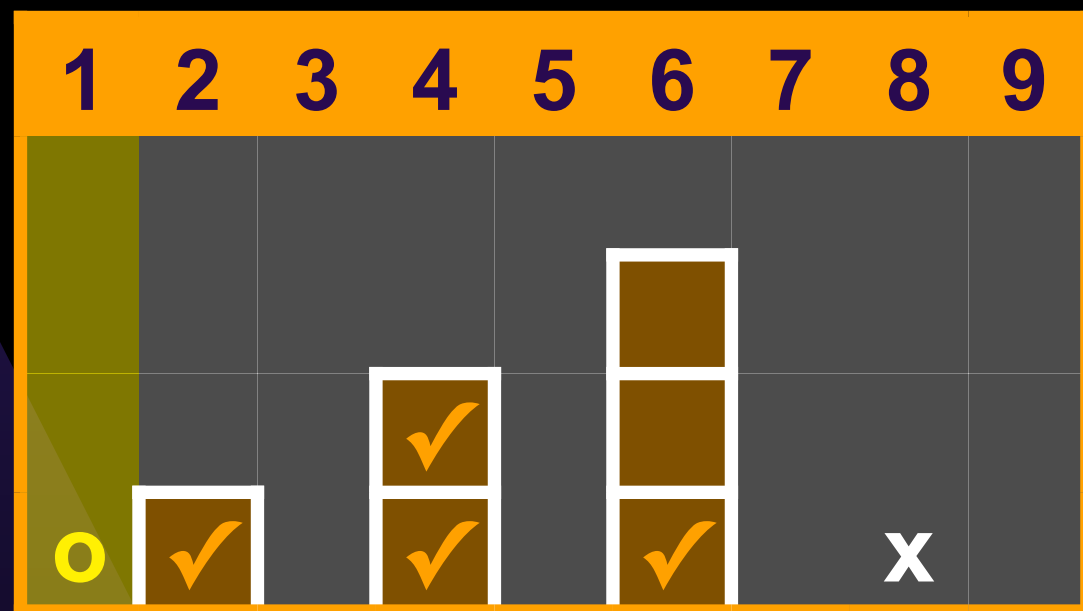
長さ = $\min(4, 6) + 1 + \min(4, 6) = 9$

666442	42641841...
666442	4
666442	24
666442	246
666442	2446
666442	12446

右側を伸ばしていく：
8はゴミ



○ = 1



○ の個数 = 1

✓ の右端までの個数 = 4

x の左端までの個数 = 6

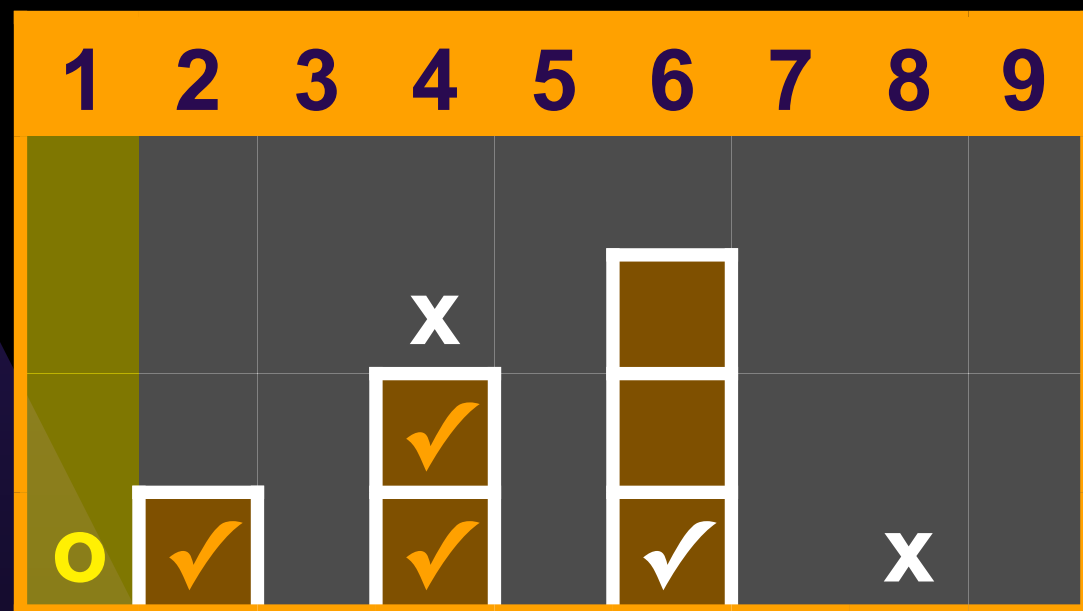
長さ = $\min(4, 6) + 1 + \min(4, 6) = 9$

666442	42641841...
666442	4
666442	24
666442	246
666442	2446
666442	12446
666442	124468



右側を伸ばしていく：
4はゴミ，大きい6もゴミに

○ = 1



○ の個数 = 1

✓ の右端までの個数 = 4

x の左端までの個数 = 3

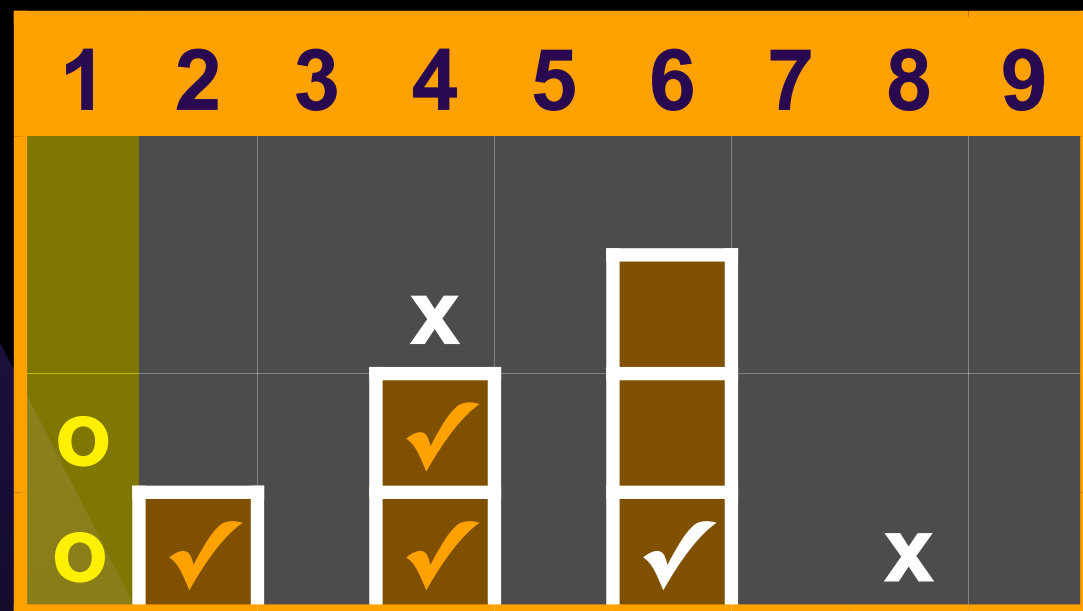
長さ = $\min(4, 3) + 1 + \min(4, 3) = 7$

666442	42641841...
666442	4
666442	24
666442	246
666442	2446
666442	12446
666442	124468
666442	1244468



右側を伸ばしていく：
ゴミが入っても○は伸びうる

○ = 1



○ の個数 = 2

✓ の右端までの個数 = 4

x の左端までの個数 = 3

長さ = $\min(4, 3) + 2 + \min(4, 3) = 8$

666442	42641841...
666442	4
666442	24
666442	246
666442	2446
666442	12446
666442	124468
666442	1244468
666442	11244468
⋮	⋮



Solve(): パターン(1)(3)

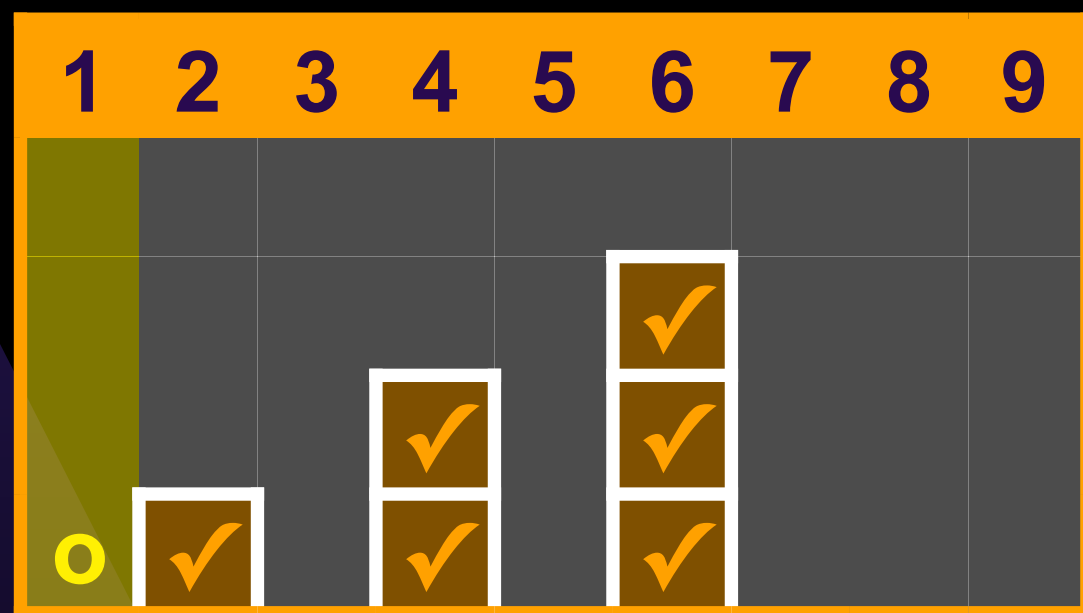
(ソート列が回文をはみ出ない)

(1)	114	664332	514415	233466	411
(2)	8776664332	514415	2334666778	889	
(3)	114	664332	000000	233466	411
(4)	8776664332	000000	2334666778	889	

はみ出るパターンと
ほぼ同じだが、



o = 1



o の個数 = 1

✓ の右端までの個数 = 6

x の左端までの個数 = 6

長さ = $\min(6, 6) + 1 + \min(6, 6) = 13$

411 666442 4264166 114
666442 1244666

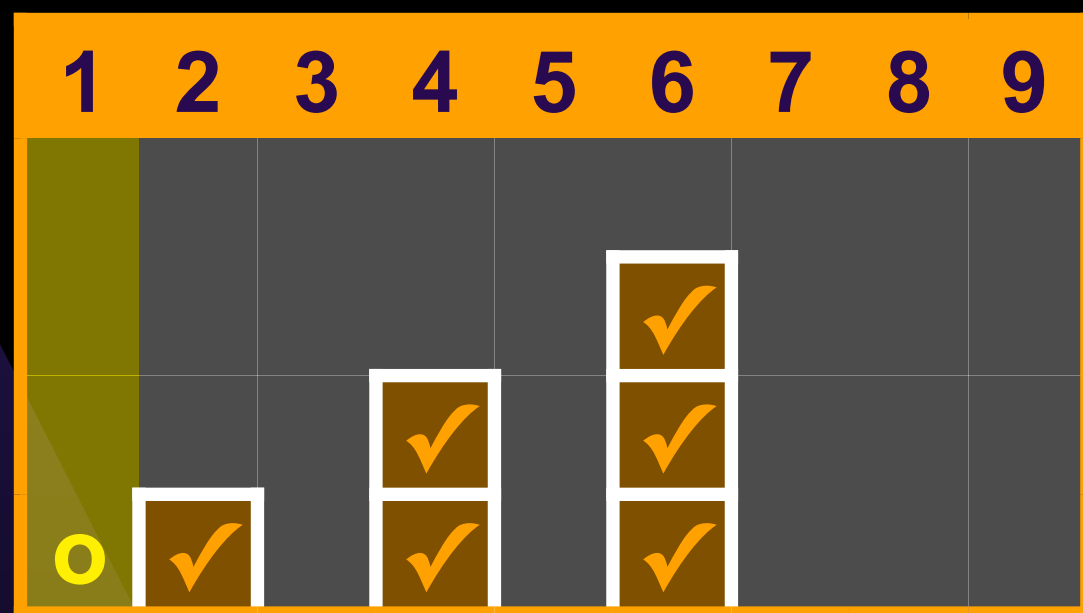
x がないとき, 伸ばせる

114 666442 1 244666 411

はみ出るパターンと
ほぼ同じだが、



o = 1



o の個数 = 1

✓ の右端までの個数 = 6

x の左端までの個数 = 6

長さ = $\min(6, 6) + 1 + \min(6, 6) = 13$

411666442 | 4264166114
666442 | 1244666

x がないとき、伸ばせる

114 666442 1 244666 411

どれくらい伸ばせるかは、
DP[←の位置][→の位置]
として $O(N^2)$ で前計算できる

やったぜ



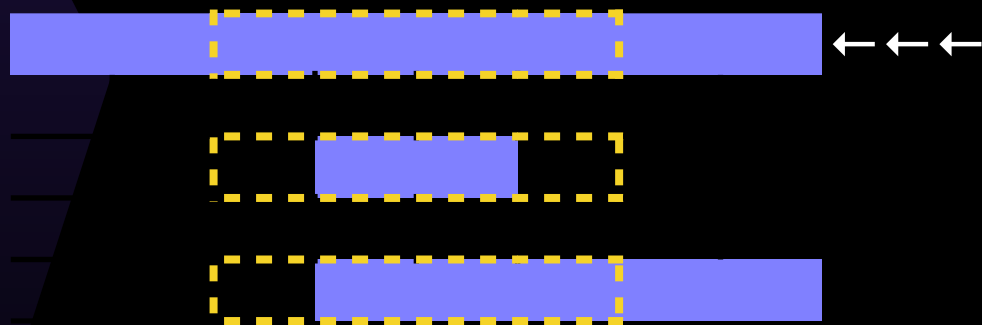
これで本当に
絶対ACするよ！

もしACしなかったら
木の下に埋めて貰っても
構わないよ



全部ソートに注意

ババーン



2124123222
→× 2121243222
→○ 1122222234

解法の概略・まとめ

- 文字列を「反転」させて2回解く
- $DP[a][b]$ = (aまで, と, bから, で一致する文字列の最長)
を $O(N^2)$ で前計算
- 回文の途中までをソートするパターン4つ
 - 初期位置を $O(N)$ 通り
 - 左側はできるだけ長い降順な列をとり,
 - 右側を順に伸ばしていく
 - どちらも $O(N)$, 合計で $O(N^2)$
- 全部ソートするパターンを忘れない

得点分布



Q Z Q Z Q Z Q Z Q Z Q Z Q Z Q Z Q Z Q

得点分布



Q Q Q Q Q Q Q Q Q Q Z Z Z Z Z Z Z Z Z Z

A = 100

Q = 10

Z = 0

とてもたのしいカードゲーム

(Card Game Is Great Fun)

解説

佐藤 竜馬 (joisino)

問題概要

- N枚のカードが1列に並んでいる
- カードには色、数字、価値が決められている
- カード列の1番目か3番目のカードを山札の一番上に出す
- 山札の一番上のカードの色または数字と一致していないと出せない
- カードが出せなくなるまで出したときの山札にあるカードの価値の合計の最大値を出力せよ

小課題1 (10点)

- $N \leq 20$

小課題1 (10点)

- 全探索
- 現在列に並んであるカード全てと山札の一番上のカードの情報持っておいてシミュレーション
- bitで管理など
- $O(N * 2^N)$

小課題2(累計25点)

- $N \leq 50$

小課題2(累計25点)

- カードは前の方から消費される



- 現在のカード列の前から3番目以降のカードはすべて列に残っている

小課題2(累計25点)

- ゲームの状態は
 - 1番目のカードの種類
 - 2番目のカードの種類
 - 3番目のカードの種類
 - 山札の一番上のカードの種類で場合が完全に定まる
- $O(N^4)$
 - 探索 / DPで間に合う

満点解法

- $N \leq 500$

満点解法1

- 3番目のカードの種類は完全に覚える必要はない
 - 1番目のカードの種類(fst)
 - 2番目のカードの種類(snd)
 - 山札の一番上のカードの種類(yama)
 - 3番目のカードの種類は $\max(yama, snd) + 1$

満点解法1

- 場のカードの情報は
 - 1番目のカードの種類
 - 2番目のカードの種類
 - 山札の一番上のカードの種類で完全に定まる
- $O(N^3)$
 - 探索 / DPで間に合う

満点解法1

$dp[yama][fst][snd] :=$ 得られる価値の最大値

$int\ trd = \max(yama, snd) + 1;$

if(fst出せる)

$dp[fst][snd][trd]$

$= \max(dp[fst][snd][trd], dp[yama][fst][snd] + v[fst]);$

if(trd出せる)

$dp[trd][fst][snd]$

$= \max(dp[trd][fst][snd], dp[yama][fst][snd] + v[trd]);$

満点解法1

- `int dp[500][500][500]`
- 500MBくらい
- 足りる...?

Card Game Is Great Fun:

とてもたのしいカードゲーム

バッチ型課題 (Batch task)

3 秒

1280MB

`-DEVAL -static -O2 -lm`

`-DEVAL -static -O2 -std=c++11`

100 点

完全

満点解法1

- 足りる
- 満点

小課題2(累計25点)

- ゲームの状態は
 - 1番目のカードの種類
 - 2番目のカードの種類
 - 3番目のカードの種類
 - 山札の一番上のカードの種類

で場合が完全に定まる

- $O(N^4)$

満点解法2

- 山札の一番上のカードの種類は完全に覚える必要はない
- 1番目のカードが出せるか、3番目のカードが出せるかがわかれば良い

満点解法2

- ゲームの状態は
 - 1番目のカードの種類 N通り
 - 2番目のカードの種類 N通り
 - 3番目のカードの種類 N通り
 - 1番目のカードが出せるか 2通り
 - 3番目のカードが出せるか 2通り

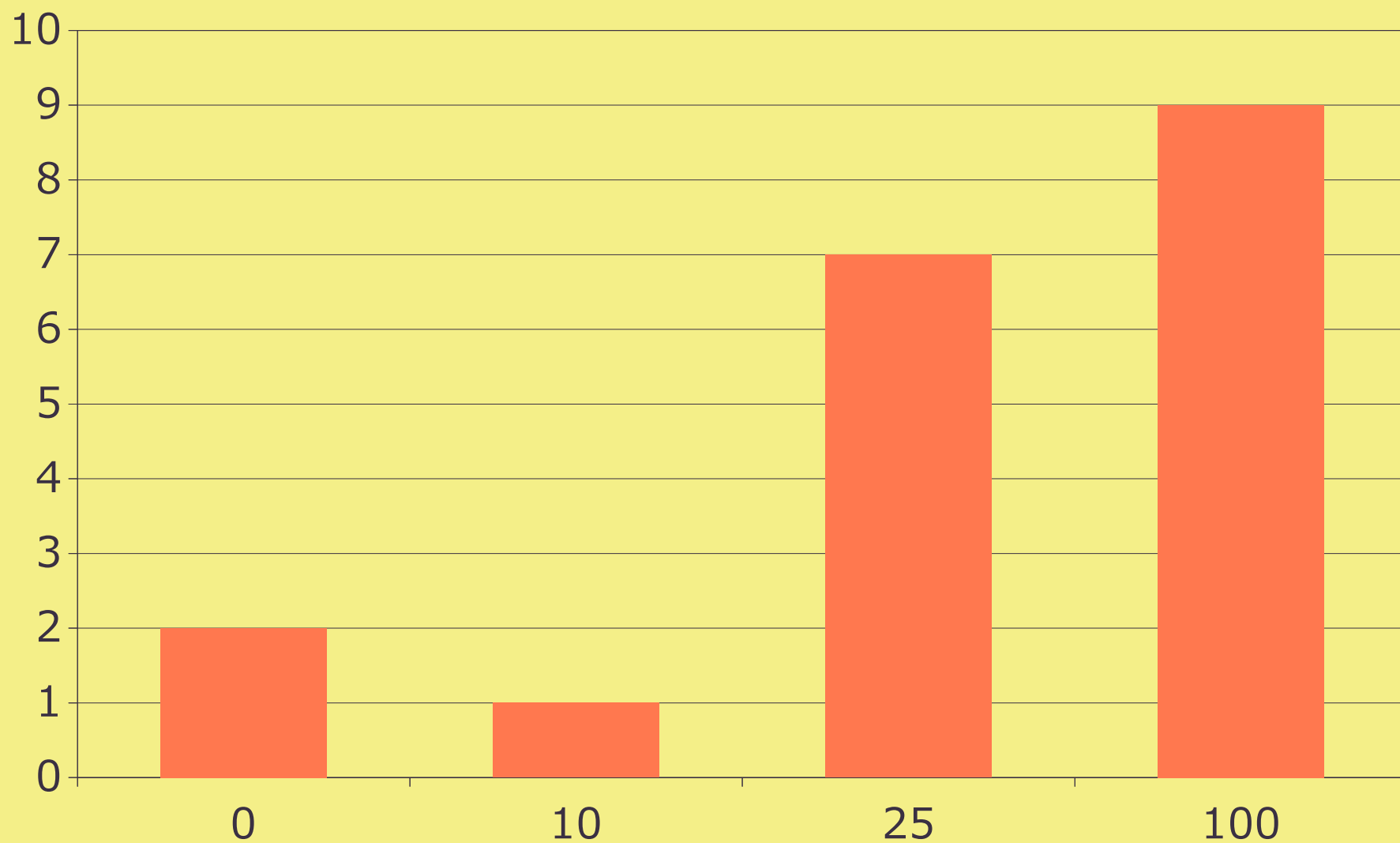
で完全に定まる

- $O(N^3)$

満点解法2

- `int dp[500][500][500][2][2]`
- 2GBくらい
- 2GBはさすがに取れない
- 節約すればなんとかなる
- `bool`なら500MBくらいなので探索なら大丈夫
- 満点

得点分布



ご清聴ありがとうございました

JOI春合宿2015

道案内(Navigation) 解説

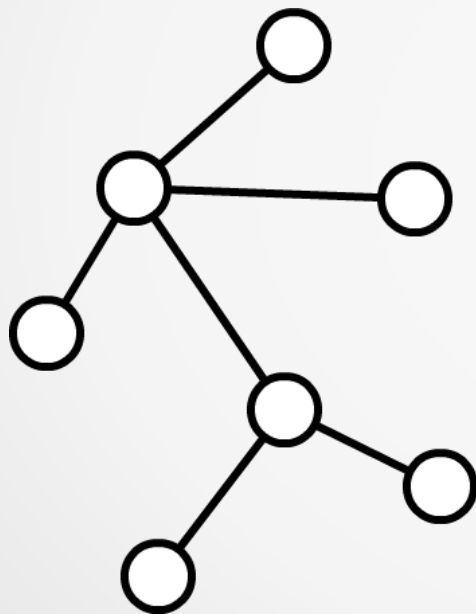
問題概要の前に

問題概要の前に

- 木(tree)
 - 閉路がない連結なグラフ

問題概要の前に

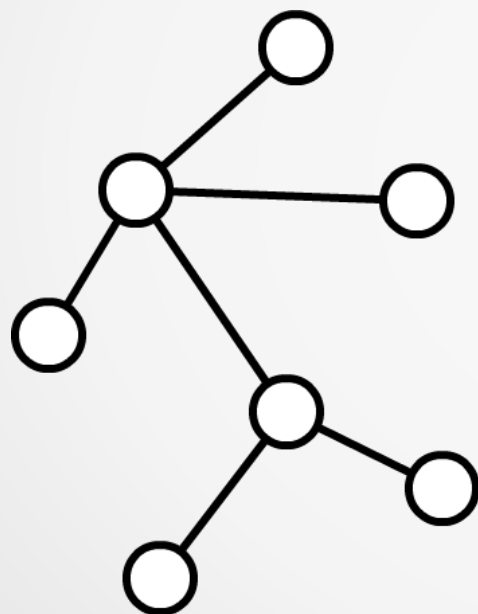
- 木(tree)
 - 閉路がない連結なグラフ



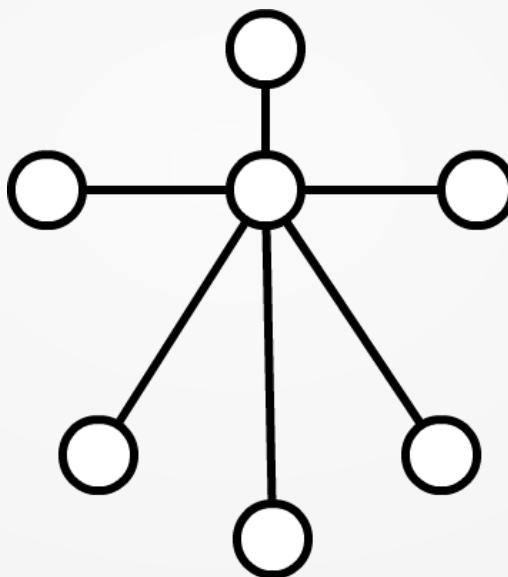
木である

問題概要の前に

- 木(tree)
 - 閉路がない連結なグラフ



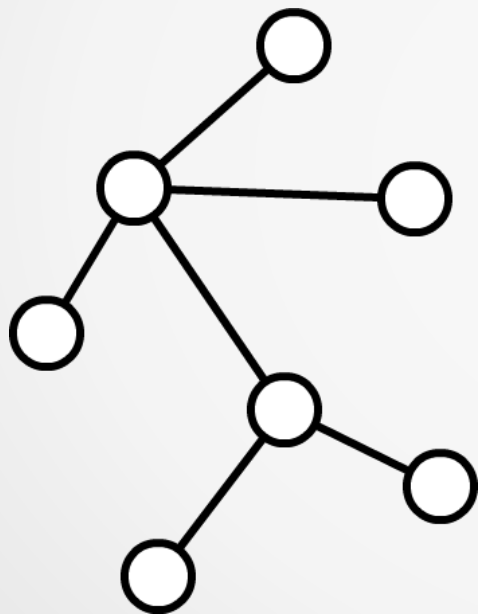
木である



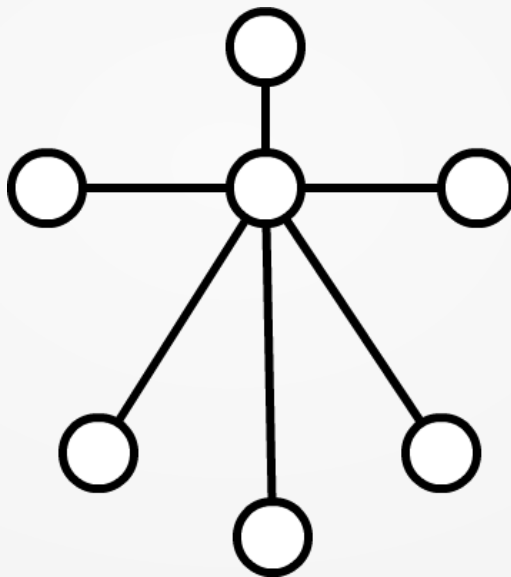
木である

問題概要の前に

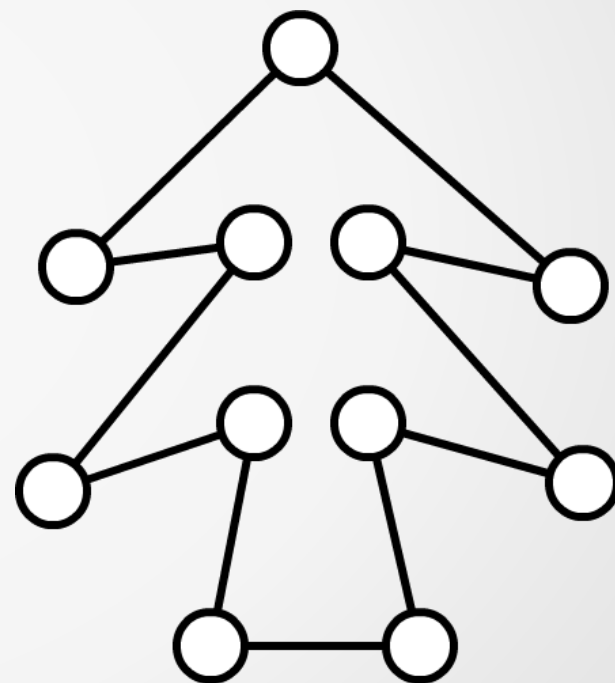
- 木(tree)
 - 閉路がない連結なグラフ



木である



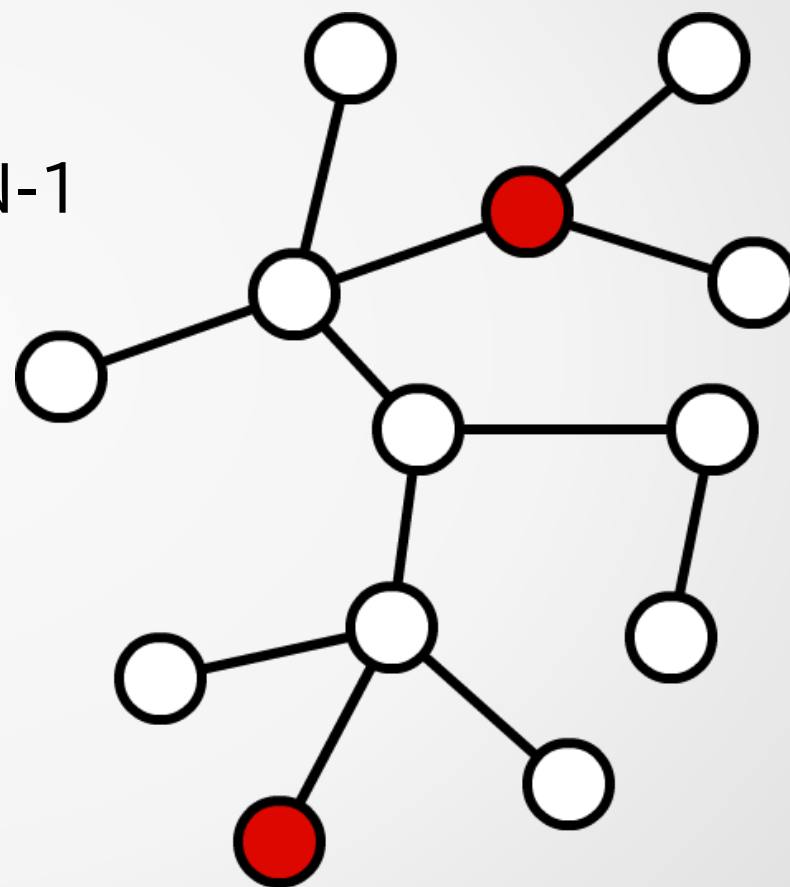
木である



木でない

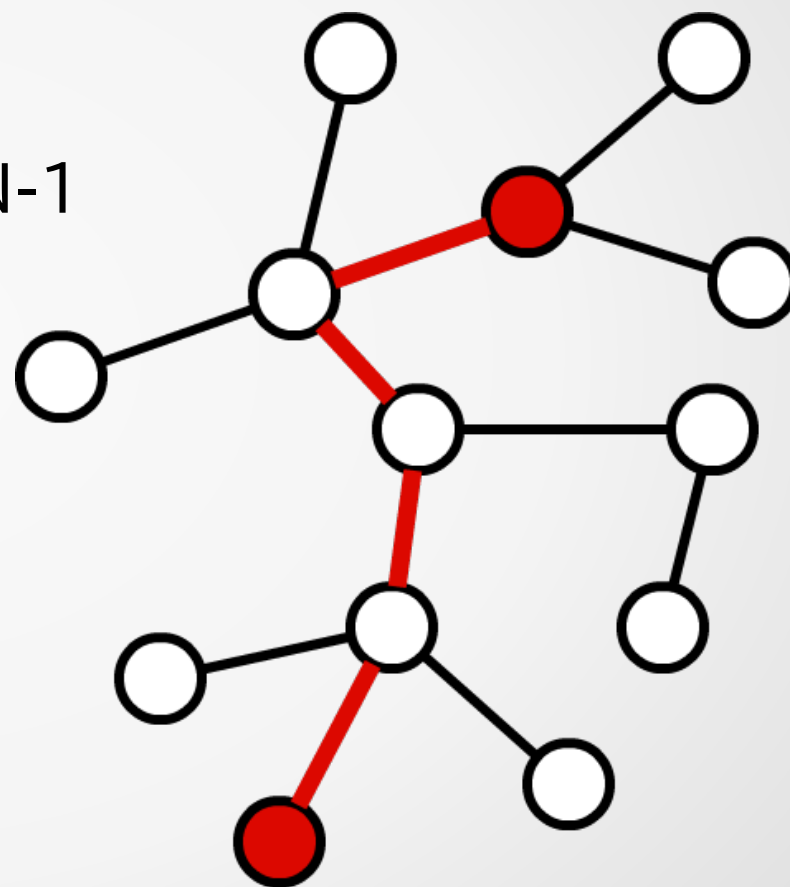
木の基本知識

- 木の性質
 - 連結である
 - 頂点数を N とすると辺の数が $N-1$
 - 二点間の経路は一意に定まる



木の基本知識

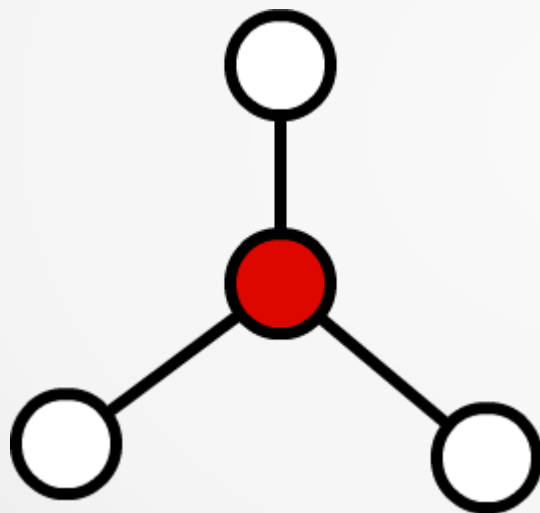
- 木の性質
 - 連結である
 - 頂点数を N とすると辺の数が $N-1$
 - 二点間の経路は一意に定まる



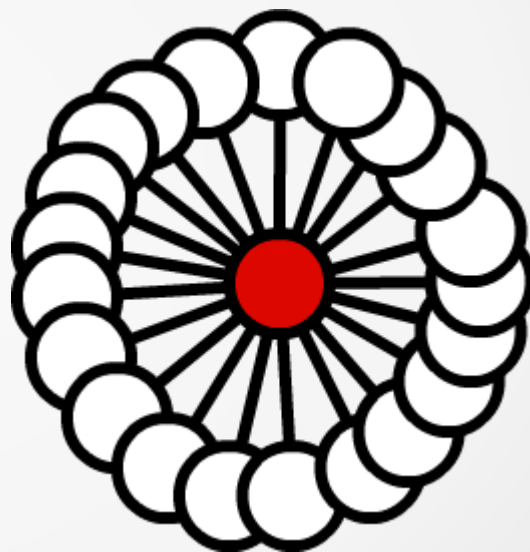
木の基本知識

- 木の用語

- ある頂点の**次数**とは、その頂点に繋がっている辺の個数



次数3

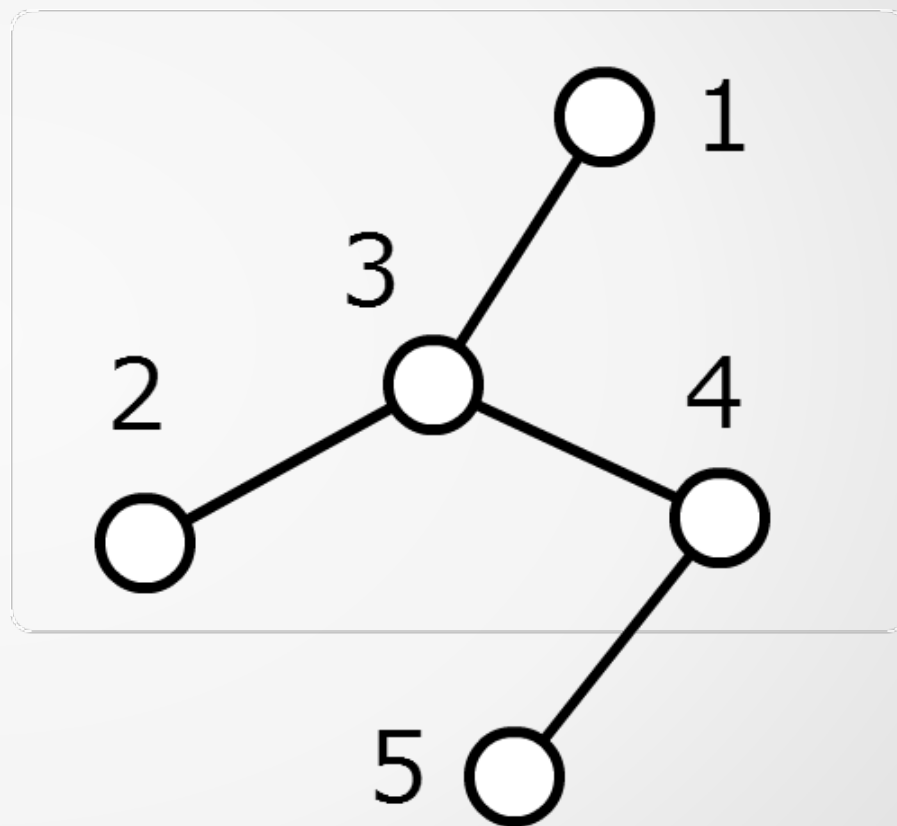


次数21

問題概要

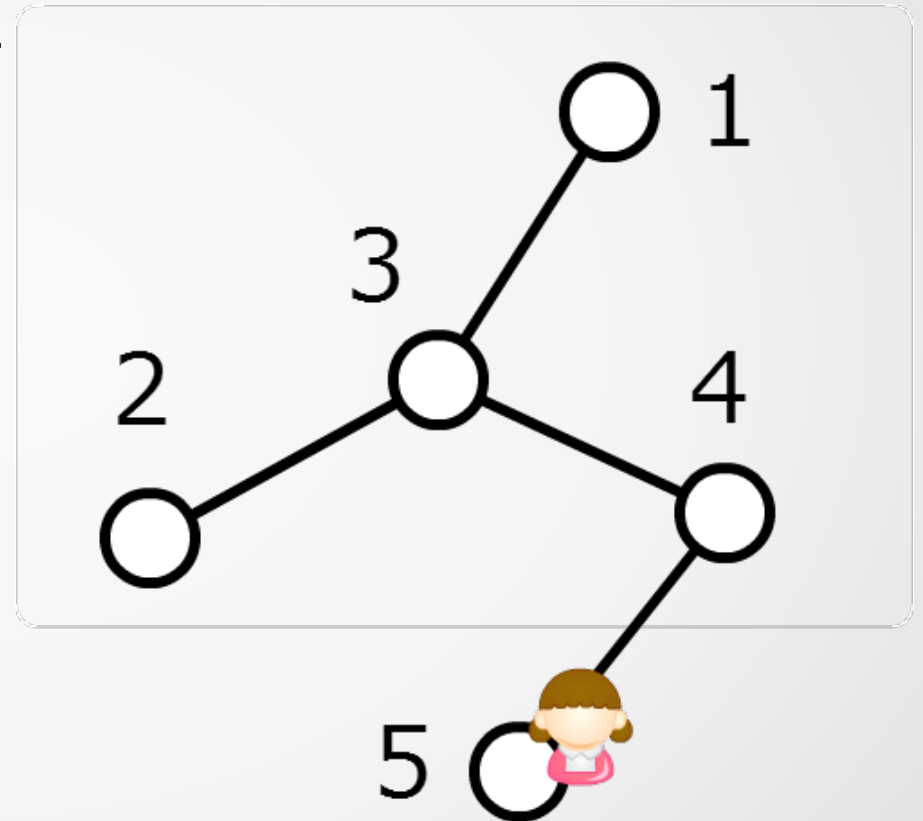
問題概要

- 頂点に1～Nの番号がつけられた木があります



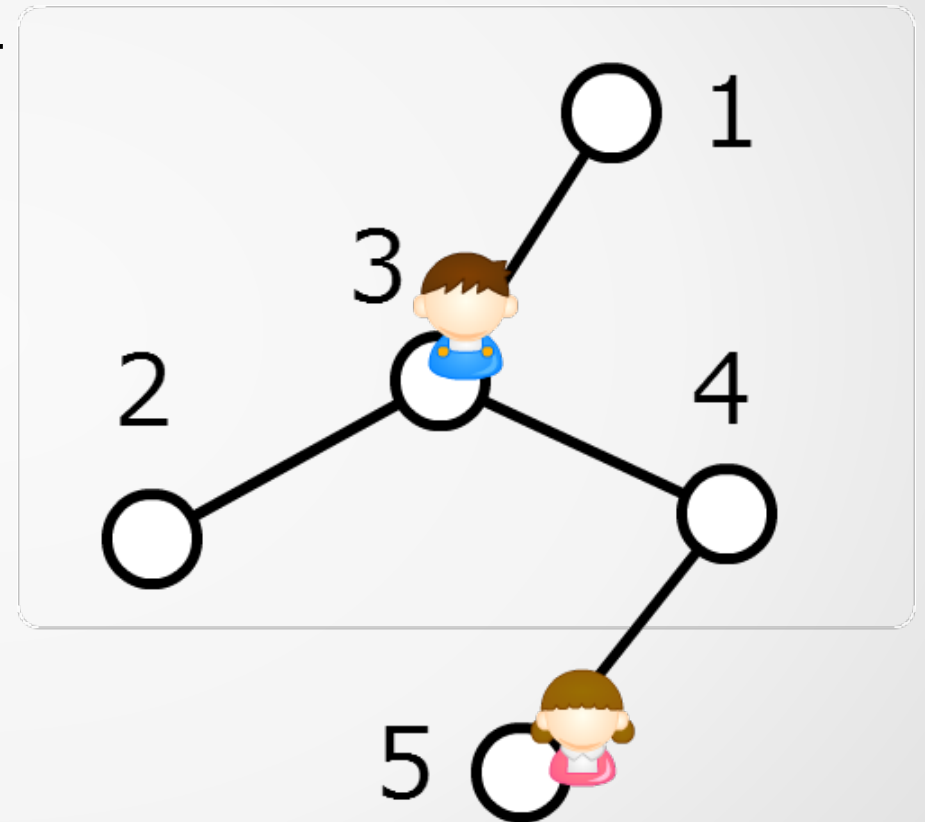
問題概要

- 頂点に1～Nの番号がつけられた木があります
- Annaは頂点Tに住んでいます



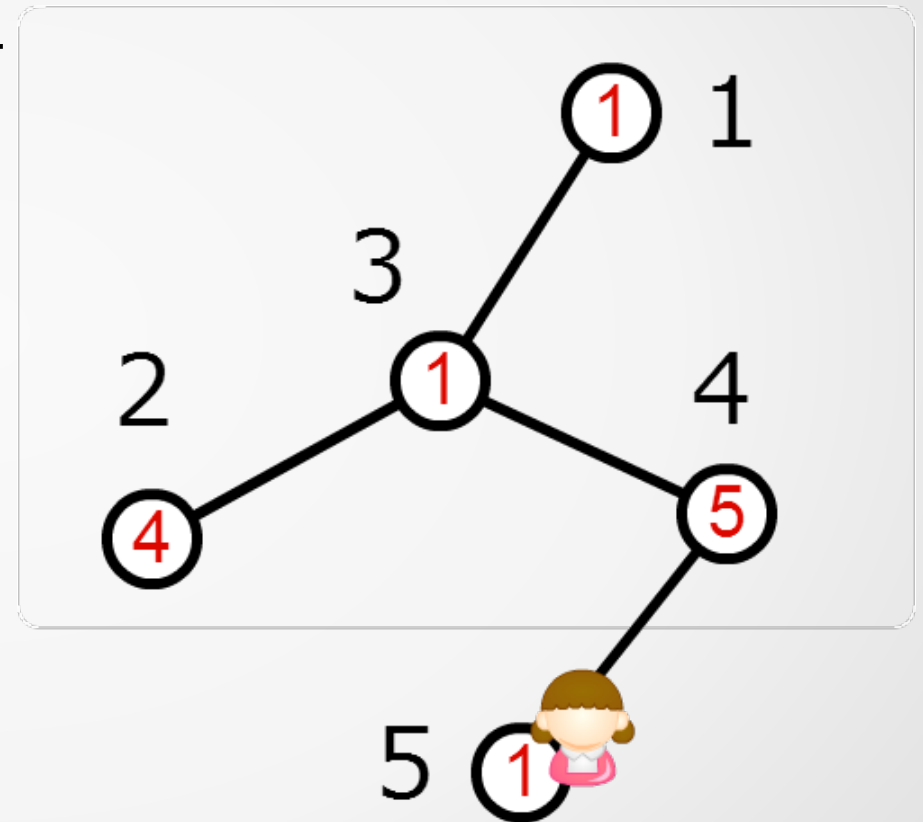
問題概要

- 頂点に1～Nの番号がつけられた木があります
- Annaは頂点Tに住んでいます
- Brunoは頂点Sに居ます



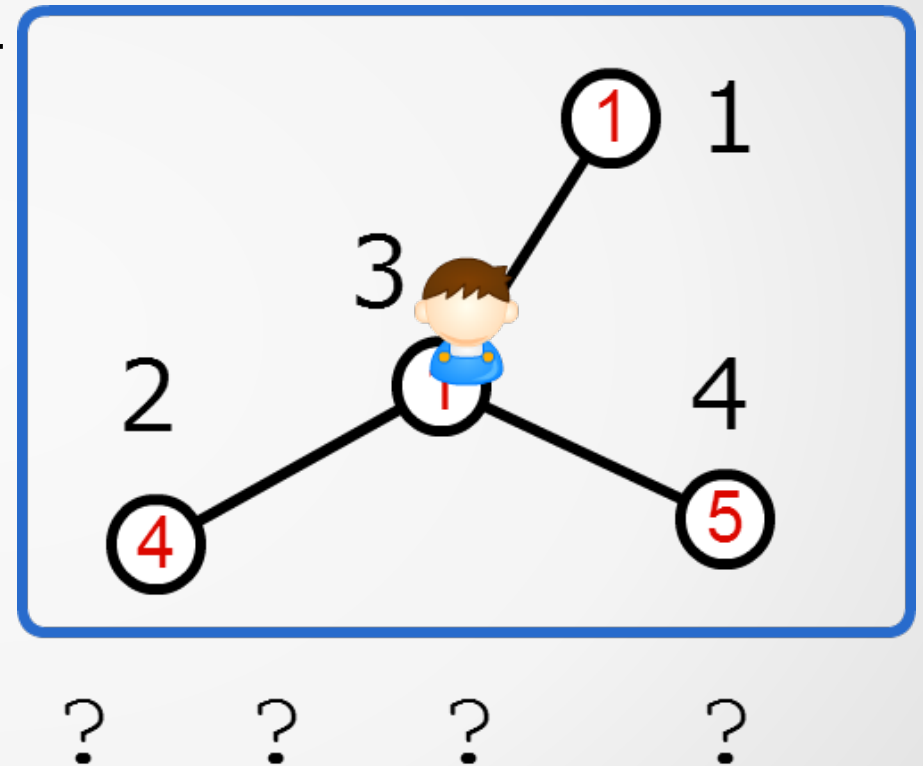
問題概要

- 頂点に1～Nの番号がつけられた木があります
- Annaは頂点Tに住んでいます
- Brunoは頂点Sに居ます
- Annaは各頂点に好きな整数を1つずつ書きます



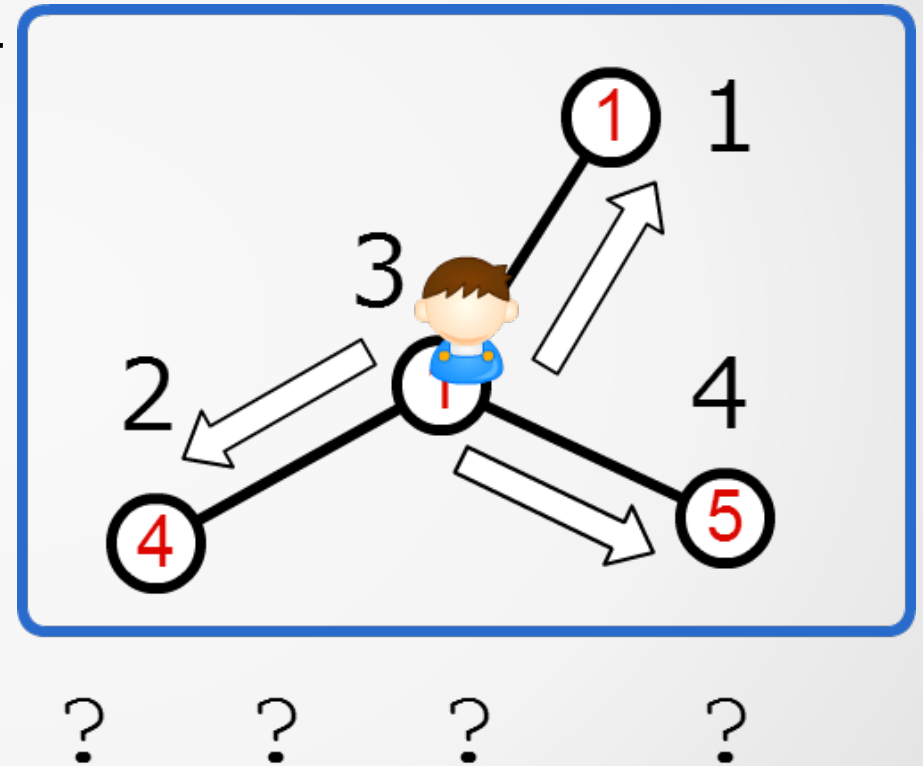
問題概要

- 頂点に1～Nの番号がつけられた木があります
- Annaは頂点Tに住んでいます
- Brunoは頂点Sに居ます
- Annaは各頂点に好きな整数を1つずつ書きます
- BrunoはSとSに隣接した頂点の情報がわかります



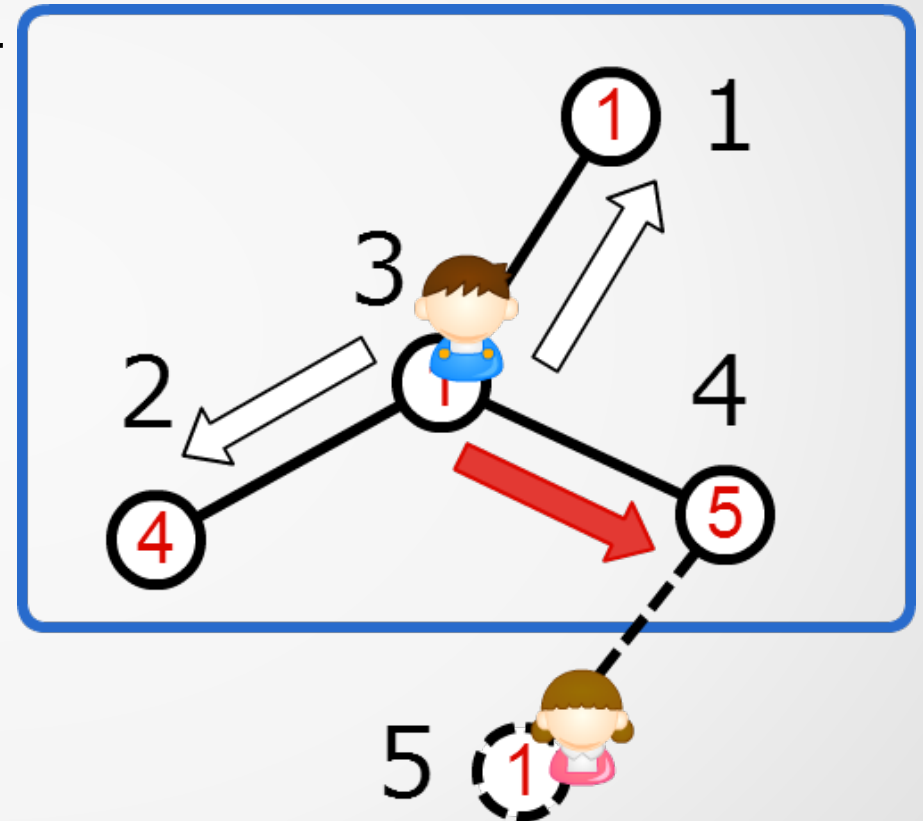
問題概要

- 頂点に1～Nの番号がつけられた木があります
- Annaは頂点Tに住んでいます
- Brunoは頂点Sに居ます
- Annaは各頂点に好きな整数を1つずつ書きます
- BrunoはSとSに隣接した頂点の情報がわかります
- BrunoがAnnaに近づくにはどちらに進むべき？



問題概要

- 頂点に1～Nの番号がつけられた木があります
- Annaは頂点Tに住んでいます
- Brunoは頂点Sに居ます
- Annaは各頂点に好きな整数を1つずつ書きます
- BrunoはSとSに隣接した頂点の情報がわかります
- BrunoがAnnaに近づくにはどちらに進むべき？



共通制約

- 頂点数 N
 - $N \leq 100,000$
- Annaが頂点に書ける整数の範囲
 - $0 \sim N$

小課題1 (10点)

小課題1

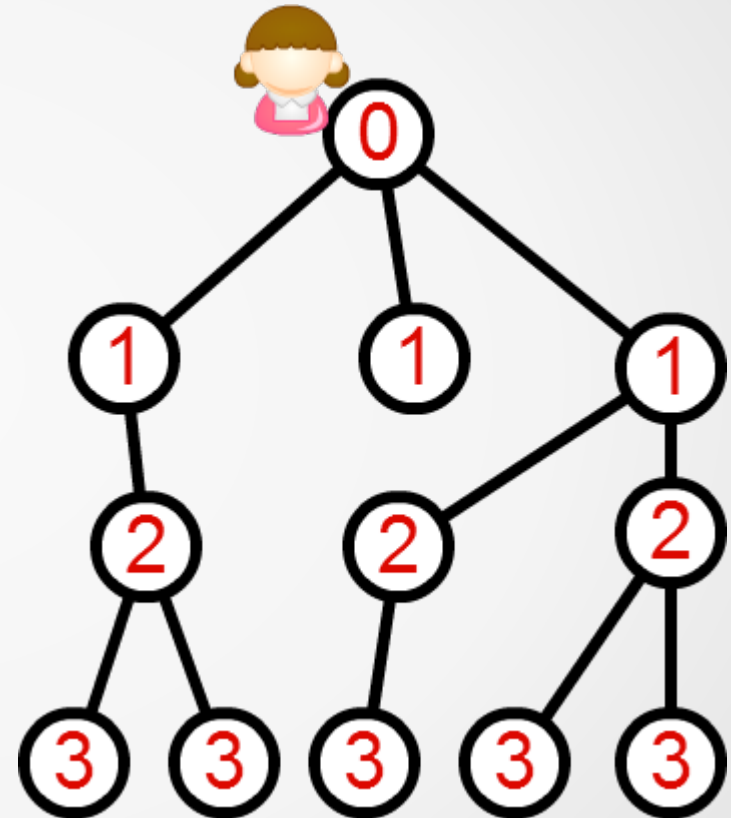
(10点)

小課題1 追加制約

- なし

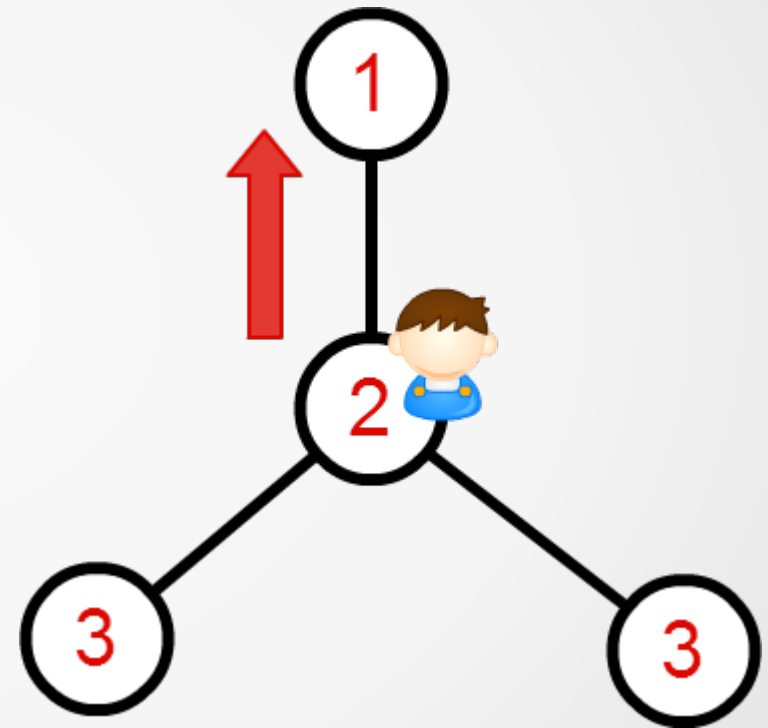
小課題1 解法

- Anna
 - 各頂点に T への距離を書く
 - 距離は T からの DFS で $O(N)$ で求められます
- Bruno
 - 今いる頂点より T に近い頂点に進めば良い
 - なければ $S = T$
 - 木なので一意に定まります



小課題1 解法

- Anna
 - 各頂点に T への距離を書く
 - 距離は T からの DFS で $O(N)$ で求められます
- Bruno
 - 今いる頂点より T に近い頂点に進めば良い
 - なければ $S = T$
 - 木なので一意に定まります



小課題2 (15点)

小課題2

(15点)

小課題2追加制約

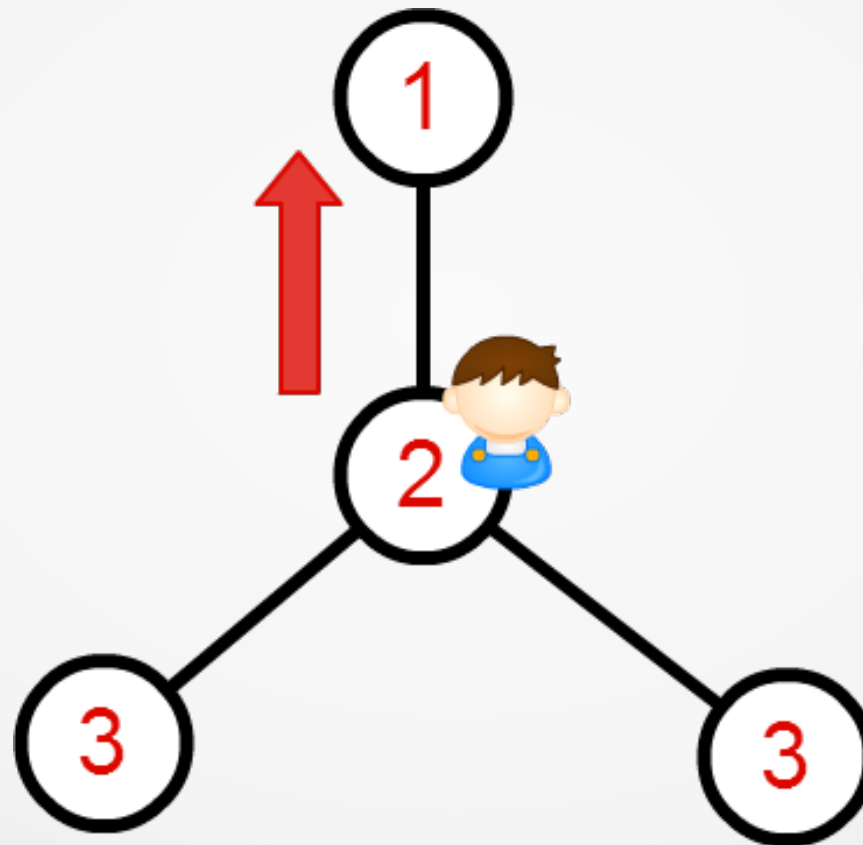
- Annaが頂点に書ける整数の範囲
 - 0, 1, 2

小課題2考察

- 小課題1でのBrunoからの眺め

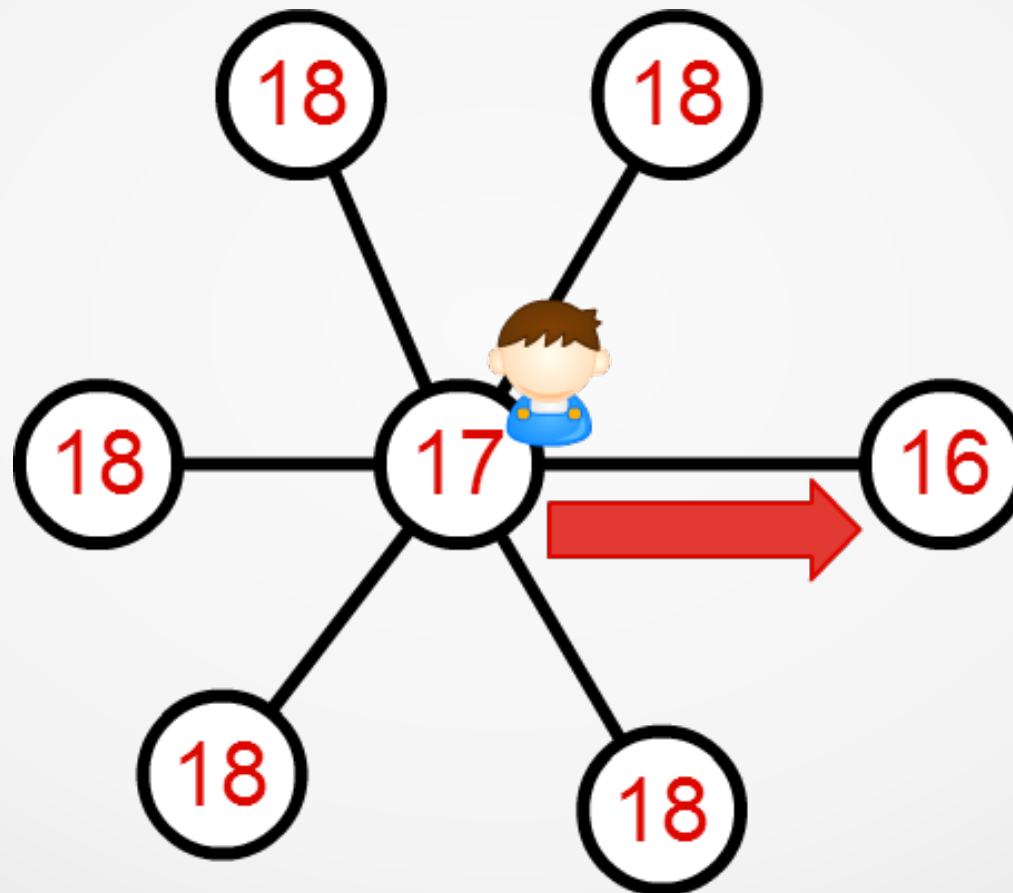
小課題2考察

- 小課題1でのBrunoからの眺め



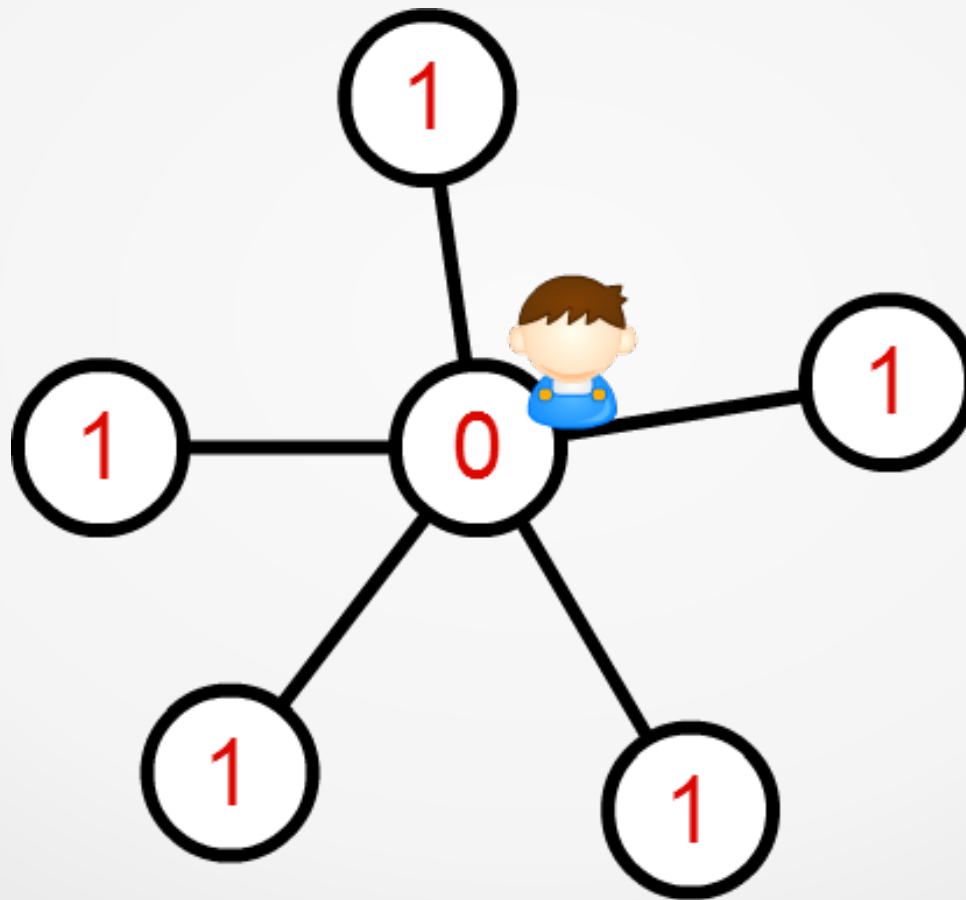
小課題2考察

- 小課題1でのBrunoからの眺め



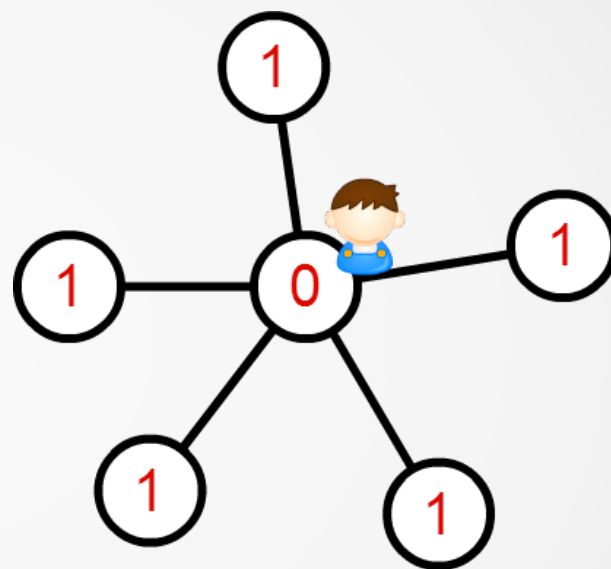
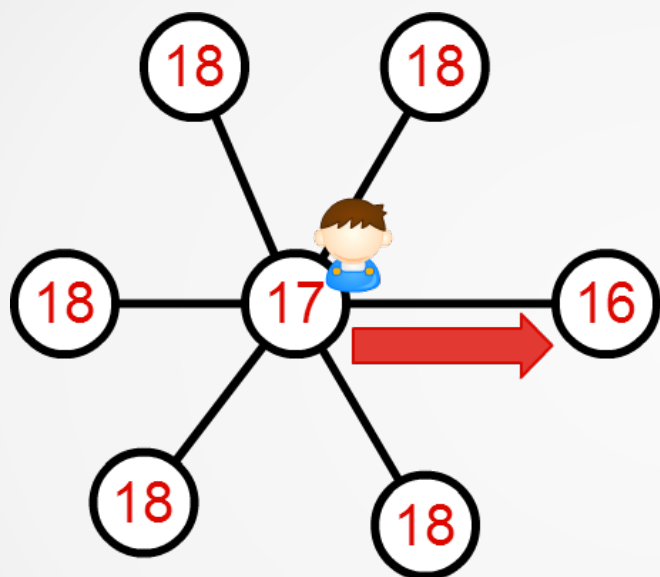
小課題2考察

- 小課題1でのBrunoからの眺め



小課題2考察

- 小課題1でのBrunoからの眺め

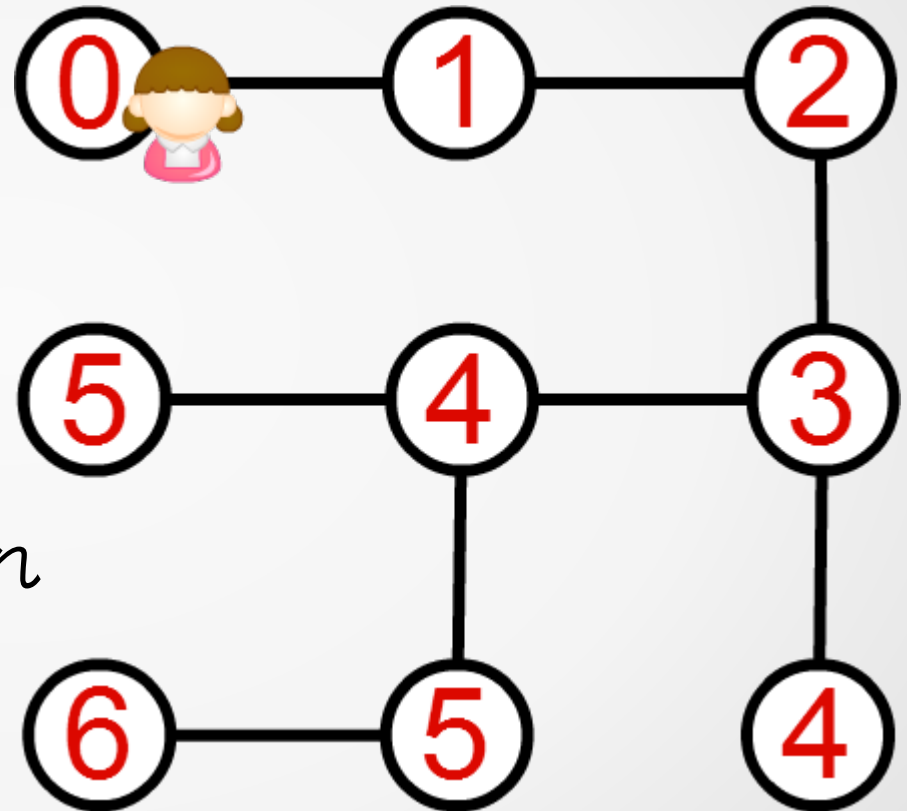


- 連続した3種類の整数しか出てこない
- ところで今回の制約もたかだか3種類しか整数がかけない

小課題2解法

- Anna

- 各頂点に T への距離を
3で割った余りを書く



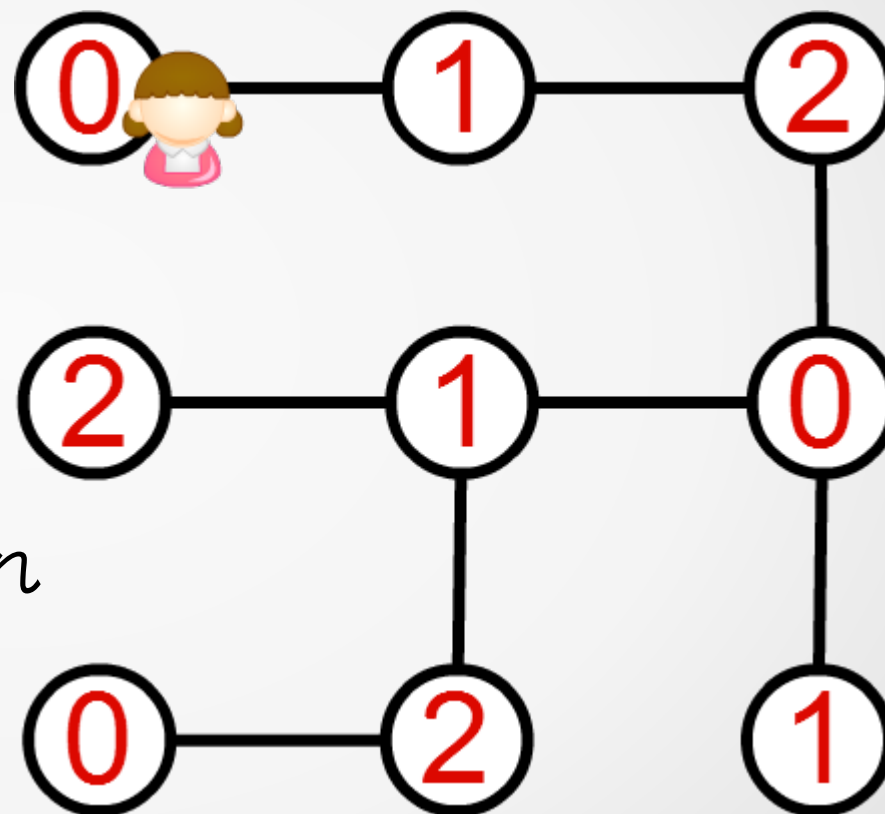
- Bruno

- 「1小さい整数」が書かれている頂点に進めば良い
- なければ $S = T$

小課題2解法

- Anna

- 各頂点に T への距離を
3で割った余りを書く

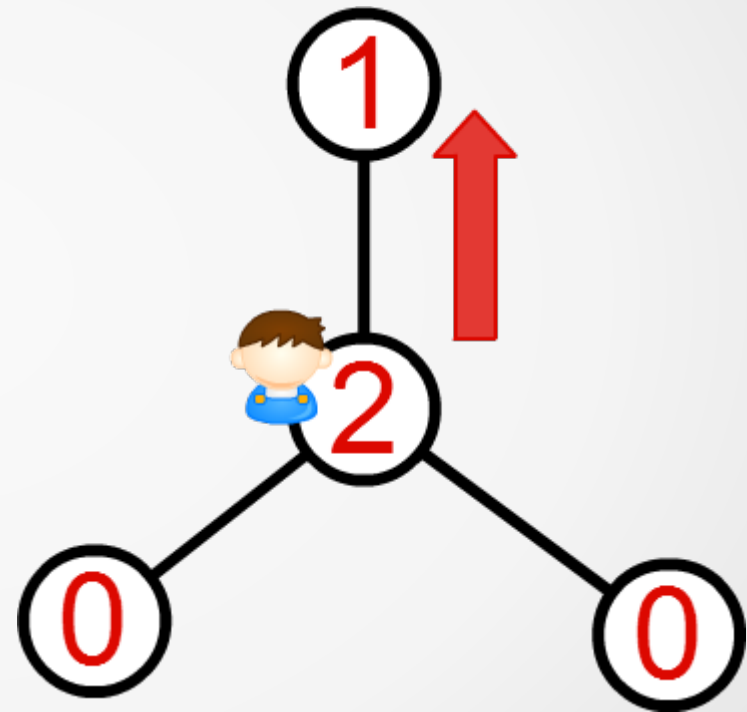


- Bruno

- 「1小さい整数」が書かれている頂点に進めば良い
- なければ $S = T$

小課題2解法

- Anna
 - 各頂点に T への距離を
3で割った余りを書く
- Bruno
 - 「1小さい整数」が書かれている頂点に進めば良い
 - なければ $S = T$



小課題2解説

- Bruno
 - 連続した3整数の真ん中の値を3で割った余りがわかる
 - 今いる頂点の整数
 - 真ん中の値がわかれば「1小さい整数」がどれか決定できる
- 真ん中が 0: $2 < 0 < 1$
- 真ん中が 1: $0 < 1 < 2$
- 真ん中が 2: $1 < 2 < 0$

0, 1, 2, 3, 4, 5, 6, 7.....

0, 1, 2, 0, 1, 2, 0, 1.....

小課題2解説

- Bruno
 - 連続した3整数の真ん中の値を3で割った余りがわかる
 - 今いる頂点の整数
 - 真ん中の値がわかれば「1小さい整数」がどれか決定できる
- 真ん中が 0: $2 < 0 < 1$
- 真ん中が 1: $0 < 1 < 2$
- 真ん中が 2: $1 < 2 < 0$

0, 1, 2, 3, 4, 5, 6, 7.....

0, 1, 2, 0, 1, 2, 0, 1.....

小課題3 (20点)

小課題3

(20点)

小課題3追加制約

- Annaが頂点に書ける整数の範囲
 - 0, 1
- 木の頂点の次数は2ではない
- $S \neq T$

小課題3追加制約

- Annaが頂点に書ける整数の範囲
 - 0, 1

厄介

- 木の頂点の次数は2ではない

わざわざ次数を制限していて怪しい

- $S \neq T$

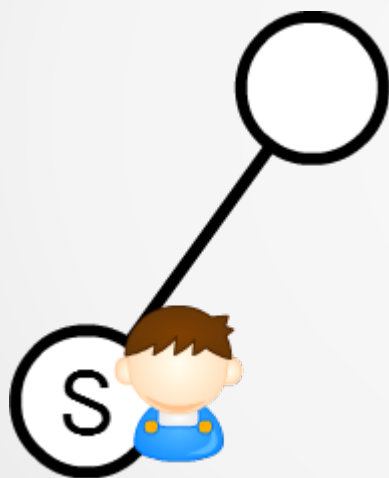
意図がまだよくわからない

小課題3考察

- 頂点は次数で3種類に大別できる

小課題3考察

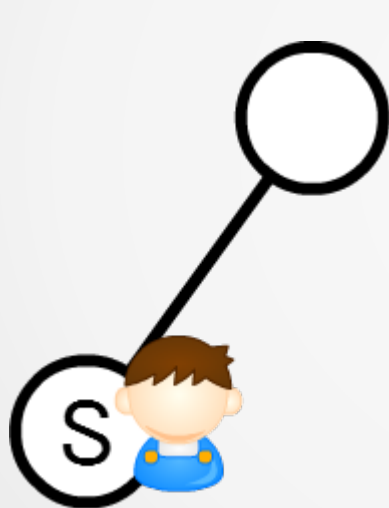
- 頂点Sは次数で3種類に大別できる
- それぞれのパターンについて別々に考える



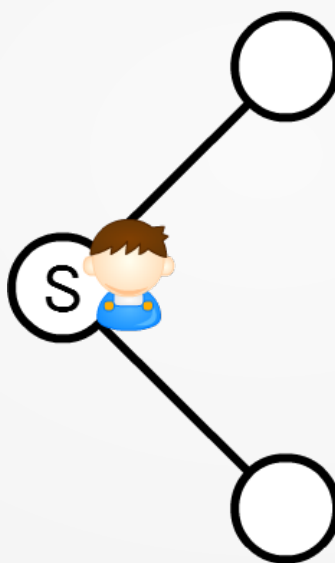
ひとつ

小課題3考察

- 頂点Sは次数で3種類に大別できる
- それぞれのパターンについて別々に考える



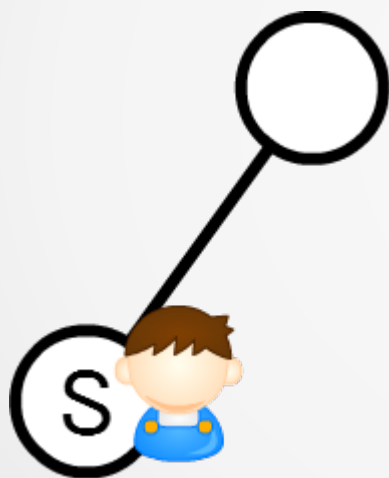
ひとつ



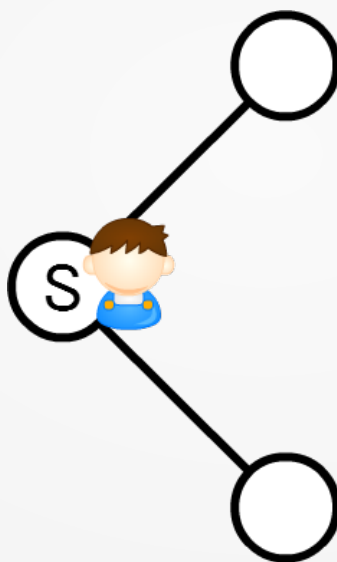
ふたつ

小課題3考察

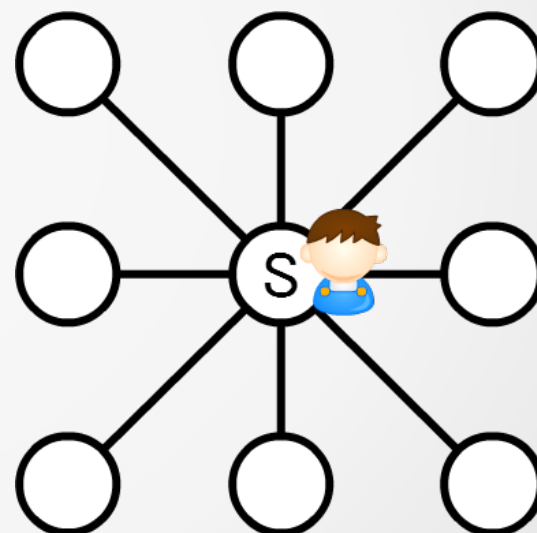
- 頂点Sは次数で3種類に大別できる
- それぞれのパターンについて別々に考える



ひとつ



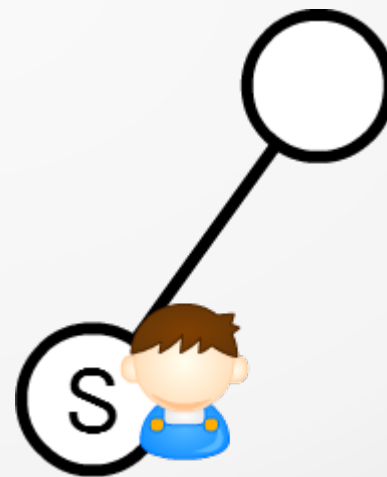
ふたつ



たくさん

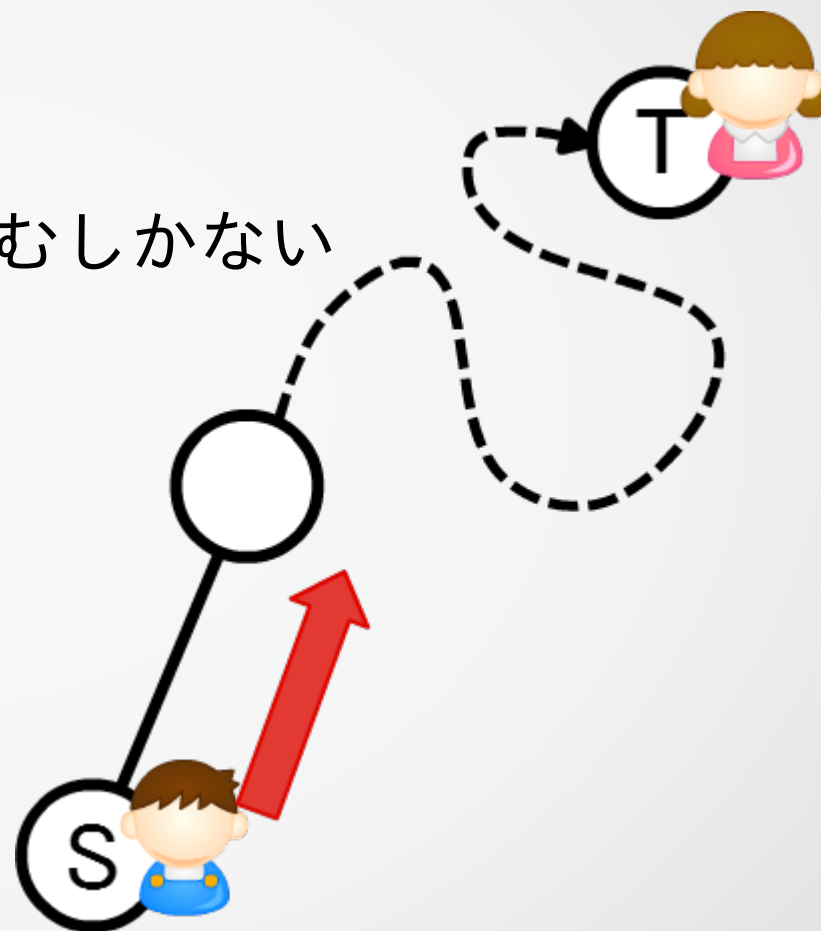
小課題3考察

- 次数が1の頂点



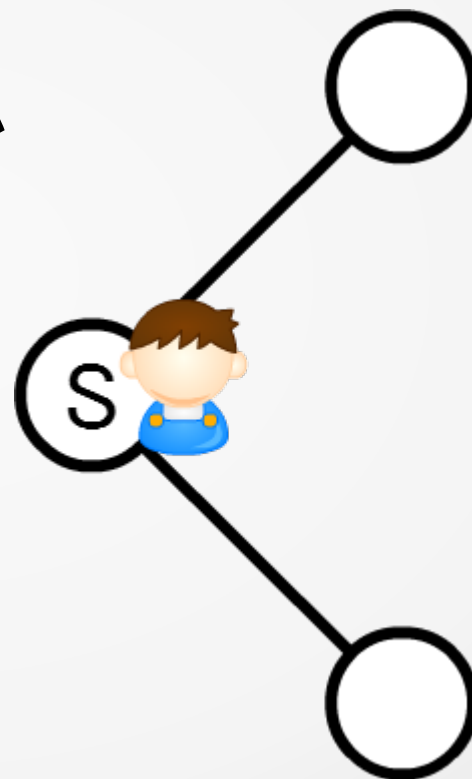
小課題3考察

- 次数が1の頂点
- $S \neq T$ なので隣接する頂点に進むしかない



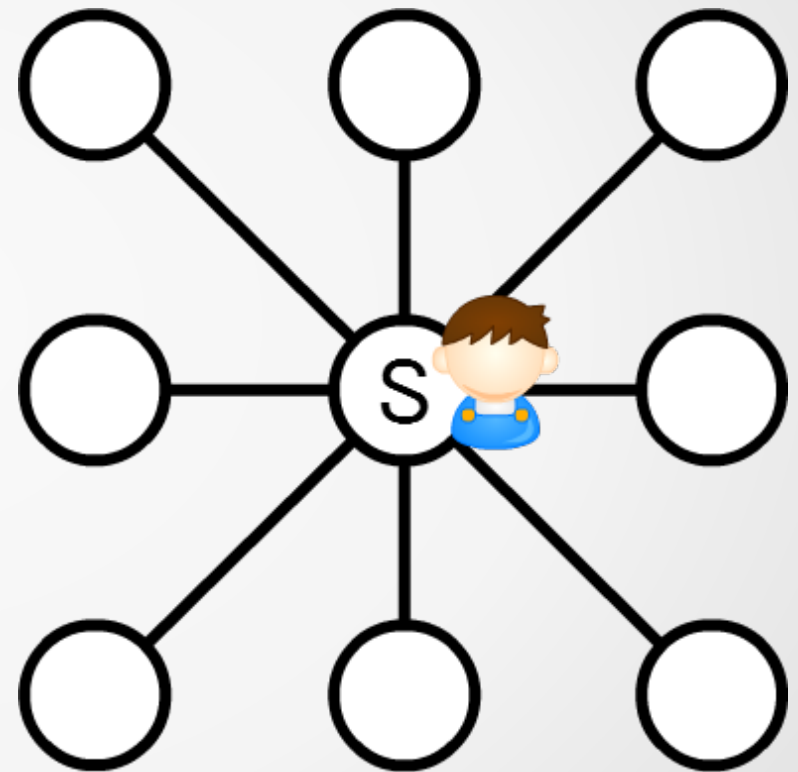
小課題3考察

- 次数が2の頂点
- 無いらしいので考えなくて良い



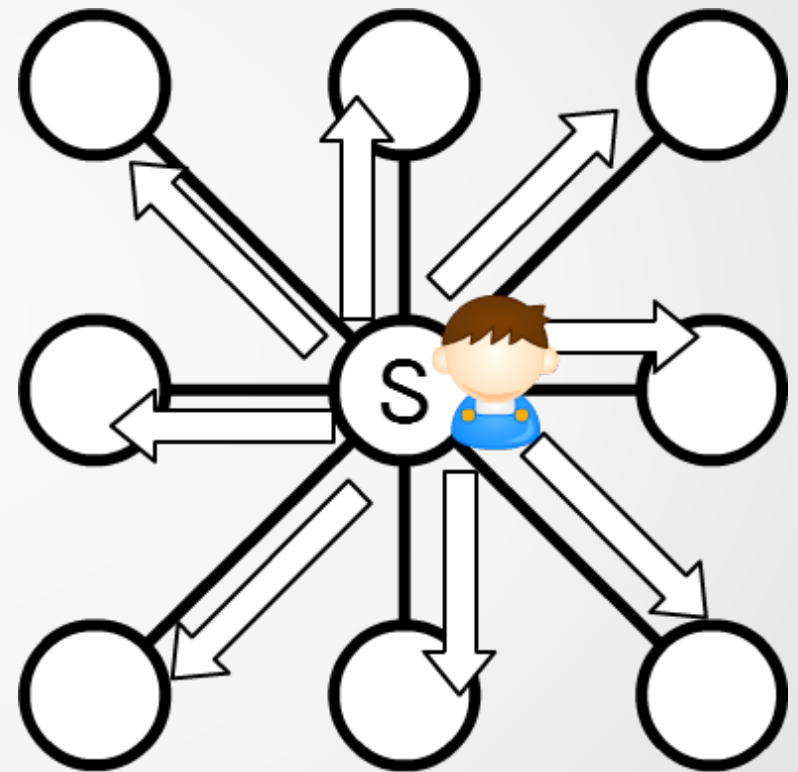
小課題3考察

- 次数がたくさん(≥ 3)の頂点



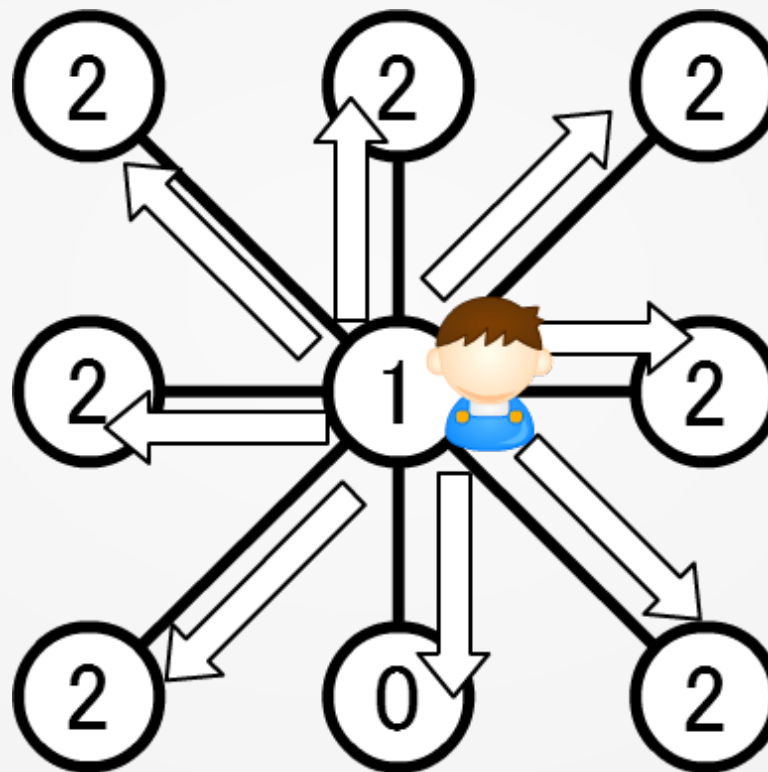
小課題3考察

- 次数がたくさん(≥ 3)の頂点
- $S \neq T$ なのでとにかくどこかに動く



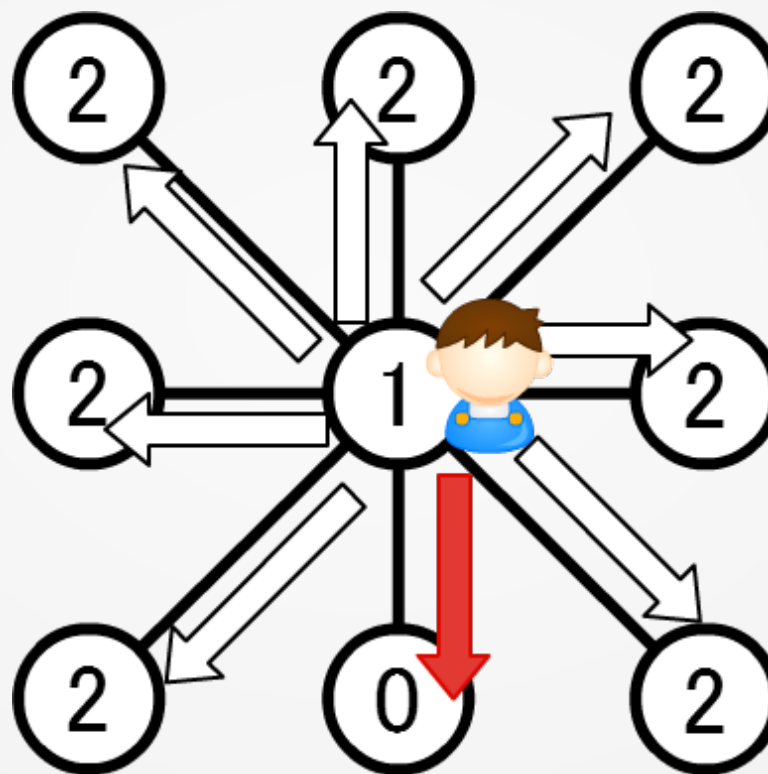
小課題3考察

- 小課題2でのBrunoからの眺め



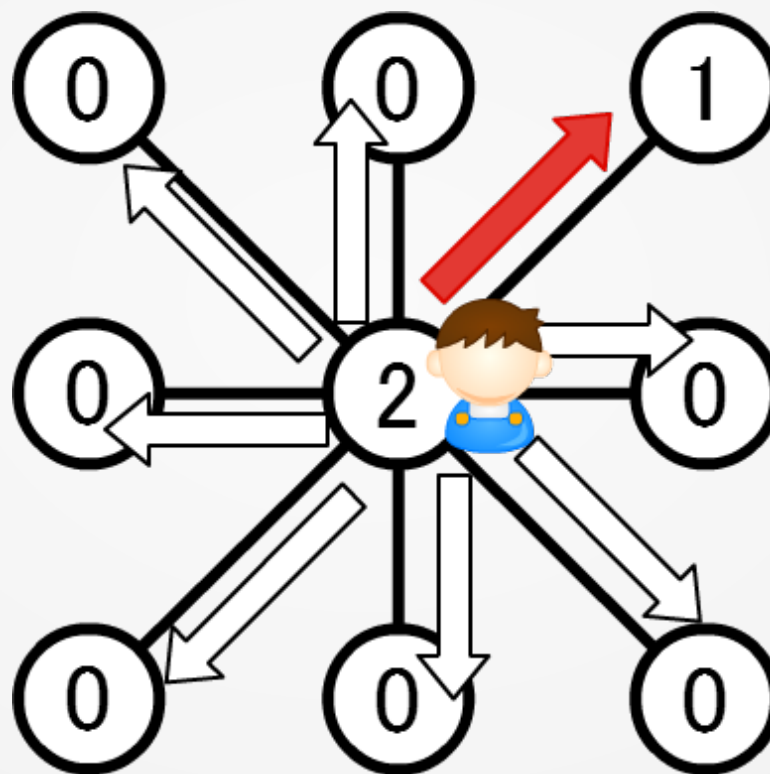
小課題3考察

- 小課題2でのBrunoからの眺め



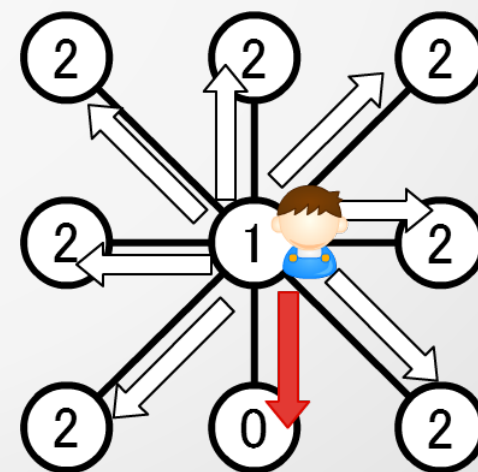
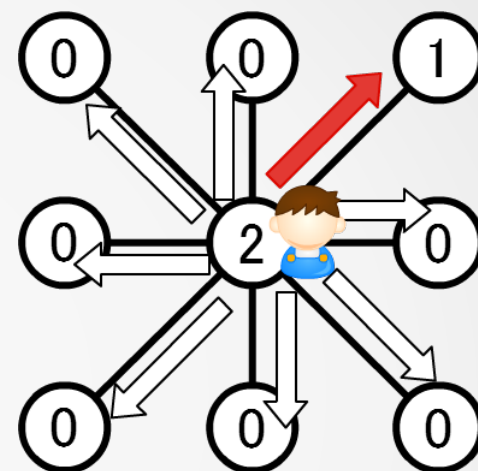
小課題3考察

- 小課題2でのBrunoからの眺め



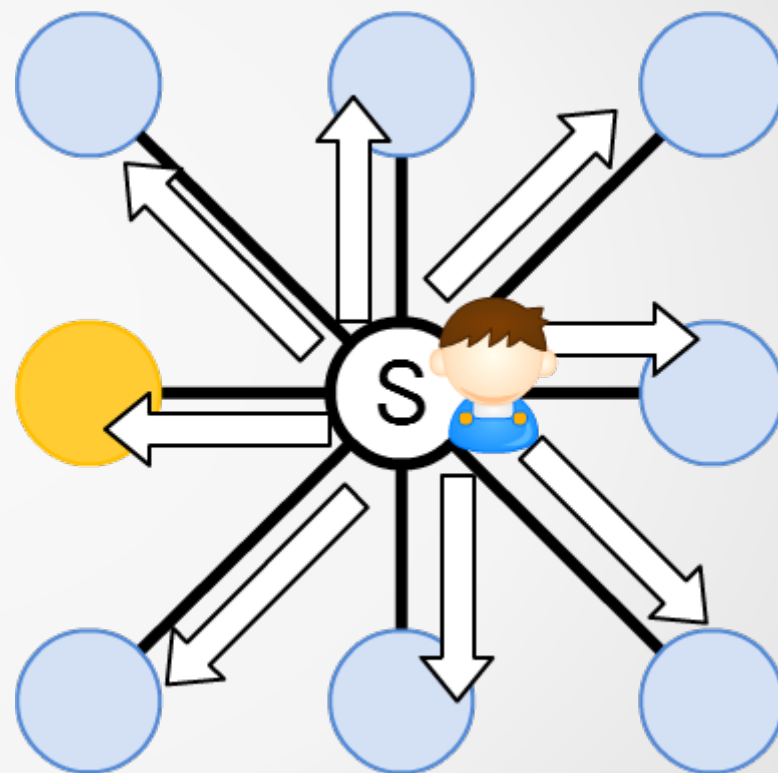
小課題3考察

- 次数がたくさん(≥ 3)の頂点
- $S \neq T$ なのでとにかくどこかに動く
- 小課題2でのBrunoからの眺め



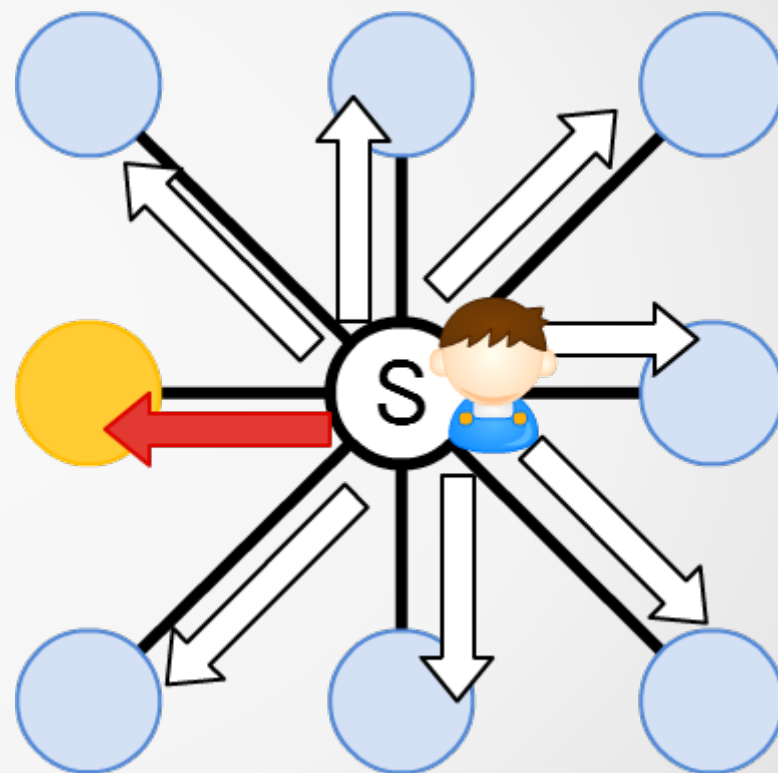
小課題3考察

- 次数がたくさん(≥ 3)の頂点
- $S \neq T$ なのでとにかくどこかに動く
- 小課題2でのBrunoからの眺め
- これは . . .



小課題3考察

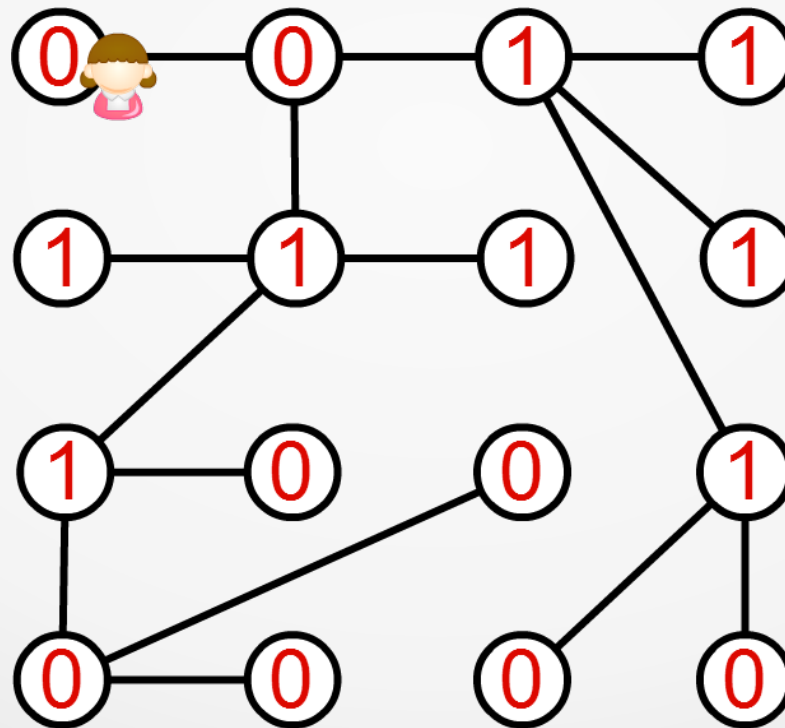
- 次数がたくさん(≥ 3)の頂点
- $S \neq T$ なのでとにかくどこかに動く
- 小課題2でのBrunoからの眺め
- 1種類しかない方に行けばよいのでは？



小課題3解法

- Anna

- 隣接する頂点の内、「進むべき頂点」と「それ以外の頂点」の整数が異なるように0と1を書く



小課題3解法

- Anna
 - 隣接する頂点の内、「進むべき頂点」と「それ以外の頂点」の整数が異なるように0と1を書く

小課題3解法

- Anna
 - 隣接する頂点の内、「進むべき頂点」と「それ以外の頂点」の整数が異なるように0と1を書く
 - どうやって？

小課題3解法

- Anna
 - 隣接する頂点の内、「進むべき頂点」と「それ以外の頂点」の整数が異なるように0と1を書く
 - どうやって？
- 「進むべき頂点」のTへの距離と「それ以外の頂点」のTへの距離の差は必ず2

小課題3解法

- Anna
 - 隣接する頂点の内、「進むべき頂点」と「それ以外の頂点」の整数が異なるように0と1を書く
 - どうやって？
- 「進むべき頂点」のTへの距離と「それ以外の頂点」のTへの距離の差は必ず2
 - $f(x) \neq f(x+2)$ となる整数から $\{0,1\}$ への変換を考える
- Before: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.....
- After: 0, 0, 1, 1, 0, 0, 1, 1, 0, 0.....

小課題3解法

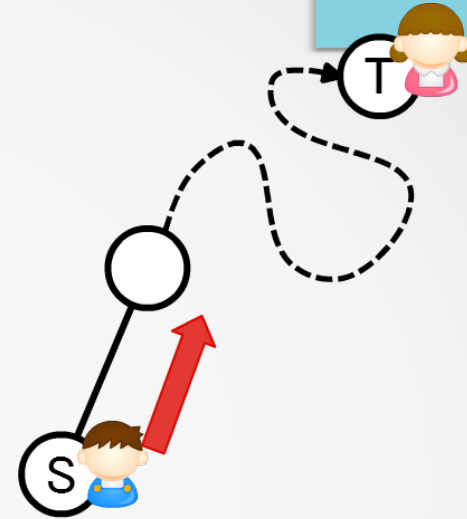
- Anna
 - 隣接する頂点の内、「進むべき頂点」と「それ以外の頂点」の整数が異なるように0と1を書く
 - どうやって？
- 「進むべき頂点」のTへの距離と「それ以外の頂点」のTへの距離の差は必ず2
 - $f(x) \neq f(x+2)$ となる整数から $\{0,1\}$ への変換を考える
- Before: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.....
- After: 0, 0, 1, 1, 0, 0, 1, 1, 0, 0.....

小課題3解法

- Anna
 - 隣接する頂点の内、「進むべき頂点」と「それ以外の頂点」の整数が異なるように0と1を書く
 - どうやって？
- 「進むべき頂点」のTへの距離と「それ以外の頂点」のTへの距離の差は必ず2
 - $f(x) \neq f(x+2)$ となる整数から $\{0,1\}$ への変換を考える
- Before: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.....
- After: 0, 0, 1, 1, 0, 0, 1, 1, 0, 0.....

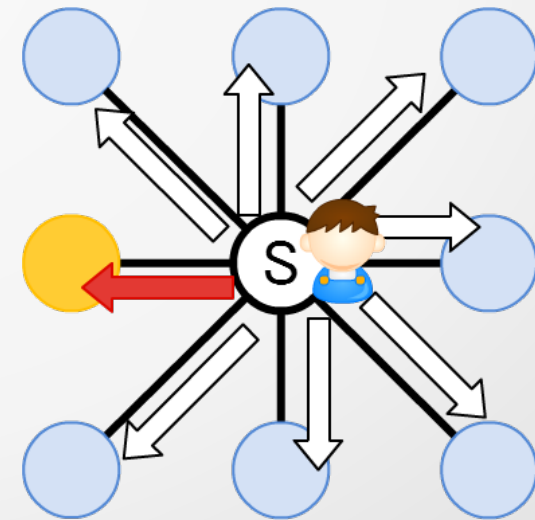
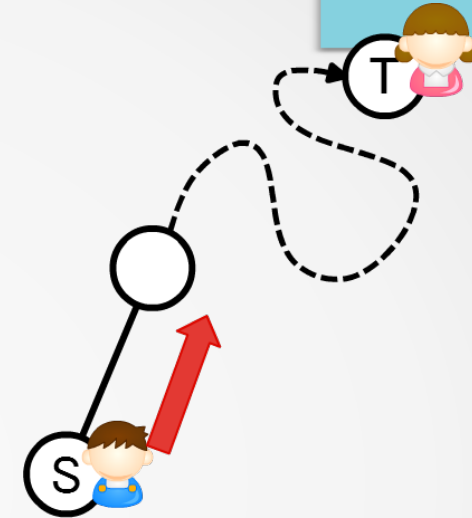
小課題3解法

- Bruno
 - 次数が1
 - とにかく動く



小課題3解法

- Bruno
 - 次数が1
 - とにかく動く
 - 次数がたくさん
 - 隣接する頂点の中で1回しか現れない整数が書かれている方に動く



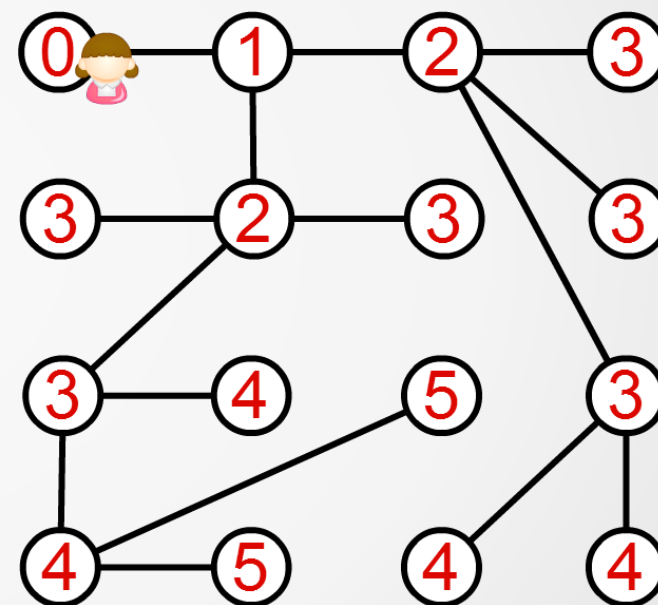
小課題3解法 まとめ

- Anna

- 各頂点にTへの距離を4で割った余りに応じて適切な整数を書く
- 距離 $\%4/2$ (あくまで一例)

- Bruno

- 次数が1
 - とにかく動く
- 次数がたくさん
 - 隣接する頂点の中で1回しか現れない整数が書かれている方に動く



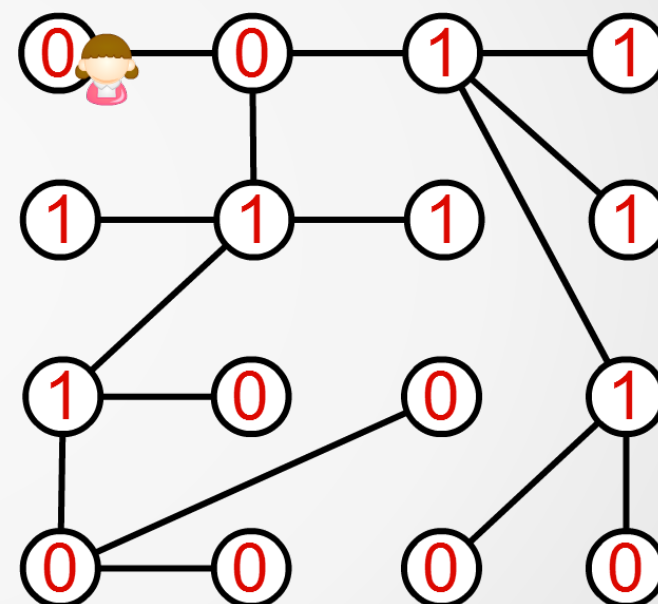
小課題3解法 まとめ

- Anna

- 各頂点にTへの距離を4で割った余りに応じて適切な整数を書く
- 距離 $\%4/2$ (あくまで一例)

- Bruno

- 次数が1
 - とにかく動く
- 次数がたくさん
 - 隣接する頂点の中で1回しか現れない整数が書かれている方に動く



小課題4追加制約

- Annaが頂点に書ける整数の範囲
 - 0, 1
- 以上

小課題4考察

- Brunoがもらえる情報が少なすぎる
 - 0,1だけというのはどうしようもない
- まだ使っていない情報があるはず
- もう一回、問題文をよく読もう

問題概要(再掲)

- 頂点に1～Nの番号がつけられた木があります
- Annaは頂点Tに住んでいます
- Brunoは頂点Sに居ます
- Annaは各頂点に好きな整数を1つずつ書きます
- BrunoはSとSに隣接した頂点の情報がわかります
- BrunoがAnnaに近づくにはどちらに進むべき？

問題概要(再掲)

- 頂点に1～Nの番号がつけられた木があります
- Annaは頂点Tに住んでいます
- Brunoは頂点Sに居ます
- Annaは各頂点に好きな整数を1つずつ書きます
- BrunoはSとSに隣接した頂点の情報がわかります
- BrunoがAnnaに近づくにはどちらに進むべき？

小課題4考察

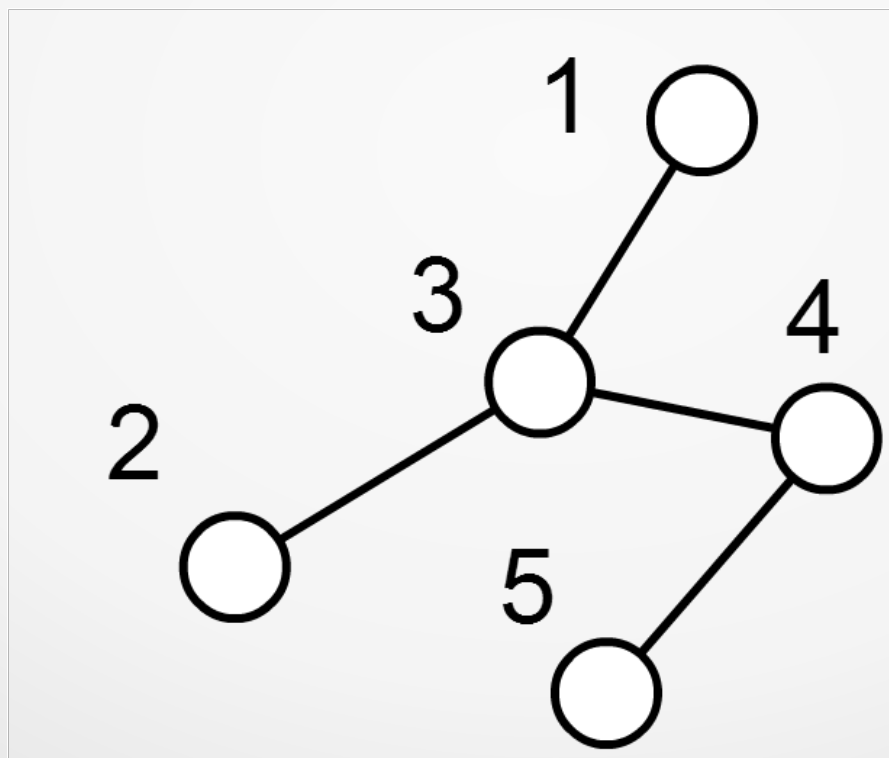
- BrunoはAnnaが書いた整数だけでなく頂点(島)の番号もわかる
- これも活用できないだろうか？

小課題4考察

- 頂点の番号の性質
 - ユニークである
 - 以上
- Brunoからの眺めを考えると頂点の番号は大小以上の情報を与えてくれなさそう
 - 和が●である、▲で割った余りが■である、などの情報を活用したくてもBrunoに与える頂点の番号は作問者が悪意をもって選ぶ事ができるので難しそう

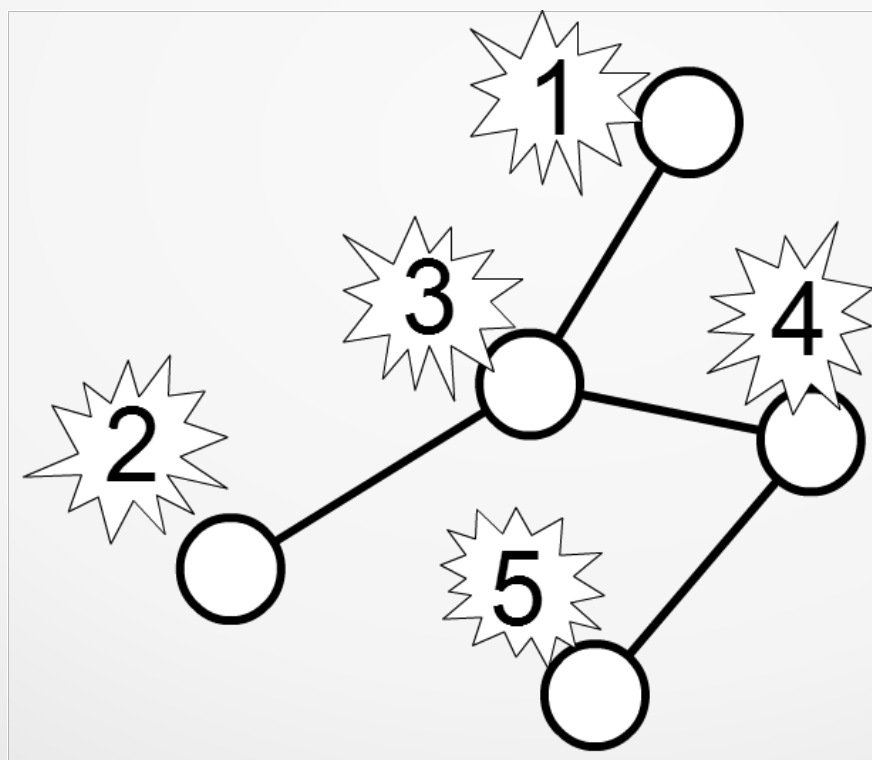
小課題4考察まとめ1

- Brunoの居る頂点と周りの頂点に書かれた整数だけでなく、頂点の番号も活用して解く
- 頂点の番号は大小関係程度の情報しか与えてくれない



小課題4考察まとめ1

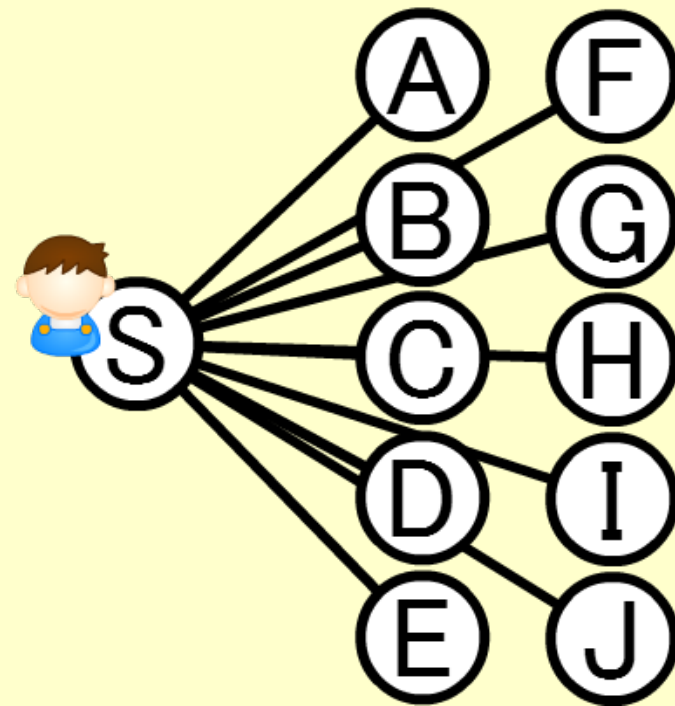
- Brunoの居る頂点と周りの頂点に書かれた整数だけでなく、頂点の番号も活用して解く
- 頂点の番号は大小関係程度の情報しか与えてくれない



小課題4考察

- どのようなアプローチでBrunoの動きを決めるか？
- いままでのアプローチ
 - 頂点を見て、Tにより近い頂点をあてる
 - 選択肢が多い

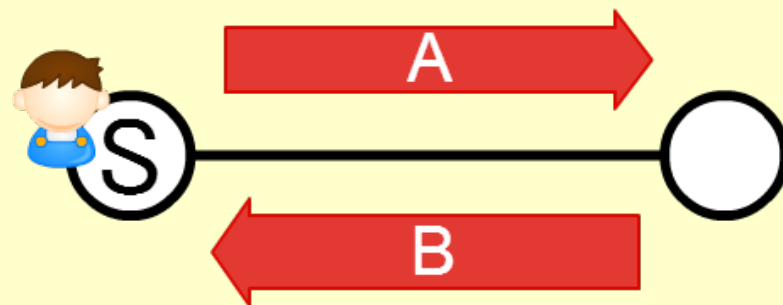
Q. どれが  に近い？



小課題4考察

- どのようなアプローチでBrunoの動きを決めるか？
- いままでのアプローチ
 - 辺をみて、どちら向きに進むべきかあてる
 - スッキリしている

Q.どっちに進むべき？



小課題4考察

- 新しいアプローチの優れている点

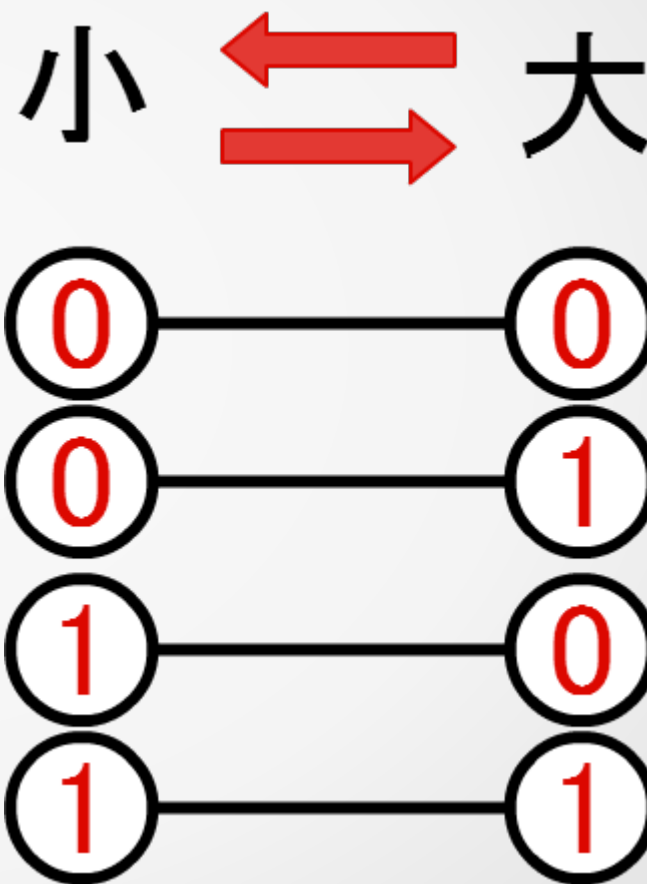
- 進むべき方向は2種類しかない

- 辺を特徴付ける情報は

- 2ビット分の整数

- 2つの整数

- 2つの整数といっても
どちらが大きいか程度
の情報と考えて良い

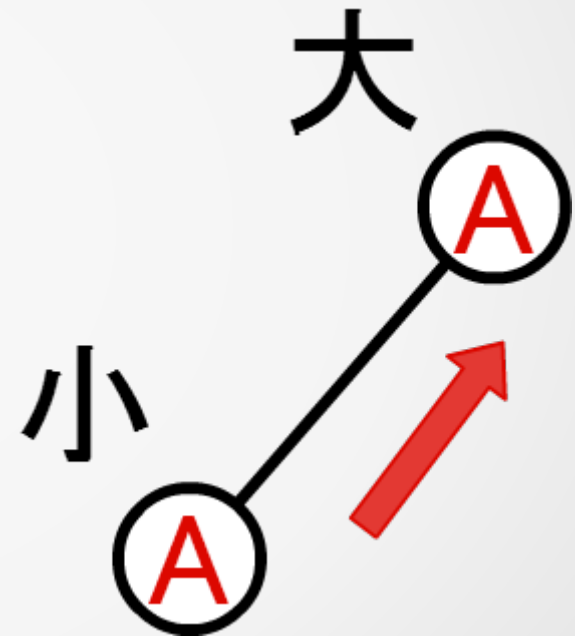


小課題4解法

- Anna
 - 各辺についてどちら向きに進むべきか求める
 - 頂点Xと頂点Yを結ぶ辺について
 - 書かれた整数が同じならば頂点番号が大きい方へ進むべき
 - 書かれた整数が違うならば頂点番号が小さい方へ進むべき
- となるように整数を書く

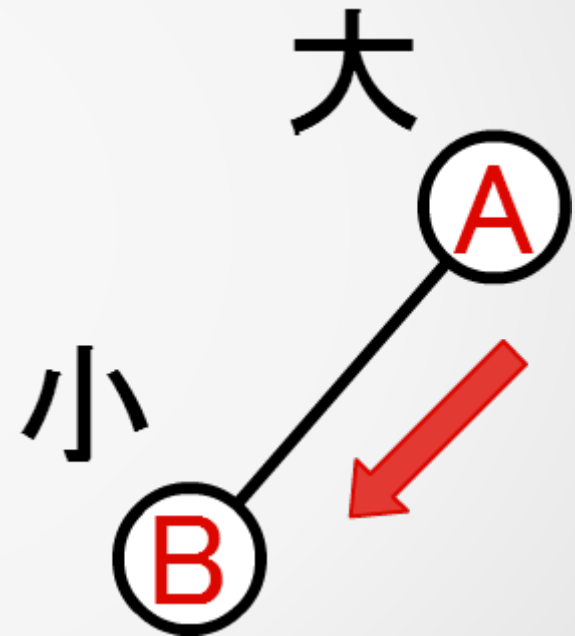
小課題4解法

- Anna
 - 各辺についてどちら向きに進むべきか求める
 - 頂点Xと頂点Yを結ぶ辺について
 - 書かれた整数が同じならば頂点番号が大きい方へ進むべき
 - 書かれた整数が違うならば頂点番号が小さい方へ進むべき
- となるように整数を書く



小課題4解法

- Anna
 - 各辺についてどちら向きに進むべきか求める
 - 頂点Xと頂点Yを結ぶ辺について
 - 書かれた整数が同じならば頂点番号が大きい方へ進むべき
 - 書かれた整数が違うならば頂点番号が小さい方へ進むべき
- となるように整数を書く

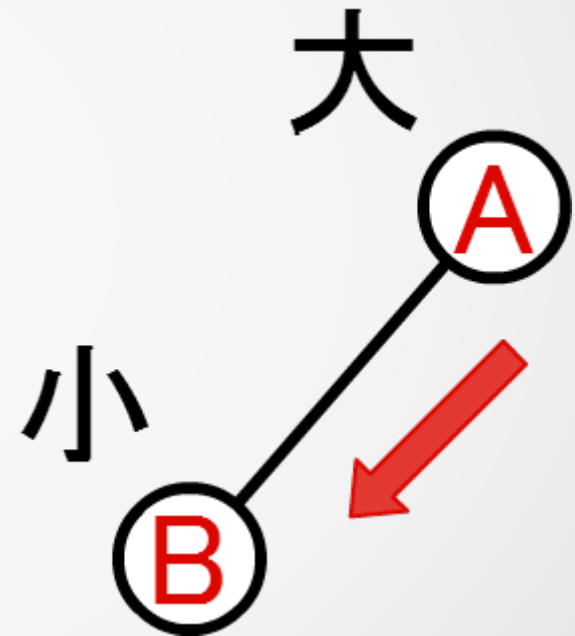


小課題4解法

- Anna
 - 各辺についてどちら向きに進むべきか求める
 - 頂点Xと頂点Yを結ぶ辺について
 - 書かれた整数が同じならば頂点番号が大きい方へ進むべき
 - 書かれた整数が違うならば頂点番号が小さい方へ進むべき

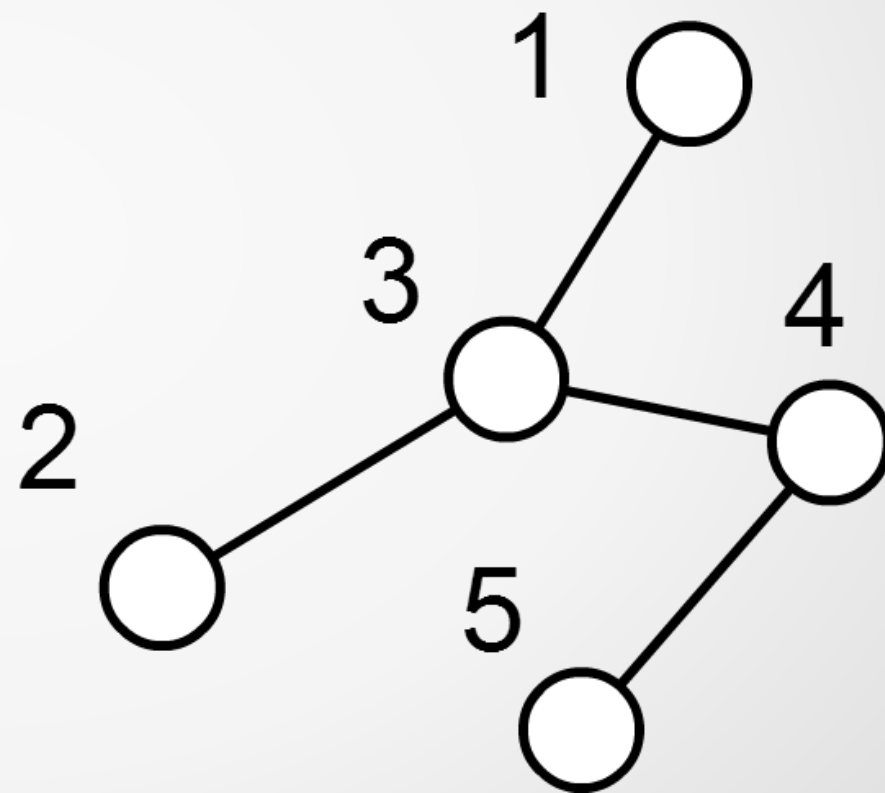
となるように整数を書く

どうやって？



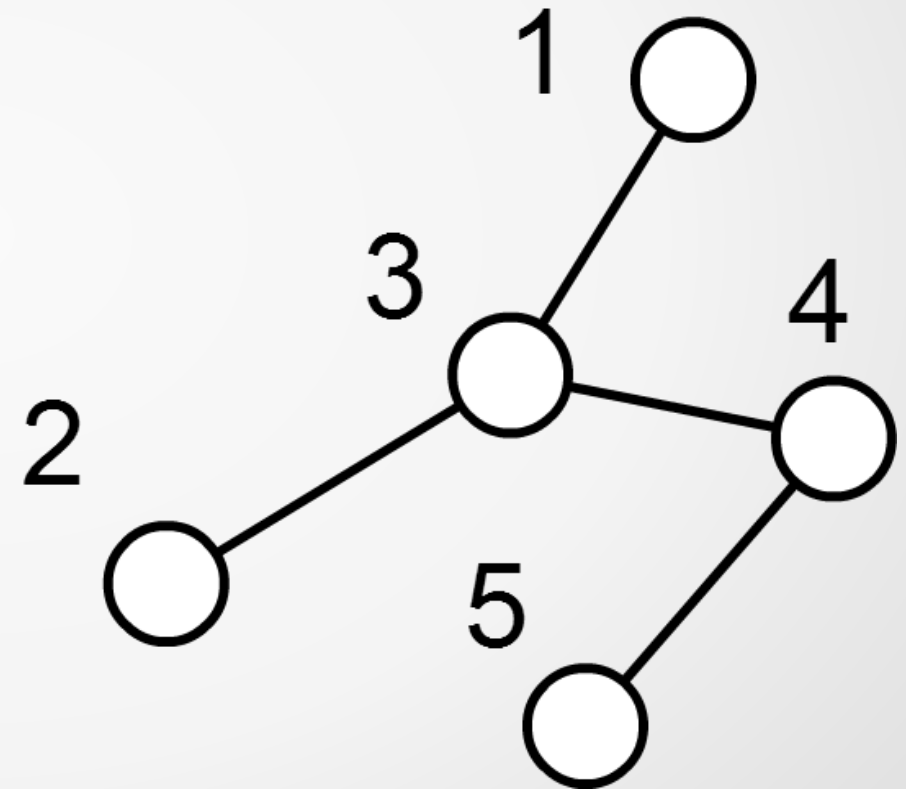
小課題4解法

- Tに近い順番に頂点に整数を書いていけば良い



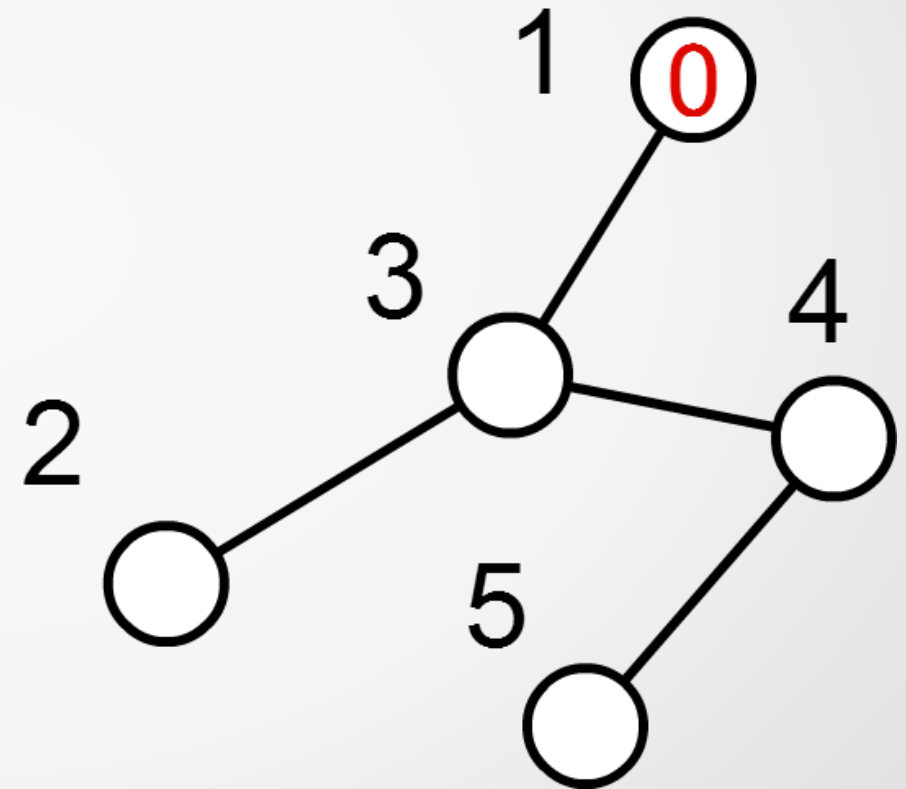
小課題4解法

- Tに近い順番に頂点に整数を書いていけば良い
- ある辺について情報を埋めたいときに必ず片方のみが決まっている
 - 2通りの埋め方が出来る
 - 0か1
 - 2通りの情報を伝えられる
 - 大きい方に進むべきか
小さい方に進むべきか



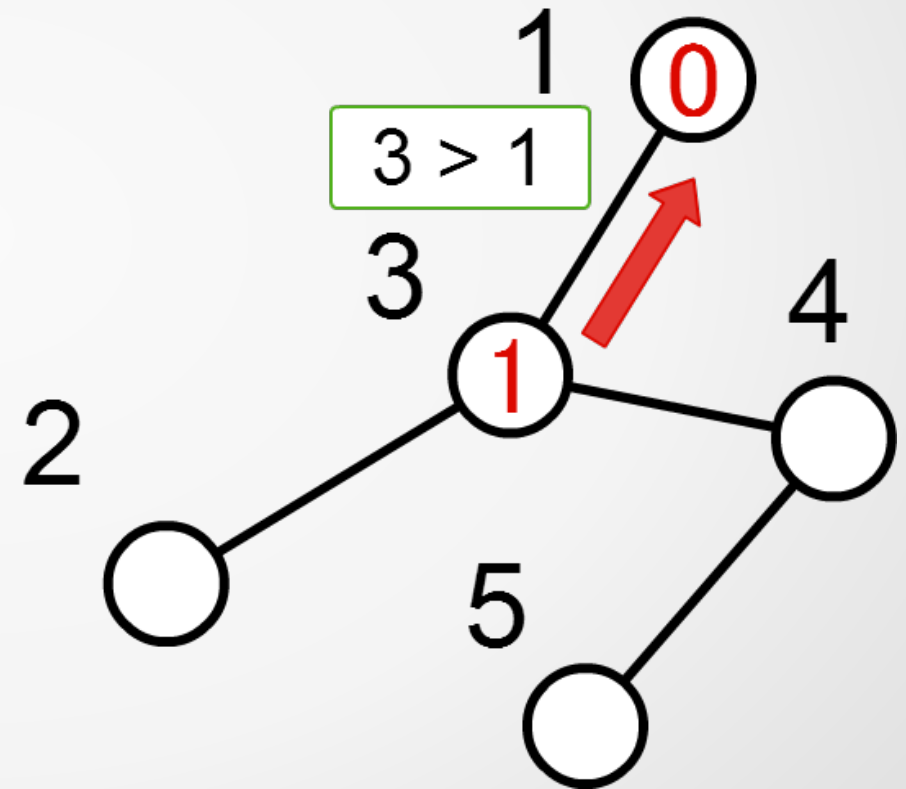
小課題4解法

- Tに近い順番に頂点に整数を書いていけば良い
- ある辺について情報を埋めたいときに必ず片方のみが決まっている
 - 2通りの埋め方が出来る
 - 0か1
 - 2通りの情報を伝えられる
 - 大きい方に進むべきか
小さい方に進むべきか



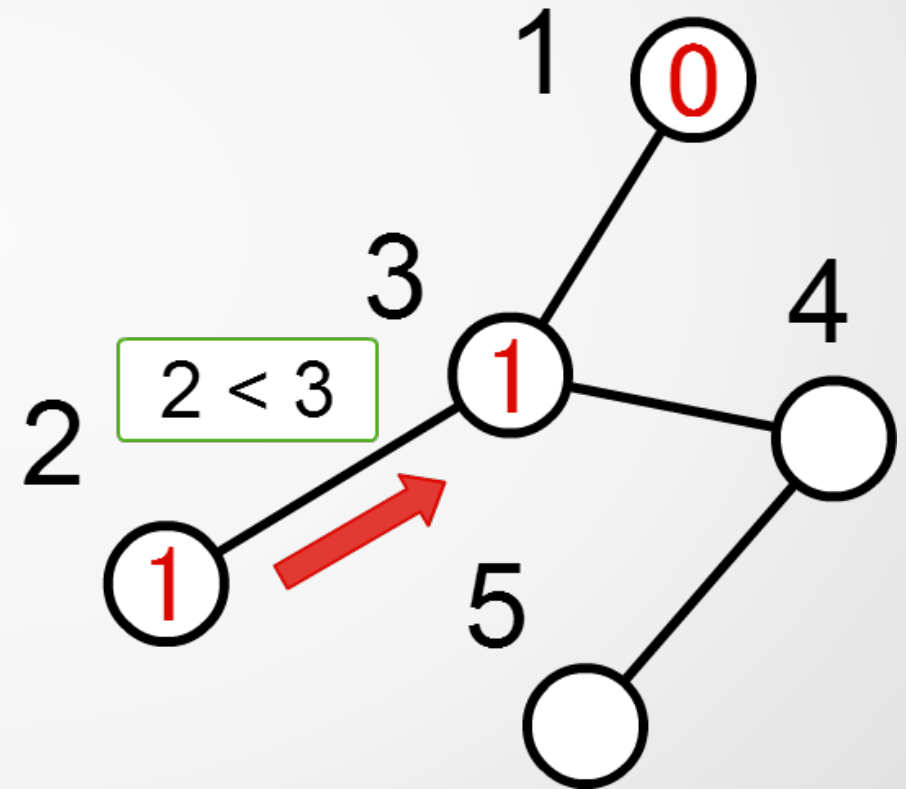
小課題4解法

- Tに近い順番に頂点に整数を書いていけば良い
- ある辺について情報を埋めたいときに必ず片方のみが決まっている
 - 2通りの埋め方が出来る
 - 0か1
 - 2通りの情報を伝えられる
 - 大きい方に進むべきか
小さい方に進むべきか



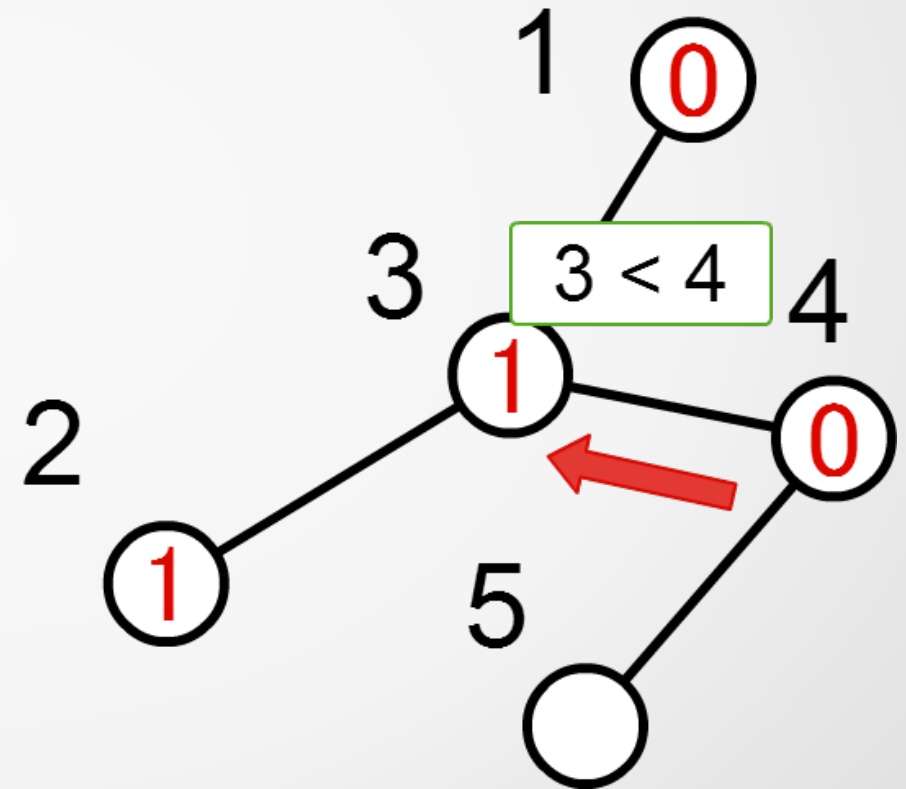
小課題4解法

- Tに近い順番に頂点に整数を書いていけば良い
- ある辺について情報を埋めたいときに必ず片方のみが決まっている
 - 2通りの埋め方が出来る
 - 0か1
 - 2通りの情報を伝えられる
 - 大きい方に進むべきか
小さい方に進むべきか



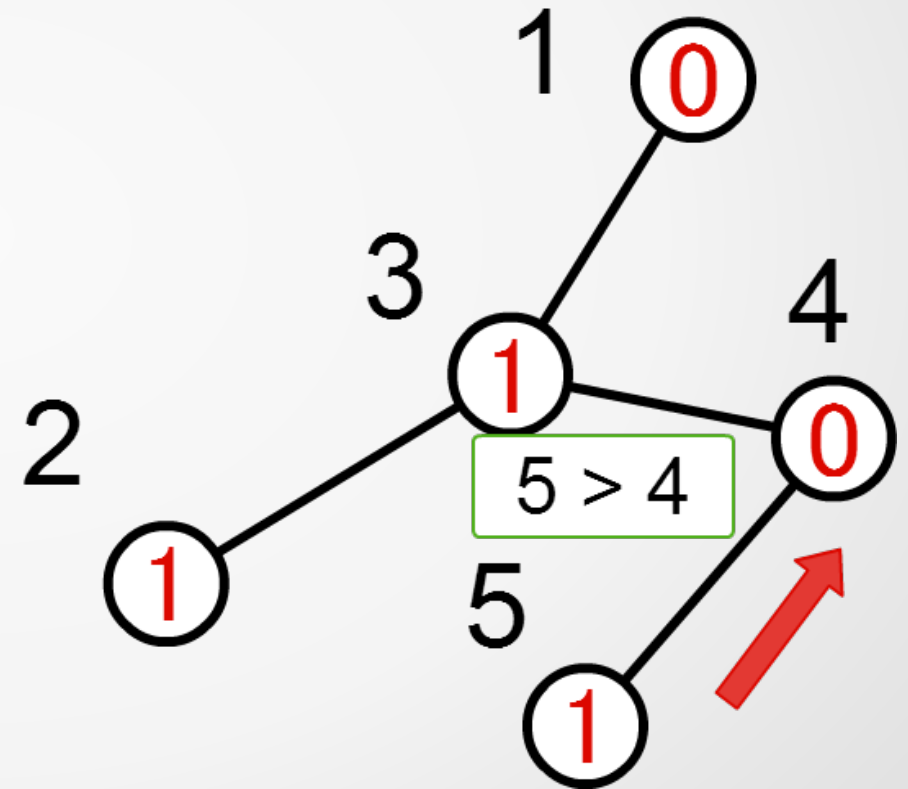
小課題4解法

- Tに近い順番に頂点に整数を書いていけば良い
- ある辺について情報を埋めたいときに必ず片方のみが決まっている
 - 2通りの埋め方が出来る
 - 0か1
 - 2通りの情報を伝えられる
 - 大きい方に進むべきか
小さい方に進むべきか



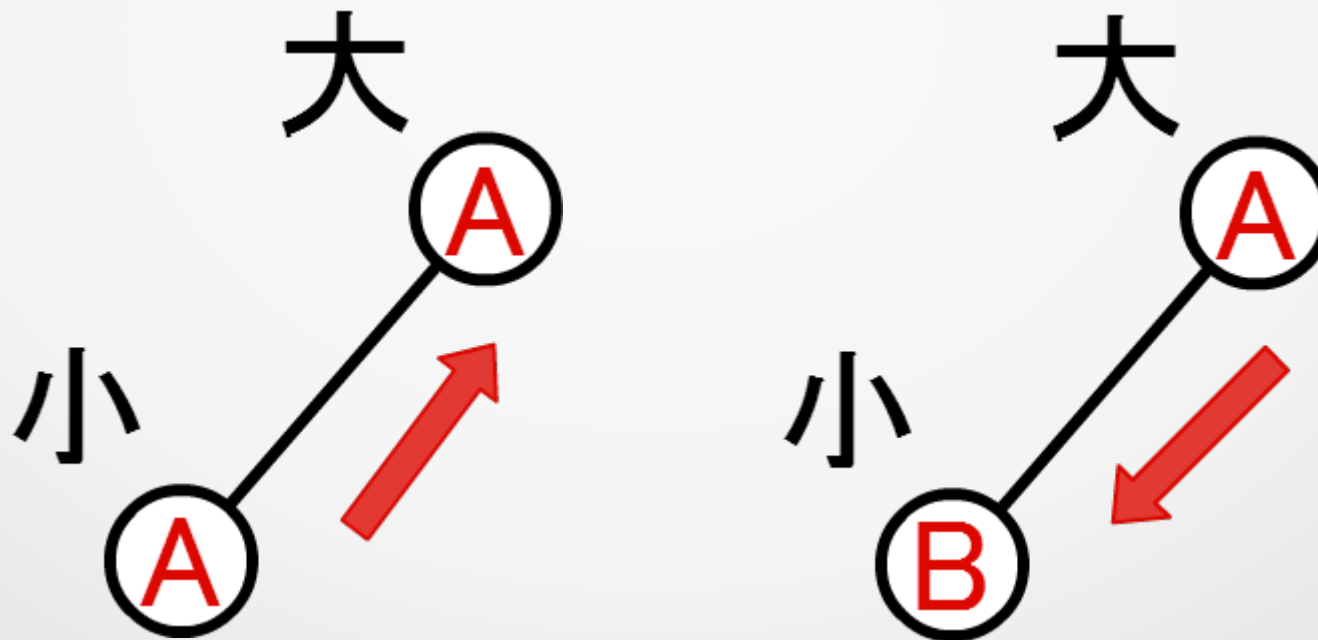
小課題4解法

- Tに近い順番に頂点に整数を書いていけば良い
- ある辺について情報を埋めたいときに必ず片方のみが決まっている
 - 2通りの埋め方が出来る
 - 0か1
 - 2通りの情報を伝えられる
 - 大きい方に進むべきか
小さい方に進むべきか



小課題4解法

- Bruno
 - 隣接する各辺について調べて、外向きに進むべき辺があるならば、その通り進む
 - そのような辺がなければ $S = T$



小課題4実装

- Anna
 - 各頂点*i*について*T*からの距離*d[i]*を求める (DFS)
 - 頂点*T*に0を書く
 - *d[i]*が小さい順に頂点をみていく
 - 隣接する辺の頂点のうちすでに整数が書かれている頂点(*b*)が一つだけある
 - その値に応じて、今いる頂点(*a*)の整数を書く
 - 頂点*x*の頂点番号を*p[x]*, 書かれた整数を*q[x]*とすると
 - $\text{if}(q[a] < q[b]) p[a] = p[b]$
 - $\text{if}(q[a] > q[b]) p[a] = !p[b]$

小課題4実装

- Anna
 - 各頂点*i*について*T*からの距離*d[i]*を求める (DFS)
 - 頂点*T*に0を書く
 - *d[i]*が小さい順に頂点をみていく
 - 隣接する辺の頂点のうちすでに整数が書かれている頂点(*b*)が一つだけある
 - その値に応じて、今いる頂点(*a*)の整数を書く
 - 頂点*x*の頂点番号を*p[x]*, 書かれた整数を*q[x]*とすると
 - $\text{if}(q[a] < q[b]) p[a] = p[b]$
 - $\text{if}(q[a] > q[b]) p[a] = !p[b]$

極めて軽実装

小課題4実装

- Bruno
 - 今いる頂点(A), 隣の頂点(B)
 - `if(p[A] == p[B] && q[A] < q[B])` Bに進む
 - `if(p[A] != p[B] && q[A] > q[B])` Bに進む
 - どこにも進まなかったら aにとどまる

小課題4実装

- Bruno
 - 今いる頂点(A), 隣の頂点(B)
 - if($p[A] == p[B] \ \&\& \ q[A] < q[B]$) Bに進む
 - if($p[A] != p[B] \ \&\& \ q[A] > q[B]$) Bに進む
 - どこにも進まなかったら aにとどまる

極めて軽実装

Navigation総評

- 考察
 - 難しい

Navigation総評

- 考察
 - 難しい
- 知識
 - 特に難しいものは必要ない

Navigation総評

- 考察
 - 難しい
- 知識
 - 特に難しいものは必要ない
- 実装
 - 極めて軽い

ところで

去年のIOIに
Game
という問題が出ました

IOI2014 Day1 Game

- 考察
 - 難しい

IOI2014 Day1 Game

- 考察
 - 難しい
- 知識
 - グラフと会話ができる程度

IOI2014 Day1 Game

- 考察
 - 難しい
- 知識
 - グラフと会話ができる程度
- 実装
 - 極めて軽い

IOI2014 Day1 Game

- 1行で解ける

じゃっぶる/wrong
@japlj⚙️Following

Game の満点解法情報です #IOI2014

```
void initialize(int n) { }  
int c[1500];  
int hasEdge(int u, int v) {  
    return ++c[max(u, v)] == max(u, v);  
}
```

[🌐 View translation](#)

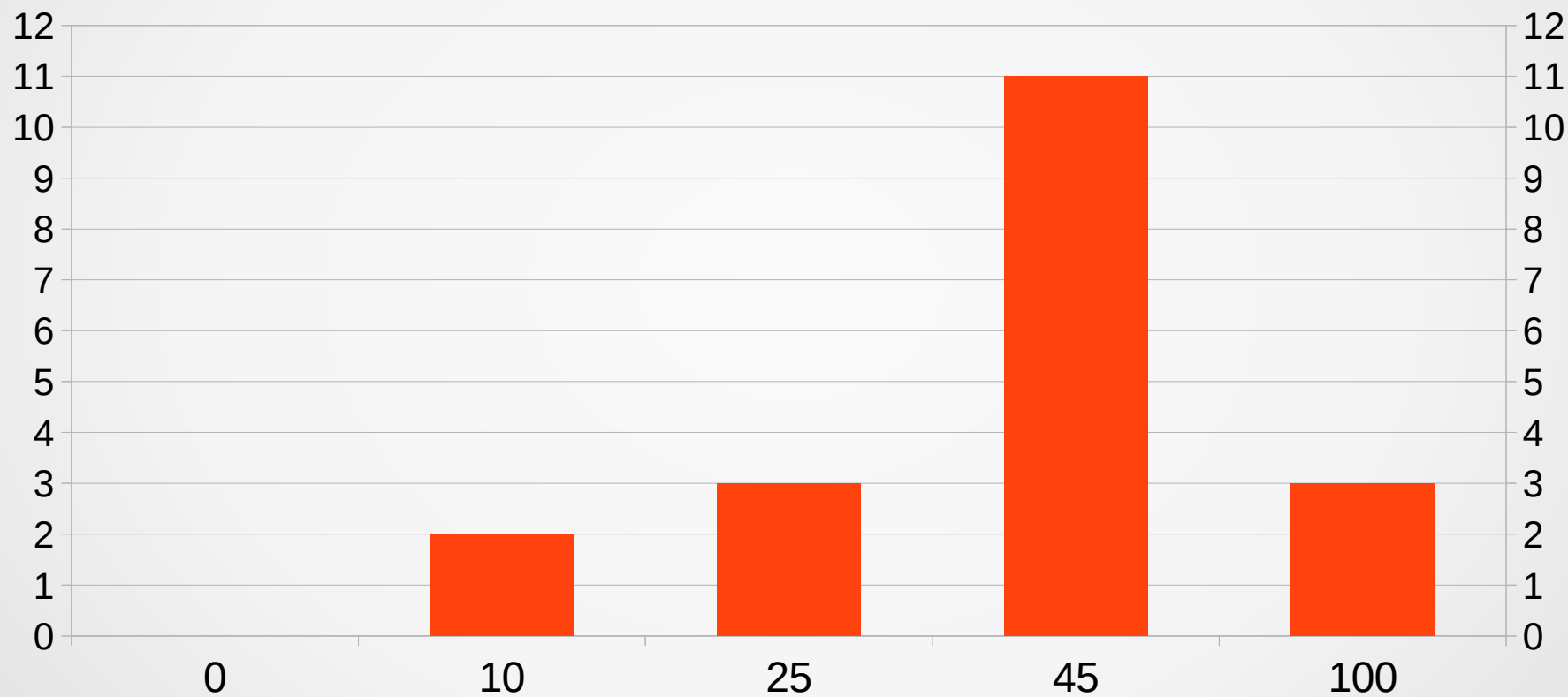
🔄 🔄 ⭐ ⋮

教訓

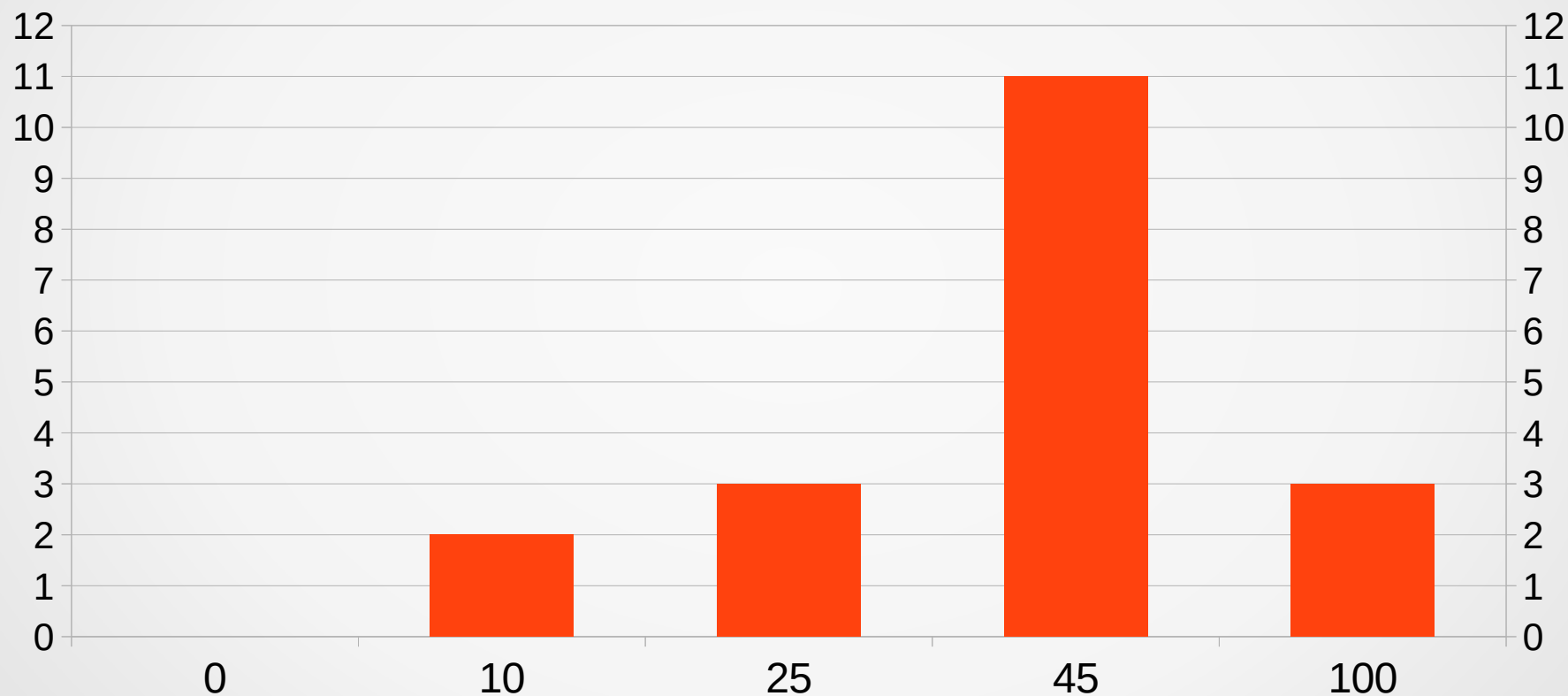
春合宿やIOIにはひらめくだけの問題
が出ることもある

実装が軽いという理由で解の候補から
外すのはよくない

得点分布



得点分布



Great!!!

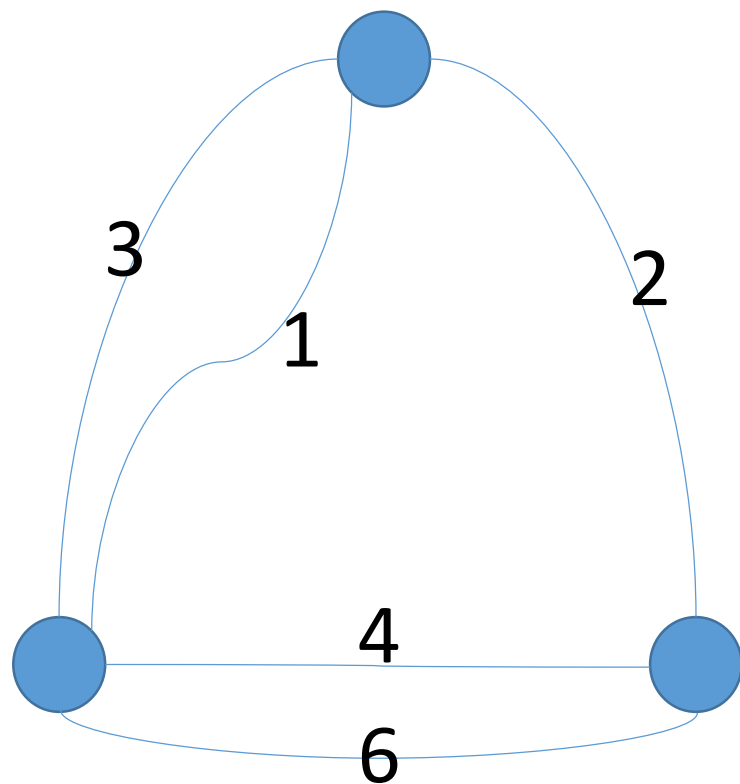
遺産相続(Inheritance)

笠浦一海

問題概要

- 頂点数 $N(\leq 1000)$ 、辺数 $M(\leq 300000)$ のグラフがある。
- 辺には重みがついてる
- 『閉路を含まない』という条件下で重み最大の辺集合を見つけ、それをグラフから取り除く
- 上を $K(\leq 10000)$ 回おこなう
- それぞれの辺が何回目に取り除かれたかをこたえよ

問題概要(例)



子供1 : $3 + 6 = 9$

子供2 : $4 + 2 = 6$

子供3 : 1

条件を言い換える

- 辺の重みはすべて正 → 取れる辺がなくなるまで辺をとる
- 『閉路を含まない』という条件でたくさん辺をとりたい
 - 各連結成分に対して全域木をとればよい
- 最大全域木をもとめる
 - 辺の重みを $\times (-1)$ して最小全域木を求める
 - Kruscal 法, Prim 法

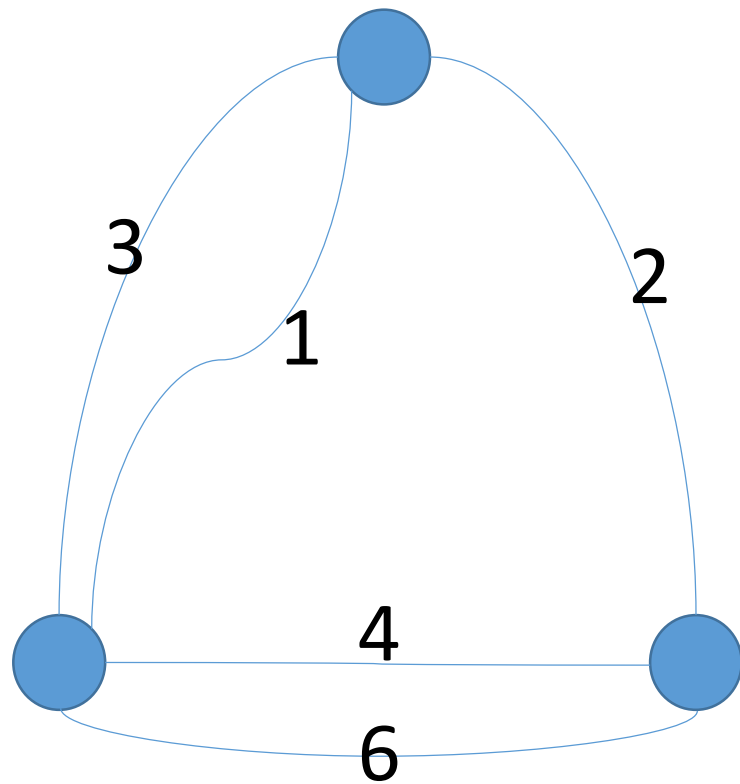
プリム法

- 詳細は蟻本などを参照
- 隣接行列で実装: $O(E + V^2)$
- vectorとpriority_queueで実装: $O(E \log V)$
- この問題では K 回行うので $O(KM + KN^2)$ か $O(KM \log N)$
- 前者はうまく実装すると $O(M \log M + KN^2)$
- 隣接行列のほうが早い
- なんにしろ15点

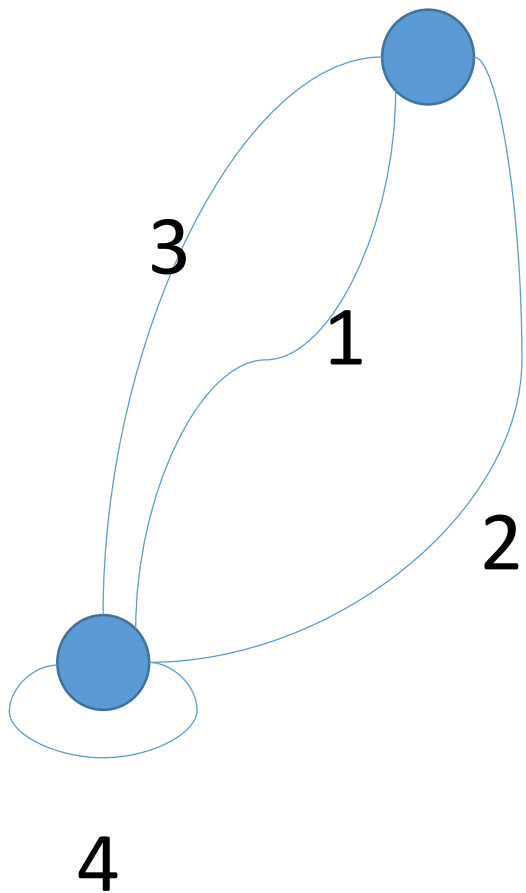
クラスカル法

- 重みの大きい辺から貪欲にとっていく
- ただし閉路ができてしまう場合は飛ばす
- 閉路ができるかの判定は Union Find を使う

クラスカル法

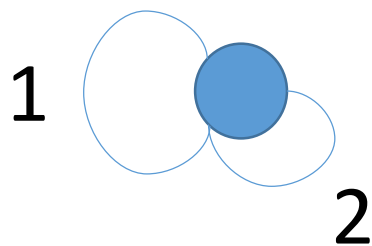


クラスカル法



子供1:6

クラスカル法



子供1: 6+3

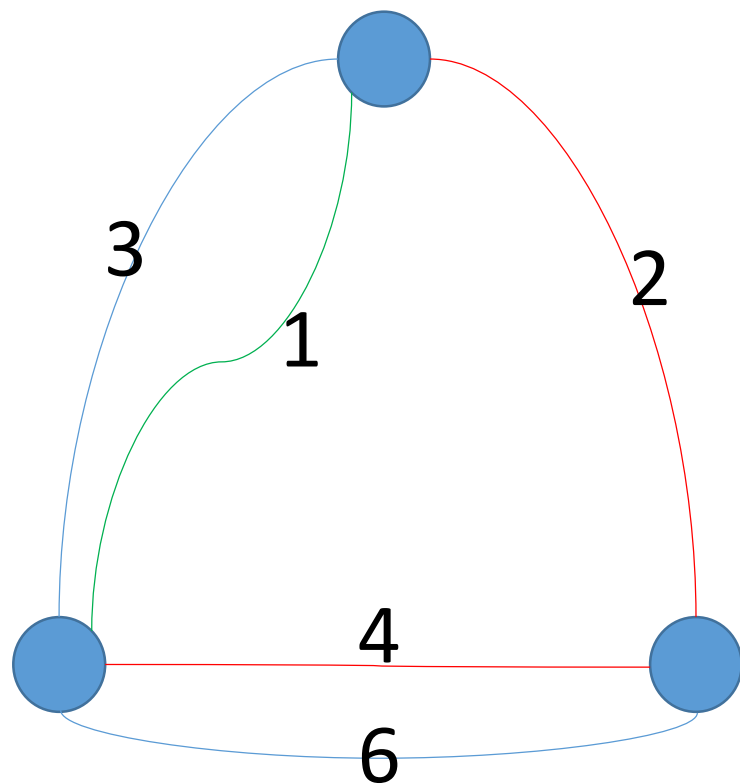
クラスカル法

- 計算量は辺のソートに $O(E \log E)$ 貪欲にとるのに $O(E \alpha(V))$
- ただし α は逆アッカーマン関数という増加の遅い関数
- 辺のソートは最初に1回行えばいいので、この問題では計算量は $O(M \log M + KM\alpha(N))$
- Subtask 1 のみ15点

クラスカル法解法を改変

- 各子供ごとに、辺を渡るループを回す
- 順番を変えて、各辺に対して、子供を渡るループを回してもよい
- すなわち重みの大きい辺から順番に、誰に相続されるかを決めていく
- 子供1から順番に見て行って、最初に相続できる人を見つける
- 相続できる $\Leftrightarrow$ 追加しても閉路ができない
- $\Leftrightarrow$ その辺が結ぶ二頂点はつながっていない

クラスカル法解法を改変(例)



辺6: 子供1○

辺4: 子供1× → 子供2○

辺3: 子供1○

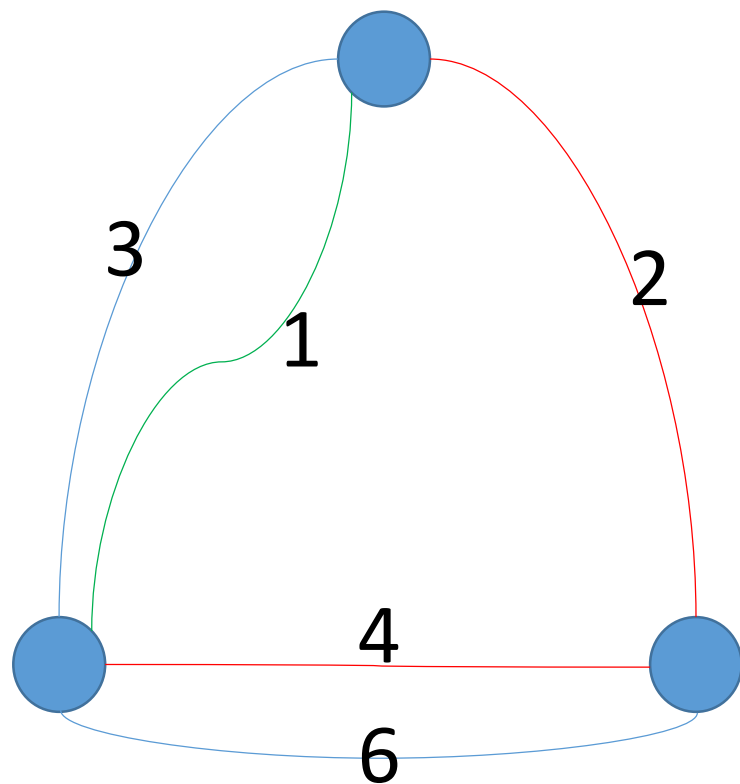
辺2: 子供1× → 子供2○

辺1: 子供1× → 子供2× → 子供3○

クラスカル法解法を改良

- K個のUF木が必要→空間計算量 $O(KN)$ が必要
- このままだとまだ計算量は $O(M \log M + KM\alpha(N))$
- 相続する人を**二分探索**で探せばよさそう

二分探索(例)



辺6: 子供2○→子供1○

辺4: 子供2○→子供1×

辺3: 子供2○→子供1○

辺2: 子供2○→子供1×

辺1: 子供2×→子供3○

二分探索の正当性

- 二分探索は正当か？
- 「子供 i が相続できる $\Rightarrow$ 子供 j も相続できる ($i < j$)」が成り立てばよい
- すでに相続先が決まっている辺のうち、子供 i が相続する辺の集合を T_i とおく
- 「 a と b が T_j によってつながっている $\Rightarrow$ a と b が T_i によってつながっている ($i < j$)」が任意の時点で成り立てばよい

二分探索の正当性の証明

- これが成り立つことを帰納法で示す
- 初期状態では明らかに成り立っている
- 頂点 c , d を結ぶ辺 α が T_k に追加されるとする
- このとき k は相続できる子供の番号の最小値なので、 c , d は $T_1, \dots, T_{k-1}$ によってつながっているはずである
- よって「 a と b が T_j によってつながっている $\Rightarrow a$ と b が T_i によってつながっている ($i < j$)」の条件は保たれている
- Q. E. D.

満点解法

- どの子供が相続するか各辺に対して二分探索を行う。
- 時間計算量: $O(M \log M + KN + M \log K \alpha(N))$
- 空間計算量: $O(M + KN)$
- 時間はOK
- $K \leq 10^5$, $N \leq 10^3$ より $KN \leq 10^7$
- UF木はintの配列だけで実装できる
- 大きさ N の int の配列 K 個で十分なので空間計算量もOK

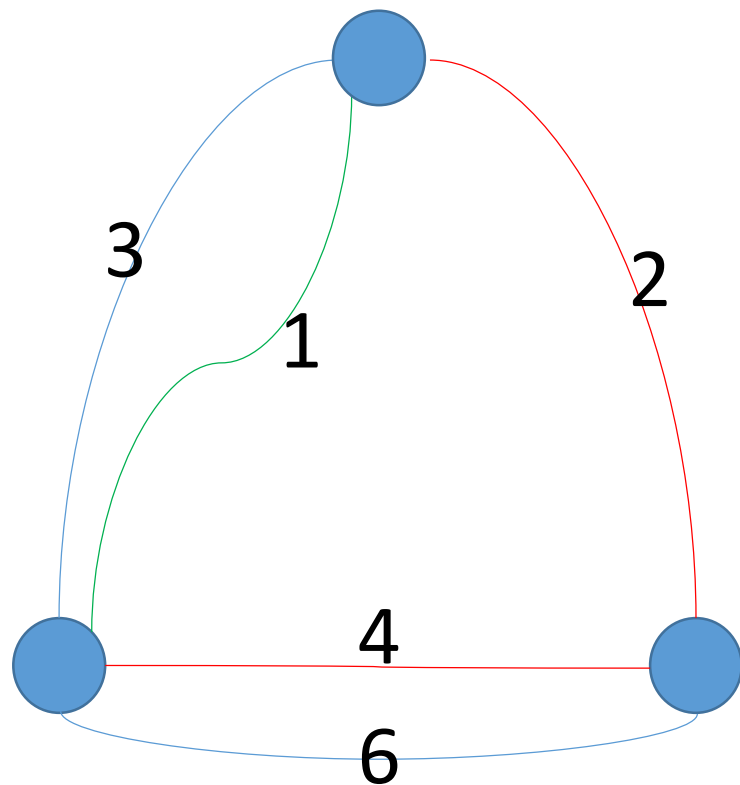
$+\alpha$

- UF木のために KN の領域を確保するが、そのうち実際の計算で使うのは $O(M \log K)$ のみ
- 動的にメモリを確保すれば節約できそう
- はじめに大きめの配列を確保し、配列の番号をmapで管理
- 各子供ごとにmapを用意すれば、それぞれのmapの大きさは高々 N
- 時間計算量: $O(M \log M + M \log K \log N)$
- 空間計算量: $O(K + M \log K)$
- 実装をうまくやらないと間に合わない

別解

- 基本方針は満点解法と同じ
- 二分探索を行う代わりに次の高速化を行う
- 頂点 a, b を結ぶ辺 α が子供 i に相続されたとする
- 次に頂点 a, b を結ぶ辺 β が出てきたとき、子供 $1, \dots, i$ は確実に相続できない！
- 各頂点のペア $\{a, b\}$ に対し、 a, b を結ぶ辺のうち、直前に出てきたものを相続した子供の番号を記録
- 次に a, b を結ぶ辺が出てきたときは、その次の番号から調べる

別解(例)



辺6: 子供1○

辺4: 子供2○

辺3: 子供1○

辺2: 子供1 × → 子供2○

辺1: 子供2 × → 子供3○

別解

- 実はこれだけで計算量がへる！
- $O(KN + N^2 + MVM\alpha(N))$ とかになる
- (評価の仕方はいろいろある)
- ポイント:「頂点 a, b を結ぶある辺を子供 i が相続するか調べる」事象を考える
- a, b, i について 1 回きり $\rightarrow O(KN^2)$ はわかる

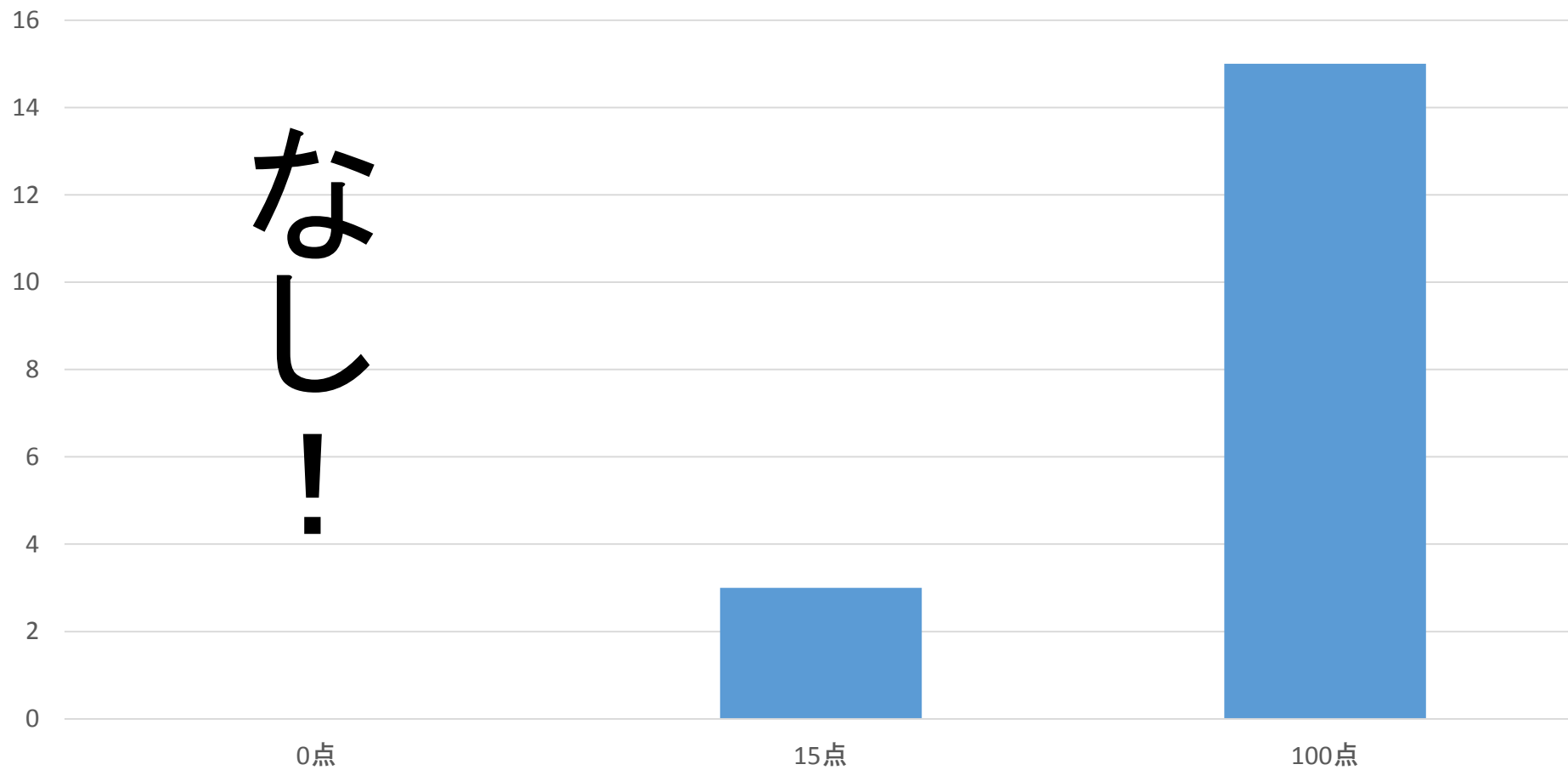
別解

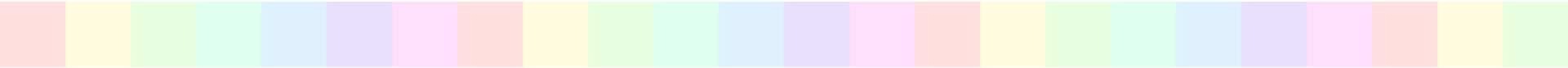
- この事象が起きるとき、最終的に子供 i が相続する辺において a, b はつながっている
- よって i を固定した時の a, b の場合の数は(子供 i が相続する辺の数 $+1$)² で抑えられる
- これを利用してうまく評価すればよい(詳細は略)

別解(いま考えた)

- $O(KN^2)$ のプリム法を高速化
- 各頂点に対し、連結している頂点の集合をsetかvectorで管理
- 同じ理由で早くなるはず.....？

得点分布

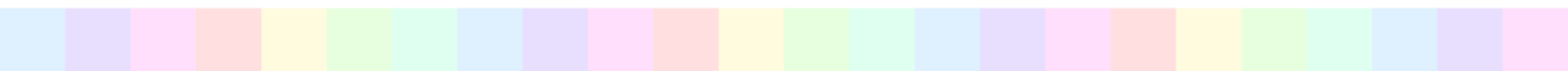




記憶縛り (Limited Memory)

JOI 春合宿 2015 Day 4

解説： 保坂 和宏



問題概要

- 良い文字列 (括弧の対応 OK)

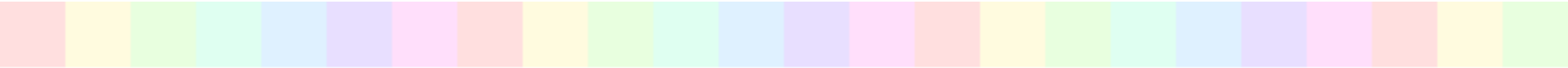
<> [] [<>] <>

- 良い文字列ではない

>< [<] >

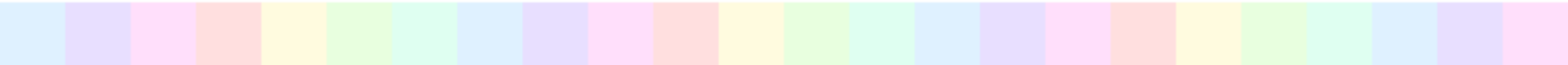
問題概要

- S が良い文字列か否か判定せよ
 - ただし 1 文字ずつしか聞けない！
 - 22 ビットしか記憶してられない
- 制約
 - 長さ $1 \leq N \leq 100$
 - 聞ける回数 15,000 回



考察

- そもそも普通に入力全部読めたら？



普通に読めたら

- 定義に従って区間 DP

- $dp[i][j] :=$ 位置 i から位置 j までの部分文字列が良い文字列であるか？

- $O(N^3)$ 時間

- 空文字列 (つまり, 長さ 0 の文字列) は良い文字列である.
- x が良い文字列ならば, $\langle x \rangle$ (つまり, x を山括弧 $\langle \rangle$ で囲った文字列) は良い文字列である.
- x が良い文字列ならば, $[x]$ (つまり, x を角括弧 $[]$ で囲った文字列) は良い文字列である.
- x, y が良い文字列ならば, xy (つまり, x, y をこの順に連結した文字列) は良い文字列である.
- 以上で良い文字列と定義されるもののみが良い文字列である.

普通に読めたら

- スタックの利用
 - 開き括弧が来たらスタックに push
 - 閉じ括弧が来たら
 - スタックが空なら NO
 - スタックの top と対応していなかったら NO
 - 対応していたら pop して続ける
- 最後にスタックが空なら YES
- $O(N)$ 時間

普通に読めたら

- スタックの利用

[<>]<>

スタック：

普通に読めたら

- スタックの利用

[<>] <>

スタック : [

普通に読めたら

- スタックの利用

[< >] < >

スタック : [<

普通に読めたら

- スタックの利用

[< >] < >

スタック : [<

普通に読めたら

- スタックの利用

[<>] <>

スタック : [

普通に読めたら

- スタックの利用

[<>]<>

スタック：<

普通に読めたら

- スタックの利用

[<>]<>

スタック : <

小課題 1 ($N \leq 8$)

- 8 文字くらいなら全部覚えられそう



今何文字目？
0 から 8 まで
(4 ビット)

文字列の内容
1 文字 4 ($= 2^2$) 種類
(16 ビット)

小課題 1 ($N \leq 8$)

Memory(N, M)

M から今何文字目かと文字列の内容を読み取る

エラーなら適当に処理する

今 N 文字目なら良い文字列か判定する

次の文字を Get する

エンコードして返す

小課題 2 ($N \leq 14$)

- スタックの利用
 - 開き括弧が来たらスタックに push
 - 閉じ括弧が来たら
 - スタックが空なら NO
 - スタックの top と対応していなかったら NO
 - 対応していたら pop して続ける
- 最後にスタックが空なら YES
- スタックだけ覚えておけばできる！

小課題 2 ($N \leq 14$)

- スタック 14 文字くらいなら全部覚えられそう



今何文字目？ スタックの長さ
0 から 14 まで 0 から 14 まで
(4 ビット) (4 ビット)

スタックの内容
1 文字 2 種類
(14 ビット)

小課題 3 ($N \leq 24$)

- スタックの利用
 - ◻ 開き括弧が来たらスタックに push
 - ◻ 閉じ括弧が来たら
 - スタックが空なら NO
 - スタックの top と対応していなかったら NO
 - 対応していたら pop して続ける

最後にスタックが空なら YES
- スタックが大きくなりすぎたら NO 確定
 - ◻ 高々 $N/2$ 文字

小課題 3 ($N \leq 24$)

- スタック 12 文字くらいなら全部覚えられそう



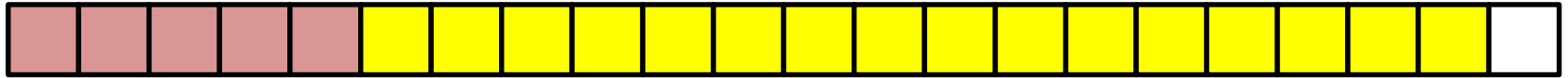
今何文字目？
0 から 24 まで
(5 ビット)

スタックの長さ
0 から 24 まで
(5 ビット)

スタックの内容
1 文字 2 種類
(12 ビット)

小課題 4 ($N \leq 30$)

- スタック 15 文字くらいなら全部覚えられそう



今何文字目？

0 から 30 まで
(5 ビット)

スタックの内容

長さ 0 から 15 まで・文字 1 文字 2 種類
(16 ビット)

小課題 4 ($N \leq 30$)

- スタック 15 文字くらいなら全部覚えられそう ???
- 0 文字以上 15 文字以下の $<$ と $[$ の列は $2^0 + 2^1 + \dots + 2^{15} = 2^{16} - 1$ 通りしかない
◻ 適切に番号を振れば 16 ビットで覚えられる

小課題 4 ($N \leq 30$)

← 空文字列

<

[

<<

<[

[<

[[

<<<

<<[

<[<

...

小課題 4 ($N \leq 30$)

← 空文字列

0

1

00

01

10

11

000

001

010

...

小課題 4 ($N \leq 30$)

00000**1**

0000**1**0

0000**1**1

000**1**00

000**1**01

000**1**10

000**1**11

00**1**000

00**1**001

00**1**010

...

小課題 5 (<, > のみ)

- スタックの中身が < だけ→長さで決まる



今何文字目？
0 から 100 まで
(7 ビット)

スタックの長さ
0 から 100 まで
(7 ビット)

満点解法

- スタックを全部覚えるのはとても無理
 - ◻ 良い文字列なら最大 50 文字

満点解法

- スタックを一部覚えればいいのか？
 - スタックの 1 文字目だけ覚える
 - スタックの 2 文字目だけ覚える
 - スタックの 3 文字目だけ覚える

.....

というのを別々にやればよい

満点解法

- スタックの 1 文字目だけ覚える

[<>]<>

スタック：

満点解法

- スタックの 1 文字目だけ覚える

[<>] <>

スタック : [

満点解法

- スタックの 1 文字目だけ覚える

[< >] < >

スタック : [?

満点解法

- スタックの 1 文字目だけ覚える

[<>]<>

スタック : [?

満点解法

- スタックの 1 文字目だけ覚える

[<>] <>

スタック : [

満点解法

- スタックの 1 文字目だけ覚える

[<>]<>

スタック : <

満点解法

- スタックの 1 文字目だけ覚える

[<>]<>

スタック : <

満点解法

- スタックの 2 文字目だけ覚える

[<>]<>

スタック：

満点解法

- スタックの 2 文字目だけ覚える

[<>]<>

スタック： ?

満点解法

- スタックの 2 文字目だけ覚える

[<>]<>

スタック : ?<

満点解法

- スタックの 2 文字目だけ覚える

[<>]<>

スタック : ?<

満点解法

- スタックの 2 文字目だけ覚える

[<>]<>

スタック： ?

満点解法

- スタックの 2 文字目だけ覚える

[<>]<>

スタック： ?

満点解法

- スタックの 2 文字目だけ覚える

[<>]<>

スタック： ?

満点解法

- スタックを一部覚えればいいのか？
 - スタックの 1 文字目だけ覚える
 - スタックの 2 文字目だけ覚える
 - スタックの 3 文字目だけ覚える
 -
- 「文字列 S を読む」を N 周
 - N^2 ステップくらい (15,000 回制限 OK)

満点解法

- k 周目はスタックの k 文字目だけ覚える



今何周目？
0 から 100 まで
(7 ビット)

今何文字目？
0 から 100 まで
(7 ビット)

スタックの長さ
0 から 100 まで
(7 ビット)

スタックの一部
1 文字 2 種類
(1 ビット)

満点解法

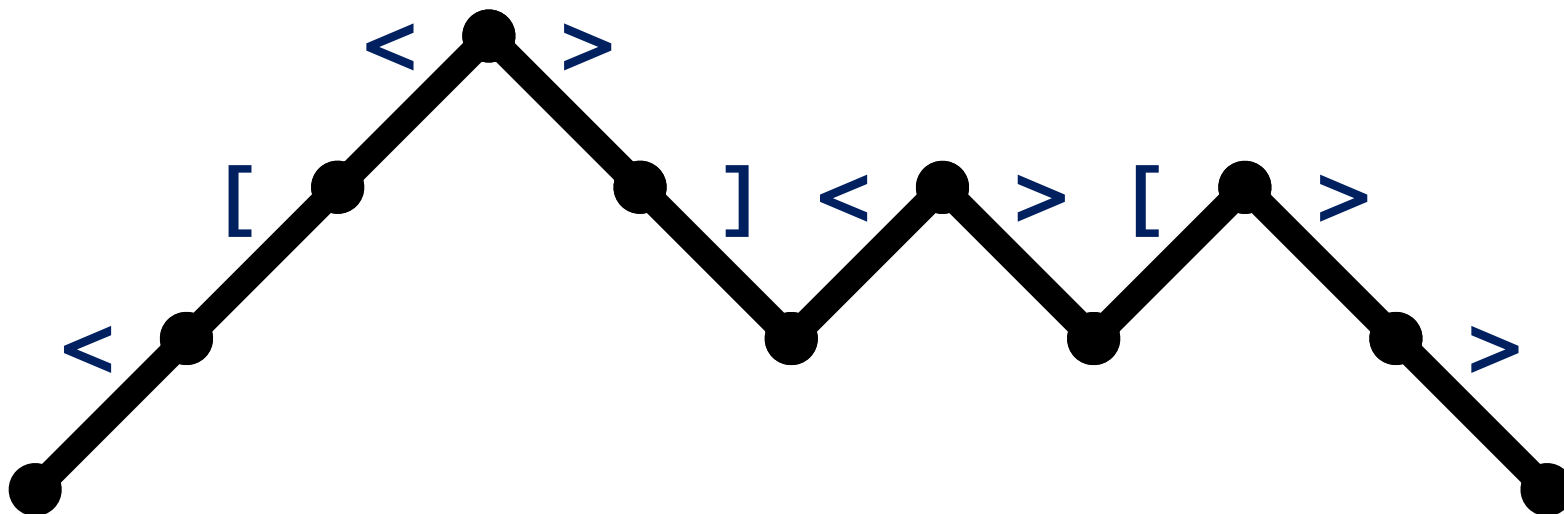
- 4 ビットくらい節約することもできる
 - スタックの長さは $N/2$ 以下としてよい
 - よって $N/2$ 周まででよい
 - $(\text{スタックの長さ}) \leq (\text{現在位置})$
 - $(\text{スタックの長さ}) \equiv (\text{現在位置}) \pmod{2}$

別解

- $k = 1, 2, \dots, N$ に対して
 - S の k 文字目が開き括弧なら
 - その開き括弧の種類を覚える
 - k 文字目から順番に見て行って、ネストの深さが対応する閉じ括弧を探す→括弧の種類が違っていたら NO
 - S の k 文字目が閉じ括弧なら
 - 左に向かって同様に
- 必要な記憶量は同じくらい

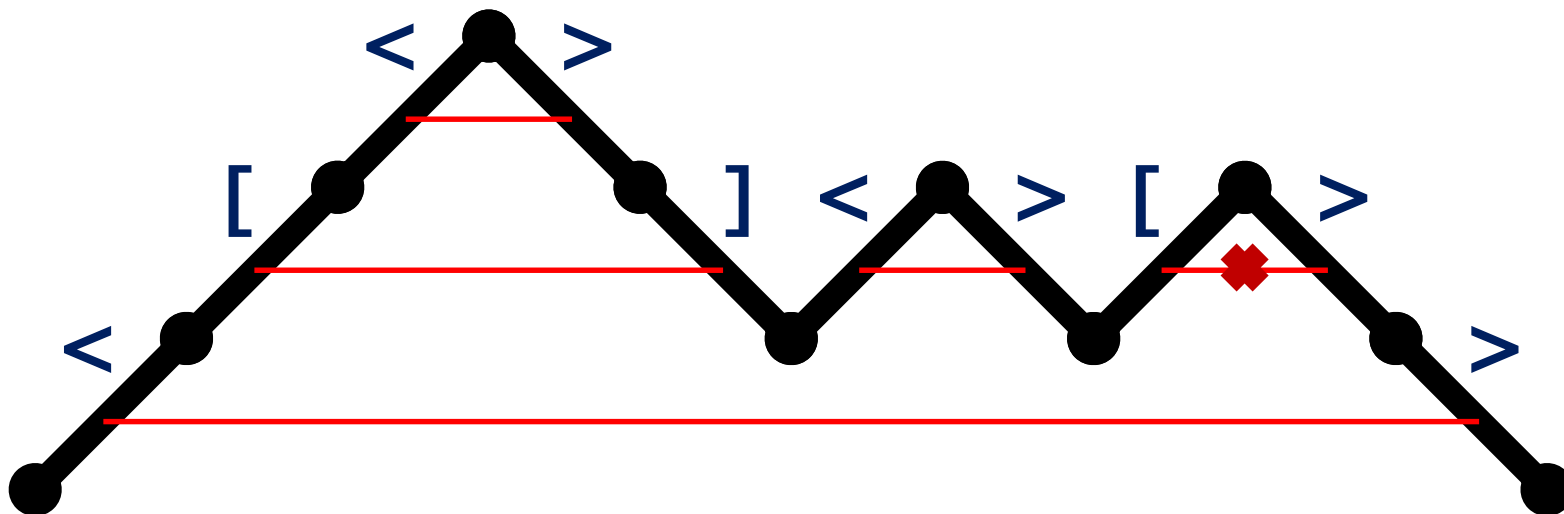
満点解法たち

$\langle [\langle \rangle] \langle \rangle [\rangle] \rangle$



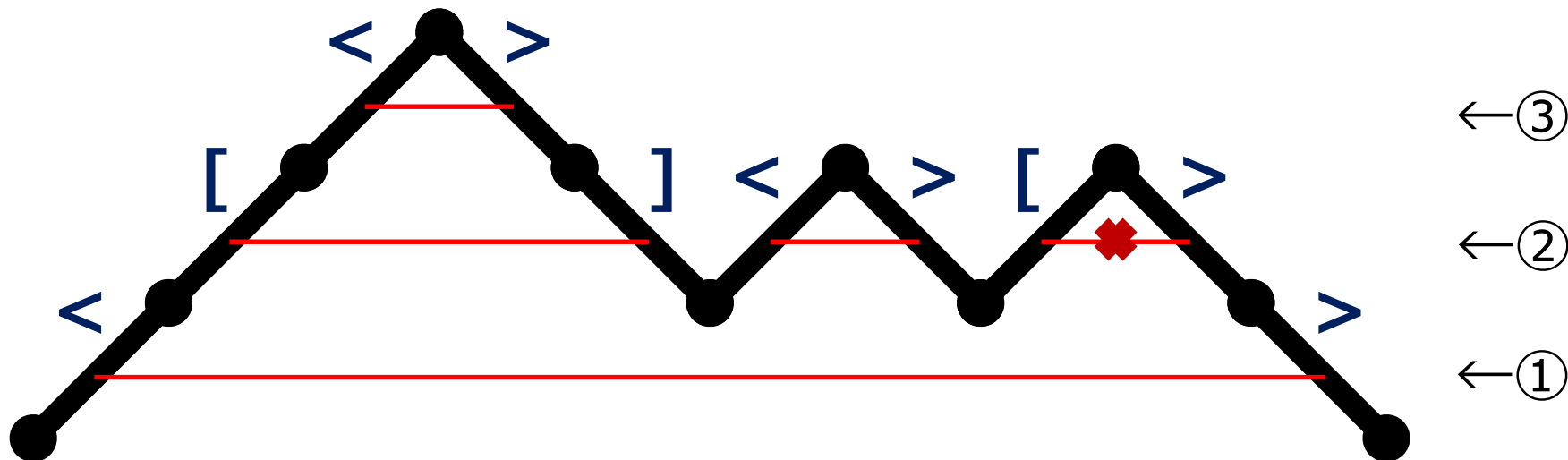
満点解法たち

$\langle [\langle \rangle] \langle \rangle [\rangle] \rangle$



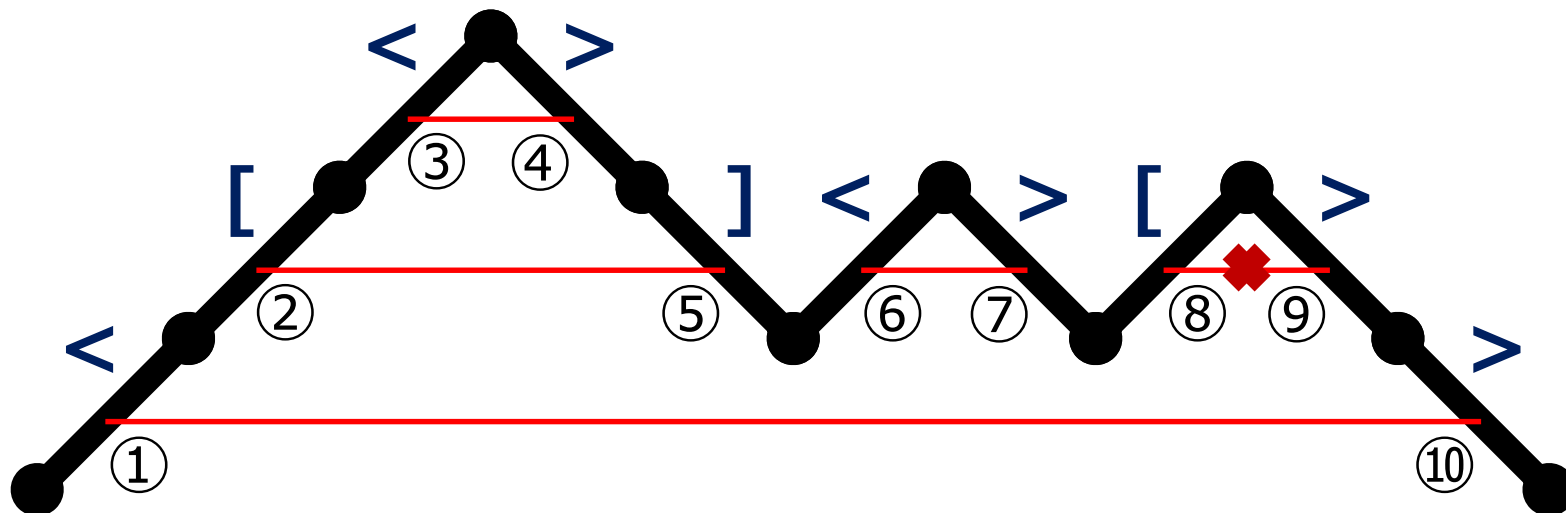
満点解法たち

$\langle [\langle \rangle] \langle \rangle [\rangle] \rangle$



満点解法たち

$\langle [\langle \rangle] \langle \rangle [\rangle] \rangle$



デバッグについて

- いろいろテストしないと正しく実装するのは難しいです
- 今回は「採点プログラムのサンプル」(a.k.a. grader) が 2 種類用意されていました
 - やりとりのデバッグ用 (grader-simple)
 - より実際の採点に近いほう (grader-strict)

デバッグについて

- 提供される grader が利用できないと不利
 - practice は重要
- ただし提供される grader 以外の機能がほしい場合もあるかも？
 - grader を改造する
 - 自分で main を書く

デバッグについて

(解答コード)

```
#ifndef EVAL
char Get(int I) {
    return '<';
}
int main(void) {
    printf("%d¥n", Memory(100, 4194303))
    return 0;
}
#endif
```

おまけ

- 空間計算量 $O(\log N)$ で括弧がとれているか判定しよう, という問題だった
- ではメモリ $O(1)$ でできるか?

おまけ

- 実は次のことが知られている

空間計算量 $O(1)$ で判定できる



正規言語である

- **正規言語** (regular language)

- 定義の例 : DFA で判定できる
- 参考 :

http://www.kmonos.net/wlog/115.html#_2300101221

おまけ

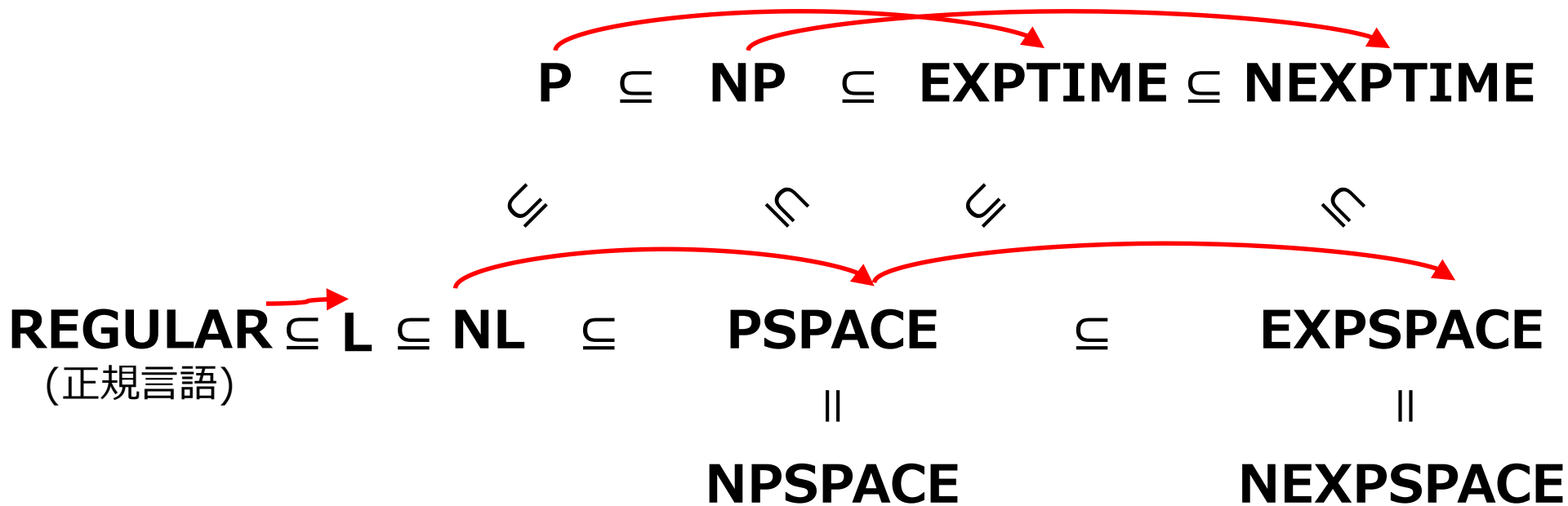
- 「括弧の対応がとれた文字列の集合」は正規言語ではない
 - ことが知られている (cf. pumping lemma)
 - 括弧の種類が 1 種類であっても正規でない
- よって空間計算量 $O(1)$ は無理
 - 実は正規言語でなければ $O(\log \log N)$ すら無理なことが知られているらしい

おまけ

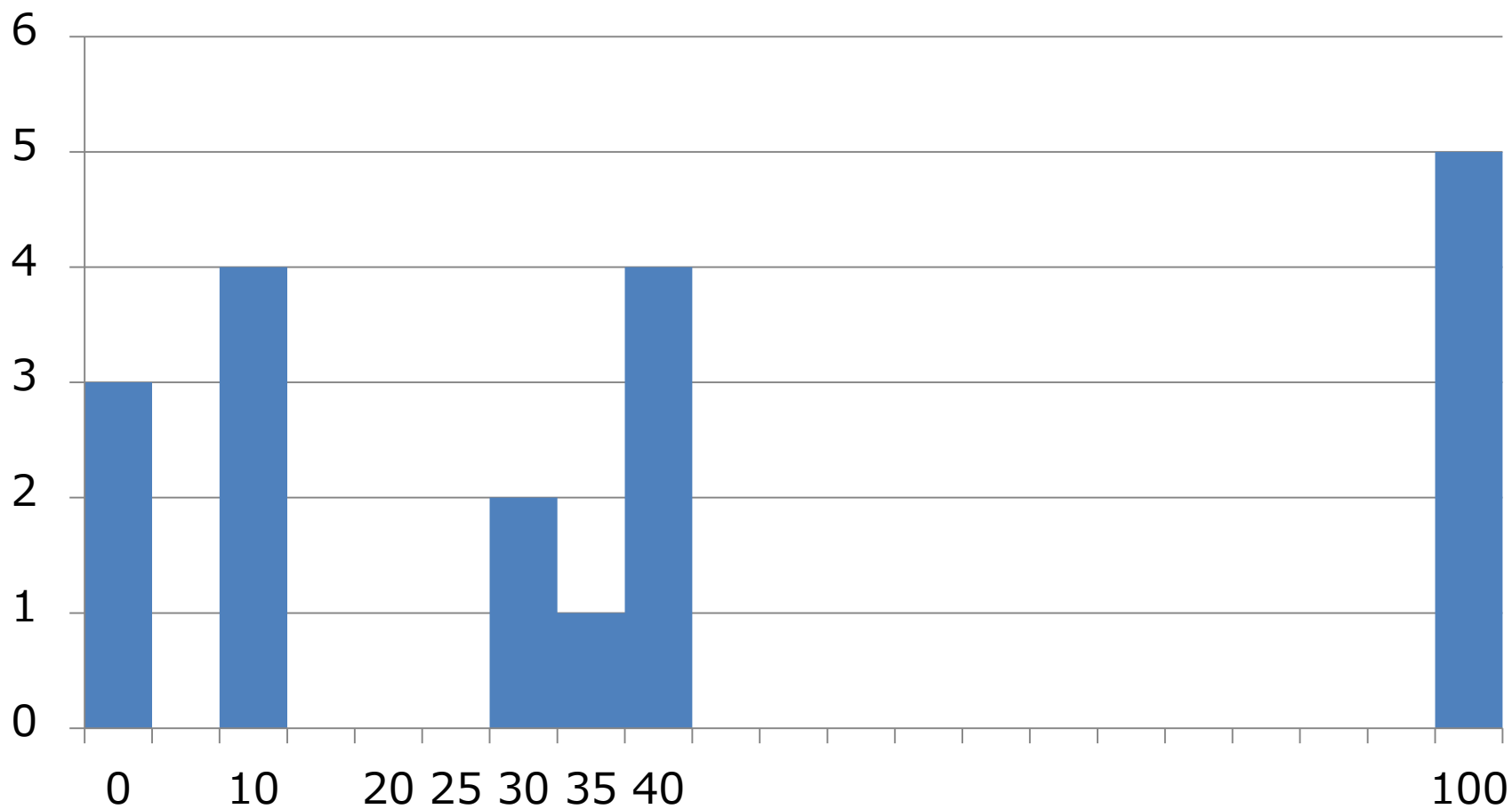
- $O(\log N)$ 空間で判定できる言語 (判定問題) のクラスは **L** と書かれる
 - **NL** というのもあります (非決定性 $O(\log N)$ 空間)
 - **P** とか **NP** とかくらい大事に違いない

おまけ

- 知られている包含関係
 - 赤矢印が真の包含 $\subsetneq$



得点分布



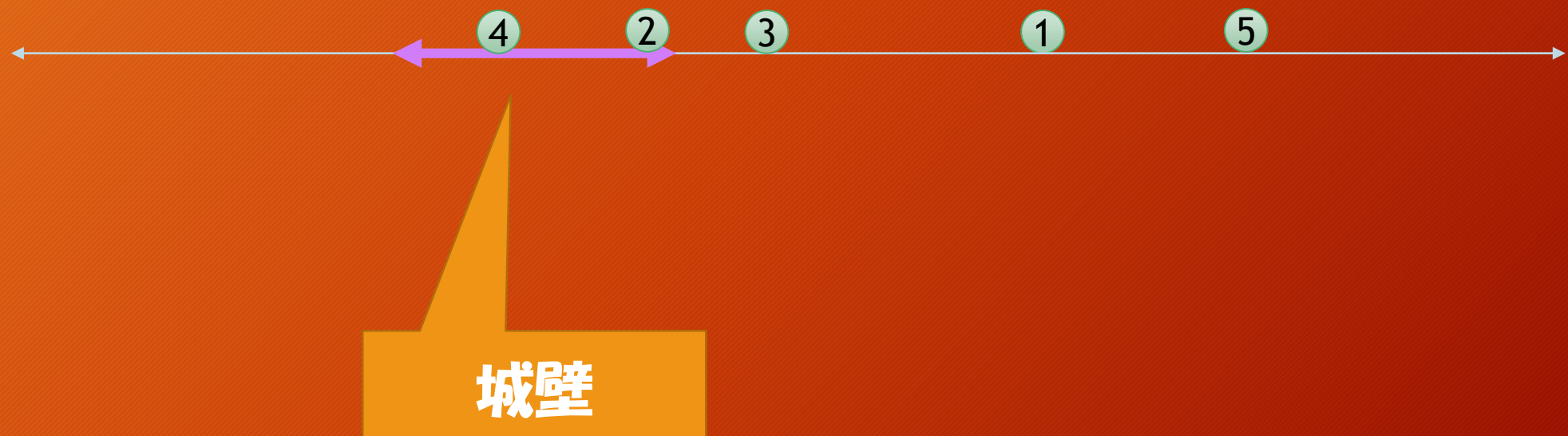
Walls(防壁) 解説

JOI 2014-2015 Spring camp Day4
semiexp

問題概要

- グリッド状の盤面がある
- 上からレーザー攻撃がたくさん行われる
- 防壁は左右に動かせるので、すべてのレーザー攻撃に対してすべての防壁をぶつけたい
- 各防壁は何回動かされるか？

問題概要(イメージ)



問題概要 (イメージ)



観察

- すべての壁をすべてのレーザーに当てる必要がある
- 壁同士は互いに関係しない
- 壁ごとに独立に解ける

自然な貪欲法

- レーザー攻撃を順番に見ていく
- 今の壁の位置で次のレーザー攻撃にぶつかるなら、動かさない
- ぶつからないなら、ぶつかるように最小限動かす

自然な貪欲法

- これでうまくいく
- 次のレーザー攻撃にぶつけるためにまっすぐに動かす必要があるのは当然
- レーザーにぶつかるところまで来たら, さらに移動する必要はない
 - 必要なら次の攻撃のときに移動させればよい

自然な貪欲法

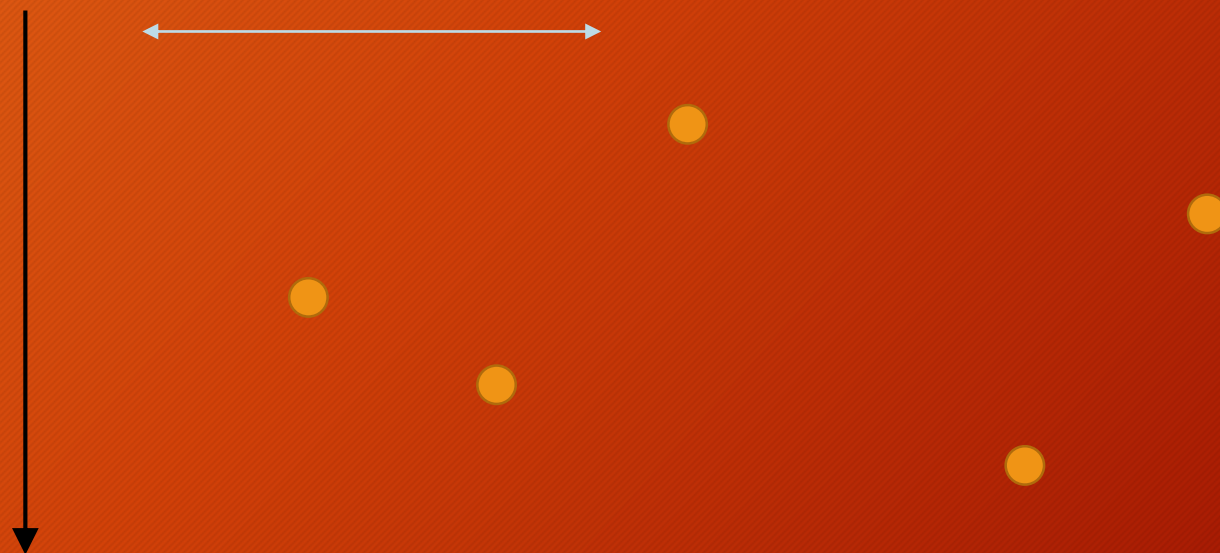
- 計算量の見積もり
- 壁ごとに独立
 - 壁ごとには, レーザー攻撃の回数だけ計算量がかかる $O(M)$
- 壁は N 個あるので $O(NM)$
- 小課題 1 が解ける (10 点)

観察

- レーザーが同じ位置に連続して来るときは、明らかに重複を除去できる
- レーザーの位置が「1→8→13」のように、連続して座標の値が増えるときには真ん中は除去できる
 - 防壁が 1 にぶつかった後 13 にぶつかるなら、間で 8 に必ずぶつかる
- レーザーの位置は、前処理で $a > b < c > d < e > \dots$ のようにできる

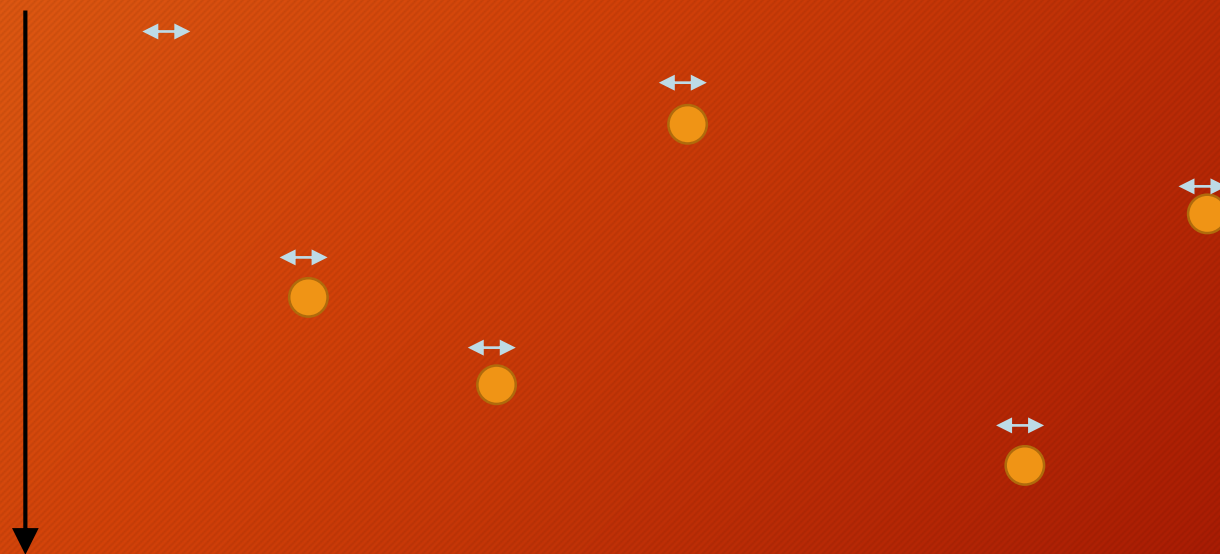
問題の見かけの転換

- 壁ごとに, いくつかの(予め決められた)点たちを順番に訪れるときの最小の移動回数を求める



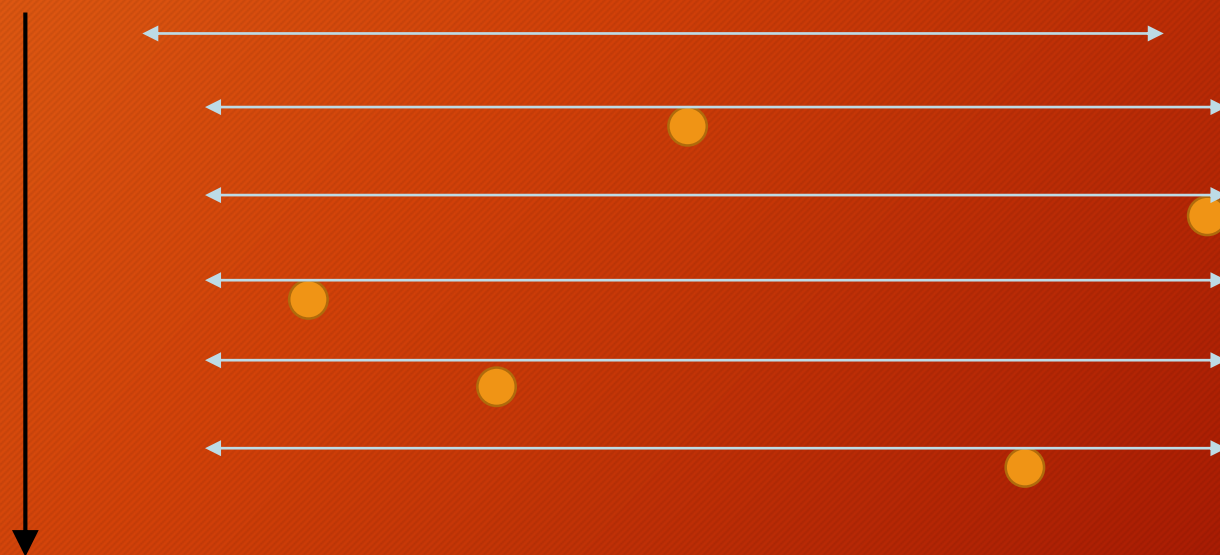
観察再び

- 壁の幅がめっちゃ狭い (0) のとき
- レーザーごとに必ず移動する必要がある



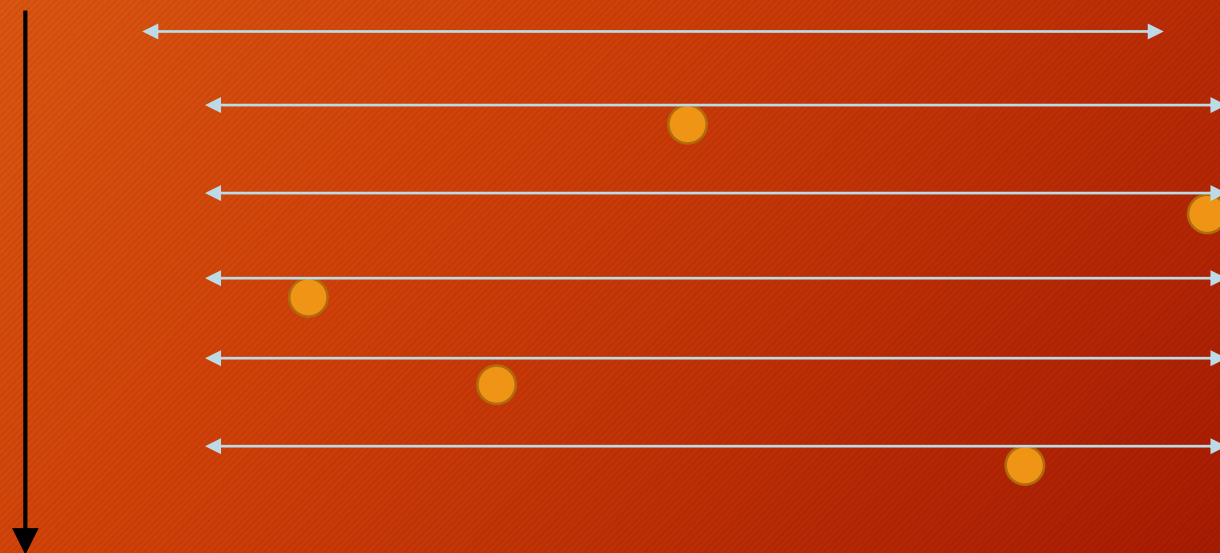
観察再び

- 壁の幅がめっちゃ広いとき
- すべてのレーザーにぶつけられるような壁の位置がある
- その位置に持ってくれば, 何度も左右に動かす必要はない



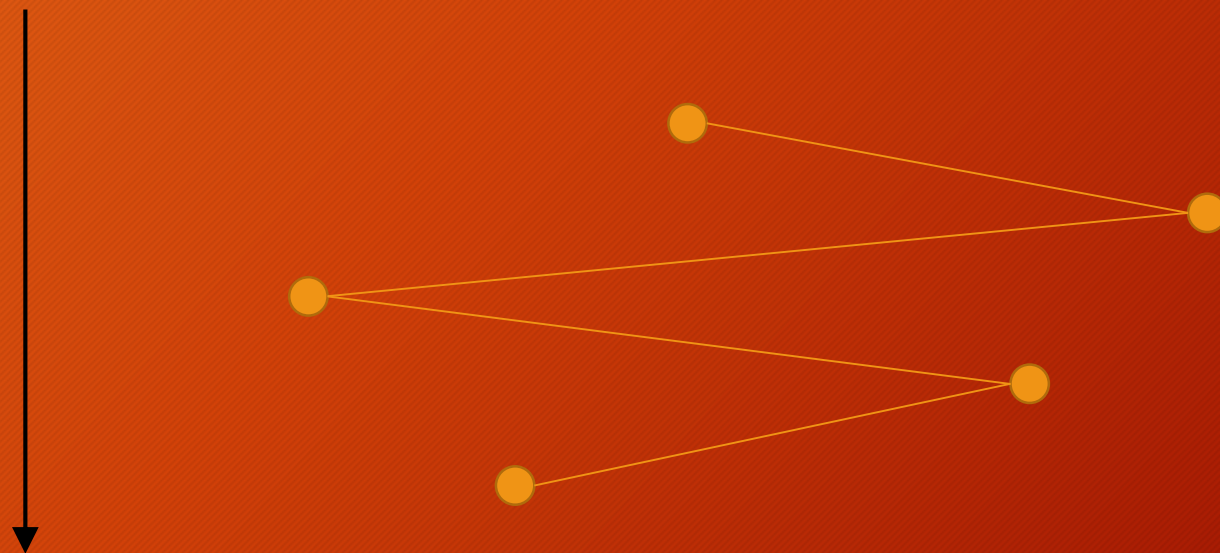
観察再び

- 防壁の幅がだんだん長くなっていくことを考えると、左右に何度も動かす必要が少なくなっていくそう



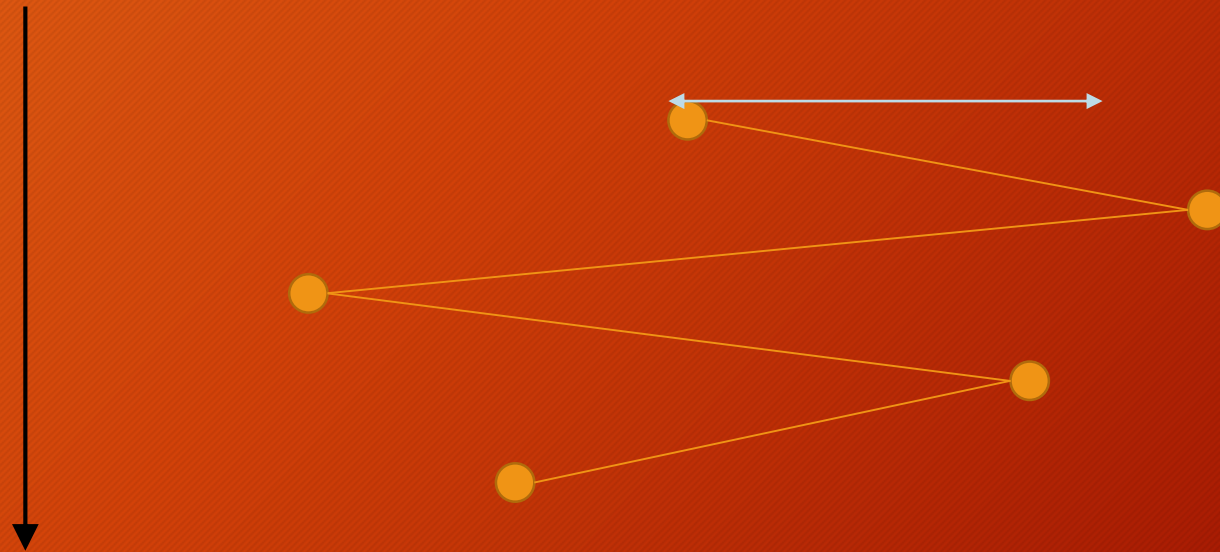
レーザーパス

- 縦軸にレーザー攻撃が何回目か，横軸にレーザー攻撃の位置をとり，連続する攻撃を結んでパスみたいなものを作る
- パスは左，右，左，右，...のような交互列みたいになる



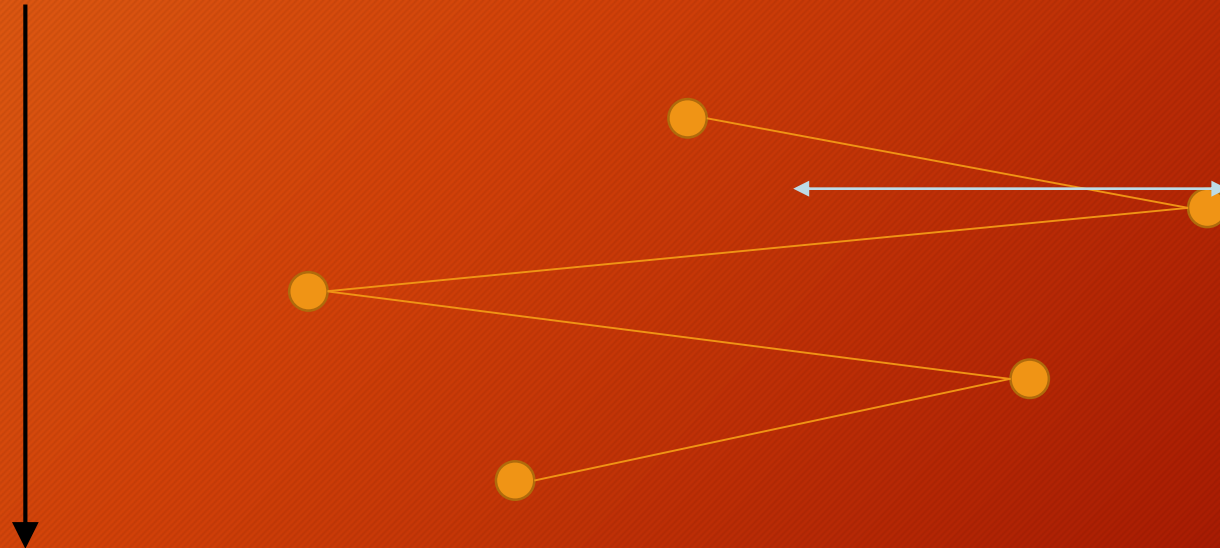
レーザーパス

- パスのある部分の長さより短い防壁は、その部分で必ず移動しないといけない



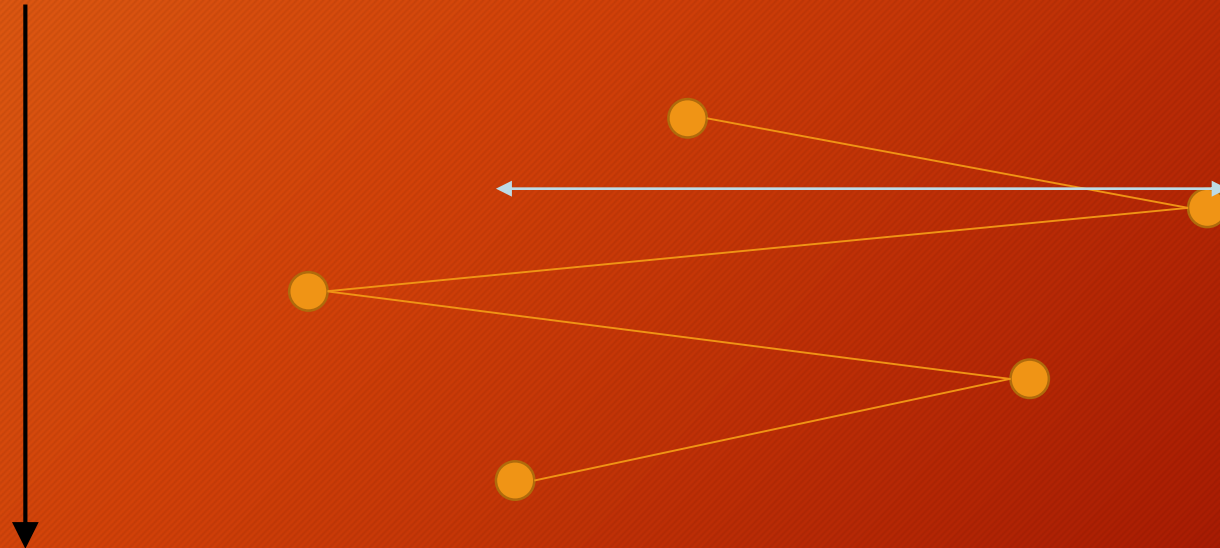
レーザーパス

- パスのある部分の長さより短い防壁は、その部分で必ず移動しないといけない



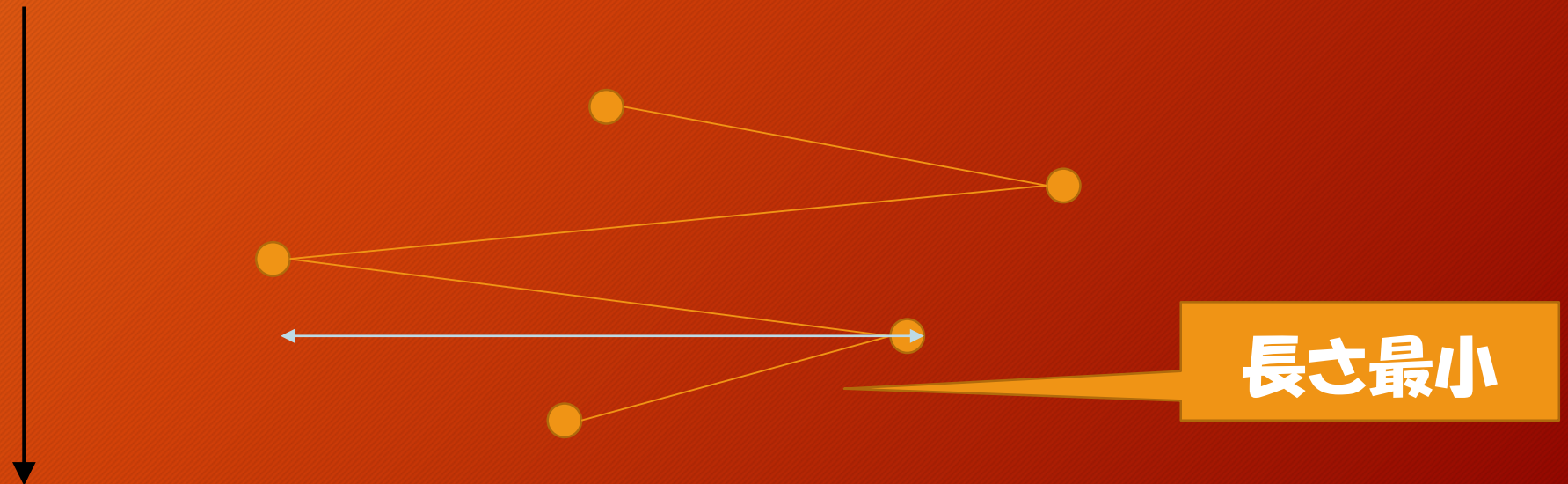
レーザーパス

- 逆に, パスのある部分の長さより「長い」防壁があったら何が起こるか？



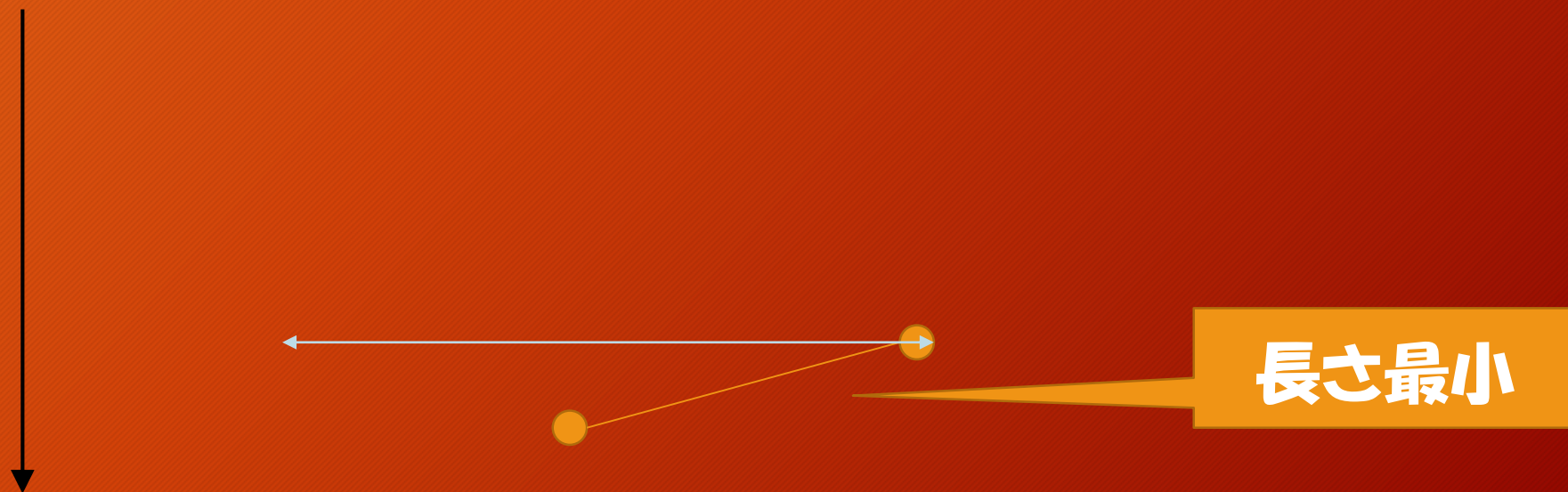
レーザーパス

- パスの部分で、長さが最小のものに着目し、それより長い防壁を動かすことを考える



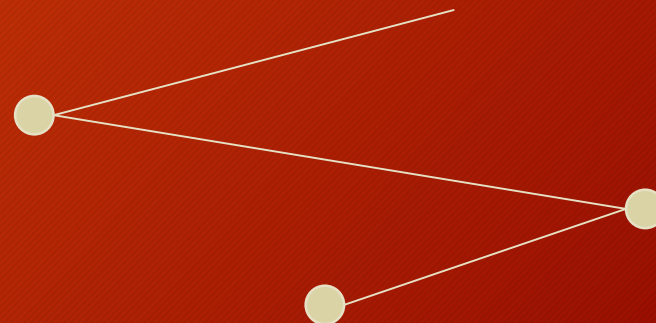
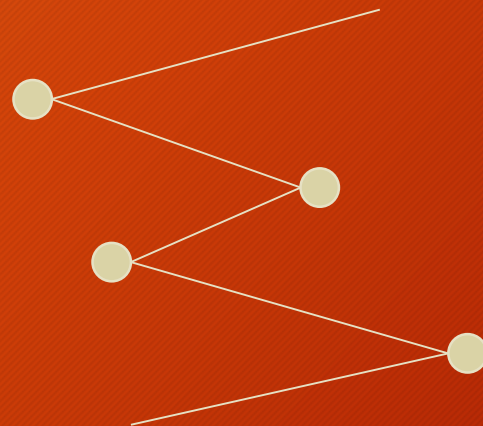
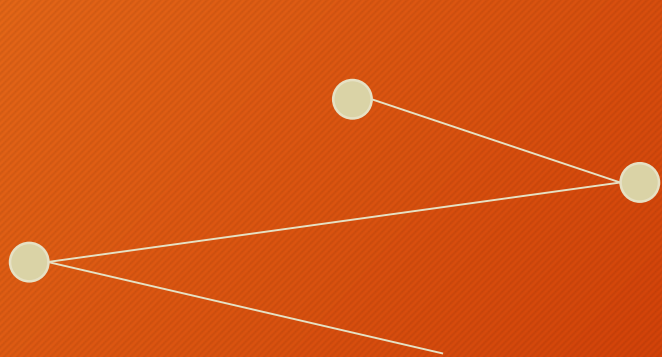
自明な場合

- パスが 1 個の部分だけからなる場合は「小課題 1」の方法で何ら問題ない



非自明な場合

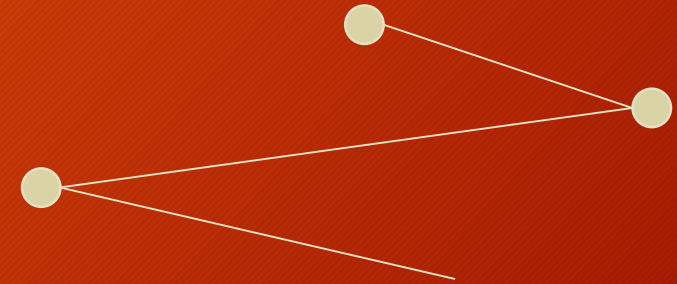
- 最小部分がパスのどこにあるかは 3 通り
 1. パスの最初
 2. 最初でも最後でもないところ
 3. パスの最後



非自明な場合

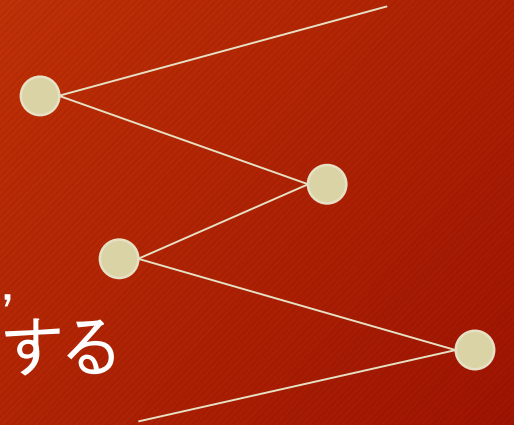
- パスの最初にある場合
- 一番最初のレーザー攻撃を単に取り除ける

- パスの 2 番目の移動を考えると, あるタイミングで
「防壁に 1, 2 番目の両方の攻撃をぶつけられる」ようになっている



非自明な場合

- パスの途中にある場合
- 攻撃 n , $n+1$ の間の部分が最短のとき, 攻撃 n , $n+1$ の両方を単に取り除ける
- 攻撃 $n-1$ に防壁をぶつけた後 $n+2$ にぶつける移動の間に, 攻撃 n , $n+1$ の両方に防壁をぶつけられるタイミングが存在する



解法の概略

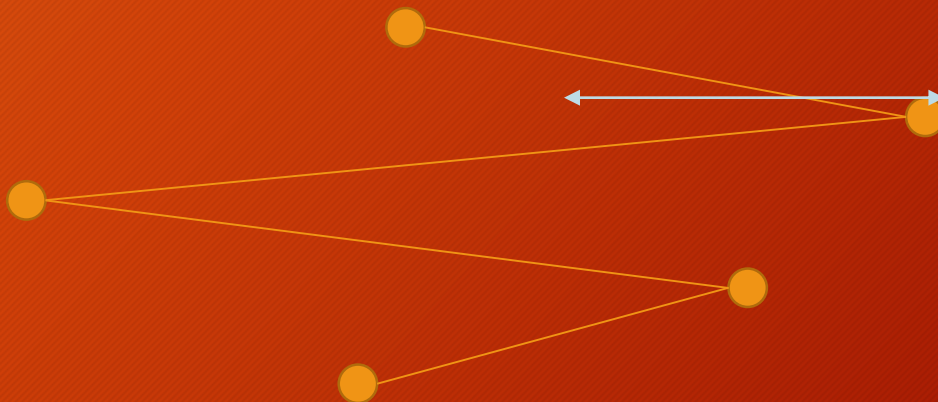
- レーザーパスを(交互列になるように)作る
- 防壁たちを幅の長さの昇順でソート
- 壁に対して, 順番に以下を行う:
 - パスの長さが短ければ何もしない
 - パスの中で, 長さ最小の部分を選び, その長さが防壁の長さ以下であれば, 適宜攻撃をなかったことにする
 - 現在のパスに対して, 壁の移動距離を求める

レーザーパスの効率よい管理

- 「長さ最小の部分を探す」と、「部分の挿入 / 削除」の操作が必要
- set を 2 つ持つと便利
 - 1 つは, (長さ, 位置) の順番で pair を作って入れる
 - もう 1 つは, (位置, 向き付き長さ) の順番で pair を作って入れる
 - 「位置」は, 必ずしも連番にならなくてもよいので, 挿入時には適当に位置の番号を選ぶ
 - 操作の際には両方の set を操作するようにする

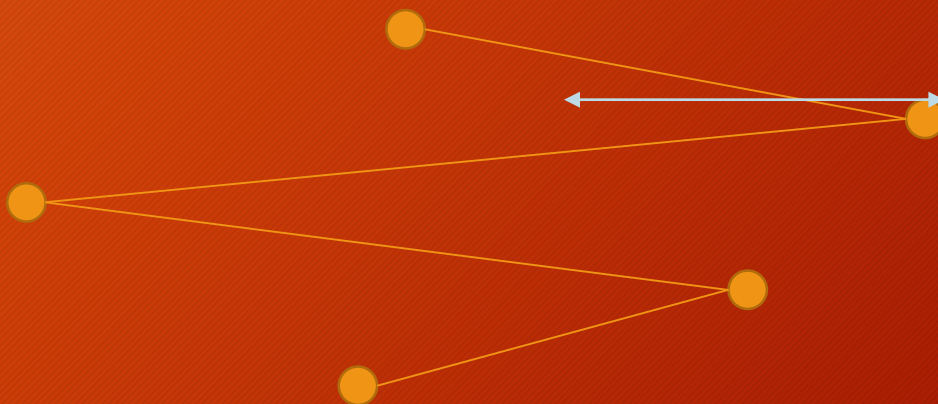
壁の移動距離の計算

- 最初の 1, 2 個のレーザー攻撃に対しては, 小課題 1 の方法で求める
- 壁がレーザーパスの各部分よりも短いので, 2 個目まで見た時には壁は攻撃を端の位置で受け止めている



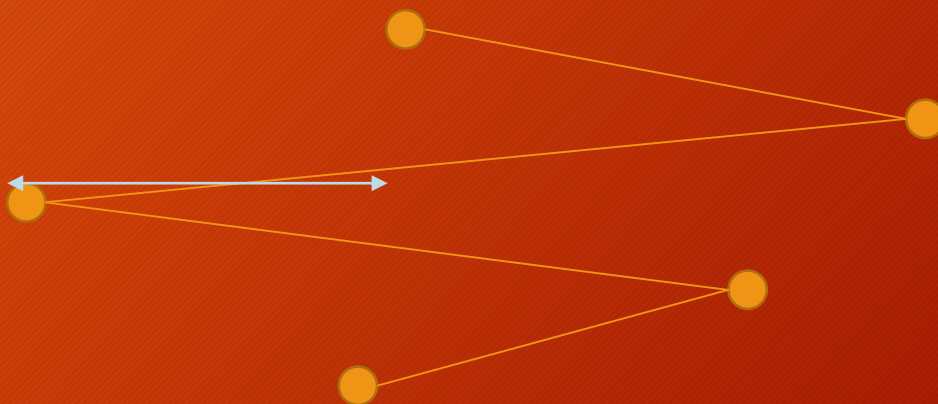
壁の移動距離の計算

- 2 個目以降については, 壁をぎりぎり端で受け止められるように左右に振るという移動になる
- 「レーザーパスの各部分の長さの和」を記録(set の更新時にこれも更新)しておけば, 計算により移動距離は求められる



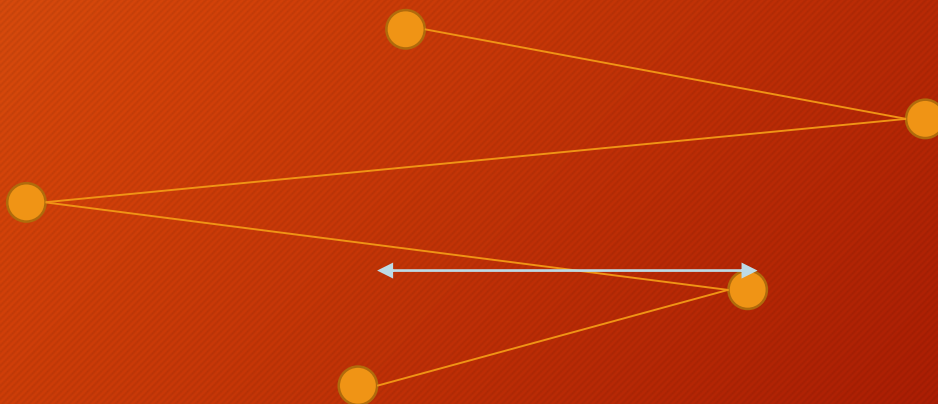
壁の移動距離の計算

- 2 個目以降については, 壁をぎりぎり端で受け止められるように左右に振るという移動になる
- 「レーザーパスの各部分の長さの和」を記録(set の更新時にこれも更新)しておけば, 計算により移動距離は求められる



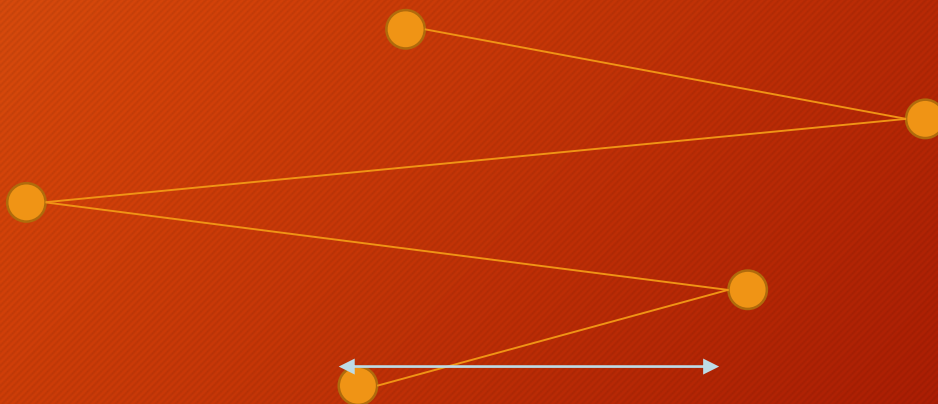
壁の移動距離の計算

- 2 個目以降については, 壁をぎりぎり端で受け止められるように左右に振るという移動になる
- 「レーザーパスの各部分の長さの和」を記録(set の更新時にこれも更新)しておけば, 計算により移動距離は求められる



壁の移動距離の計算

- 2 個目以降については, 壁をぎりぎり端で受け止められるように左右に振るという移動になる
- 「レーザーパスの各部分の長さの和」を記録(set の更新時にこれも更新)しておけば, 計算により移動距離は求められる



計算量

- レーザーパスの簡略化は $O(M)$ 回
 - 1 回の簡略化で必ず 1 個以上の攻撃がなかったことにされる
- 各簡略化は set を $O(1)$ 回操作するので計算量 $O(\log M)$
- 壁たちのソートに $O(N \log N)$
- 結局 $O(N \log N + M \log M)$
- 満点が得られる

得点分布

